Algorithms and Computation in Mathematics 27

[ACM]

Antonio Montes

The Gröbner Cover

Springer

Algorithms and Computation
in Mathematics • Volume 27

Editors

D. Eisenbud M. F. Singer
B. Sturmfels M. Braverman
B. Viray

More information about this series at http://www.springer.com/series/3339

Antonio Montes

The Gröbner Cover

Springer

Antonio Montes
Universitat Politècnica de Catalunya
Barcelona, Spain

ISSN 1431-1550
Algorithms and Computation in Mathematics
ISBN 978-3-030-03903-5 ISBN 978-3-030-03904-2 (eBook)
https://doi.org/10.1007/978-3-030-03904-2

Library of Congress Control Number: 2018962627

Mathematics Subject Classification (2010): 13-02, 13-04, 11C08, 13B25, 13A15, 13F20, 13P10, 13P15, 13P25, 68T15, 03B35

© Springer Nature Switzerland AG 2018
This work is subject to copyright. All rights are reserved by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.
The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.
The publisher, the authors and the editors are safe to assume that the advice and information in this book are believed to be true and accurate at the date of publication. Neither the publisher nor the authors or the editors give a warranty, express or implied, with respect to the material contained herein or for any errors or omissions that may have been made. The publisher remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

This Springer imprint is published by the registered company Springer Nature Switzerland AG
The registered company address is: Gewerbestrasse 11, 6330 Cham, Switzerland

To Rosa
for her love and support

Preface

The role of computers in mathematics has created a new perspective in which the development of algorithms becomes central. This was the reason for developing Computer Algebra Software systems (CAS). In 1965, Bruno Buchberger in his thesis [14], introduced Gröbner Basis (GB) and his famous algorithm providing the canonical representation of ordinary polynomial systems. It allows for solving problems concerning polynomial equations. Many problems in mathematics, engineering and other scientific domains can be modelled by polynomial equations.

Since then, Gröbner Basis has become a classical subject of the Computer Algebra curriculum at an undergraduate level due to the excellent text books of Cox, Little, O'Shea [25], Becker-Weispfenning [6], Adams-Loustaunau [2], Eisenbud [30], and Buchberger's algorithm has been implemented in many CAS systems.

Between the CAS systems implementing Gröbner bases, Singular [62], developed by Greuel, Pfister and Schönemann, at Kaiserslautern Technical University is one of the most efficient. The books [28, 39], contribute also to its relevance.

Ordinary Gröbner Bases can solve polynomial systems that depend only on variables but not on parameters. As the polynomial equations appearing in many technical problems contain parameters, a new approach was necessary. Volker Weispfenning [83] introduced Comprehensive Gröbner Systems (CGS) and Basis (GCB) to tackle polynomial systems with parameters. Since then, many other mathematicians have developed and improved new algorithms for the same purpose [41, 42, 42, 54–56, 66, 69, 75, 84].

In 2007 Michael Wibmer, in his thesis [85], proved a central theorem for parametric polynomial systems. Using it, in 2010, Wibmer and I [63] developed the Gröbner Cover, which is the canonical algorithm for solving parametric polynomial systems and is the analogue of the Reduced Gröbner basis for parametric systems.

Starting in 2008, the author with the priceless help of Hans Schönemann implemented the SINGULAR "*grobcov.lib*" library. Since then, the library has continuously been improved to solve new kinds of problems in which parametric polynomial systems are central.

This book gives a complete description of the Canonical Gröbner Cover. It also serves as a *User's Manual for the* SINGULAR "*grobcov.lib*" library, as it contains

many examples and comments. All the algorithms described are implemented in the library.

It is assumed that the reader is familiar with ordinary Gröbner basis theory of multivariate polynomials.

A first preliminary chapter of the book is devoted to some basic results (without proof or completeness) of ordinary Gröbner basis theory, which are widely used in this text. The objective is to introduce the notations and to recall some central elements of the theory. However, it should not, of course, be considered as a complete description of ordinary Gröbner basis theory. We also present a simple example of the use of the Gröbner Cover (GC) to give an idea of its usefulness.

Next, the book is divided into two parts. Part I: Theory has four chapters: Chaps. 2, 3, 4, contain the basic tools needed in the construction of the Gröbner Cover, which is described in Chap. 5.

Chapter 2 is devoted to constructible sets. In collaboration with Josep M. Brunat, we have recently established the canonical form of constructible sets [13]. It is described in this chapter and used later in the applications to loci computations. Nevertheless, for the Gröbner Cover it is not necessary to use the whole canonical algorithm, because Wibmer's Theorem [85] ensures that, for homogeneous polynomials, the union of all the segments of a Comprehensive Gröbner System corresponding to the same set of lpp (leading power products) is locally closed. Thus, it suffices to use a simpler algorithm called LCUnion.

Chapter 3 describes the algorithm for computing a disjoint and reduced Comprehensive Gröbner System (CGS), which is the previous step for the GC. When the GC was developed in 2008, we used our own algorithm, called BuildTree, which is described in the Appendix. In 2010, Kapur, Sun, Wang [43] designed a better algorithm for this purpose, which is described in this chapter; this is the CGS algorithm included in the current Singular library as part of the instruments needed for the GC.

Chapter 4 describes I-regular functions, which are functions defined on a locally closed set, and determined by one or more polynomials. They are essential for Wibmer's Theorem [85].

Chapter 5 is the central part describing the Gröbner Cover [63]. In it, Wibmer's Theorem and the existence of the Canonical Gröbner Cover are proved, and the algorithms provided. The theory and algorithms developed in Chaps. 2, 3 and 4 are used for this purpose. In a first reading one can go directly to this chapter and consult the previous chapters when needed.

Even though there are many possible different applications of the GC, we have chosen only three for which the Gröbner Cover gives new insight, that we have developed in collaboration with M.A. Abánades, F. Botana, T. Recio.

These three applications are the contents of Part II: Applications.

Chapter 6 is an application to Automatic Deduction of Geometric Theorems (ADGT). It was originally developed in collaboration with T. Recio [60, 61], and it has been improved in this chapter.

Chapter 7 is devoted to the GC in Dynamical Geometry (DG) for the computation and classification of geometrical loci. This is an interesting application that has been

tested in the widely known DG software GeoGebra. M.A. Abánades, F. Botana and Z. Kovacs have developed applications to GeoGebra, and together with T. Recio and the author [1], we defined a taxonomy for geometrical loci using GC. In this chapter, we also extend the definitions and theorems for locus taxonomy to n-dimensional space.

Chapter 8 is devoted to envelopes of families of hypersurfaces that can also be used in DG. A generalized definition of envelope is given that leads to solving new kinds of problems. New algorithms for obtaining the associated tangent family element at a certain point of the envelope, as well as determining all the family elements that pass at a certain point of the envelope, are developed. They can help analyze the structure of the real algebraic envelope components. This is a new development that has been improved and rewritten in this book. F. Botana, Z. Kovacs and T. Recio, have also developed applications to GeoGebra.

Algorithms and commands of the library are included in the book with different fonts. For example, GROBCOV is the algorithm and `grobcov` the implemented command of the library. Not every algorithm has a corresponding command since some algorithms are auxiliary and not public. Moreover, their names can be slightly different. For example EXTEND and `extendpoly` and `extendGC`. All the algorithms are described but only the implemented commands of the library are used in the examples.

Barcelona, Spain Antonio Montes

Acknowledgements

I am indebted and grateful to many people. First, to Prof. Tomás Recio (University of Cantabria, Spain), who initiatiated of the Spanish group of Computer Algebra to which I was invited from the beginning in 1994 during the MEGA'94 meeting (Effective Methods in Algebraic Geometry) at Cantabria University. Since then, I have received continuous scientific support from him. We have also collaborated in the development of interesting applications of the Gröbner Cover that are described in this book. We founded, together with other Spanish colleagues, the EACA committee (Encuentros de Algebra Computacional y Aplicaciones) and the EACA network, which has become a met in the international community of Computer Algebra. In MEGA'94 I also met Prof. Volker Weispfenning of Passau University, who is the next person to whom I am indebted. I was applying Gröbner bases for electrical networks, and he suggested using his recently introduced new algorithm for parametric polynomial systems, which is where my interest in the subject comes from. He invited me to Passau University and we established a warm relationship.

In the next years, I developed new algorithms for improving the Weispfenning CGS algorithm. My Ph.D. student Montserrat Manubens collaborated in this development. Together we developed first our BuildTree algorithm for obtaining a CGS, and then we designed another algorithm which, taking it as a starting point, combines segments with the same set of lpp allowing for a common basis. Thus the complete algorithm generates a very compact discussion [56]. I am also indebted to Prof. Yosuke Sato of Tokyo University who invited me to go to Japan and Korea to discuss the subject.

Most especially, I am indebted to Michael Wibmer, who proposed his central Theorem in 2007. He suggested that I develop an algorithm to implement it. The result was the Gröbner Cover (GC) [63], which starting in 2008, we developed at Heidelberg University. Without his ideas, it would not have been possible to develop the GC. In 2008, I was invited by Prof. Gert Martin Greuel to Kaiserslautern Technical University for a sabbatical period, where I collaborated with Prof. Hans Schönemann in the implementation of the GC algorithm using SINGULAR and creating the "*grobcov.lib*" library. The library has evolved, and has now become a robust package [62]. In 2016 I was once more invited to Kaiserslautern by Prof.

Wolfram Decker and Hans Schönemann to put the finishing touches on the library, thus enabling us to prepare this book.

I am also indebted to M.A. Abánades (Universidad Complutense de Madrid), F. Botana (Universidad de Vigo) and Zoltan Kovacs (Private University College of Education of the Diocese of Linz) who together with T. Recio and me developed applications of the Gröbner Cover in Dynamical Geometry for loci and envelope computations [1, 9, 10, 60, 61]. Chapter 7 is deeply indebted to Francisco Botana.

I also thank Prof. Deepak Kapur and his collaborators Yao Sun and Dingkang Wang for having developed his new algorithm for obtaining a disjoint classical CGS. It provides a reduced partition of the parameter space, and is more efficient than our own original BUILDTREE algorithm [43], thus enabling us to introduce it in the "*grobcov.lib*" library [59].

I would like to thank the following friends and colleagues: Josep M. Brunat (Universitat Politècnica de Catalunya), with whom I have collaborated in many projects (some of which are included in this book), and who has read and revised parts of the book, and to Pelegrí Viader (Universitat Pompeu Fabra) who has always listened to my doubts in writing the book. I thank also Lenny Tornero who has graciously helped me in the English revision of the text.

Last but not least, I am truly grateful to my wife Rosa Boada, who has freed me from many familiar requirements and has always encouraged me to write this book.

Contents

1 Preliminaries 1
- 1.1 Basic Notions for Multivariate Polynomials and Notations 1
- 1.2 Ordinary Gröbner Bases 4
- 1.3 Characterization of the Solutions of a Polynomial System 8
- 1.4 Parametric Systems and Example of the Gröbner Cover 10
- 1.5 Remark: Triangular Sets 12

Part I Theory

2 Constructible Sets 15
- 2.1 Introduction 15
- 2.2 Locally Closed Sets 16
- 2.3 Canonical Representation of Constructible Sets 20
- 2.4 Algorithms 26
- 2.5 Examples 32

3 Comprehensive Gröbner Systems and Bases 41
- 3.1 Introduction 41
- 3.2 Parametric Polynomial Systems and Specializations 42
- 3.3 Comprehensive Gröbner Systems 53
- 3.4 Suzuki-Sato Algorithm 55
- 3.5 Kapur-Sun-Wang Proof of the Existence of a CGS 56
- 3.6 Kapur-Sun-Wang Algorithm for Computing a CGS 58
- 3.7 Examples 60

4 I-Regular Functions on a Locally Closed Set 67
- 4.1 Regular and I-Regular Functions on a Locally Closed Set 67
- 4.2 Representations of Regular and I-Regular Functions 69

5 The Canonical Gröbner Cover 79
- 5.1 Introduction 79
- 5.2 Canonical Partition of the Parameter Space 80

5.3 Algorithms for Computing the Canonical Gröbner Cover 88
5.4 The SINGULAR Library and the grobcov Command 90

Part II Applications

6 Automatic Deduction of Geometric Theorems 99
6.1 Steiner-Lehmus Theorem 100
6.2 Automatic Discovery Using the Gröbner Cover 110
6.3 Orthic Triangle 113
6.4 Automatic Demonstration of Geometric Theorems 119

7 Geometric Loci 125
7.1 Elementary Loci 125
7.2 Locus of a Geometric Construction 126
7.3 Taxonomy of Loci 131
7.4 Algorithm LOCUS2 137
7.5 LOCUS Algorithm 153
7.6 Some Other Examples 169
7.7 Web Implementation in GeoGebra 177
7.8 Exercises 180

8 Geometric Envelopes 181
8.1 Envelopes 181
8.2 Definitions and Theorems 182
8.3 Algorithms 185
8.4 Simple Examples 187
8.5 New Examples 211
8.6 Open Problems 228
8.7 Loci with Two Movers 245
8.8 Exercises 251

A The BUILDTREE Algorithm 253
A.1 R-Representation of Special Locally Closed Sets 253
A.2 Algorithms for Splitting the Discussion 255
A.3 BUILDTREE Algorithm 259
A.4 Comparison of BUILDTREE and KAPUR-SUN-WANG Algorithms 262
A.5 Conclusions 266

Bibliography 269

Index 273

Chapter 1
Preliminaries



It is assumed that the reader is familiar with ordinary Gröbner Basis theory (GB). As GB is the support of the Gröbner Cover (GC), in Sects. 1.1, 1.2 and 1.3 we introduce some notations and recall central elements of the theory without proofs. This introduction should not be considered as a description of ordinary GB theory, for which we refer to the excellent books of Cox, Little, O'Shea (1992 1st ed, 2007 3rd ed) [25], Becker-Weispfenning [6], Adams-Loustaunau [2] and Eisenbud (1994 1st ed, 1998 3rd ed) [30]. For further relevant studies we refer to the books of Cox, Little, O'Shea [26] and Mora [64, 65].

In Sect. 1.4 we show the result of an application of the Gröbner Cover to a simple example. Even if GC is not described here, this example can be useful to understand its objective and usefulness.

Finally, we also make a brief reference to the theory of Triangular Sets, which is not developed in this book.

1.1 Basic Notions for Multivariate Polynomials and Notations

Throughout this book, we use the notations $\subseteq$ and $\subset$ to represent inclusion and strict inclusion, respectively.

Some notations used for multivariate polynomials over a field K are provided

- $\mathbf{x} = (x_1, \ldots, x_n)$ is the set of *variables*,
- $\mathbf{u} = (u_1, \ldots, u_m)$ is the set of *parameters*,
- $\alpha = (\alpha_1, \ldots, \alpha_n) \in \mathbb{Z}^n$ is an *exponent vector*,
- K is a computable field; L an algebraically closed extension of K.
- $K[\mathbf{x}]$ is the *polynomial ring* with variables $\mathbf{x}$ over the field K.
- $K[\mathbf{u}][\mathbf{x}]$ is the polynomial ring over the ring $K[\mathbf{u}]$ of the parameters,
- $\mathbf{x}^\alpha = x_1^{\alpha_1} \ldots x_n^{\alpha_n}$ is a *monomial* or *power product*,

© Springer Nature Switzerland AG 2018

A. Montes, *The Gröbner Cover*, Algorithms and Computation
in Mathematics 27, https://doi.org/10.1007/978-3-030-03904-2_1

- $\mathbf{c}_\alpha \in K[\mathbf{u}]$ is a *coefficient*,
- $\mathbf{c}_\alpha \mathbf{x}^\alpha$ is a *term*,
- $\deg(\mathbf{x}^\alpha) = |\alpha| = \alpha_1 + \cdots + \alpha_n$ is the *degree* of the monomial $\mathbf{x}^\alpha$,
- $f = \sum_\alpha \mathbf{c}_\alpha \mathbf{x}^\alpha \in K[\mathbf{u}][\mathbf{x}]$ is a *parametric polynomial*,
- $\text{supp}(f)$ = *support* of $f \in K[\mathbf{u}][\mathbf{x}]$ is the set of all monomials in the terms of f with non-null coefficients,
- $\deg(f) =$ the maximum degree of the monomials in $\text{supp}(f)$.

Let K be a computable field.

Definition 1.1 (Ideal) A subset $I \subset K[\mathbf{x}]$ is an *ideal* if it satisfies

(i) $0 \in I$;
(ii) If $f, g \in I$, then $f + g \in I$;
(iii) If $f \in I$ and $h \in K[\mathbf{x}]$, then $hf \in I$.

Definition 1.2 (Basis of an ideal) We set

$$\langle f_1, \ldots, f_s \rangle = \left\{ \sum_{i=1}^{s} h_i f_i \ : \ \forall i, h_i \in K[\mathbf{x}] \right\}.$$

Then $\langle f_1, \ldots, f_s \rangle$ is an ideal, and we say that the set $B = \{f_1, \ldots, f_s\}$ is a basis of I.

There exist infinitely many bases of an ideal. Given an ideal I, we say that f belongs to $\sqrt{I}$ if and only if there exists some integer n for which $f^n \in I$. The set $\sqrt{I}$ is an ideal and we denote it to be the radical of I.

The set of points in which all the polynomials of an ideal vanish is called a *variety*. We consider the varieties defined over L^n, where L is an extension field of the algebraic closure $\overline{K}$ of K. An ideal $I = \langle p_1, \ldots, p_r \rangle$ defines the variety:

$$\begin{aligned} \mathbf{V}(I) =& \{\mathbf{u} \in L^n : p_1(\mathbf{u}) = 0, \ldots, p_r(\mathbf{u}) = 0\} \\ =& \{\mathbf{u} \in L^n : f(\mathbf{u}) = 0, \forall f \in I\}, \end{aligned}$$

representing the solutions of the system of equations B.

Different ideals can define the same variety. Over an algebraically closed field L, the Nullstellensatz establishes a one-to-one correspondence between radical ideals and varieties. If I is an ideal and $\mathfrak{a} = \sqrt{I}$ is its radical, then we have the bijection between $\mathbf{V}(I)$ and $\sqrt{I} = \mathbf{I}(V)$:

$$V = \mathbf{V}(I) = \mathbf{V}(\mathfrak{a}) \longleftrightarrow \mathfrak{a} = \sqrt{I} = \mathbf{I}(V).$$

Thus, the best ideal for defining a given variety over an algebraically closed field L is the radical ideal $\mathfrak{a} = \sqrt{I}$, which is also the greatest ideal defining the variety and is also called the *ideal of the variety* $\mathfrak{a} = \mathbf{I}(V)$.

This is the reason for using the K-Zariski topology of L^n, (see [3]), where the closed sets are the varieties of L^n defined by polynomials over K, where K is a computable field and L is an algebraically closed extension of K. We need K to be computable in order to be able to decide when two polynomials are equal, etc.

The ideal $\langle 1 \rangle$ represents the whole polynomial ring.

Using the K-Zariski topology of L^n, any proper ideal different from $\langle 1 \rangle$ has solutions in L^n. From here on, we shall always use this topology and consider particularly the field K to be the field $\mathbb{Q}$ of the rational numbers and the field L to be $\mathbb{C}$, the field of complex numbers. But the study can be extended to any computable field K and an algebraic closed extension of it.

We also define a variety V to be *irreducible* if it cannot be decomposed as a union of smaller varieties V_1 and V_2 with $V_i \subset V$. Irreducible varieties correspond to prime ideals. An ideal is prime if when it contains a product $fg \in I$, then $f \in I$ or $g \in I$.

A *monomial order* or *term order* $\succ$ is a relation on the set of monomials $\mathbf{x}^\alpha$ (or equivalently $>$ in the set of exponent vectors α) with the following properties:

(1) $\succ$ is a total order,
(2) it is compatible with multiplication, i.e., if $\mathbf{x}^\alpha \succ \mathbf{x}^\beta$ and $\mathbf{x}^\gamma$ is any monomial, then $\mathbf{x}^\alpha \mathbf{x}^\gamma = \mathbf{x}^{\alpha+\gamma} \succ \mathbf{x}^{\beta+\gamma} = \mathbf{x}^\beta \mathbf{x}^\gamma$,
(3) $\succ$ is a well ordering, i.e., every non-empty collection of monomials has a smallest element.

The most frequently used monomial orders are lexicographical order (lex) and graded reverse lexicographical order (grevlex). The lex order is defined by $\alpha > \beta$ if the first non-null entry of $\alpha - \beta$ beginning on the left is positive. The grevlex order is a graded order, and is defined by $\alpha > \beta$ if $\deg(\alpha) > \deg(\beta)$ or when $\deg(\alpha) = \deg(\beta)$, then the first non-null entry of $\alpha - \beta$ beginning on the right is negative. For example,

$$x^3y^5 \succ_{\mathrm{lex}(x>y)} xy^8, \quad \text{but} \quad x^3y^5 \prec_{\mathrm{grevlex}(x>y)} xy^8$$

because the first variable has greater exponent in the first monomial, but the second monomial has a total degree greater than the first.

Given a monomial order $\succ$ on the monomials of $\mathbb{Q}[\mathbf{x}]$ and a polynomial $f \in \mathbb{Q}[\mathbf{x}]$ we denote

(1) $\mathrm{lpp}_\succ(f)$ = *leading power product* of f: is the highest monomial in the support of f;
(2) $\mathrm{lc}_\succ(f)$ = *leading coefficient* of f, i.e., $\mathrm{lc}_\succ(f) = \mathrm{coef}(f, \mathrm{lpp}_\succ(f))$;
(3) $\mathrm{lt}_\succ(f)$ = *leading term* of f, i.e., $\mathrm{lt}_\succ(f) = \mathrm{lc}_\succ(f)\ \mathrm{lpp}_\succ(f)$.

1.2 Ordinary Gröbner Bases

Gröbner bases were introduced in 1960 in the thesis of Bruno Buchberger [14, 15], the Austrian mathematician Wolfgang Gröbner(1899–1980) being his advisor. We refer to [25] for the basic topics defined in this section and used in this text.

When a monomial order $\succ$ is fixed, we can use a pseudo-division algorithm REDUCE of a polynomial $f \in \mathbb{Q}[\mathbf{x}]$ by the divisors $\{g_1, \ldots, g_s\} \in \mathbb{Q}[\mathbf{x}]$. The result is a remainder $r \in \mathbb{Q}[\mathbf{x}]$ and a set of quotients $q_1, \ldots, q_r \in \mathbb{Q}[\mathbf{x}]$ with the following properties.

(1) $f = \sum_{i=1}^{s} q_i g_i + r$;
(2) no monomial in $\mathrm{supp}(r)$ can be divided by $\mathrm{lpp}(g_i)$ for any i;
(3) $\mathrm{lpp}(f) \succeq \mathrm{lpp}(q_i g_i)$ for any i.

The algorithm is a generalization of the ordinary division algorithm on $K[x]$. But now instead of a single divisor, there are a list of divisors. We must fix the order in which they will be used and try to divide by the first divisor whose lpp divides the lpp of the partial residue p. If no $\mathrm{lpp}(g_i)$ divides $\mathrm{lpp}(p)$, then $\mathrm{lt}(p)$ passes to the remainder and so on. For example,

Example 1.3 Divide $f = x^2 y + x y^2 + y^2$ by $g_1 = x y - 1$ and $g_2 = y^2 - 1$ in $\mathrm{lex}_{x \succ y}$ order and considering first g_1 then g_2.

We set $p = f$ and dispose the result as follows (with the understanding that we use the symbol $\sqrt{\ }$ only to highlight the dividend);

$$
\begin{array}{ll}
 & \underline{\quad q \quad} \qquad\qquad \underline{\quad r \quad} \\
g_1 = x y - 1 : & \\
g_2 = y^2 - 1 : & \\
 & \\
\sqrt{x^2 y + x y^2 + y^2} &
\end{array}
$$

We test, in order the first $\mathrm{lm}(g_i)$ that divides $\mathrm{lm}(p)$. If so, we set the partial quotient $q_i = \mathrm{lm}(p)/\mathrm{lm}(g_i)$ in place and subtract $q_i g_i$ from p. If none of the $\mathrm{lm}(g_i)$ divides $\mathrm{lm}(p)$, then $\mathrm{lt}(p)$ passes to the remainder.

The first step $\mathrm{lm}(g_1)$ divides $\mathrm{lm}(p)$ and the first partial quotient is x:

$$
\begin{array}{ll}
 & \underline{\quad q \quad} \qquad\qquad \underline{\quad r \quad} \\
g_1 = x y - 1 : & x \\
g_2 = y^2 - 1 : & \\
 & \\
\sqrt{x^2 y + x y^2 + y^2} & \\
-x^2 y + x & \\
\hline
x y^2 + x + y^2 &
\end{array}
$$

The second step $\mathrm{lm}(g_1)$ divides $\mathrm{lm}(p)$ and the second partial quotient is y:

$$\begin{array}{lll} & \underline{q} & \underline{r} \\ g_1 = x\,y - 1: & x + y & \\ g_2 = y^2 - 1: & & \end{array}$$

$$\begin{array}{l} \sqrt{x^2\,y + x\,y^2 + y^2} \\ \underline{-x^2\,y + x} \\ x\,y^2 + x + y^2 \\ \underline{-xy^2 + y} \\ x + y^2 + y \end{array}$$

The third step no $\mathrm{lm}(g_i)$ divides $\mathrm{lm}(p)$, and so x passes to the remainder:

$$\begin{array}{lll} & \underline{q} & \underline{r} \\ g_1 = x\,y - 1: & x + y & \\ g_2 = y^2 - 1: & & \end{array}$$

$$\begin{array}{ll} \sqrt{x^2\,y + x\,y^2 + y^2} & \\ \underline{-x^2\,y + x} & \\ x\,y^2 + x + y^2 & \\ \underline{-xy^2 + y} & \\ x + y^2 + y & \\ \underline{-x} & x \\ y^2 + y & \end{array}$$

The fourth step $\mathrm{lm}(g_2)$ divides $\mathrm{lm}(p)$, being 1 the third partial quotient:

$$\begin{array}{lll} & \underline{q} & \underline{r} \\ g_1 = x\,y - 1: & x + y & \\ g_2 = y^2 - 1: & 1 & \end{array}$$

$$\begin{array}{ll} \sqrt{x^2\,y + x\,y^2 + y^2} & \\ \underline{-x^2\,y + x} & \\ x\,y^2 + x + y^2 & \\ \underline{-xy^2 + y} & \\ x + y^2 + y & \\ \underline{-x} & x \\ y^2 + y & \\ \underline{-y^2 + 1} & \\ y + 1 & \end{array}$$

Finally, neither y nor 1 are divisible by any of the $\mathrm{lm}(g_i)$ and they both pass to the remainder:

$$
\begin{array}{lll}
 & \underline{q} & \underline{r} \\
g_1 = x\,y - 1: & x + y & \\
g_2 = y^2 - 1: & 1 &
\end{array}
$$

$$
\begin{array}{l}
\sqrt{x^2\,y + x\,y^2 + y^2} \\
\underline{-x^2\,y + x} \\
x\,y^2 + x + y^2 \\
\underline{-xy^2 + y} \\
x + y^2 + y \\
\underline{-x} \qquad\qquad\qquad\

It is easy to prove that given an ideal I and a monomial order $\succ$, the following statements are equivalent:

(i) $G = \{g_1, \ldots, g_s\}$ is a Gröbner basis of I;
(ii) the remainder of the pseudo-division of any polynomial by G is unique.

So, using Gröbner bases we recover the property that allow us to know wether or not a polynomial $f \in \mathbb{Q}[\mathbf{x}]$ belongs to the ideal I. Let G be a Gröbner basis of I. Therefore f belongs to I if and only if the remainder of the division by G is zero.

Define the S-polynomial of f and g as

$$S(f,g) = \mathrm{lc}(g)\frac{\mathrm{lcm}(\mathrm{lpp}(f),\mathrm{lpp}(g))}{\mathrm{lpp}(g)}f - \mathrm{lc}(f)\frac{\mathrm{lcm}(\mathrm{lpp}(f),\mathrm{lpp}(g))}{\mathrm{lpp}(f)}g.$$

The S-polynomial of (f, g) is defined to cancel the leading terms of f and g, in trying to obtain a new polynomial of the ideal $\langle f, g\rangle$ with a smaller lpp.

With a fixed monomial ordering, the famous Buchberger's algorithm ([25], p. 88) computes a Gröbner basis of a given ideal. Given an ideal $I = \langle g_1, g_2, \ldots, g_s\rangle$, the algorithm consists of obtaining new polynomials by computing all possible S-polynomials of the basis and reducing them modulo the other ones, until any new S-polynomial reduces to zero. The result is a Gröbner basis of I. Then we can reduce each polynomial in the resulting basis by the remaining ones, and moreover divide each polynomial by its leading coefficient to make it monic. We denote this algorithm REDUCEDGRÖBNERBASIS, and refer to [25].

To illustrate the concepts, let us consider the following example.

Example 1.5 Let

$$I = \langle g_1, g_2\rangle = \langle xy + x - 4y + 2, g_2 = x - y + 2\rangle,$$

and apply Buchberger's algorithm using the $\mathrm{lex}_{x\succ y}$ monomial order. We compute

$$p = S(g_1, g_2) = g_1 - yg_2 = x + y^2 - 6y + 2$$

and reduce it modulo $\{g_1, g_2\}$. We have

$$g_3 = \overline{p}^{\{g_1,g_2\}} = y^2 - 5y.$$

In this example the remaining S-polynomials $S(g_1, g_3)$, and $S(g_2, g_3)$ both reduce to zero. Thus, we obtain a Gröbner basis

$$B = \{g_1, g_2, g_3\} = \{xy + x - 4y + 2, x - y + 2, y^2 - 5y\}.$$

From a given Gröbner basis, it is easy to obtain the unique reduced basis, by dividing each polynomial in it by the remaining set of polynomials and normalizing the

polynomials. Here we have

$$G = \{g_2, g_3\} = \{x - y + 2, y^2 - 5y\}.$$

The reduced Gröbner basis is the canonical object identifying an ideal, since it does not depend on the algorithm but only on the fixed monomial order, two ideals being equal if and only if they have the same reduced Gröbner basis.

As we can see in this example, the reduced Gröbner basis gives helpful information about the solutions $\mathbf{V}(I)$. We have here obviously

$$\mathbf{V}(I) = \{(-2, 0), (3, 5)\}.$$

1.3 Characterization of the Solutions of a Polynomial System

Let G be the reduced Gröbner basis of the ideal I. Denote $\mathrm{lpp}(G)$ the set of the lpp's of the basis, which by definition of Gröbner Basis is a minimal generating set of $\mathrm{lpp}(I)$.

The most relevant types of solutions of a polynomial system are:

(1) The system has no solution, i.e., $\mathbf{V}(I) = \emptyset$.
(2) The system has a finite number of solutions. In that case we want to determine the number of solutions $|\mathbf{V}(I)|$.
(3) $\mathbf{V}(I)$ is infinite. In that case we want to determine the dimension $\dim(\mathbf{V}(I))$ of $\mathbf{V}(I)$.

Case (1). As we consider the solutions on an algebraic closed field, we can use the Nullstellensatz and so the system has no solution if and only if $G = \{1\}$.

Case (2) is described by the following Theorem ([25], p. 232, Theor. 6).

Theorem 1.6 (Finite number of solutions) *Let $I \subset \mathbb{Q}[\mathbf{x}]$ be an ideal, $\succ$ a monomial order and G a Gröbner basis of I. Then the following statements are equivalent:*

(i) $\mathbf{V}(I)$ *is a finite set.*
(ii) *For every $1 \leq i \leq n$, there exists some $r_i > 0$ such that $x_i^{r_i} = \mathrm{lpp}(g)$, for some $g \in G$.*
(iii) *The number of solutions of the system, including multiplicity, is equal to the dimension of the $\mathbb{Q}$-vector space $\mathbb{Q}[\mathbf{x}]/I$, that is equal to the number of monomials α such that $\mathbf{x}^\alpha \notin \langle \mathrm{lpp}(I) \rangle$.*

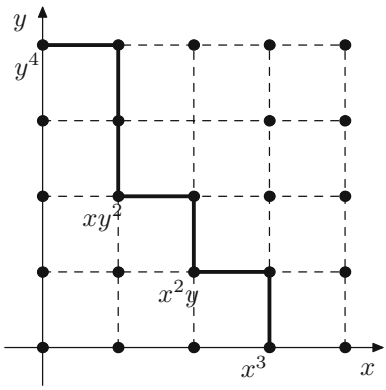


Fig. 1.1 Principal monomial of $\mathbb{Q}[x]/I$ of Example 1.7

Example 1.7 Let G be the reduced Gröbner basis of an ideal with respect to lex(x, y) order:

$$\begin{aligned} G = \{&y^4 - 7y^3 + 11y^2 + 7y - 12,\\ &xy^2 - x - 2y^3 + 4y^2 + 2y - 4,\\ &4x^2y - 4x^2 - 9y^3 + 24y^2 + 5y - 20,\\ &8x^3 - 24x^2 - 8x - 11y^3 + 36y^2 + 11y - 12\}. \end{aligned}$$

We have $\text{lpp}(G) = \{y^4, xy^2, x^2y, x^3\}$. As $\text{lpp}(G)$ contains x^3 and y^4, the variety has a finite number of solutions. The basis of $\mathbb{Q}[\mathbf{x}]/I$ is the set of monomials that are not divisible by any element of $\text{lpp}(G)$, i.e., $B_v = \{1, y, y^2, y^3, x, xy, x^2\}$ (see Fig. 1.1). As a consequence, by Theorem 1.6, it has 7 solutions counting multiplicities, i.e., it has at most 7 different solutions. In this example it is easy to see that the solutions are $\{(4, 4), (2, 3), (3, 1), (1, 1), (-1, 1), (1, -1), (-1, -1)\}$ that are effectively 7.

Case (iii). Finally consider the case of varieties with infinitely many solutions. First, consider monomial ideals, i.e., ideals generated by monomials. The following Proposition ([25], p. 411, Prop. 3) gives its dimension:

Proposition 1.8 (Dimension of a monomial ideal) *Let $I = \langle \mathbf{x}^{\alpha_1}, \ldots, \mathbf{x}^{\alpha_s} \rangle \subset \mathbb{Q}[\mathbf{x}]$ be a monomial ideal, generated by the monomials $\mathbf{x}^{\alpha_i}$. Denote*

$$M_j = \{k \in \{1, \ldots, n\} : x_k \textit{ divides the monomial } \mathbf{x}^{\alpha_j}\}$$

having positive exponent in $\mathbf{x}^{\alpha_j}$. (Observe that every M_j is non-empty whenever $I \neq \mathbb{Q}[\mathbf{x}]$). Setting

$$M = \{J \subset \{1, \ldots, n\} : J \cap M_j \neq \emptyset \textit{ for all } 1 \leq j \leq s\},$$

then

$$\dim(\mathbf{V}(I)) = n - min\{|J| : J \in M\}.$$

The above proposition allow us to determine the dimension of the variety of a general ideal when we know a Gröbner basis using the following Theorem ([25], p. 428, Prop. 4, and p. 431, Theor. 8):

Theorem 1.9 (Dimension of a Variety) *Let $I \subset \mathbb{Q}[\mathbf{x}]$ be an ideal, $\succ$ a graded monomial order and G a Gröbner basis of I. Then*

$$\dim(\mathbf{V}(I)) = \dim(V(\mathrm{lpp}(I))) = \dim(V(\mathrm{lpp}(G)).$$

Example 1.10 Let us give an example with dimension greater than 0.

Let $G = \{x^2z + y^2z - z, x^3y + xy^3 - xy\}$ be the reduced Gröbner basis of an ideal in the graded order grevlex(x, y, z). We have $\mathrm{lpp}(G) = \{x^2z, x^3y\}$, so that, $M_1 = \{1, 3\}$, $M_2 = \{1, 2\}$. The subset of $\{1, 2, 3\}$ with fewer elements having non-null intersection with all the M_j, is $M = \{1\}$, which has a unique element. Thus, $\dim(\mathbf{V}(G)) = 3 - 1 = 2$.

Remark 1.11 Using the primary decomposition algorithm [36], we can see that the variety can be decomposed into three irreducible varieties: a cylinder and two lines, namely: $V = \mathbf{V}(x^2 + y^2 - 1) \cup \mathbf{V}(x, z) \cup \mathbf{V}(x, y)$. So its dimension is the dimension of the greatest irreducible variety inside, i.e. the cylinder, whose dimension is 2.

1.4 Parametric Systems and Example of the Gröbner Cover

Let us consider a parametric polynomial system of equations

$$\begin{cases} p_1(u_1, \ldots, u_m; x_1, \ldots, x_n) = 0 \\ \qquad\qquad\qquad\qquad\qquad\qquad \ldots \\ p_r(u_1, \ldots, u_m; x_1, \ldots, x_n) = 0 \end{cases}$$

with coefficients in the computable field K, where $\mathbf{x} = x_1, \ldots, x_n$ are the variables and $\mathbf{u} = u_1, \ldots, u_m$ are the parameters. The system can be denoted $F = \{p_1, \ldots, p_r\} \subset K[\mathbf{u}][\mathbf{x}]$. For polynomials $f \in K[\mathbf{u}][\mathbf{x}]$ we can extend the monomial order defined in $K[\mathbf{x}]$ and all the concepts of leading monomials, terms and coefficients. Now $\mathrm{lc}(f) \in K[\mathbf{u}]$, $\mathrm{lpp}(f) \in K[\mathbf{x}]$ and $\mathrm{lt}(f) \in K[\mathbf{u}][\mathbf{x}]$.

The problem is obtaining the solutions $\mathbf{x} \in L^n$, where L is an algebraically closed extension field of K for every fixed value of the parameters $\mathbf{u} \in L^m$. In practice, we deal with the fields $K = \mathbb{Q}$ of the rational numbers for the coefficients and the field $L = \mathbb{C}$ of the complex numbers for the values, but the reader can generalize it to other fields K and L.

The book is devoted to studying of this problem, and in the following chapters all the necessary tools are developed. As a preview, from an algebraic perspective, the best result that can be obtained is the Gröbner Cover (GC) of the ideal $\langle F\rangle$. The algorithm GROBCOV, introduced in [63] and described in Chap. 5, computes the GC. The Gröbner Cover is the objective of this book and is largely discussed here. Interesting applications are also shown.

To show the powerfulness of the method, let us give an example.

Example 1.12 Consider the equation $y^2 = x^3 + ax + b$ of a cubic curve in Weierstrass form. We intend to determine for which parameter values it has singular points, its position and characteristics. Thus we have to add the derivatives with respect to the variables. The ideal corresponding to the problem is $S = \langle y^2 - x^3 - ax - b, -3x^2 - a, 2y\rangle$. We want to know for which values of the parameters (a, b) the cubic has singular points, and what its nature is.

Applying the `grobcov` command of the SINGULAR "grobcov.lib" library [62], we obtain an output that can be summarized in the following table:

1. lpp $= \{1\}$. Basis: $\{1\}$ Segment: $\mathbb{C}^2 \setminus \mathbf{V}(27b^2 + 4a^3)$: Generic segment Description: Non-degenerate cubic without singular points.
2. lpp $= \{x, y\}$ Basis: $\{2ax + 3b, y\}$ Segment: $\mathbf{V}(27b^2 + 4a^3) \setminus \mathbf{V}(a, b)$ Description: 1 singular point, intersection of two branches.
3. lpp $= \{y, x^2\}$ Basis: $\{y, x^2\}$ Segment: $\mathbf{V}(a, b)$ Description: 1 double singular point, cusp.

The output of the GROBCOV command contains a list of segments with fixed lpp (lpp-segments), each of which has 3 elements:

(1) The set of the lpp's (leading power products) of the basis.
(2) The basis, that specializes to the reduced Gröbner basis on the segment.
(3) The subset of the parameter space (segment).

Let us describe the output in the example. We have 3 lpp segments. In this simple example the first segment with lpp=$\langle 1\rangle$ corresponds to no solution, i.e., no singular points of the cubic, and this happen for all values of (a, b) except when $27b^2 + 4a^3 = 0$.

The second segment has lpp=$\langle x, y\rangle$, and corresponds to the solution $x = -\frac{3b}{2a}$, $y = 0$, which happens only for values of (a, b) verifying $27b^2 + 4a^3 = 0$,

except for the point $(0, 0)$. Thus for the values $a = -(\frac{27}{4}b^2)^{1/3}$ we have a simple singular point in position $(x, y) = (-\frac{3b}{2a}, 0)$, the intersection of two branches.

Finally, the third segment corresponds to $(a, b) = (0, 0)$ where it exists a double singular point (cusp) at $(x, y) = (0, 0)$.

We obtain a complete canonical discussion of the singular points of the cubic in Weierstrass form. For the real case, we can obtain a singular point in the cubic, for example, for the curve $y^2 = x^3 - 3x + 2$ at the point $(x, y) = (1, 0)$.

1.5 Remark: Triangular Sets

There exists a parallel approach to Gröbner bases for solving parametric polynomial systems, namely Triangular Sets. In 1999, Philippe Aubry, Daniel Lazard and Marc Moreno-Maza [5] published the first presentation of these theories. There has been much progress on the subject. In 2007 Daniel Lazard and Fabrice Rouillier [48] applied these techniques to parametric polynomial systems. Since then, many developments have occurred [17–19, 21, 22].

We will not be discussing this interesting approach, because it would require a complete development of its own, and this is not the object of this book.

Part I
Theory

Chapter 2
Constructible Sets



2.1 Introduction

Constructible sets and particularly locally closed sets are central in describing subsets of the parameter space for which a parametric polynomial system has the same type of solution. Moreover, canonical forms of these subsets are needed to obtain nice algorithms for dealing with parametric ideals. We devote this chapter to describing these subsets and their canonical representations.

A reader interested in the Gröbner Cover when first going through this book may only look at Sect. 2.2 of this chapter. Then, remember that this chapter also contains the algorithm LCUNION that allows for adding together a set of locally closed sets when its union is certified by a theorem to be locally closed, as Wibmer's Theorem [85] does for the Gröbner Cover. But for the applications, particularly for automatic deduction of geometric theorems, the complete algorithm for constructible sets is used.

The canonical form of constructible sets were introduced in [4] in the context of general topology. More recently, J. O'Halloran and M. Schilmoeller [67] have given a description of invariant sequences of constructible sets in the Zariski topology. Recently, J.M. Brunat and A. Montes [13] have given an algorithm for computing the canonical form of constructible sets which is the basis of Sect. 2.3 of this chapter.

In Sect. 2.2, we recall the two canonical representations of locally closed sets, namely the C-representation (CREP) and the P-representation (PREP), and an algorithm PCREP for computing both. These representations were introduced in [63].

In Sect. 2.3, we recall the canonical structure of constructible sets introduced by J. O'Halloran and M. Schillmoeller in [67], complementing it with dimension characteristics and an effective formula. This formula allows us to give an algorithm in Sect. 2.4 to build the canonical representation of constructible sets, using the CREP for locally closed sets. In Sect. 2.4 all the algorithms needed are described.

Finally, in Sect. 2.5 simple and illustrative examples are shown.

© Springer Nature Switzerland AG 2018

A. Montes, *The Gröbner Cover*, Algorithms and Computation
in Mathematics 27, https://doi.org/10.1007/978-3-030-03904-2_2

2.2 Locally Closed Sets

Some remarks about notation. If $r \geq 1$ is an integer, the symbol $[r]$ indicates the set $[r] = \{i \in \mathbb{N} : 1 \leq i \leq r\}$. For a set $S \subseteq \mathbb{C}^n$, the complementary set $\mathbb{C}^n \setminus S$ of S is denoted by S^c. The closure $\overline{S}$ of S is the smallest variety including S. Finally $A \uplus B$ means disjoint reunion, that is, $A \cup B$ with the additional information that $A \cap B = \emptyset$.

A set $S \subseteq \mathbb{C}^n$ is *locally closed* if it is the intersection of an open set and a closed set.

Remark 2.1 The concept of locally closed set admits different but equivalent definitions. Indeed, the following conditions are easily shown to be equivalent:

(a) The set S is locally closed;
(b) the set S is the difference between two closed sets;
(c) the set S is open in the closure $\overline{S}$ of S;
(d) the set $\overline{S} \setminus S$ is closed.

Let S be an open (resp. closed) set. Since $\mathbb{C}^n$ is closed (resp. open), then $S = S \cap \mathbb{C}^n$ is a locally closed set. Thus, open sets and closed sets are locally closed.

We restrict the study to the $\mathbb{Q}$-Zariski topology of $\mathbb{C}^n$ (see Sect. 1.1 of Chap. 1), even if it can be easily generalized.

Let us introduce now the canonical C-*representation* of a locally closed set S. Let S be a locally closed set. Since $\overline{S}$ and $\overline{S} \setminus S$ are closed, there exist radical ideals $\mathfrak{a}$ and $\mathfrak{b}$ such that

$$\overline{S} = \mathbf{V}(\mathfrak{a}) \qquad \text{and} \qquad \overline{S} \setminus S = \mathbf{V}(\mathfrak{b}).$$

These ideals satisfy $\mathfrak{a} \subset \mathfrak{b}$ and

$$S = \overline{S} \setminus (\overline{S} \setminus S) = \mathbf{V}(\mathfrak{a}) \setminus \mathbf{V}(\mathfrak{b}). \tag{2.1}$$

Taking into account the one-to-one correspondence between radical ideals and varieties, the ideals $\mathfrak{a} = \mathbf{I}(\overline{S})$ and $\mathfrak{b} = \mathbf{I}\left(\overline{S} \setminus S\right)$ are uniquely determined by S. The pair $\textsc{Crep}(S) = [\mathfrak{a}, \mathfrak{b}]$ is called the C-*canonical representation* of the locally closed set S. It is canonical in the sense that it does not depend on how the locally closed set S is given: it depends only on S. The set $\mathbf{V}(\mathfrak{a})$ (or $\mathfrak{a}$) is called the *top* of S, whereas $\mathbf{V}(\mathfrak{b})$ (or $\mathfrak{b}$) is called the *hole* of S.

Remark 2.2 If $[\mathfrak{a}, \mathfrak{b}] = \textsc{Crep}(S)$, then S is closed if and only if $\mathfrak{b} = \langle 1 \rangle$. On the contrary, S is open if and only if $\mathfrak{a} = \langle 0 \rangle$.

Remark 2.3 Note that if S is empty, then $\mathfrak{a} = \langle 1 \rangle$, $\mathfrak{b} = \langle 1 \rangle$.

The following proposition explains how to obtain $\textsc{Crep}(S) = [\mathfrak{a}, \mathfrak{b}]$ for a locally closed set $S = \mathbf{V}(P) \setminus \mathbf{V}(Q)$ given by two ideals P and Q. It computes the decomposition of $\mathbf{V}(P)$ into irreducible varieties, using the Gianni-Trager-Zacharias

[36] PRIMEDECOMP algorithm for obtaining the *prime decomposition* of $\sqrt{P}$ into distinct prime ideals.

Proposition 2.4 *Let $S = \mathbf{V}(P) \setminus \mathbf{V}(Q)$ be a locally closed set given by two ideals P and Q, and let $\{\mathfrak{p}'_1, \ldots, \mathfrak{p}'_s\}$ be the prime decomposition of P. Consider the subset $\{\mathfrak{p}_1, \ldots, \mathfrak{p}_r\} = \{\mathfrak{p}'_i : \mathbf{V}(\mathfrak{p}'_i) \not\subseteq \mathbf{V}(Q)\}$. Then, the C-representation $[\mathfrak{a}, \mathfrak{b}]$ of S satisfies*

(i) $\mathfrak{a} = \bigcap_{i=1}^{r} \mathfrak{p}_i$;
(ii) $\mathfrak{b} = \text{RAD}(\mathfrak{a} + Q)$;
(iii) *if S is non-empty then $\mathfrak{a} \subset \mathfrak{b}$.*

Proof

(i) For $i \in [s]$ let $V_i = \mathbf{V}(\mathfrak{p}'_i)$. Then $\mathbf{V}(P) = V_1 \cup \cdots \cup V_s$ is the decomposition of $\mathbf{V}(P)$ into irreducible varieties. We have

$$S = \mathbf{V}(P) \setminus \mathbf{V}(Q) = \left(\bigcup_{i=1}^{s} V_i\right) \setminus \mathbf{V}(Q) = \bigcup_{i=1}^{s} (V_i \setminus (\mathbf{V}(Q) \cap V_i)) .$$

Let $J = \{i \in [s] : \mathbf{V}(\mathfrak{p}_i) \not\subseteq \mathbf{V}(Q)\}$. If $i \in [s] \setminus J$, then $V_i \cap \mathbf{V}(Q) = V_i$ and so $V_i \setminus (\mathbf{V}(Q) \cap V_i) = \emptyset$. Thus the set $V_i \setminus (\mathbf{V}(Q) \cap V_i)$ does not contribute to S and can be excluded from the union, obtaining

$$S = \bigcup_{i \in J} (V_i \setminus (\mathbf{V}(Q) \cap V_i)) .$$

For $i \in J$, we have $\mathbf{V}(Q) \cap V_i \subset V_i$. As V_i is irreducible, the closure of $V_i \setminus (\mathbf{V}(Q) \cap V_i)$ is V_i. Therefore,

$$\overline{S} = \bigcup_{i \in J} V_i = \bigcap_{i=1}^{r} \mathbf{V}(\mathfrak{p}_i), \tag{2.2}$$

$$\mathfrak{a} = \mathbf{I}(\overline{S}) = \bigcap_{i \in J} \mathbf{I}(V_i) = \bigcap_{i \in J} \mathfrak{p}'_i = \bigcap_{i=1}^{r} \mathfrak{p}_i . \tag{2.3}$$

(ii) To obtain $\mathfrak{b} = \mathbf{I}(\overline{S} \setminus S)$, note that

$$S = \bigcup_{i \in J} (V_i \setminus (\mathbf{V}(Q) \cap V_i)) = \bigcup_{i \in J} (V_i \setminus \mathbf{V}(Q)) = \left(\bigcup_{i \in J} V_i\right) \setminus \mathbf{V}(Q) = \overline{S} \setminus V(Q),$$

$$\overline{S} \setminus S = \overline{S} \setminus (\overline{S} \setminus \mathbf{V}(Q)) = \overline{S} \cap \mathbf{V}(Q) = \mathbf{V}(\mathfrak{a}) \cap \mathbf{V}(Q) = \mathbf{V}(\mathfrak{a} + Q),$$

so that, $\mathfrak{b} = \mathbf{I}(\overline{S} \setminus S) = \text{RAD}(\mathfrak{a} + Q)$.

(iii) From $\mathfrak{b} = \text{RAD}(\mathfrak{a} + Q)$, clearly $\mathfrak{b} \supseteq \mathfrak{a}$. Now $\mathfrak{b} = \mathfrak{a}$ implies $S = \emptyset$ and in this case $\mathfrak{a} = \mathfrak{b} = \langle 1 \rangle$. Therefore, if S is non-empty, then $\mathfrak{b} \supset \mathfrak{a}$. □

We can further decompose $\text{CREP}(S) = [\mathfrak{a}, \mathfrak{b}]$ and obtain another representation of S. Let $\{\mathfrak{p}_i : i \in [r]\}$ be the prime decomposition of $\mathfrak{a}$ and for $i \in [r]$ let $\{\mathfrak{p}_{ij} : j \in [r_i]\}$ be the prime decomposition of $\mathfrak{p}_i + \mathfrak{b}$. The set

$$\text{PREP}(S) = \{[\mathfrak{p}_i, \{\mathfrak{p}_{ij} : j \in [r_i]\}] : i \in [r]\} \tag{2.4}$$

is called the *canonical P-representation* of S. Note that it only depends on S. Each $[\mathfrak{p}_i, \{\mathfrak{p}_{ij} : j \in [r_i]\}]$ is called a *component* of S, from which $\mathbf{V}(\mathfrak{p}_i)$ (or $\mathfrak{p}_i$) is the *top* and $\mathbf{V}(\mathfrak{p}_{ij})$ (or $\mathfrak{p}_{ij}$) with $j \in [r_i]$ its *holes*.

Remark 2.5 Note that if $S = \emptyset$, then $\text{PREP}(S) = \{[\langle 1 \rangle, \{\langle 1 \rangle\}]\}$.

Proposition 2.6 *Let* $[\mathfrak{a}, \mathfrak{b}]$ *and* $\{[\mathfrak{p}_i, \{\mathfrak{p}_{ij} : j \in [r_i]\}] : i \in [r]\}$ *be respectively the C and P-representations of a locally closed set S. Then*

(i) *If* $S \neq \emptyset$, *then* $\mathfrak{p}_i \subset \mathfrak{p}_{ij}$, *for all* $i \in [r]$ *and* $j \in [r_i]$;

(ii) $\mathfrak{a} = \bigcap_{i=1}^{r} \mathfrak{p}_i$;

(iii) $\mathfrak{b} = \bigcap_{i=1}^{r} \bigcap_{j=1}^{r_i} \mathfrak{p}_{ij}$;

(iv) $S = \bigcup_{i=1}^{r} \left(\mathbf{V}(\mathfrak{p}_i) \setminus \left(\bigcup_{j=1}^{r_i} \mathbf{V}(\mathfrak{p}_{ij}) \right) \right)$.

Proof

(i) Is a consequence of the definition of P-representation, as $\mathfrak{b} \not\subset \mathfrak{p}_i$ for non empty S.

(ii) Is a consequence of Proposition 2.4.

(iii) From $\text{RAD}(\mathfrak{p}_i + \mathfrak{b}) = \mathfrak{p}_{i_1} \cap \cdots \cap \mathfrak{p}_{i_{r_i}}$ we have

$$T = \bigcap_{i=1}^{r} \bigcap_{j=1}^{r_i} \mathfrak{p}_{ij} = \bigcap_{i=1}^{r} \text{RAD}(\mathfrak{p}_i + \mathfrak{b}) = \text{RAD}\left(\bigcap_{i=1}^{r} (\mathfrak{p}_i + \mathfrak{b}) \right)$$

and

$$\mathbf{V}(T) = \mathbf{V}\left(\bigcap_{i=1}^{r} (\mathfrak{p}_i + \mathfrak{b}) \right) = \bigcup_{i=1}^{r} \mathbf{V}(\mathfrak{p}_i + \mathfrak{b}) = \bigcup_{i=1}^{r} (\mathbf{V}(\mathfrak{p}_i) \cap \mathbf{V}(\mathfrak{b}))$$

$$= \left(\bigcup_{i=1}^{r} \mathbf{V}(\mathfrak{p}_i) \right) \cap \mathbf{V}(\mathfrak{b}) = \mathbf{V}(\mathfrak{a}) \cap \mathbf{V}(\mathfrak{b}) = \mathbf{V}(\mathfrak{b}).$$

Taking ideals of the varieties and using the Nullstellensatz, we have $\mathfrak{b} = T$, so that (iii) is proved.

(iv) As $[\mathfrak{a}, \mathfrak{b}]$ is the C-representation of S, we have $S = \mathbf{V}(\mathfrak{a}) \setminus \mathbf{V}(\mathfrak{b})$. Then, by taking varieties in (ii) and (iii), we have

$$\begin{aligned} S =& \mathbf{V}(\mathfrak{a}) \setminus \mathbf{V}(\mathfrak{b}) = \left(\bigcup_{i=1}^{r} \mathbf{V}(\mathfrak{p}_i)\right) \setminus \left(\bigcup_{j=1}^{r}\bigcup_{k=1}^{r_j} \mathbf{V}(\mathfrak{p}_{jk})\right) \\ =& \bigcup_{i=1}^{r}\left(\mathbf{V}(\mathfrak{p}_i) \setminus \bigcup_{j=1}^{r} \mathbf{V}(\mathfrak{p}_j + \mathfrak{b})\right) = \bigcup\left(\mathbf{V}(\mathfrak{p}_i) \setminus \bigcup_{j=1}^{r} \mathbf{V}(\mathfrak{p}_i + \mathfrak{p}_j + \mathfrak{b})\right) \\ =& \bigcup_{i=1}^{r}(\mathbf{V}(\mathfrak{p}_i) \setminus \mathbf{V}(\mathfrak{p}_i + \mathfrak{b})) = \bigcup_{i=1}^{r}\left(\mathbf{V}(\mathfrak{p}_i) \setminus \bigcup_{j=1}^{r_i} \mathbf{V}(\mathfrak{p}_{ij})\right) \end{aligned}$$

□

Proposition 2.7 *Let S be a non empty locally closed set with*

$$\text{CREP}(S) = [\mathfrak{a}, \mathfrak{b}] \text{ and } \text{PREP}(S) = \{[\mathfrak{p}_i, \{\mathfrak{p}_{ij} : j \in [r_i]\}] : i \in [r]\}. \text{Then}$$

(i) $\dim\ \mathbf{V}(\mathfrak{p}_{ij}) < \dim\ \mathbf{V}(\mathfrak{p}_i)$ *for all* $i \in [r]$ *and* $j \in [r_i]$;
(ii) $\dim \mathbf{V}(\mathfrak{b}) < \dim \mathbf{V}(\mathfrak{a})$.

Proof

(i) Since $\mathfrak{p}_i \subset \mathfrak{p}_{ij}$ are prime ideals and thus correspond to irreducible varieties, the result is obvious.
(ii) From Proposition 2.6 (iii) we have

$$\begin{aligned} \dim V(\mathfrak{b}) &= \dim \bigcup_{i=1}^{r}\bigcup_{j=1}^{r_i} \mathbf{V}(\mathfrak{p}_{ij}) \\ &= \max\{\dim \mathbf{V}(\mathfrak{p}_{ij}) : i \in [r],\ \ j \in [r_i]\} \\ &< \max\{\dim \mathbf{V}(\mathfrak{p}_i) : i \in [r]\} = \dim V(\mathfrak{a}). \end{aligned}$$

□

Corollary 2.8 *Let V and W be varieties and $S = V \setminus W$. If $W \subset V$ and $V = \overline{S}$, then* $\text{CREP}(S) = [\mathbf{I}(V), \mathbf{I}(W)]$ *and* $\dim W < \dim V$.

Proof If $\overline{S} = V$, then $\mathfrak{a} = \mathbf{I}(\overline{S}) = \mathbf{I}(V)$. Moreover,

$$\overline{S} \setminus S = \overline{S} \cap (\overline{S}^c \cup W) = \overline{S} \cap W = W.$$

Thus $\mathfrak{b} = \mathbf{I}(W)$. The dimension relation is a consequence of Proposition 2.7. □

[CREP, PREP] ← **CPrep**(P, Q)
Input:
  $[P, Q]$: a pair of ideals representing the set $S = \mathbf{V}(P) \setminus \mathbf{V}(Q)$
Output:
  $[\mathfrak{a}, \mathfrak{b}]$ the C-representation of S and
  $\left\{[\mathfrak{p}_i, \{\mathfrak{p}_{i1}, \ldots, \mathfrak{p}_{ir_i}\}] : 1 \le i \le r\right\}$ the P-representaction of S
begin
  $\mathfrak{a} = \langle 1 \rangle$; PREP $= \emptyset$; $i = 0$
  $\{\mathfrak{p}'_1, \ldots, \mathfrak{p}'_s\} =$ PRIMEDECOMP(P)
  for $j = 1$ to s **do**
    if $Q \not\subseteq \mathfrak{p}'_j$ **then**
      $i = i + 1$; $\mathfrak{p}_i = \mathfrak{p}'_j$; $\mathfrak{a} = \mathfrak{a} \cap \mathfrak{p}_i$
      $\{\mathfrak{p}_{i1}, \ldots, \mathfrak{p}_{ir_i}\} =$ PRIMEDECOMP$(Q + \mathfrak{p}_i)$
      PREP $=$ PREP $\cup \{[\mathfrak{p}_i, \{\mathfrak{p}_{i1}, \ldots, \mathfrak{p}_{ir_i}\}]\}$
    end if
  end for
  $\mathfrak{b} =$ RAD$(Q + \mathfrak{a})$
  CREP $= [\mathfrak{a}, \mathfrak{b}]$
  return([CREP, PREP])
end

Algorithm 2.1: CPREP

Proposition 2.4 and the definition of P-representation justify Algorithm 2.1: CPREP for obtaining the canonical representations CREP and PREP of a locally closed set $S = \mathbf{V}(P) \setminus \mathbf{V}(Q)$ given by a pair of ideals $[P, Q]$.

Remark 2.9 If the set S is empty, the algorithm for obtaining CPREP(S) will return CPREP$(S) = [[\langle 1 \rangle, \langle 1 \rangle], \{[\langle 1 \rangle, \langle 1 \rangle], \{[\langle 1 \rangle, \{\langle 1 \rangle\}]\}\}]$.

It is obvious how to simplify the CPREP Algorithm for obtaining only CREP(S) or only PREP(S). Moreover, we can use Proposition 2.6 (ii) and (iii) for obtaining CREP$(S) = [\mathfrak{a}, \mathfrak{b}]$ from PREP$(S) = \{[\mathfrak{p}_i, \{\mathfrak{p}_{ij} : j \in [r_i]\}] : i \in [r]\}$.

2.3 Canonical Representation of Constructible Sets

A set $S \subseteq \mathbb{C}^n$ is *constructible* if it is a finite union of locally closed sets. In particular, locally closed sets are constructible. Constructible sets appear naturally in solving parametric polynomial systems of equations. Many authors give special representations for constructible sets [19, 20, 44, 49, 67], which are adequate for their goals. Our goal is developing the invariant sequence of a constructible set described in [67] thus setting the outlook on its effective computation to generalize the CREP of a locally closed set.

The next lemma recalls the behavior of locally closed sets and constructible sets with respect to union, intersection and complementation. We omit the proofs which are straightforward.

Lemma 2.10

(i) *If S is locally closed, then S^c is constructible;*
(ii) *If S_1 and S_2 are locally closed, then $S_1 \cup S_2$ is constructible and $S_1 \cap S_2$ is locally closed;*
(iii) *If S_1 is locally closed and S_2 is constructible, then $S_1 \cup S_2$ and $S_1 \cap S_2$ are constructible;*
(iv) *If S_1 and S_2 are constructible, then $S_1 \cup S_2$ and $S_1 \cap S_2$ are constructible.*
(v) *if S is constructible, then S^c is constructible.*
(vi) *if S_1 and S_2 are constructible, then $S_1 \setminus S_2$ is constructible.*

In the following, $\mathcal{L}$ denotes the family of locally closed sets and $\mathcal{C}$ the family of constructible sets.

Remark 2.11 According to Lemma 2.10, if S_1 and S_2 are constructible, then $S_1 \cup S_2$, $S_1 \cap S_2$ and S_1^c are constructible sets as well. Thus $\mathcal{C}$ is a Boolean algebra of subsets of $\mathbb{C}^n$ containing $\mathcal{L}$. On the other hand, if a Boolean algebra $\mathcal{A}$ contains $\mathcal{L}$, then it must contain the finite union of locally closed sets, that is, $\mathcal{C} \subseteq \mathcal{A}$. We conclude that $\mathcal{C}$ is the Boolean algebra generated by $\mathcal{L}$. Let $\mathcal{T}$ be the union of the family of open sets and the family of closed sets. The Boolean algebra generated by $\mathcal{T}$ contains $\mathcal{L}$, so $\mathcal{C}$ is also the Boolean algebra generated by $\mathcal{T}$.

The first step to construct the canonical structure of the constructible set S given as a union of locally closed sets is to separate $\overline{S}$ into two disjoint sets: $\overline{S} = S \uplus C$ where C is the complement of S with respect to $\overline{S}$. Having this in mind, we define:

$$\mathbf{C}(S) = \overline{S} \setminus S, \quad \mathbf{L}(S) = \overline{S} \setminus \overline{\mathbf{C}(S)}; \tag{2.5}$$

(If the set S is clear from the context, we often write C and L instead of $\mathbf{C}(S)$ and $\mathbf{L}(S)$ respectively).

If $S \in \mathcal{C}$, then $\overline{S}$ and S^c are constructible and $\mathbf{C}(S) = \overline{S} \setminus S$, being difference of constructible sets, is constructible again. We define the maps

$$\begin{array}{lll} \mathbf{C}\colon \mathcal{C} \to & \mathcal{C} & \qquad \mathbf{L}\colon \mathcal{C} \to & \mathcal{L} \\ \quad S \mapsto \mathbf{C}(S) = \overline{S} \setminus S & & \quad S \mapsto \mathbf{L}(S) = \overline{S} \setminus \overline{C} & \end{array}$$

which are obviously well defined.

Lemma 2.12 *If $S \in \mathcal{C}$, then,*

(i) $\overline{S} = S \uplus \mathbf{C}(S)$;
(ii) $\overline{S} = \mathbf{L}(S) \uplus \overline{C}$.
(iii) $\mathbf{L}(S) \subseteq S$

(iv) *S is closed if and only if* $\mathbf{C}(S) = \emptyset$*;*
(v) *S is locally closed if and only if* $\mathbf{C}(S)$ *is closed.*

Proof (i), (ii) (iv) and (v) are immediate. To proof (iii) we have

$$\mathbf{L}(S) = \overline{S} \setminus \overline{C} = \overline{S} \setminus \left(\overline{\overline{S} \setminus S}\right) \subseteq \overline{S} \setminus \left(\overline{S} \setminus S\right) = S. \qquad \square$$

For a constructible set S, the set $\mathbf{L}(S)$ can be characterized as the largest locally closed set included in S. The next proposition provides an explicit expression of C as a union of locally closed sets in terms of S.

Proposition 2.13 *Let $S = S_1 \cup \cdots \cup S_r$ be a constructible set with each S_i locally closed. For $i \in [r]$, let* $\textsc{Crep}(S_i) = [\mathfrak{a}_i, \mathfrak{b}_i]$*, $V_i = \mathbf{V}(\mathfrak{a}_i)$ and $W_i = \mathbf{V}(\mathfrak{b}_i)$. Then,*

$$\begin{aligned} C = \overline{S} \setminus S &= \bigcup_{T \subset [r]} \left(\left(\bigcap_{j \in T} V_j^c \right) \cap \left(\bigcap_{j \notin T} W_j \right) \right) \\ &= \bigcup_{T \subset [r]} \left(\left(\bigcap_{j \notin T} W_j \right) \setminus \left(\bigcup_{j \in T} V_j \right) \right). \end{aligned} \tag{2.6}$$

Proof We have

$$\begin{aligned} S =& (V_1 \setminus W_1) \cup \cdots \cup (V_r \setminus W_r) = (V_1 \cap W_1^c) \cup \cdots \cup (V_r \cap W_r^c) \\ =& \bigcap_{T \subseteq [r]} \left(\left(\bigcup_{j \in T} V_j \right) \cup \left(\bigcup_{j \notin T} W_j^c \right) \right), \end{aligned}$$

and thus

$$S^c = \bigcup_{T \subseteq [r]} \left(\left(\bigcap_{j \in T} V_j^c \right) \cap \left(\bigcap_{j \notin T} W_j \right) \right).$$

For a subset $T \subseteq [r]$, let

$$Z_T = \left(\bigcap_{j \in T} V_j^c \right) \cap \left(\bigcap_{j \notin T} W_j \right),$$

so that $S^c = \bigcup_{T\subseteq[r]} Z_T$. With this notation, the equality to prove is $\overline{S} \setminus S = \bigcup_{T\subset[r]} Z_T$. For a set $T \subseteq [r]$ and an index $\ell \in T$, we have

$$V_\ell \cap Z_T \subseteq V_\ell \cap \bigcap_{j\in T} V_j^c \subseteq V_\ell \cap V_\ell^c = \emptyset.$$

In particular, $V_\ell \cap Z_{[r]} = \emptyset$ and, if $\ell \notin T$, then $W_\ell \subset V_\ell$ and

$$V_\ell \cap \bigcap_{j\notin T} W_j = \bigcap_{j\notin T} W_j,$$

and we have $V_\ell \cap Z_T = Z_T$. Therefore, using the distributive law,

$$\overline{S} \setminus S = (V_1 \cup \cdots \cup V_r) \cap S^c = (V_1 \cup \cdots \cup V_r) \cap \bigcup_{T\subseteq[r]} Z_T$$

$$= \bigcup_{\ell=1}^{r} \bigcup_{T\subseteq[r]} (V_\ell \cap Z_T) = \bigcup_{T\subset[r]} Z_T. \qquad \square$$

Proposition 2.13 provides an explicit formula of $C = \overline{S} \setminus S$, as a union of locally closed sets. We can compute the CREP of each one of these subsets of C and obtain an expression that allows us to handle $C \subset \overline{S}$ in the same way as we have done with S. This provides an iterative method to build the canonical representation of S. The next proposition summarizes the basic properties of the first step in the recursive construction.

Proposition 2.14 *Let $S \neq \emptyset$ be a constructible set, $C = \mathbf{C}(S)$, $L = \mathbf{L}(S)$, $\mathfrak{a} = \mathbf{I}(S)$ and $\mathfrak{b} = \mathbf{I}(C)$. Then,*

(i) $C \subset \overline{S}$;
(ii) $C \subseteq \overline{S} \setminus L$;
(iii) $\overline{S} = \overline{L}$;
(iv) *if S is non-empty, then $\overline{C} \subset \overline{S}$;*
(v) $[\mathfrak{a}, \mathfrak{b}] = [\mathbf{I}(S), \mathbf{I}(C)] = [\mathbf{I}(\overline{S})], \mathbf{I}(\overline{C})]$ *is the C-representation of L;*
(vi) $\dim C < \dim S$.

Proof

(i) Let $S = S_1 \cup \cdots \cup S_r$ with S_i locally closed. For $i \in [r]$, let $\text{CREP}(S_i) = [\mathfrak{a}_i, \mathfrak{b}_i]$, $V_i = \mathbf{V}(\mathfrak{a}_i)$ and $W_i = \mathbf{V}(\mathfrak{b}_i)$. Then, $S = \bigcup_{i=1}^r (V_i \setminus W_i)$ with $W_i \subset V_i$. By taking closures, this results in $\overline{S} = \bigcup_{i=1}^r V_i$. Now, from formula (2.6) of Proposition 2.13, we have

$$C \subseteq \bigcup_{i=1}^{r} W_i \subset \bigcup_{i=1}^{r} V_i = \overline{S}.$$

(ii) We have $C = \overline{S} \setminus S = \overline{S} \cap S^c$. But $L \subseteq S$ and so $L^c \supseteq S^c$. Thus $C \subseteq \overline{S} \cap L^c = \overline{S} \setminus L$.

(iii) From $\overline{C} \subseteq \bigcup_{j=1}^{r} W_j$, we have

$$L = \overline{S} \setminus \overline{C} \supseteq \overline{S} \setminus \left(\bigcup_{j=1}^{r} W_j \right) = \left(\bigcup_{i=1}^{r} V_i \right) \setminus \left(\bigcup_{j=1}^{r} W_j \right) = \bigcup_{i=1}^{r} \bigcup_{k=1}^{r_i} \left(V_{ik} \setminus \bigcup_{j=1}^{r} W_j \right),$$

where $V_i = \bigcup_{k=1}^{r_i} V_{ik}$ is the decomposition of V_i into irreducible varieties. If some irreducible variety V_{ik} of V_i of the segment i is cancelled by some W_j of a segment j, i.e., $W_j \supseteq V_{ik}$, then $V_j \supset W_j \supseteq V_{ik}$, and in this case the variety V_{ik} is included in V_j. So, V_{ik} does not cancel neither in the closure of L nor of S. Thus $\overline{L} \supseteq \bigcup_{i=1}^{r} V_i = \overline{S}$. Since $L \subseteq S$, we also have $\overline{L} \subseteq \overline{S}$, and the inclusion is proved.

(iv) Of course $\overline{C} \subseteq \overline{S}$. If both should be equal, then $L = \overline{S} \setminus \overline{C}$ would be empty. By (iii), also S must be empty, and the result follows.

(v,vi) From (iii) and (iv) the expression $L = \overline{S} \setminus \overline{C}$ satisfies the conditions of Corollary 2.8, and thus (iv) and (v) follow. □

We proceed now to describe the method for obtaining the canonical representation. Let S be a constructible set. Define the sequence (A_i) by

$$A_1 = S, \quad A_{i+1} = \mathbf{C}(A_i).$$

By Proposition 2.14 (iv) and (vi), if $A_i \neq \emptyset$, we have $\overline{A_i} \supset \overline{A_{i+1}}$ and $\dim \overline{A_i} > \dim \overline{A_{i+1}}$. Therefore, there exists an integer $k \geq 1$ such that $A_{k+1} = \emptyset$ and A_k is closed. Consider the finite sequences

$$\begin{aligned} &S = A_1, A_2, \ldots, A_k, A_{k+1} = \emptyset, \\ &\overline{S} = \overline{A_1} \supset \overline{A_2} \supset \cdots \supset \overline{A_{k+1}} = \emptyset, \\ &\dim(S) = \dim(A_1) > \dim(A_2) > \ \ldots \ > \dim(A_{k+1}) = -1. \end{aligned} \tag{2.7}$$

By construction $A_2 = \mathbf{C}(A_1) = \overline{S} \setminus S$ is disjoint with $S = A_1$. But $A_3 = \overline{A_2} \setminus A_2$ is disjoint with A_2 and a subset of S. Thus, we have two decreasing and disjoint subsequences

$$\begin{aligned} S =& A_1 \supset A_3 \supset \cdots \supset A_{2\ell \pm 1}, \\ C =& A_2 \supset A_4 \supset \cdots \supset A_{2\ell}. \end{aligned}$$

Applying $\mathbf{L}$ to sequence (2.7), i.e. $L_i = \overline{A_i} \setminus \overline{A_{i+1}}$, we get a new sequence of disjoint sets that fill the whole $\overline{S}$,

$$L_1 = \overline{A_1} \setminus \overline{A_2}, \quad L_2 = \overline{A_2} \setminus \overline{A_3}, \ldots, L_k = \overline{A_k} \setminus \overline{A_{k+1}} = \overline{A_k}$$

so that

$$\overline{S} = \overline{A_1} = \overline{A_1} \setminus \overline{A_{k+1}} = L_1 \uplus L_2 \uplus \cdots \uplus L_k.$$

Since the L_i belong alternatively to S and to C, the previous sequence is divided into

$$S = L_1 \uplus L_3 \uplus \cdots \uplus L_{2\ell \pm 1}, \tag{2.8}$$

$$C = L_2 \uplus L_4 \uplus \cdots \uplus L_{2\ell}. \tag{2.9}$$

The odd disjoint locally closed subsets $L_1, L_3 \ldots L_{2\ell \pm 1}$ in which S is decomposed by the above procedure form the *canonical structure of the constructible set* S and is independent of the initially locally closed sets defining S. We also obtain the canonical structure of the complement $C = \overline{S} \setminus S$ as the union of the even locally closed subsets $L_2 \uplus L_4 \uplus \cdots \uplus L_{2\ell}$. Thus (2.8) and (2.9) are the *canonical representations* of S and C respectively, since the levels are already given by their CREP. Figure 2.1 shows a scheme of the varieties $\overline{A_i}$ (limited by the red circles) defining the canonical levels (blue sectors) of the constructible S and its complement C.

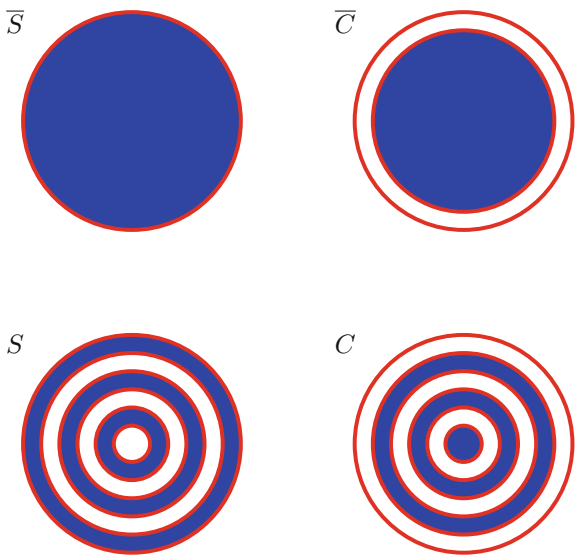


Fig. 2.1 Scheme of the canonical structure of a constructible set

For $i \in [k]$, define the ideals $\mathfrak{a}_i = \mathbf{I}(\overline{A_i})$. By using Proposition 2.14 (iv) and (vi), this results in the following:

$$\begin{aligned}
&L_i = \mathbf{V}(\mathfrak{a}_i) \setminus \mathbf{V}(\mathfrak{a}_{i+1}),\\
&\textsc{Crep}(L_i) = [\mathfrak{a}_i, \mathfrak{a}_{i+1}],\\
&\dim \mathbf{V}(\mathfrak{a}_i) > \dim \mathbf{V}(\mathfrak{a}_{i+1}),\\
&\mathbf{I}(S) = \mathfrak{a}_1 \subset \mathfrak{a}_2 \subset \cdots \subset \mathfrak{a}_{k+1} = \langle 1 \rangle,\\
&\overline{S} = \mathbf{V}(\mathfrak{a}_1) \supset \mathbf{V}(\mathfrak{a}_2) \supset \mathbf{V}(\mathfrak{a}_3) \supset \cdots \supset \mathbf{V}(\mathfrak{a}_{k+1}) = \emptyset.
\end{aligned}$$

Remark 2.15 In $\mathbb{Q}[x_1, \ldots, x_n]$, taking into account the decreasing dimensions of the levels of a constructible set, we have:

(i) The maximum number of levels of S and C is $n+1$, and this will occur when

$$\dim(L_1) = n,\ \dim(L_2) = n-1,\ \dim(L_3) = n-2,\ \ldots,\ \dim(L_{n+1}) = 0.$$

(ii) The maximum number of levels of S is $\lfloor \frac{n}{2} \rfloor + 1$.
(iii) $\dim(L_{2i-1}) \geq \dim(L_{2i+1}) + 2$.

2.4 Algorithms

The algorithms work with ideals, whereas the definitions of $\mathbf{C}$ and $\mathbf{L}$ as well as the formulas in the previous sections are given using varieties. To set down the algorithms, we must consider the one-to-one correspondence between ideals of varieties and varieties.

To make the language flexible, if $S = S_1 \cup \cdots \cup S_r$ is a constructible set with each S_i locally closed, we call the sets S_i the *segments* of S. Algorithm 2.2: FIRSTLEVEL corresponds to Proposition 2.13. Given a constructible set S, we apply the algorithm CREP to its segments; the resulting set of pairs of ideals is the input of FIRSTLEVEL.

FIRSTLEVEL applied to A_i returns $[\mathfrak{a}, A_{i+1}]$, following Proposition 2.13, where $A_{i+1} = \mathbf{C}(A_i)$ is given by the set of CREP of its segments, and $\mathfrak{a}$ is the ideal corresponding to the top of A_i.

Algorithm 2.3: CONSLEVELS iterates a call to FIRSTLEVEL(A_i), obtaining $[\mathfrak{a}, A_{i+1}]$, separating the top $\mathfrak{a}$ and repeating the call with the next A_{i+1}. (We denote $C = A_{i+1}$ inside the algorithm). The iteration finishes when $C = \emptyset$, obtaining the succession of closures of the levels

$$L = [\mathfrak{a}_1, \ldots, \mathfrak{a}_{s+1}].$$

$[\mathfrak{a}, C] \leftarrow$ **FirstLevel**(A)
Input:
$A = \{[\mathfrak{a}_1, \mathfrak{b}_1], \ldots, [\mathfrak{a}_r, \mathfrak{b}_r]\}$
a set of CREP of the segments defining a constructible set A
Output:
$\mathfrak{a}$: the closure of A and
C: a set of CREP of the segments defining $\mathbf{C}(A)$

begin
$\mathfrak{a} = \bigcap_{i=1}^{r} \mathfrak{a}_i$
$\mathfrak{p} = \langle 0 \rangle$; $\mathfrak{q} = \langle 1 \rangle$; $C = \emptyset$;
for all $T \subset [r]$ **do**
for $j \in [r]$ **do**
if $j \in T$ **then** $\mathfrak{p} = \mathfrak{p} + \mathfrak{b}_j$ **else** $\mathfrak{q} = \mathfrak{q} \cap \mathfrak{a}_j$ **end if**
end for
$[\mathfrak{a}, \mathfrak{b}] =$ CREP$(\mathfrak{p}, \mathfrak{q})$
$C =$ APPEND$([\mathfrak{a}, \mathfrak{b}]$ to $C)$
end do
$C =$ SIMPLIFYUNION(C) # facility for reducing terms
return $([\mathfrak{a}, C])$
end

Algorithm 2.2: FIRSTLEVEL

$L \leftarrow$ **ConsLevels**(A)
Input:
$A = \{[\mathfrak{p}_1, \mathfrak{q}_1], \ldots, [\mathfrak{p}_r, \mathfrak{q}_r]\}$
a set of CREP of the segments of a constructible set
$S = \bigcup_{i=1}^{r} (\mathbf{V}(\mathfrak{p}_i) \setminus \mathbf{V}(\mathfrak{q}_i))$
Output:
$L = [\mathfrak{a}_1, \mathfrak{a}_2, \ldots \mathfrak{a}_s]$
the set of closures of the canonical levels of S.

begin
$L = \emptyset$; $C = A$;
while $C \neq \emptyset$ **do**
$[\mathfrak{a}, C] =$ FIRSTLEVEL(C)
$L =$ APPEND$(\mathfrak{a}$ to $L)$
end while
return(L)
end

Algorithm 2.3: CONSLEVELS

Moreover, the algorithms can be accelerated. Formula (2.6) of Proposition 2.13 for computing the complement $C = \mathbf{C}(A) = \overline{A} \setminus A$ can contain many terms as CREP of locally closed sets, since it considers all the subsets of $[r]$. Observe that if there are two different segments of C such that CREP$(C_i) = [\mathfrak{a}_i, \mathfrak{b}_i]$ and CREP$(C_j) = [\mathfrak{a}_j, \mathfrak{b}_j]$ being $\mathfrak{b}_i = \mathfrak{a}_j$, then

$$C_i \cup C_j = (\mathbf{V}(\mathfrak{a}_i) \setminus \mathbf{V}(\mathfrak{b}_i)) \cup (\mathbf{V}(\mathfrak{b}_i) \setminus \mathbf{V}(\mathfrak{b}_j)) = \mathbf{V}(\mathfrak{a}_i) \setminus \mathbf{V}(\mathfrak{b}_j),$$

```
L ← Levels(C)
Input:
   C = [𝔞_1, ..., 𝔞_{s+1}]
      the set of canonical closures of the levels obtained by ConsLevels
Output:
   L = [[1, 𝔞_1, 𝔞_2], [3, 𝔞_3, 𝔞_4], ..., [s, 𝔞_s, 𝔞_{s+1}]]
      the set of canonical locally closed levels of S in Crep.

begin
   L = ∅;
   for ℓ = 1 to s + 1 do
      𝔟 = C_ℓ            # C_ℓ = 𝔞_ℓ
      if ℓ mod 2 = 1 then
         𝔞 = 𝔟
      else
         L = Append([𝔞, 𝔟] to L)
      end if
   end do
   if s mod 2 = 0 then L = Append([s, 𝔞, ⟨1⟩] to L)
   return(L)
end
```

Algorithm 2.4: Levels

so that $\text{Crep}(C_i \cup C_j) = [\mathfrak{a}_i, \mathfrak{b}_j]$. This can be tested for every (i, j). After this process, more than one segment that has become closed can appear. All of them can be summarized in a single one, taking the intersection of the corresponding ideals of varieties. In doing so, we can reduce the number of segments in C, which will result in an acceleration of the algorithm ConsLevels. The acceleration Algorithm 2.5: SimplifyUnion is to be used inside the FirstLevel after obtaining C.

To obtain the true levels from the output C of ConsLevels, it suffices to call the Algorithm 2.4: Levels(L) that generates a new list L where it collects the ideals in C by pairs. If there are an even number of ideals in C, it adds $[s, \mathfrak{a}, \langle 1\rangle]$ as last level.

As we have commented, for the construction of the Gröbner Cover in Chap. 5, we have to add disjoint locally closed sets whose union is certified to be locally closed by Wibmer's Theorem. In this case, algorithm ConsLevels can be replaced by the simpler Algorithm 2.6: LCUnion which is more effective. It works with Prep of the locally closed sets instead of Crep.

Proposition 2.16 *Let $S_1, \ldots, S_r$ be pairwise disjoint, locally closed subsets of $\mathbb{C}^m$ whose union $S = S_1 \cup \cdots \cup S_r$ is certified to be also locally closed. Then* LCUnion *computes the* Prep *of S.*

Proof We assume that S_i are given in Prep:

$$S_i : \left(\left(\mathfrak{p}^i_j, (\mathfrak{p}^i_{jk} : 1 \le k \le r^i_j)\right) : 1 \le j \le r^i\right).$$

Note: If A is a list and J a set of indices, DELETE(A, J) *means delete from A all the elements in positions $j \in J$.*
$A' \leftarrow$ **SimplifyUnion**(A) # implementation facility for reducing terms
`Input:`
$A = \{[\mathfrak{a}_1, \mathfrak{b}_1], \ldots, [\mathfrak{a}_r, \mathfrak{b}_r]\}$
a set of CREP of the locally closed sets defining $T = \bigcup_{i=1}^{r} (\mathbf{V}(\mathfrak{a}_i) \setminus \mathbf{V}(\mathfrak{b}_i))$
`Output:`
A': a simpler set of CREP of the T

begin
$A' = A$
$i = 1$
while $i \leq \#A'$ **do**
$j = 1$
while $j \leq \#A'$ **do**
if $j \neq i$ **and** $A'_{i,2} = A'_{j,1}$ **do**
$A'_i = [A'_{i,1}, A'_{j,2}]$
$A' =$ DELETE$(A', \{j\})$
if $j < i$ **then** $i = i - 1$ **end if**
$j = j - 1$
end if
$j = j + 1$
end while
$i = i + 1$
end while
$J = \{j \in \#A' : A'_{j,2} = 1\}$
$\mathfrak{a} = \bigcap_{j \in J} A'_{j,1}$
$A' =$ DELETE(A', J)
$A' =$ APPEND$([\mathfrak{a}, \langle 1 \rangle]$ to $A')$
return(A')
end

Algorithm 2.5: SIMPLIFYUNION

Since $\overline{S} = \overline{S_1} \cup \cdots \cup \overline{S_r} = \mathbf{V}(\cap_{i,j} \mathfrak{p}^i_j)$, let

$$P = \text{minimal ideals of } [\mathfrak{p}^i_j : \ 1 \leq i \leq r, \ 1 \leq j \leq r_j] = [[[i_k, j_k], \mathfrak{p}_k] : k \in [m]].$$

P will contain exactly all the tops of the resulting locally closed set S. (We add the information $[i_k, j_k]$ of the position of $\mathfrak{p}_k = \mathfrak{p}^{j_k}_{i_k}$, which will be useful in Chap. 5 for the algorithm GCOVER to associate a basis to $\mathfrak{p}_k$). It remains to be proved that it also yields the correct holes. For this, fix a component of S with top $\mathfrak{p}$. As seen above, $\mathfrak{p}$ is also a component of some S_{i_0}. Since the S_i are pairwise disjoint, this S_{i_0} is uniquely determined. We have to show that Algorithm 2.7: ADDPART transforms the holes $H = \{\mathfrak{q}_1, \ldots, \mathfrak{q}_s\}$ of $\mathfrak{p}$ with respect to S_{i_0} into the holes of $\mathfrak{p}$ with respect to S. In Algorithm 2.6 LCUNION, we start from

$$C = \left\{[\mathfrak{p}^i_j, \ \left\{\mathfrak{p}^i_{j1}, \ldots, \mathfrak{p}^i_{jr^i_j}\right\}] : i \in [r] \setminus \{i_0\}, j \in [r_i]\right\},$$

$T \leftarrow$ **LCUnion**$(S_1, \ldots, S_r)$
Input:
$S_1, \ldots, S_r$; pairwise disjoint locally closed subsets of L^m whose union is certified to be locally closed and where the S_i are given in the P-representation.

$$S_i : \left\{\left[\mathfrak{p}^i_j, (\mathfrak{p}^i_{jk} : k \in [r^i_j])\right] : 1 \le j \in [r^i]\right\}.$$

Output:
The P-representation of $S = S_1 \cup \cdots \cup S_r$

begin
$P = \text{SELECTMINIDEALS}\left(\mathfrak{p}^i_j : i \in [r], j \in [r^i]\right)$
$\#\, P = [(i_k, j_k), \mathfrak{p}_k) : k \in [s]]$
where (i_k, j_k) =(segment,component) of the selected $\mathfrak{p}_k = \mathfrak{p}^{i_k}_{j_k}$
$T = \emptyset$
for $k \in [s]$ **do**
Let $H = (\mathfrak{q}^k_1, \ldots, \mathfrak{q}^k_{k_t})$ be the holes of $\mathfrak{p}_k$.
$C = \left\{\left[\mathfrak{p}^i_j,\ (\mathfrak{p}^i_{j1}, \ldots, \mathfrak{p}^i_{jr^i_j})\right] : i \in [r], i \ne i_k, j \in [r_i]\right\}$
$T = T \cup \{[(i_k, j_k), \mathfrak{p}_k], \text{ADDPART}(H, C)\}$
end for
end

Algorithm 2.6: LCUNION algorithm

$Q \leftarrow$ **AddPart**(H, C)
Input:
$H = \{\mathfrak{q}_1, \ldots, \mathfrak{q}_s\}$ set of prime ideals (the holes of some component $\mathfrak{p}$ of some S_i)
$C = \{C_j : 1 \le j \le l\}$ where $C_j = \left[\mathfrak{p}_j, \{\mathfrak{p}_{jk} : k = 1 \ldots r_j\}\right]$ are the PREP which will be used to "fill the holes".
Output:
$Q = \{\mathfrak{q}'_1, \ldots, \mathfrak{q}'_{s'}\}$ the holes of the component $\mathfrak{p}$ of $S = S_1 \cup \cdots \cup S_r$
begin
$Q = H$
while there exists $\mathfrak{q} \in Q$ and $j \in \{1, \ldots, l\}$ with $\mathfrak{q} = \mathfrak{p}_j$ **do**
$Q = \text{SELECTMINIDEALS}\left((Q \setminus \{\mathfrak{q}\}) \cup \{\mathfrak{p}_{j1}, \ldots, \mathfrak{p}_{jr_j}\}\right)$
end while
while there exists $\mathfrak{q} \in Q$ and $j \in \{1, \ldots, l\}$ with $\mathfrak{q} \supset \mathfrak{p}_j$ and $\mathfrak{q} \not\supseteq \mathfrak{p}_{jk}$ for $k = 1, \ldots, r_j$ **do**
$Q = \text{SELECTMINIDEALS}\left((Q \setminus \{\mathfrak{q}\}) \bigcup_{k=1}^{r_j} \text{PRIMEDECOMP}(\mathfrak{q} + \mathfrak{p}_{ik})\right)$
end while
end

Algorithm 2.7: AddPart algorithm

and apply ADDPART(H, C), to determine them. Let

$$\{\mathfrak{q}'_1, \ldots, \mathfrak{q}'_{s'}\} = \text{ADDPART}(H, C).$$

We have to show that $\mathbf{V}(\mathfrak{q}'_1) \cup \cdots \cup \mathbf{V}(\mathfrak{q}'_{s'})$ is the minimal decomposition of $(\overline{S} \setminus S) \cap \mathbf{V}(\mathfrak{p})$ into irreducible closed sets. Because of the usage of SELECTMINIDEALS, there are no inclusions between the $\mathfrak{q}'_j$ and therefore it suffices to show that

$$(\overline{S} \setminus S) \cap \mathbf{V}(\mathfrak{p}) = \mathbf{V}(\mathfrak{q}'_1) \cup \cdots \cup \mathbf{V}(\mathfrak{q}'_{s'}). \tag{2.10}$$

Inside Algorithm 2.7: ADDPART, the set Q of prime ideals gets modified at every step. When a new element, say $\mathfrak{p}'$, is being added to Q, then it always satisfies $\mathfrak{p}' \supsetneqq \mathfrak{q}$ for some $\mathfrak{q}$ which is being deleted from Q. In particular, at every step, the closed set $\cup_{\mathfrak{q}\in Q}\mathbf{V}(\mathfrak{q})$ gets strictly smaller. This shows that ADDPART will terminate. Since we also have $\mathfrak{p} \subsetneqq \mathfrak{q}_i$ for the "initial" holes $\{\mathfrak{q}_1, \ldots, \mathfrak{q}_s\}$ of $\mathfrak{p}$ with respect to S_{i_0}, thus $\mathfrak{p} \subsetneqq \mathfrak{q}$ for every $\mathfrak{q} \in Q$ at every step. In particular, $\mathbf{V}(\mathfrak{q}'_i) \subset \mathbf{V}(\mathfrak{p})$ for $i = 1, \ldots, s'$.

Dually, the set $\mathbf{V}(\mathfrak{p}) \setminus (\cup_{\mathfrak{q}\in Q}\mathbf{V}(\mathfrak{q}))$ gets strictly larger at every step of ADDPART and the algorithm works in such a way that $\mathbf{V}(\mathfrak{p}) \setminus (\cup_{\mathfrak{q}\in Q}\mathbf{V}(\mathfrak{q}))$ will always be a subset of S because at every step the set $\mathbf{V}(\mathfrak{p}) \setminus (\cup_{\mathfrak{q}\in Q}\mathbf{V}(\mathfrak{q}))$ is only enlarged with elements contained in some S_i. In particular, $\mathbf{V}(\mathfrak{p}) \setminus (\mathbf{V}(\mathfrak{q}'_1) \cup \cdots \cup \mathbf{V}(\mathfrak{q}'_{s'})) \subseteq S$, or equivalently

$$(\overline{S} \setminus S) \cap \mathbf{V}(\mathfrak{p}) = \mathbf{V}(\mathfrak{p}) \setminus S \subseteq \mathbf{V}(\mathfrak{q}'_1) \cup \cdots \cup \mathbf{V}(\mathfrak{q}'_{s'}).$$

It remains to prove the inclusion "$\supseteq$" of Eq. (2.10). We have already observed above that $\mathbf{V}(\mathfrak{q}'_i) \subset \mathbf{V}(\mathfrak{p})$. So, suppose as a contradiction that there exists $\mathfrak{q}' \in \{\mathfrak{q}'_1, \ldots, \mathfrak{q}'_{s'}\}$ such that $\mathbf{V}(\mathfrak{q}')$ is not contained in $\overline{S} \setminus S$, or equivalently $\mathbf{V}(\mathfrak{q}') \cap S \neq \emptyset$. But then, as S is locally closed, $\mathbf{V}(\mathfrak{q}')$ is a non-empty open subset of $\mathbf{V}(\mathfrak{q}')$ and therefore

$$\mathbf{V}(\mathfrak{q}') = \overline{\mathbf{V}(\mathfrak{q}') \cap S} = \bigcup_{i=1}^{r} \overline{\mathbf{V}(\mathfrak{q}') \cap S_i}.$$

Since $\mathbf{V}(\mathfrak{q}')$ is irreducible, we must have $\mathbf{V}(\mathfrak{q}') = \overline{\mathbf{V}(\mathfrak{q}') \cap S_i}$ for some $i \in \{1, \ldots, r\}$. As $\mathbf{V}(\mathfrak{q}') \subset \mathbf{V}(\mathfrak{q}_j)$ for some hole $\mathfrak{q}_j$ of $\mathfrak{p}$ with respect to S_{i_0}, we have $\mathbf{V}(\mathfrak{q}') \cap S_{i_0} = \emptyset$ and therefore $i \neq i_0$. Furthermore since

$$S_i = \bigcup_{j=1}^{r^i} \left(\mathbf{V}(\mathfrak{p}^i_j) \setminus \mathbf{V}\big(\mathfrak{p}^i_{j1} \cap \cdots \cap \mathfrak{p}^i_{jr^i_j}\big) \right)$$

it follows that

$$\mathbf{V}(\mathfrak{q}') = \bigcup_{j=1}^{r^i} \overline{(\mathbf{V}(\mathfrak{q}') \cap \mathbf{V}(\mathfrak{p}^i_j)) \setminus \mathbf{V}(\mathfrak{p}^i_{j1} \cap \cdots \cap \mathfrak{p}^i_{jr^i_j})}$$

and again by irreducibility of $\mathbf{V}(\mathfrak{q}')$ we obtain

$$\mathbf{V}(\mathfrak{q}') = \overline{(\mathbf{V}(\mathfrak{q}') \cap \mathbf{V}(\mathfrak{p}^i_j)) \setminus \mathbf{V}(\mathfrak{p}^i_{j1} \cap \cdots \cap \mathfrak{p}^i_{jr^i_j})} \tag{2.11}$$

for some $j \in \{1, \ldots, r^i\}$. In particular, $\mathbf{V}(\mathfrak{q}') \subset \mathbf{V}(\mathfrak{q}') \cap \mathbf{V}(\mathfrak{p}^i_j)$ so that $\mathfrak{p}^i_j \subset \mathfrak{q}'$. Furthermore $\mathbf{V}(\mathfrak{q}') \nsubseteq \mathbf{V}(\mathfrak{p}^i_{jk})$ for $k = 1, \ldots, r^i_j$ because otherwise the right-hand side of Eq. (2.11) would be the empty set. Summarily we have found $i, j, (i \neq i_0)$ such that $\mathfrak{q}' \supset \mathfrak{p}^i_j$ and $\mathfrak{q}' \not\supseteq \mathfrak{p}^i_{jk}$ for $k = 1, \ldots, r^i_j$. This contradicts our assumption that ADDPART has terminated. □

2.5 Examples

The algorithms CREP, PREP, PTOCREP, FIRSTLEVEL, CONSLEVELS SIMPLIFYUNION, LEVELS and LCUNION are included in the "grobcov.lib" SINGULAR library [62]. The accessible public commands are `Crep`, `Prep`, `PtoCrep`, `ConsLevels` and `Levels`. Interesting examples using these commands can be seen in the following chapters. Here we give only some simple examples to show how they work.

Example 2.17 Use algorithms CREP ans PREP with the following locally closed sets:

(1) $S_1 = \mathbf{V}(P) \setminus \mathbf{V}(Q) = \mathbf{V}(xyz) \setminus \mathbf{V}(x, z-1)$,
(2) $S_2 = \mathbf{V}(P) \setminus \mathbf{V}(Q_1) = \mathbf{V}(xyz) \setminus \mathbf{V}(x(z-1))$.

As this is the first example in the book of the use of the SINGULAR "grobcov.lib" library [62], we comment on how we present the examples in the whole book. To execute a command of the library, we first enter `Singular` and write

```
> LIB "grobcov.lib";
```

to which SINGULAR answers, indicating all the libraries that are used inside. Often we add the `short` command to obtain a more readable output. The input commands are preceded by the symbol ">" which is the symbol that SINGULAR shows on attending a command. The output is directly written in the same way as SINGULAR. Of course, the answer to the input `> LIB "grobcov.lib";` is not reproduced in the text.

We show the application of the library to the given examples.

```
> LIB "grobcov.lib";
> ring R=0,(x,y,z),lp;
> ideal P=x*y*z;
> ideal Q=x,z-1;
> Prep(P,Q);
[1]:
   [1]:
      _[1]=z
```

```
   [2]:
      [1]:
         _[1]=1
[2]:
   [1]:
      _[1]=y
   [2]:
      [1]:
         _[1]=z-1
         _[2]=y
         _[3]=x
[3]:
   [1]:
      _[1]=x
   [2]:
      [1]:
         _[1]=z-1
         _[2]=x
> Crep(P,Q);
[1]:
   _[1]=xyz
[2]:
   _[1]=z-1
   _[2]=x
> ideal Q1=x*(z-1);
> Prep(P,Q1);
[1]:
   [1]:
      _[1]=z
   [2]:
      [1]:
         _[1]=z
         _[2]=x
[2]:
   [1]:
      _[1]=y
   [2]:
      [1]:
         _[1]=z-1
         _[2]=y
      [2]:
         _[1]=y
         _[2]=x
> Crep(P,Q1);
[1]:
   _[1]=yz
[2]:
   _[1]=yz
   _[2]=xz-x
   _[3]=xy
```

The output is to be read as follows:

$\textsc{Prep}(P, Q) = \{[\langle z\rangle, \{\langle 1\rangle\}], [\langle y\rangle, \{\langle z-1, y, x\rangle\}], [\langle x\rangle, \{\langle z-1, x\rangle\}]\}$

representing $(\mathbf{V}(z) \setminus \mathbf{V}(1)) \cup (\mathbf{V}(y) \setminus \mathbf{V}(z-1, y, x)) \cup (\mathbf{V}(x) \setminus \mathbf{V}(z-1, x))$.

$\textsc{Crep}(P, Q) = [\langle xyz\rangle, \langle z-1, x\rangle]$

representing $\mathbf{V}(xyz) \setminus \mathbf{V}(z-1, x)$.

$\textsc{Prep}(P, Q_1) = \{[\langle z\rangle, \{\langle z, x\rangle\}], [\langle y\rangle, \{\langle z-1, y\rangle, \langle y, x\rangle\}]\}$

representing $(\mathbf{V}(z) \setminus \mathbf{V}(z, x)) \cup (\mathbf{V}(y) \setminus (\mathbf{V}(z-1, y) \cup \mathbf{V}(y, x)))$.

$\textsc{Crep}(P, Q_2) = [\langle yz\rangle, \langle yz, xz-x, xy\rangle]$

representing $\mathbf{V}(yz) \setminus \mathbf{V}(yz, xz-x, xy)$.

The algorithm PTOCREP applied to PREP(S) returns, in both examples, the corresponding CREP(S), as expected.

Example 2.18 This example represents a simple geometric problem in 3-dimensional space with a nice geometrical interpretation. Consider the constructible set $S = S_1 \cup S_2 \cup S_3$ (see Fig. 2.2), where

$$S_1 = \mathbf{V}(x^2 + y^2 + z^2 - 1) \setminus \mathbf{V}(z, x^2 + y^2 - 1),$$
$$S_2 = \mathbf{V}(y, x^2 + z^2 - 1) \setminus \mathbf{V}(z(z+1), y, x + z + 1),$$
$$S_3 = \mathbf{V}(x) \setminus \mathbf{V}(5z - 4, 5y - 3, x).$$

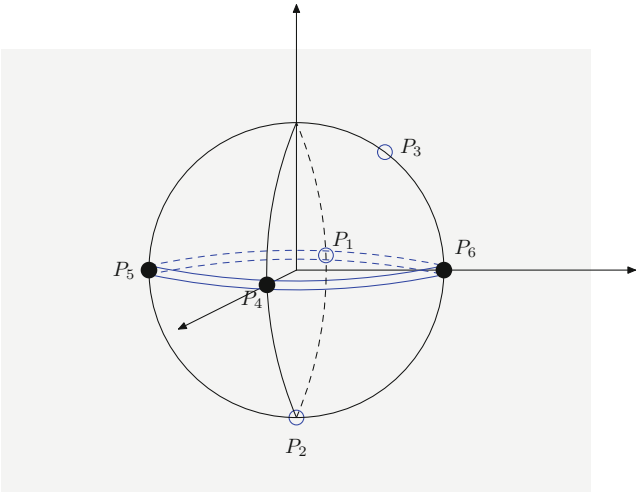


Fig. 2.2 Constructible set S of Example 2.18

The set S_1 is the sphere B of radius 1 minus its maximum circle for $z = 0$. S_2 is the maximum circle of B for $y = 0$ minus points $P_1 = (-1, 0, 0)$ and $P_2 = (0, 0, -1)$. And S_3 is the plane $x = 0$ minus the point $P_3 = (0, \frac{3}{5}, \frac{4}{5})$ inside B.

To become familiar with using of the "grobcov.lib" library, we compute the levels using the commands `ConsLevels` and `Levels` :

```
> LIB "grobcov.lib";
> ring R=0,(x,y,z),lp;
> short=0;
> list S=list(list(ideal(x^2+y^2+z^2-1),ideal(z,x^2+y^2-1)),
              list(ideal(y,x^2+z^2-1),ideal(z*(z+1),y,x+z+1)),
              list(ideal(x),ideal(5*z-4,5*y-3)));
> S;
[1]:
   [1]:
      _[1]=x^2+y^2+z^2-1
   [2]:
      _[1]=z
      _[2]=x^2+y^2-1
[2]:
   [1]:
      _[1]=y
      _[2]=x^2+z^2-1
   [2]:
      _[1]=z^2+z
      _[2]=y
      _[3]=x+z+1
[3]:
   [1]:
      _[1]=x
   [2]:
      _[1]=5*z-4
      _[2]=5*y-3
> def CL=ConsLevels(S);
> CL;
[1]:
   _[1]=x^3+x*y^2+x*z^2-x
[2]:
   _[1]=z
   _[2]=x^2+y^2-1
[3]:
   _[1]=z
   _[2]=x+y^2-1
   _[3]=x*y
   _[4]=x^2-x
[4]:
   _[1]=1
> def L=Levels(CL);
> L;
[1]:
   [1]:
      1
   [2]:
```

```
      [1]:
         _[1]=x^3+x*y^2+x*z^2-x
      [2]:
         _[1]=z
         _[2]=x^2+y^2-1
[2]:
   [1]:
      3
   [2]:
      [1]:
         _[1]=z
         _[2]=x+y^2-1
         _[3]=x*y
         _[4]=x^2-x
      [2]:
         _[1]=1
> def L11=Prep(L[1][2][1],L[1][2][2]);
L11;
[1]:
   [1]:
      _[1]=x^2+y^2+z^2-1
   [2]:
      [1]:
         _[1]=z
         _[2]=x^2+y^2-1
[2]:
   [1]:
      _[1]=x
   [2]:
      [1]:
         _[1]=z
         _[2]=y-1
         _[3]=x
      [2]:
         _[1]=z
         _[2]=y+1
         _[3]=x
> def L12=Prep(L[2][2][1],L[2][2][2]);
L12
[1]:
   [1]:
      _[1]=z
      _[2]=y-1
      _[3]=x
   [2]:
      [1]:
         _[1]=1
[2]:
   [1]:
      _[1]=z
      _[2]=y
      _[3]=x-1
   [2]:
      [1]:
```

```
          _[1]=1
[3]:
   [1]:
      _[1]=z
      _[2]=y+1
      _[3]=x
   [2]:
      [1]:
         _[1]=1
```

The procedure CONSLEVELS returns as output not the proper levels but simply the closures. To obtain the levels (in C-representation), it suffices to group them two-by-two, or use LEVELS. We can also determine their P-representations. The output has two levels for S, and is to be read as follows:

$$\begin{aligned} L_1 =& \mathbf{V}(x^3 + xy^2 + xz^2 - x) \setminus \mathbf{V}(z, x^2 + y^2 - 1) \\ =& L_{11} \cup L_{12} \\ =& (\mathbf{V}(x) \setminus \mathbf{V}(z, x^2 + y^2 - 1)) \cup (\mathbf{V}(z, y - 1, x)) \cup (\mathbf{V}(z, y + 1, x)) \\ L_3 =& \mathbf{V}(z, x + y^2 - 1, xy, x^2 - x) \\ =& \mathbf{V}(z, y - 1, x) \cup \mathbf{V}(z, y, x - 1) \cup \mathbf{V}(z, y + 1, x). \end{aligned}$$

We now give the geometrical interpretation of the obtained levels: S_2 is included in S_1, except for point $P_4 = (1, 0, 0)$ since its holes P_1 and P_2 are included in the hole of S_1. Thus $S_1 \cup S_2 = S_1 \cup P_4$. Now adding S_3, the plane $x = 0$ must be added to the first level and it includes two points $P_5 = (0, -1, 0)$ and $P_6 = (0, 1, 0)$ in the hole of $S_1 \cup S_2$ that must be added to the second level of S. The point $P_3 = (0, 3/5, 4/5)$, not in S_2, is already included in S_1 and thus can be ignored. Hence finally $S_1 \cup S_2 \cup S_3$ will have as closure $\mathbf{V}(x(x^2 + y^2 + z^2 - 1)$ from which the maximal circle $\mathbf{V}(z, x^2 + y^2 - 1)$ must be retrieved, forming the first level $L_1 = (\mathbf{V}(x) \setminus \mathbf{V}(z, x^2 + y^2 - 1)) \cup (\mathbf{V}(z, y - 1, x)) \cup (\mathbf{V}(z, y + 1, x)))$. But points P_4, P_5 and P_6 must be added. They form the second level of S which is $L_3 = P_4 \cup P_5 \cup P_6 = \mathbf{V}(z, y - 1, x) \cup \mathbf{V}(z, y, x - 1) \cup \mathbf{V}(z, y + 1, x)$.

We can also determine the C- and P-representations of the complementary set C. We have:

$$\begin{aligned} C = CL_2 \setminus CL_3 &= \mathbf{V}(z, x^2 + y^2 - 1) \setminus \mathbf{V}(z, x + y^2 - 1, xy, x^2 - x) \\ &= \mathbf{V}(z, x^2 + y^2 - 1) \setminus (\mathbf{V}(z, y - 1, x) \cup \mathbf{V}(z, y, x - 1) \cup \mathbf{V}(z, y + 1, x)) \end{aligned}$$

Example 2.19 We consider now a trivial example of locally closed sets that are disjoint and whose union is locally closed. Let

$$S = (\mathbf{V}(x) \setminus \mathbf{V}(x, y)) \uplus (\mathbf{V}(x, y) \setminus \mathbf{V}(x, y, z)) \uplus (\mathbf{V}(x, y, z) \setminus \mathbf{V}(1)).$$

Applying CONSLEVELS to the segments, the result is $S = \mathbf{V}(x)$. Moreover, expressing the segments of S in P-representation and applying LCUNION, the result is also $S = \mathbf{V}(x)$.

Example 2.20 Let us now present an example proceeding from Comprehensive Gröbner Systems, which will be studied in the next chapter. Consider the ring $R = \mathbb{Q}(a_0, b_0, c_0, a_1, b_1, c_1)[x, y]$, and the parametric system

$$S = \{a_0x^2 + b_0x + c_0, a_1x^2 + b_1x + c_1\}.$$

Assume that we have already computed a CGS (Comprehensive Gröbner System) of S, using the Kapur-Sun-Wang algorithm [43], described in the next Chap. 3. As a result, (see Table 2.1), the parameter space is divided into 11 disjoint segments $S_1, \ldots, S_{11}$, and for each segment S_i, a basis B_i specializing to the reduced Gröbner basis on the whole segment is also given. The segments in the CGS are algorithm depending, and can change if we use another algorithm for computing the CGS.

Let us now add together the segments with the same set of lpp (leading power product of the bases), using CONSLEVELS algorithm. There are four different sets of lpp in the 11 cases, namely $\{1\}$, $\{x\}$, $\{x^2\}$ and $\{0\}$. We apply CONSLEVELS to each group of segments corresponding to the same lpp, and deduce from them their canonical levels. The result is shown in Table 2.2.

The canonical levels of the constructible sets so obtained do not any longer depend on the CGS algorithm that was used, as each of these segments correspond to a canonical level of all the points of the parameter space with the fixed value of the lpp. In the next chapter we shall take advantage of this situation.

We can verify the union of the segments corresponding to lpp $= \{1\}$. The result using CONSLEVELS is

$$S_{\{1\}} = S_1 \cup S_3 \cup S_5 \cup S_8 = L_{\{1\},1} \cup L_{\{1\},3} \cup L_{\{1\},5}.$$

Observe that $S_1 = L_{\{1\},1}$ and $S_8 = L_{\{1\},5}$. So we have to verify only that

$$S_3 \cup S_5 = L_{\{1\},3}.$$

We have

$$\begin{aligned} S_3 &= \mathbf{V}(a_1, a_0) \setminus \mathbf{V}(a_1, b_0b_1c_1 - c_0b_1^2, a_0) \\ &= \mathbf{V}(a_1, a_0) \setminus (\mathbf{V}(b_1, a_1, a_0) \cup \mathbf{V}(a_1, b_0c_1 - c_0b_1, a_0)) \\ S_5 &= \mathbf{V}(b_1, a_1, a_0) \setminus \mathbf{V}(c_1, b_1, a_1, a_0). \end{aligned}$$

Table 2.1 Segments and bases of Example 2.20

$S_1 =$	$\mathbf{V}(0)$
	$\setminus \mathbf{V}(a_0^2c_1^2 - a_0b_0b_1c_1 - 2a_0c_0a_1c_1 + a_0c_0b_1^2 + b_0^2a_1c_1$
	$-b_0c_0a_1b_1 + c_0^2a_1^2)$
$B_1 =$	$\{1\}$
$\mathrm{lpp}_1 =$	$\{1\}$
$S_2 =$	$\mathbf{V}(a_0^2c_1^2 - a_0b_0b_1c_1 - 2a_0c_0a_1c_1 + a_0c_0b_1^2 + b_0^2a_1c_1$
	$-b_0c_0a_1b_1 + c_0^2a_1^2)$
	$\setminus \mathbf{V}(b_0a_1c_1 - c_0a_1b_1,\ a_0a_1c_1 - c_0a_1^2,\ a_0c_0a_1b_1 - b_0c_0a_1^2,$
	$a_0^2c_1^2 - a_0b_0b_1c_1 + a_0c_0b_1^2 - c_0^2a_1^2)$
$B_2 =$	$\{(b_0a_1c_1 - c_0a_1b_1)x + (-a_0c_1^2 + b_0b_1c_1 + c_0a_1c_1 - c_0b_1^2)\}$
$\mathrm{lpp}_2 =$	$\{x\}$
$S_3 =$	$\mathbf{V}(a_1, a_0) \setminus \mathbf{V}(a_1, b_0b_1c_1 - c_0b_1^2, a_0)$
$B_3 =$	$\{1\}$
$\mathrm{lpp}_3 =$	$\{1\}$
$S_4 =$	$\mathbf{V}(a_1, a_0c_1^2 - b_0b_1c_1 + c_0b_1^2) \setminus \mathbf{V}(b_1, a_1, a_0c_1)$
$B_4 =$	$\{b_1x - c_1\}$
$\mathrm{lpp}_4 =$	$\{x\}$
$S_5 =$	$\mathbf{V}(b_1, a_1, a_0) \setminus \mathbf{V}(c_1, b_1, a_1, a_0)$
$B_5 =$	$\{1\}$
$\mathrm{lpp}_5 =$	$\{1\}$
$S_6 =$	$\mathbf{V}(c_1, b_1, a_1) \setminus \mathbf{V}(c_1, b_1, a_1, a_0)$
$B_6 =$	$\{a_0x^2 + b_0x + c_0\}$
$\mathrm{lpp}_6 =$	$\{x^2\}$
$S_7 =$	$\mathbf{V}(c_1, b_1, a_1, a_0) \setminus \mathbf{V}(c_1, b_1, a_1, b_0, a_0)$
$B_7 =$	$\{b_0x + c_0\}$
$\mathrm{lpp}_7 =$	$\{x\}$
$S_8 =$	$\mathbf{V}(c_1, b_1, a_1, b_0, a_0) \setminus \mathbf{V}(c_1, b_1, a_1, c_0, b_0, a_0)$
$B_8 =$	$\{1\}$
$\mathrm{lpp}_8 =$	$\{1\}$
$S_9 =$	$\mathbf{V}(c_1, b_1, a_1, c_0, b_0, a_0) \setminus \mathbf{V}(1)$
$B_9 =$	$\{0\}$
$\mathrm{lpp}_9 =$	$\{0\}$
$S_{10} =$	$\mathbf{V}(c_1, c_0) \setminus \mathbf{V}(c_1, c_0, a_0b_1 - b_0a_1)$
$B_{10} =$	$\{a_0b_1 - b_0a_1)x\}$
$\mathrm{lpp}_{10} =$	$\{x\}$
$S_{11} =$	$\mathbf{V}(b_0c_1 - c_0b_1, a_0c_1 - c_0b_1, a_0b_1 - b_0, a_1)$
	$\setminus \mathbf{V}(a_1, b_0c_1 - c_0b_1, a_0c_1, a_0b_1)$
$B_{11} =$	$\{a_1x^2 + b_1x + c_1\}$
$\mathrm{lpp}_{11} =$	$\{x^2\}$

Table 2.2 Levels of the canonical representation of Example 2.20

$S_{\{1\}} = S_1 \cup S_3 \cup S_5 \cup S_8 = L_{\{1\},1} \cup L_{\{1\},3} \cup L_{\{1\},5}$
$L_{\{1\},1} = \mathbf{V}(0) \setminus \mathbf{V}(a_0^2c_1^2 - a_0b_0b_1c_1 - 2a_0c_0a_1c_1 + a_0c_0b_1^2 + b_0^2a_1c_1$ $-b_0c_0a_1b_1 + c_0^2a_1^2)$
$L_{\{1\},3} = \mathbf{V}(a_1, a_0) \setminus \mathbf{V}(a_1, a_0, -b_0c_1 + c_0b_1)$
$L_{\{1\},5} = \mathbf{V}(b_1, a_1, b_0, a_0) \setminus \mathbf{V}(c_1, b_1, a_1, c_0, b_0, a_0)$
$S_{\{x\}} = S_2 \cup S_4 \cup S_7 \cup S_{10} = L_{\{x\},1} \cup L_{\{x\},2}$
$L_{\{x\},1} = \mathbf{V}(a_0^2c_1^2 - a_0b_0b_1c_1 - 2a_0c_0a_1c_1 + a_0c_0b_1^2 + b_0^2a_1c_1 - b_0c_0a_1b_1$ $+c_0^2a_1^2) \setminus \mathbf{V}(-a_0c_1 + c_0a_1, -a_0b_1 + b_0a_1, -a_0b_0c_1 + a_0c_0b_1)$
$L_{\{x\},3} = \mathbf{V}(a_1, a_0, -b_0c_1 + c_0b_1) \setminus \mathbf{V}(b_1, a_1, b_0, a_0)$
$S_{\{x^2\}} = S_6 \cup S_{11} = L_{\{x^2\},1}$
$L_{\{x^2\},1} = \mathbf{V}(-b_0c_1 + c_0b_1, -a_0c_1 + c_0a_1, -a_0b_1 + b_0a_1)$ $\setminus \mathbf{V}(a_1, a_0, -b_0c_1 + c_0b_1)$
$S_{\{0\}} = S_9$
$L_{\{0\},1} = \mathbf{V}(c_1, b_1, a_1, c_0, b_0, a_0) \setminus \mathbf{V}(1)$

Thus adding S_5 to S_3 will eliminate the variety $\mathbf{V}(b_1, a_1, a_0)$ from the hole of S_3, and replace it by $\mathbf{V}(c_1, b_1, a_1, a_0)$. But since

$$\mathbf{V}(c_1, b_1, a_1, a_0) \subset \mathbf{V}(a_1, b_0c_1 - c_0b_1, a_0),$$

finally $\mathbf{V}(c_1, b_1, a_1, a_0)$ can also be eliminated from the hole, resulting in

$$S_3 \cup S_5 = \mathbf{V}(a_1, a_0) \setminus \mathbf{V}(a_1, b_0c_1 - c_0b_1, a_0) = L_{\{1\},3}.$$

Alternatively, if we use LCUNION to add $S_3 \cup S_5$, using their P-representations, the result is $L_{\{1\},3}$ as well. But LCUNION can only be used when we know that the union is locally closed.

We propose as an exercise to verify, the union of the segments corresponding to $\mathrm{lpp} = \{x\}$, and to $\mathrm{lpp} = \{x^2\}$.

Chapter 3
Comprehensive Gröbner Systems and Bases



3.1 Introduction

From now on we will develop the study of parametric polynomial systems of equations. In this chapter, after a brief introduction describing the progress in area, we will center our attention on Comprehensive Gröbner Systems (CGS) and Basis (CGB). Computing a Comprehensive Gröbner System is a necessary previous step for obtaining the complete canonical discussion provided by the Gröbner Cover, which is described in Chap. 5, after the introduction of I-regular functions in Chap. 4 that are also required.

Comprehensive Gröbner Systems for parametric ideals were introduced by Volker Weispfenning [83] in 1992. He presented the theory, introduced the concept of *specialization*, gave a non-constructive proof of the existence of a CGS and a CGB, and a constructive proof going back to a first algorithm for computing both. His algorithm was implemented by his students (first by M. Pesh, and then by A. Dolzman and T. Sturm [29]). The next algorithm for computing CGS was developed by D. Kapur [42] in 1995.

The author and his Ph.D student Montserrat Manubens developed new algorithms starting in 2002 [55, 56, 58], which have been the basis for the ulterior development of the Gröbner Cover. The final version of the authors BUILDTREE algorithm for computing a CGS is described in the Appendix. It was used when the Canonical Gröbner Cover was introduced in [63].

In 1997, M. Kalkbrenner [41] gave a *stability criterion* whose usefulness was rediscovered 5 years later during which time no important development was made. Based on this stability criterion, a series of new algorithms for computing a CGS appeared: Suzuki-Sato [80, 81] in 2006 and 2007, Nabeshima [66].

The objective for developing new algorithms for computing a CGS are obtaining: (1) fewer number of segments, (2) disjoint segments, (3) more efficient or speedup algorithms. In 2010, Kapur-Sun-Wang [43] developed a new algorithm improving on Suzuki-Sato's algorithm and that is clearly better.

© Springer Nature Switzerland AG 2018

A. Montes, *The Gröbner Cover*, Algorithms and Computation in Mathematics 27, https://doi.org/10.1007/978-3-030-03904-2_3

In 2010, Montes-Wibmer [63] published the Canonical Gröbner Cover, that, starting with a disjoint reduced CGS, (for which we used BUILDTREE), produces the simplest form of the canonical discussion. Simultaneously, starting in 2008, the author developed the SINGULAR library "grobcov.lib" [62] implementing all of the algorithms involved in the Gröbner Cover. All of these algorithms are described in the book, and the commands of the algorithms that are implemented, are used in the examples, so that the book can also be used as a kind of users manual of the library. In its actual implementation, the `grobcov` command uses the Kapur-Sun-Wang algorithm (KSW) for the previous CGS step.

The chapter begins by describing basic concepts used for parametric polynomial systems: specializations, PNORMALF and pseudo-division PDIVI algorithms, that are needed for building Comprehensive Gröbner Systems and Basis.

Then, the KSW algorithm for computing a CGS is fully described. As a tail of completeness, our own algorithm BUILDTREE is described in the Appendix, where one can also find and compare the effectiveness between both algorithms for their use in the Gröbner Cover.

3.2 Parametric Polynomial Systems and Specializations

We follow the notations of Chap. 1, Sect. 3.2. Let us consider a parametric polynomial system of equations,

$$\begin{cases} p_1(u_1, \ldots, u_m, x_1, \ldots, x_n) = 0 \\ \qquad\qquad \cdots \\ p_r(u_1, \ldots, u_m, x_1, \ldots, x_n) = 0 \end{cases}$$

with coefficients in the computable field K, where $\mathbf{x} = x_1, \ldots, x_n$ are the variables and $\mathbf{u} = u_1, \ldots, u_m$ are the parameters. The system can be denoted $F = \{p_1, \ldots, p_r\} \subset K[\mathbf{u}][\mathbf{x}]$ and generates the ideal $I = \langle F \rangle$. We use the K-Zariski topology of L^n (see Chap. 1, Sect. 1.1), where K is a computable field, and L is an algebraically closed extension of K, as in the whole book.

The problem is decomposing the parameter space L^m into sets S_i having the same type of solutions. More precisely, the segments S_i must verify that when the parameters $\mathbf{u}$ are substituted by points inside S_i, the resulting set of polynomials is a Gröbner basis of the specialized ideal of $L[\mathbf{x}]$.

We consider subsets S_i of the parameter space that are locally closed $S_i = \mathbf{V}(\mathfrak{a}_i) \setminus \mathbf{V}(\mathfrak{b}_i)$, and, if desired, can be given in a canonical C or P-representation.

From now one, we shall consider two monomial orders $\succ_{\mathbf{x}}$ and $\succ_{\mathbf{u}}$ for the power products in the variables $\mathbf{x}$ and $\mathbf{u}$ respectively. We also consider the product order $\succ_{\mathbf{x},\mathbf{u}}$ with $\mathbf{x} \succ \mathbf{u}$.

Following Weispfenning [83], we define *specialization* to be a ring homomorphism. We shall consider the following specializations:

Definition 3.1 A *specialization* $\sigma_{\mathfrak{a}}$ on an ideal $\mathfrak{a} \subset K[\mathbf{u}]$ is a ring homomorphism

$$\begin{aligned} \sigma_{\mathfrak{a}} \colon K[\mathbf{u}] &\to K[\mathbf{u}]/\mathfrak{a} \\ f(\mathbf{u}) &\mapsto f_{\mathfrak{a}}(\mathbf{u}), \end{aligned}$$

where $\mathfrak{a} = \ker \sigma_{\mathfrak{a}}$ is the kernel of σ, i.e., $f_{\mathfrak{a}}(\mathbf{u})$ is the equivalence class of polynomials of $K[\mathbf{u}]$:

$$f_{\mathfrak{a}}(\mathbf{u}) = \{g \in K[\mathbf{u}] : f - g \in \mathfrak{a}\}.$$

Definition 3.2 Let $S = \mathbf{V}(\mathfrak{a}) \setminus \mathbf{V}(\mathfrak{b})$ be a locally closed set. A *specialization* $\sigma_{\mathbf{u}^{(0)}}$ on a point $\mathbf{u}^{(0)} \in S \subset L^m$ is a ring homomorphism

$$\begin{aligned} \sigma_{\mathbf{u}^{(0)}} \colon K[\mathbf{u}] &\to L \\ f(\mathbf{u}) &\mapsto f(\mathbf{u}^{(0)}), \end{aligned}$$

that can be defined by giving the images $u_i^{(0)} \in L$ of the variables u_i for all i and considering $\sigma | K = \mathrm{id}_K$. In other words, this corresponds to replacing $\mathbf{u}$ by $\mathbf{u}^{(0)} \in L^m$.

Consider also the specialization

$$\begin{aligned} \sigma'_{\mathbf{u}^{(0)}} \colon K[\mathbf{u}]/\mathfrak{a} &\to L \\ f_{\mathfrak{a}}(\mathbf{u}) &\mapsto f(\mathbf{u}^{(0)}). \end{aligned}$$

The result of applying $\sigma'_{\mathbf{u}^{(0)}}$ to $f_{\mathfrak{a}}$ is the same for each element of $f_{\mathfrak{a}}$, and in particular to $f \in f_{\mathfrak{a}}$. So we shall denote $\sigma'_{\mathbf{u}^{(0)}}$ simply as $\sigma_{\mathbf{u}^{(0)}}$.

All of these specializations can be extended to the polynomial ring on $\mathbf{x}$ and denoted with the same label σ. We consider the specializations

$$\begin{array}{ccccc} & & \sigma_{\mathfrak{a}} & & \sigma_{\mathbf{u}^{(0)}} \\ K[\mathbf{u}][\mathbf{x}] & \longrightarrow & (K[\mathbf{u}]/\mathfrak{a})[\mathbf{x}] & \longrightarrow & L[\mathbf{x}] \\ f & \mapsto & f_{\mathfrak{a}} & \mapsto & f_{\mathbf{u}^{(0)}}. \end{array}$$

Defining $0_{\mathfrak{a}} = \{\sum_{\alpha} c_{\alpha} \mathbf{x}^{\alpha} : c_{\alpha} \in \mathfrak{a} \subset K[\mathbf{u}]\}$, we have

$$f_{\mathfrak{a}} = \{g \in K[\mathbf{u}][\mathbf{x}] : f - g \in 0_{\mathfrak{a}}\}.$$

Given an ideal $I \subset K[\mathbf{u}][\mathbf{x}]$, its projection on an ideal $\mathfrak{a}$ is the specialization

$$\sigma_{\mathfrak{a}}(I) = \{\sigma_{\mathfrak{a}}(f) : f \in I\}.$$

Exercise 3.1 Let R be a domain with unity and φ a ring homomorphism $\varphi \colon R \to R'$. Prove that R' is a domain if and only if $\ker(\varphi)$ is a prime ideal.

3.2.1 Representing Polynomials Under Specialization

There exist a distinguished element to represent the equivalence classes produced by
$\sigma_{\mathfrak{a}}\colon K[\mathbf{u}] \to K[\mathbf{u}]/\mathfrak{a}$ modulo the radical ideal $\mathfrak{a}$. It can be obtained as follows. Let A be the reduced Gröbner basis of $\mathfrak{a}$ wrt $\succ_{\mathbf{u}}$, and let $f(\mathbf{u}) \in K[\mathbf{u}]$. Then we can represent $\sigma_{\mathfrak{a}}(f)$ by the remainder r of the pseudo-division $\textsc{Reduce}(f, A)$. We denote the remainder r by $f^r_{\mathfrak{a}}$. It has the following properties:

(1) $f^r_{\mathfrak{a}} \in \sigma_{\mathfrak{a}}(f) = f_{\mathfrak{a}} \subset K[\mathbf{u}][\mathbf{x}]$;
(2) $f^r_{\mathfrak{a}}(\mathbf{u}^{(0)}) = f(\mathbf{u}^{(0)})$ for all $\mathbf{u}^{(0)} \in \mathbf{V}(\mathfrak{a})$;
(3) No monomial of $f^r_{\mathfrak{a}}$ is divisible by no $\mathrm{lpp}_{\mathbf{u}}$ of the polynomials in A.
(4) $f^r_{\mathfrak{a}} = 0$ if and only if $f_{\mathfrak{a}} = 0_{\mathfrak{a}}$.

Given a parametric polynomial $p = \sum_{\alpha} c_{\alpha}\mathbf{x}^{\alpha} \in K[\mathbf{u}][\mathbf{x}]$, it can be considered as a distributive polynomial where $c_{\alpha} \in K[\mathbf{u}]$ are the coefficients and $\mathbf{x}^{\alpha}$ are the power products of the variables $\mathbf{x}$. To compute the specialization of p on a variety $\mathbf{V}(E)$, first we have to compute $\mathfrak{a} = \textsc{Radical}(E)$, then $A = \textsc{ReducedGröbnerBasis}(\mathfrak{a}, \succ_{\mathbf{u}})$ and then reduce (using the ordinary division of multivariate polynomials) each coefficient c_{α} wrt to A. We have

$$\textsc{Reduce}(p, A) = p^r_{\mathfrak{a}} = \sum_{\alpha} c^r_{\alpha}\mathbf{x}^{\alpha},$$

where $c^r_{\alpha} = \textsc{Reduce}(c_{\alpha}, A)$ is the remainder of the division of c_{α} by A.

Some coefficients can become 0 in the specialization. Even if some coefficient is non-zero as a polynomial, it can become zero on a subset of points of $\mathbf{V}(E)$. If, for example, we want to ensure that the $\mathrm{lc}_{\mathbf{x}}(p)$ is non-zero on a segment whose top is E, we can eliminate from $\mathbf{V}(E)$ the variety $\mathbf{V}(N)$ of the points where the specialized $\mathrm{lc}_{\mathbf{x}}(p^r_{\mathfrak{a}})$ is zero. The result will be $\mathbf{V}(E) \setminus \mathbf{V}(\mathrm{lc}_{\mathbf{x}}(p^r_{\mathfrak{a}}))$.

The representation of $\sigma_{\mathfrak{a}}(p)$ by $p^r_{\mathfrak{a}}$ is called *non-faithful*, whereas the representation by p itself is called *faithful* following Weispfenning. Both representatives are elements of $\sigma_{\mathfrak{a}}(p)$.

3.2.2 NONNULL *Algorithm*

On many occasions we need to ensure that the leading coefficient of a polynomial is different from zero by the specialization. It is thus necessary to have a condition for deciding whether a polynomial in $K[\mathbf{u}]$ cannot become zero on a locally closed set $S = \mathbf{V}(E) \setminus \mathbf{V}(N)$. We have the following lemma:

Lemma 3.3 (Non-null) *Let $S = \mathbf{V}(E) \setminus \mathbf{V}(N) \subset L^m$ be a locally closed set determined by the pair of ideals $E = \langle E_1, \ldots, E_s\rangle$ and $N = \langle N_1, \ldots, N_t\rangle$ of $K[\mathbf{u}]$, and $g \in K[\mathbf{u}]$. Then, $g(\mathbf{u}^{(0)}) \neq 0$ for all $\mathbf{u}^{(0)} \in S$ if for all $j \in [t]$ the ideal of*

$\subset K[\mathbf{u}, t]$

$$I_j = \langle E_1, \ldots, E_s, g, 1 - tN_j \rangle = \langle 1 \rangle. \tag{3.1}$$

Proof Using Rabinowitz's trick, we prove that if $I_j = \langle 1 \rangle$, then

$$N_j \in \text{RADICAL}(\langle E_1, \ldots, E_s, g \rangle).$$

We proceed as follows. If $I_j = \langle 1 \rangle$, then $\mathbf{V}(I_j) = \emptyset$. Let $\mathbf{u}^{(0)}$ be any common zero of $E_1, \ldots, E_s, g$. As polynomials in $\mathbf{u}, t$ they are also 0 in $(\mathbf{u}^{(0)}, t)$ for all values of t. If $N_j(\mathbf{u}^{(0)})$ were non-zero, then for $t = t_0 = 1/N_j(\mathbf{u}^{(0)})$, also the last polynomial $1 - tN_j$ would become zero, and so $\mathbf{V}(I_j)$ would not be empty, which is impossible. Thus N_j vanishes at every common zero of $E_1, \ldots, E_s, g$, and by Hilbert Nullstellensatz $N_j \in \text{RADICAL}(E_1, \ldots, E_s, g)$.

As a consequence if condition (3.1) is verified for all $j \in [r]$, then

$$N \subseteq \text{RADICAL}(\langle E_1, \ldots, E_s, g \rangle).$$

Thus, for all $\mathbf{u}^{(0)} \in \mathbf{V}(E)$, if $g(\mathbf{u}^{(0)}) = 0$, then also $N_j(\mathbf{u}^{(0)}) = 0$ for all $j \in [r]$, which implies that $\mathbf{u}^{(0)} \notin \mathbf{V}(E) \setminus \mathbf{V}(N)$. And thus, if the condition is verified, we can ensure that for all $\mathbf{u}^{(0)} \in S$ is $g(\mathbf{u}^{(0)}) \neq 0$. □

Lemma 3.3 justifies Algorithm 3.1: NONNULL.

Example 3.4 Consider the ring $R = \mathbb{Q}[a, b, c]$ and the locally closed set $S = \mathbf{V}(a) \setminus \mathbf{V}(a, b)$. Test whether the polynomials b and c can or cannot be null on a point in S.

For b, NONNULL algorithm will consider the ideals $I_1 = \langle a, b, 1 - ta \rangle$ and $I_2 = \langle a, b, 1 - tb \rangle$. Computing their Gröbner bases both are equal to $\{1\}$. Thus b is non-null on S.

$t \leftarrow$ **Nonnull**$(c, \mathfrak{a}, \mathfrak{b})$
`Input:`
  $c \in K[\mathbf{u}]$: polynomial to decide if it is nonnull in $S = \mathbf{V}(\mathfrak{a}) \setminus \mathbf{V}(\mathfrak{b})$
  $\mathfrak{a}$, $\mathfrak{b}$ the C- (or R)-representation of the locally closed set S.
`Output`: t: is 1 if $c \neq 0$ on all points of S, t is 0 if c can be zero on some points of S

begin
  $t = 1$
  for all $b_j \in \mathfrak{b}$ **while** $t = 1$ **do**
    $I = \langle a_1, \ldots, a_s, c, 1 - tb_i \rangle$ # being $\mathfrak{a} = \langle a_1, \ldots, a_s \rangle$
    $G = \text{REDUCEDGRÖBNERBASIS}(I)$
    if $G \neq \{1\}$ **then** $t = 0$
  end for
  return(t)
end

Algorithm 3.1: NONNULL Algorithm

For c, NONNULL algorithm will consider the ideals $I_1 = \langle a, c, 1 - ta\rangle$ and $I_2 = \langle a, c, 1 - tb\rangle$. Computing their Gröbner bases we obtain $G_1 = \{1\}$ but $G_2 = \{c, bt - 1, a\} \neq \{1\}$. Thus c can be null on S.

Comment: Obviously b cannot be 0 on S, because the point $a = 0, b = 0$ is excluded from S. But c can be zero on S, as the point $a = 0, c = 0$ is not excluded from S. We verify the correctness of Lemma 3.3 in the example.

3.2.3 PNORMALF *of a Polynomial in* $S = \mathbf{V}(\mathfrak{a}) \setminus \mathbf{V}(\mathfrak{b})$

Usually we consider monic polynomials and assume that the leading coefficient of a polynomial in $K[\mathbf{u}][\mathbf{x}]$ is to be considered as the denominator for which we have to divide the polynomial to make it really monic. So, if a polynomial has a factor in $K[\mathbf{u}]$ and we can ensure that it is non-null in S, we can divide by it, thus simplifying the result. So, let $[\mathfrak{a}, \mathfrak{b}]$ be the C-representation of $S = \mathbf{V}(\mathfrak{a}) \setminus \mathbf{V}(\mathfrak{b})$. We define the PNORMALF$(p)$ on S as the result of reducing p by $A =$ REDUCEDGRÖBNERBASIS$(\mathfrak{a})$ and dividing by the non-null factors of the gcd of all the coefficients of p. This is obtained using Algorithm 3.2: PNORMALF.

Example 3.5 Consider the polynomial $f = (a-c)x^2y+cx^2+bcy$ and the segment $S = \mathbf{V}(a-c) \setminus \mathbf{V}(a-c, c)$. Determine its normal-form on S. We use the command `pnormalf` of the SINGULAR library "grobcov.lib".

$g \leftarrow$ **Pnormalf**$(f, \mathfrak{a}, \mathfrak{b})$
`Input:`
  $f \in K[\mathbf{u}][\mathbf{x}]$: polynomial to be specialized
  $\mathfrak{a}, \mathfrak{b}$ the C-representation of the locally closed set S.
`Output:` g the normal form of the specialization of f on S

begin
  Let $p = \sum_\alpha c_\alpha \mathbf{x}^\alpha$
  $g = \sum_\alpha$ REDUCE$(c_\alpha, \mathfrak{a}) = \sum_\beta c'_\beta x^\beta$
    # where the c'_β are the coefficients that have not been reduced
      to zero by specialization.
  $m =$ GCD$(c'_{\beta_1}, \ldots, c'_{\beta_\ell})$
  $\{m_1^{n_1}, \ldots, m_s^{n_s}\} =$ FACTORS(m)
  $k = 1$
  for $i = 1$ **to** s **do**
    if NONNULL$(m_i, \mathfrak{a}, \mathfrak{b})$ **then** $k = km_i^{n_i}$ **end if**
  end do
  $g = g/k$
  return(g)
end

Algorithm 3.2: PNORMALF algorithm

```
> LIB "grobcov.lib";
> ring R=(0,a,b,c),(x,y),dp;
> short=0;
> ideal E=a-c;
> ideal N=a-c,c;
> poly f=(a-c)*x^2*y+c*x^2+b*c*y;
> pnormalf(f,E,N);
x^2+(b)*y;
```

As a first step, the algorithm reduces the coefficients modulo $\langle a - c \rangle$. The result is $cx^2 + bcy$. Then the algorithm considers the gcd of the coefficients, which is c, and applies to it the algorithm NONNULL that returns 1, i.e., it verifies that c is non-null on S. Then the algorithm divides the result by c obtaining $x^2 + by$, that is the normal-form of f on S.

3.2.4 PDIVI *Algorithm*

Let us introduce the pseudo-division algorithm in $K[\mathbf{u}][\mathbf{x}]$, called PDIVI. Let $f, g_1, \ldots, g_s \in K[\mathbf{u}][\mathbf{x}]$, and let $K(\mathbf{u})$ be the quotient field of $K[\mathbf{u}]$. Consider the ordinary pseudo-division REDUCE of f by $g_1, \ldots, g_s$ considering the polynomials over the quotient field, i.e., the pseudo-division in $K(\mathbf{u})[\mathbf{x}]$. This generates quotients $q_1, \ldots, q_s$ and remainder r verifying:

$$f = q_1 g_1 + \cdots + q_s g_s + r \tag{3.2}$$

(see [25]), where

(1) $q_1, \ldots, q_s, r \in K(\mathbf{u})[\mathbf{x}]$,
(2) No power product in the support of r is divisible by any $\mathrm{lpp}_{\mathbf{x}}(g_i)$,
(3) $\mathrm{lpp}_{\mathbf{x}}(f) \succeq_{\mathbf{x}} \mathrm{lpp}_{\mathbf{x}}(q_i g_i)$ for $1 \leq i \leq s$,
(4) the denominators of coefficients of $q_1, \ldots, q_s, r \subset K(\mathbf{u})$ are only factors of the $\mathrm{lc}_{\mathbf{x}}(g_i)$.

We can thus multiply (3.2) by $\mu \in K[\mathbf{u}]$, the lcm of all the denominators, and obtain

$$\mu f = q'_1 g_1 + \cdots + q'_s g_s + r' \tag{3.3}$$

where

(1) $q'_1, \ldots, q'_s, r \in K[\mathbf{u}][\mathbf{x}]$,
(2) No power product in the support of r' is divisible by no $\mathrm{lpp}_{\mathbf{x}}(g_i)$,
(3) $\mathrm{lpp}_{\mathbf{x}}(f) \succeq_{\mathbf{x}} \mathrm{lpp}_{\mathbf{x}}(q'_i g_i)$ for $1 \leq i \leq s$,
(4) μ is the product of some factors of the $\mathrm{lc}_{\mathbf{x}}(g_i)$.

The remainder $r \in K[\mathbf{u}][\mathbf{x}]$ is a reduced form of f wrt to the divisors. Just as for the ordinary pseudo-division REDUCE, the result is not unique and depends on the

$r, Q \leftarrow$ **Pdivi**(f, G)
`Input:`
$f \in K[\mathbf{u}][\mathbf{x}]$: polynomial dividend
$G = [g_1, \ldots, g_s] \subset K[\mathbf{u}][\mathbf{x}]$: ideal of divisors (in a specific order)
The pseudo-division is made on $K(\mathbf{u})[\mathbf{x}]$
`Output:` $r, [q_1, \ldots, q_s]$
r: the remainder of the pseudo-division
$[q_1, \ldots, q_s]$: the quotients of the pseudo-division
$\mu \in K[\mathbf{u}]$ the factor that multiplies f to have polynomials in $K[\mathbf{u}][\mathbf{x}]$

begin
$p = f, r = 0,$ **for** $i \in [s]$ **do** $q_i = 0$ **end if**
while $p \neq 0$ **do**
$t = 1, i = 1$
while $i \leq s$ and $t = 1$ **do**
if $\mathrm{lpp}_{\mathbf{x}}(g_i)$ divides $\mathrm{lpp}_{\mathbf{x}}(p)$ **then** # division step
$\psi = \dfrac{\mathrm{lc}_{\mathbf{x}}(p)}{\mathrm{lc}_{\mathbf{x}}(g_i)} \dfrac{\mathrm{lpp}_{\mathbf{x}}(p)}{\mathrm{lpp}_{\mathbf{x}}(g_i)}$
$q_i = q_i + \psi$
$p = p - \psi\, g_i$
$t = 0$
end if
$i = i + 1$
end while
if $t = 1$ **then** # remainder step
$r = r + \mathrm{lm}_{\mathbf{x}}(p)$
$p = p - \mathrm{lm}_{\mathbf{x}}(p)$
end if
end while
$\mu = \mathrm{lcm}(\mathrm{denoms}(q_1, \ldots, q_s, r))$
for $i \in [s]$ **do** $q_i = \mu q_i$ **end if**; $r = \mu r$
return$(r, [q_1, \ldots, q_s], \mu)$
end

Algorithm 3.3: PDIVI algorithm

order in which the polynomials of the divisor are considered. PDIVI algorithm is described as Algorithm 3.3.

Example 3.6 Consider the division of the following polynomial f and the three polynomials in G:

$$\begin{aligned} f =& (a^2b + a^2 + b^2 + c^2 + c)x^2y + (a^2 + b^2)x^2 \\ &+ (a + b + c)x + (a^2 + b^2)y^2, \\ G =& \left[(a^2 + c)x^2y + cx,\ cx^2y - b,\ a^3x + c\right]. \end{aligned}$$

We use the command `pdivi` of the SINGULAR "grobcov.lib" library;

```
> LIB "grobcov.lib";
> ring R=(0,a,b,c),(x,y),lp;
> short=0;
> poly f=(a^2*b+a^2+b^2+c^2+c)*x^2*y+(a^2+b^2)*x^2+(a+b+c)*x
     +(a^2+b^2)*y^2;
> ideal G=(a^2+c)*x^2*y+c*x, c*x^2*y-b, a^3*x+c;
> def P=pdivi(f,G); P;
[1]:
   (a^10+a^8*b^2+a^8*c+a^6*b^2*c)*y^2+(-a^6*c+a^5*b*c^2-a^5*b*c
   +a^3*b^2*c^2-a^3*b*c^2+a^3*c^4+a^2*b^2*c^2+a^2*c^3+b^2*c^3)
[2]:
   _[1]=(a^8*b+a^8+a^6*b^2+a^6*c^2+a^6*c)
   _[2]=0
   _[3]=(a^7+a^5*b^2+a^5*c+a^3*b^2*c)*x+(a^6-a^5*b*c+a^5*b
   -a^3*b^2*c+a^3*b*c-a^3*c^3-a^2*b^2*c-a^2*c^2-b^2*c^2)
[3]:
   (a^8+a^6*c)
def r=P[1]; def Q=P[2]; def mu=P[3];
> mu*f-Q[1]*G[1]-Q[2]*G[2]-Q[3]*G[3]-r;
0
```

We can see that the identity (3.3) is verified and that no monomial in r is divisible by any leading monomial of the divisors. The result is to be read as follows:

$$
\begin{aligned}
r =& (a^{10} + a^8b^2 + a^8c + a^6b^2c)y^2 \\
&+ (-a^6c + a^5bc^2 - a^5bc + a^3b^2c^2 - a^3bc^2 + a^3c^4 + a^2b^2c^2 + a^2c^3 + b^2c^3) \\
q_1 =& (a^8b + a^8 + a^6b^2 + a^6c^2 + a^6c) \\
q_2 =& 0 \\
q_3 =& (a^7 + a^5b^2 + a^5c + a^3b^2c)x \\
&+ (a^6 - a^5bc + a^5b - a^3b^2c + a^3bc - a^3c^3 - a^2b^2c - a^2c^2 - b^2c^2) \\
\mu =& a^6(a^2 + c).
\end{aligned}
$$

We will now study division under specialization.

Theorem 3.7 *Let* $f, g_1, \ldots, g_s \in K[\mathbf{u}][\mathbf{x}]$, *and* $\sigma_{\mathfrak{a}} : K[\mathbf{u}] \to K[\mathbf{u}]/\mathfrak{a}$, *a specialization satisfying* $\sigma_{\mathfrak{a}}(\mathrm{lc}_{\mathbf{x}}(g_i)) \neq 0$ *and* $\sigma_{\mathbf{u}^{(0)}}(\mathrm{lc}_{\mathbf{x}}(g_i)) \neq 0$ *for all* $\mathbf{u}^{(0)} \in \mathbf{V}(\mathfrak{a}) \setminus \mathbf{V}(\mathfrak{b}) \subset L^m$ *and for* $1 \leq i \leq s$.

Let $q_1, \ldots, q_s, r \in K[\mathbf{u}][\mathbf{x}]$, *and* $\mu \in K[\mathbf{u}]$ *be the quotients, remainder and* μ*-factor in the* PDIVI *division of* f *by* $g_1, \ldots, g_s$. *Then* $\sigma_{\mathfrak{a}}(q_1), \ldots, \sigma_{\mathfrak{a}}(q_s)$ *and* $\sigma_{\mathfrak{a}}(r)$ *make sense for all points* $\mathbf{u}^{(0)} \in \mathbf{V}(\mathfrak{a}) \setminus \mathbf{V}(\mathfrak{b})$ *and verify that*

$$\sigma_{\mathfrak{a}}(\mu)\sigma_{\mathfrak{a}}(f) = \sum_{i=1}^{s} \sigma_{\mathfrak{a}}(q_i)\sigma_{\mathfrak{a}}(g_i) + \sigma_{\mathfrak{a}}(r). \tag{3.4}$$

Moreover, the above expression corresponds to the ordinary pseudo-division REDUCE *in* $L[\mathbf{x}]$ *of* $\sigma_{\mathbf{u}^{(0)}}(\mu)\sigma_{\mathbf{u}^{(0)}}(f)$ *by* $\sigma_{\mathbf{u}^{(0)}}(g_1), \ldots, \sigma_{\mathbf{u}^{(0)}}(g_s)$ *for every* $\mathbf{u}^{(0)} \in \mathbf{V}(\mathfrak{a}) \setminus \mathbf{V}(\mathfrak{b})$.

We note that some terms of $\sigma_{\mathfrak{a}}(q_i)$ can specialize to 0 for particular points $\mathbf{u}^{(0)} \in \mathbf{V}(\mathfrak{a}) \setminus \mathbf{V}(\mathfrak{b})$, but the result is still valid for these points.

Proof Formula (3.4) is the result of applying $\sigma_{\mathfrak{a}}$ to Eq. (3.3), taking into account that the specialization is a ring homomorphism.

We use induction on the divisions step in $K(\mathbf{u})[\mathbf{x}]$. Call $(p^{(j)}, {q_1}^{(j)}, \ldots, {q_s}^{(j)}, r^{(j)})$ and $(p'^{(k)}, {q'_1}^{(k)}, \ldots, {q'_s}^{(k)}, r'^{(k)})$ the partial (divisors, quotients and remainder) at steps j and k respectively of the divisions in $K(\mathbf{u})[\mathbf{x}]$ and in $L[\mathbf{x}]$.

We set out to prove that for each step j of the division of f by G, there is some step k in the division of $\sigma_{\mathfrak{a}}(f)$ by $\sigma_{\mathfrak{a}}(G)$, so that $\sigma_{\mathfrak{a}}(p^{(j)}) = p'^{(k)}$, $\sigma_{\mathfrak{a}}(q_i^{(j)}) = q_i'^{(k)}$ for $i = 1, \ldots, s$ and $\sigma_{\mathfrak{a}}(r^{(j)}) = r'^{(k)}$, specializing as well for every $\mathbf{u}^{(0)} \in \mathbf{V}(\mathfrak{a}) \setminus \mathbf{V}(\mathfrak{b})$. It is true, by construction, for $j = 0$ by picking $k = 0$. Suppose we are at step j in the division in $K(\mathbf{u})[\mathbf{x}]$, and let us assume the induction hypothesis.

Assume that j is a division step and let i_j be the first index for which $\mathrm{lpp}_{\mathbf{x}}(g_{i_j})$ divides $\mathrm{lpp}_{\mathbf{x}}(p^{(j)})$. The new partial quotient and the new partial remainder will be

$$\begin{aligned} \psi_{i_j} &= \frac{\mathrm{lm}_{\mathbf{x}}(p^{(j)})}{\mathrm{lm}_{\mathbf{x}}(g_{i_j})} \\ p^{(j+1)} &= p^{(j)} - \psi_{i_j} g_{i_j}. \end{aligned} \tag{3.5}$$

Observe that by hypothesis, $\sigma_{\mathfrak{a}}$ makes sense on ψ_{i_j} and $p^{(j+1)}$ for all points $\mathbf{u}^{(0)} \in \mathbf{V}(\mathfrak{a}) \setminus \mathbf{V}(\mathfrak{b})$. So, specializing, for a point $\mathbf{u}^{(0)} \in \mathbf{V}(\mathfrak{a}) \setminus \mathbf{V}(\mathfrak{b})$, two cases can occur:

Case 1. $\sigma_{\mathbf{u}^{(0)}}(\mathrm{lc}_{\mathbf{x}}(p^{(j)})) = 0$. In this case $\sigma_{\mathbf{u}^{(0)}}(\psi_{i_j}) = 0$ and $\sigma_{\mathbf{u}^{(0)}}(p^{(j+1)}) = \sigma_{\mathbf{u}^{(0)}}(p^{(j)})$, and the same is true for the partial quotients. So the same k applies to $j + 1$.

Case 2. $\sigma_{\mathbf{u}^{(0)}}(\mathrm{lc}_{\mathbf{x}}(p^{(j)})) \neq 0$. Then $\sigma_{\mathbf{u}^{(0)}}(\psi_{i_j}) \neq 0$. Since by hypothesis all $\mathrm{lpp}_{\mathbf{x}}(g_i)$ remain stable under specialization, i_j is the first index for which $\mathrm{lpp}_{\mathbf{x}}(\sigma_{\mathbf{u}^{(0)}}(g_{i_j}))$ divides $\mathrm{lpp}_{\mathbf{x}}(p'^{(k)})$, so

$$\begin{aligned} \psi'_{i_j} &= \frac{\mathrm{lm}_{\mathbf{x}}(p'^{(k)})}{\mathrm{lm}_{\mathbf{x}}(\sigma_{\mathbf{u}^{(0)}}(g_{i_j}))} \\ p'^{(k+1)} &= p'^{(k)} - \psi'_{i_j} \sigma_{\mathbf{u}^{(0)}}(g_{i_j}). \end{aligned}$$

Using the induction hypothesis for j and specializing Eqs. (3.5) yields:

$$\begin{aligned} \sigma_{\mathbf{u}^{(0)}}(\psi_{i_j}) &= \frac{\sigma_{\mathbf{u}^{(0)}}(\mathrm{lm}(p^{(j)}))}{\sigma_{\mathbf{u}^{(0)}}(\mathrm{lm}_{\mathbf{x}}(g_{i_j}))} = \frac{\mathrm{lm}_{\mathbf{x}}(p'^{(k)})}{\mathrm{lm}_{\mathbf{x}}(\sigma_{\mathbf{u}^{(0)}}(g_{i_j}))} = \psi'_{i_j} \\ \sigma_{\mathbf{u}^{(0)}}(p^{(j+1)}) &= p'^{(k)} - \psi'_{i_j} \sigma_{\mathbf{u}^{(0)}}(g_{i_j}) = p'^{(k+1)}, \end{aligned}$$

so in this case we pick $k + 1$. Note that the partial quotients also remain stable.

If step j is a remainder step, we have $r^{(j+1)} = r^j + \mathrm{lm}_{\mathbf{x}}(p^{(j)})$ and $p^{(j+1)} = p^j - \mathrm{lm}_{\mathbf{x}}(p^{(j)})$. Specializing and using the induction hypothesis, if $\sigma_{\mathbf{u}^{(0)}}(\mathrm{lc}_{\mathbf{x}}(p^{(j)})) \neq 0$, then we have

$$\begin{aligned}\sigma_{\mathbf{u}^{(0)}}(r^{(j+1)}) &= r^{(j)} + \sigma_{\mathbf{u}^{(0)}}(\mathrm{lm}_{\mathbf{x}}\, p^{(j)}) \\ &= r'^{(k)} + \sigma_{\mathbf{u}^{(0)}}(\mathrm{lm}_{\mathbf{x}}\, p'^{(k)}) = r'^{(k+1)},\end{aligned}$$

proving the hypothesis for $k+1$. If $\sigma_{\mathbf{u}^{(0)}}(\mathrm{lc}_{\mathbf{x}}(p^{(j)}) = 0$, nothing changes and k does not change for step $j+1$.

Since the remainders (and quotients) in the divisions are the last partial remainders (and quotients) after multiplying by the non-null μ-factor, the proposition is proved.

Under the hypothesis of Theorem 3.7, we can multiply by the μ-factor that will be non-null in the specialization and will only change the normalization. If the condition on the leading monomials is satisfied, Theorem (3.7) guarantees that PDIVI specializes automatically with the non-null factor $\sigma_{\mathfrak{a}}(\mu) \neq 0$ for all points $\mathbf{u}^{(0)} \in \mathbf{V}(\mathfrak{a}) \setminus \mathbf{V}(\mathfrak{b})$. □

Example 3.8 Consider the pseudo-division of

$$\begin{aligned}f &= ax^2y + bxy^2 + 5x + (c^2-1)y + 1; \\ G &= [cx^2 + dy + a,\ dy - 4];\end{aligned}$$

and the specialization on a point $\mathbf{u}^{(0)} \in \mathbf{V}(a,b) \setminus \mathbf{V}(cd)$.

$$\begin{aligned}f' = \sigma(f) &= 5x + (c^2-1)y + 1; \\ G' = \sigma(G) &= [cx^2 + dy,\ dy - 4].\end{aligned}$$

Table 3.1 describes in parallel the steps of the PDIVI algorithm applied to the ordinary division and the specialized one, showing the correspondence. From the table, the result for the non-specialized division (before multiplying by μ) is

$$\begin{aligned}r &= ((16b + 5d^2)/d^2)x + (-4a^2 - 16a + 4c^3 + cd - 4c)/(cd), \\ q_1 &= (a/c)y, \\ q_2 &= (b/d)xy + (4b/d^2)x + (-a/c)y + ((-a^2 - 4a + c^3 - c)/(cd)),\end{aligned}$$

and for the specialized one:

$$\begin{aligned}r' &= 5x + (4c^2 + d - 4c)/d, \\ q'_1 &= 0, \\ q'_2 &= (c^2-1)/d,\end{aligned}$$

Table 3.1 Corresponding steps of non-specialized and specialized pseudo-divisions in Example 3.8

Non-specialized division steps	Specialized division steps
$p^{(0)} = ax^2y + bxy^2 + 5x + (c^2-1)y + 1,$	$p'(0) = 5x + (c^2-1)y + 1$
$\psi(1) = (a/c)y$ $p^{(1)} = bxy^2 + 5x - (ad/c)y^2$ $+((-a^2+c^3-c)/c)y + 1$ $q_1 = (a/c)y$	
$\psi^{(2)} = (b/d)xy$ $p^{(2)} = ((4b)/d)xy + 5x + ((-ad)/(c))y^2$ $+((-a^2+c^3-c)/(c))y + 1$ $q_2 = (b/d)xy$	
$\psi^{(3)} = (4b/d^2)x$ $p^{(3)} = ((16b+5d^2)/d^2)x + ((-ad)/c)y^2$ $+((-a^2+c^3-c)/c)y + 1$ $q_2 = (b/d)xy + (4b/d^2)x$	
$r^{(4)} = ((16b+5d^2)/d^2)x$ $p^{(4)} = ((-ad)/(c))y^2$ $+((-a^2+c^3-c)/c))y + 1$	$r'^{(1)} = 5x$ $p'^{(1)} = (c^2-1)y + 1$
$\psi^{(5)} = (-a/c)y$ $p^{(5)} = ((-a^2-4a+c^3-c)/c)y + 1$ $q_2 = (b/d)xy + (4b/d^2)x + (-a/c)y$	
$\psi^{(6)} = (-a^2-4a+c^3-c)/(cd)$ $p^{(6)} = (-4a^2-16a+4c^3+cd-4c)/(cd)$ $q_2 = (b/d)xy + (4b/d^2)x + (-a/c)y$ $+((-a^2-4a+c^3-c)/(cd))$	$\psi'^{(2)} = (c^2-1)/d$ $p'^{(2)} = (4c^2+d-4)/d$ $q'_2 = (c^2-1)/d$
$r^{(7)} = ((16b+5d^2)/d^2)x$ $+(-4a^2-16a+4c^3+cd-4c)/(cd)$ $p^{(7)} = 0$	$r'^{(3)} = 5x + (4c^2+d-4)/d$ $p'^{(3)} = 0$

which corresponds well to the specialization of the non-specialized division. For the non-specialized division, $\mu = cd^2$, whereas for the specialized division $\mu = d$ suffices.

Observe that on the points of

$$\mathbf{u}^{(0)} \in \mathbf{V}(a, b, c^2-1) \setminus \mathbf{V}(cd) \subset \mathbf{V}(a, b) \setminus \mathbf{V}(cd),$$

step $k = 2$ does not exist, but nevertheless the specialization is still valid.

Now compare the result with the effective computation using the `pdivi` command of the SINGULAR "grobcov.lib" library:

```
> LIB "grobcov.lib";
> ring R=(0,a,b,c,d),(x,y),lp;
> short=0;
> poly f=a*x^2*y+b*x*y^2+5*x+(c^2-1)*y+1;
> ideal G=c*x^2+d*y+a,d*y-4;
> pdivi(f,G);
[1]:
   (16*b*c+5*c*d^2)*x+(-4*a^2*d-16*a*d+4*c^3*d+c*d^2-4*c*d)
[2]:
   _[1]=(a*d^2)*y
   _[2]=(b*c*d)*x*y+(4*b*c)*x+(-a*d^2)*y+(-a^2*d-4*a*d+c^3*d-c*d)
[3]:
   (c*d^2)
> poly f0=5*x+(c^2-1)*y+1;
> ideal G0=c*x^2+d*y, d*y-4;
> pdivi(f0,G0);
[1]:
   (5*d)*x+(4*c^2+d-4)
[2]:
   _[1]=0
   _[2]=(c^2-1)
[3]:
   (d)
```

3.3 Comprehensive Gröbner Systems

Given a parametric ideal $I \subset K[\mathbf{u}][\mathbf{x}]$, we aim to obtain locally closed subsets $S_i \subseteq L^m$, (that can be determined by their $\text{CREP}(S_i) = [\mathfrak{a}_i, \mathfrak{b}_i]$), such that there exist a set of polynomials $G_i \subset I$ over S_i which by specializing them on each point $\mathbf{u}^{(0)} \in S_i$, the resulting set $\sigma_{\mathbf{u}^{(0)}}(G_i)$ is a Gröbner basis of the ideal obtained by replacing $\mathbf{u}$ by $\mathbf{u}^{(0)}$ in I, i.e., by the specialization $\sigma_{\mathbf{u}^{(0)}} : (K[\mathbf{u}]/\mathfrak{a})[\mathbf{x}] \to L[\mathbf{x}]$ of $I_{\mathfrak{a}}$ on $\mathbf{u}^{(0)}$.

The polynomials of G_i can be specialized in $\mathfrak{a}$ and represented by their reduced form. Denote B_i to be the reduced representation of the specialized polynomials $\sigma_{\mathfrak{a}}(G_i)$. The G_i are *faithful* (they belong to I), whereas the reduced form of the specialization are *non-faithful* (they do no longer belong to I).

Definition 3.9 (Comprehensive Gröbner System (CGS)) Given a parametric ideal $I \subset K[\mathbf{u}][\mathbf{x}]$, a Comprehensive Gröbner System (CGS) of I is a set of pairs

$$\text{CGS}(F) = \{[S_i, B_i] : 1 \leq i \leq s\}$$

where

(1) $S_i = \mathbf{V}(\mathfrak{a}_i) \setminus \mathbf{V}(\mathfrak{b}_i)$ are locally closed subsets of L^m that we shall consider represented by its $\text{CREP}(S_i) = [\mathfrak{a}_i, \mathfrak{b}_i]$;
(2) the B_i are subsets of $(K[\mathbf{u}]/\mathfrak{a})[\mathbf{x}]$ that specialize for every point $\mathbf{u}^{(0)} \in S_i$ to a Gröbner basis of $\sigma_{\mathbf{u}^{(0)}}(I)$;
(3) $\bigcup_{i=1}^{s} = L^m$.

The B_i can be faithful, i.e., polynomials in I, or non-faithful, i.e., reduced modulo $\mathfrak{a}_i$, and represented by their reduced form as simple polynomials of $K[\mathbf{u}][\mathbf{x}]$.

First, we have to prove the existence of a CGS, and then we turn our attention to obtaining a CGS with *disjoint* S_i and bases B_i of reduced polynomials modulo $\mathfrak{a}_i$ specializing to the reduced Gröbner basis of $\sigma_{\mathbf{u}^{(0)}}(\langle F \rangle)$ for every point $\mathbf{u}^{(0)} \in S_i$. The subsets S_i will be called *segments*.

Assume that we consider a subset $\mathbf{V}(\mathfrak{a}_i) \subseteq L^m$ for which we try to find a subset $G_i \subset I$ whose reduced specialization B_i on $\mathbf{V}(\mathfrak{a}_i)$ specializes to a Gröbner basis for every point of $\mathbf{V}(\mathfrak{a}_i)$. By construction, we choose polynomials of B_i in such a form that the leading coefficients are not zero as elements of $K[\mathbf{u}]/\mathfrak{a}$. But they can become zero in some subset of $\mathbf{V}(\mathfrak{a}_i)$ and so the lpp of the polynomials in B_i do not remain stable on the whole $\mathbf{V}(\mathfrak{a}_i)$. This is the reason for reducing the set $\mathbf{V}(\mathfrak{a}_i)$ removing a variety containing all the points of $\mathbf{V}(\mathfrak{a}_i)$ where some leading coefficient of a polynomial in B_i specializes to zero, i.e. $\mathfrak{b} = \text{RAD}(\prod_j \text{lc}_{\mathbf{x}}(B_{ij}))$. We will develop this idea when building a CGS.

We will obtain bases B_i with reduced polynomials, and thus are non-faithful. The polynomials in each specialized class $f_{\mathfrak{a}_i,ij} \in (K[\mathbf{u}]/\mathfrak{a}_i)[\mathbf{x}]$ are represented by its reduced form $f^r_{\mathfrak{a}_i,ij}$ (non-faithful). But we can also conserve the faithful expression f_{ij}, equivalent to $f^r_{\mathfrak{a}_i,ij}$ modulo $\mathfrak{a}_i$. Considering all the faithful polynomials equivalent to the polynomials in the B_i, we can obtain a *Comprehensive Gröbner Basis B* that specializes to a Gröbner basis of the specialized ideal $\sigma_{\mathbf{u}^{(0)}}(I)$ for every point $\mathbf{u}^{(0)} \in L^m$.

Definition 3.10 (Comprehensive Gröbner basis (CGB)) Given a parametric ideal $I \subset K[\mathbf{u}][\mathbf{x}]$, a Comprehensive Gröbner Basis (CGB) of I is a finite subset $G \subset I$ that specializes for every point $\mathbf{u}^{(0)} \in L^m$ to a Gröbner basis of $\sigma_{\mathbf{u}^{(0)}}(I)$.

As mentioned above, if we are able to obtain a CGS, considering the faithful polynomials, anti-images of all the non-faithful polynomials in the bases of the CGS, the result is a CGB.

We now begin to develop the algorithm for obtaining a CGS. In our initial description of the Gröbner Cover [63], we used our own algorithm BUILDTREE, which is appropriate for the GC, as it generates a disjoint reduced CGS with small number of segments.

Nevertheless, in the actual implementation of the library "grobcov.lib", it has been replaced by the more efficient Kapur-Sun-Wang (KSW) algorithm [43]. In order to be thorough, we have included the description of BUIDTREE in Appendix A.

Now, we proceed to describe the KSW algorithm. It starts by considering the Suzuki-Sato algorithm, but it is a deep improvement.

3.4 Suzuki-Sato Algorithm

This section is devoted to a historical family of relevant algorithms that were the starting point of the actual Kapur-Sun-Wang algorithm described in the next section and which we consider the actual algorithm to be used for computing a CGS.

Kalkbrenner's stabilization Theorem 3.1 in [41] was the starting point for the new family of algorithms for computing a CGS. The first algorithm based on these ideas was proposed by Suzuki-Sato [75]. We summarize briefly the key lemmas giving rise to the SS algorithm.

In what follows we consider $\succ_{\mathbf{x}\cdot\mathbf{u}}$ to be an admissible block order, equal to the product orders $\succ_{\mathbf{x}} \cdot \succ_{\mathbf{u}}$, so that $\mathbf{x} \succ \mathbf{u}$.

Lemma 3.11 (Suzuki-Sato) *Let $F \subset K[\mathbf{u}][\mathbf{x}]$ be a parametric ideal. Denote $\sigma = \sigma_{\mathbf{u}^{(0)}}$ the specialization in a point $\mathbf{u}^{(0)} \in L^m$. If*

(i) $G = \mathrm{GB}(\langle F\rangle, \succ_{\mathbf{x}\cdot\mathbf{u}})$, # GB=Gröbner Basis
(ii) $G_1 = \{g \in G : \sigma(\mathrm{lc}_{\mathbf{x}}(g)) \neq 0\}$,

then $\sigma(G_1)$ is a $\mathrm{GB}(\langle\sigma(F)\rangle, \succ_{\mathbf{x}})$ iff $\mathrm{Red}(\sigma(g), \sigma(G_1)) = 0$ for every $g \in G$.

Lemma 3.12 (Suzuki-Sato) *If*

(i) $G = \mathrm{GB}(\langle F\rangle, \succ_{\mathbf{x}\cdot\mathbf{u}})$,
(ii) $\sigma(\mathrm{lc}_{\mathbf{x}}(g)) \neq 0\}$ *for each $g \in G \setminus (G \cap K[\mathbf{u}])$,*

then $\sigma(G)$ is a $\mathrm{GB}(\langle\sigma(F)\rangle, \succ_{\mathbf{x}})$.

The main idea of the SS algorithm is to first compute a reduced Gröbner basis $G = \mathrm{GB}(\langle F\rangle, \succ_{\mathbf{x}\cdot\mathbf{u}})$, and consider for which specializations it stabilizes to a Gröbner basis. Let $\{h_1, \ldots, h_t\} = \{\mathrm{lc}_{\mathbf{x}}(g) : g \in G \setminus (G \cap K[\mathbf{u}])\}$. By Lemma 3.12, $(G \cap K[\mathbf{u}], \mathbf{V}(h_1) \cup \cdots \cup \mathbf{V}(h_t), G)$ forms a branch of a CGS for F, i.e., for any $\mathbf{u}^{(0)} \in \mathbf{V}(G \cap K[\mathbf{u}]) \setminus (\mathbf{V}(h_1) \cup \cdots \cup \mathbf{V}(h_t))$, $\sigma_{\mathbf{u}^{(0)}}(G)$ this is a Gröbner basis of $\langle\sigma_{\mathbf{u}^{(0)}}(F)\rangle$ wrt to $\succ_{\mathbf{x}}$. For computing other branches corresponding to the specialization on $\mathbf{u}^{(0)} \in \mathbf{V}(h_1) \cup \cdots \cup \mathbf{V}(h_t)$, Lemma 3.12 is used for each $F \cup \{h_i\}$ repeating the steps until the algorithm terminates.

To implement this algorithm we only need an efficient implementation of a Gröbner basis algorithm on a polynomial ring over a field. It has very good performances since it takes advantage of well-known fast implementations for them.

Nevertheless, the algorithm suffers from certain weaknesses. It does not check whether $\mathbf{V}(G \cap K[\mathbf{u}]) \setminus \mathbf{V}(h)$ is empty or not; This results in many redundant and non-disjoint branches. In [80], an improved version of the algorithm is reported which removes redundant branches. Nabeshima proposed a speed-up algorithm in [66]. The main idea of that algorithm is to explore inequality constraints in a parametric

setting. For every lpp in $G \setminus K[\mathbf{u}]$ that is a nontrivial multiple of any other product in it, a branch is generated by asserting its leading coefficient h_i to be nonzero. The corresponding polynomial is made monic using Rabinowitz's trick to introduce a new variable to handle the inequality $h_i \neq 0$, and the Gröbner basis computation is performed again, simplifying polynomials whose lpp are multiples, including their parametric coefficients.

3.5 Kapur-Sun-Wang Proof of the Existence of a CGS

Kapur-Sun-Wang [43] gives an algorithm that avoids any unnecessary branches in the SS algorithm by using radical membership check for parametric constraints asserted to be nonzero. The resulting algorithm has *disjoint* branches and has fewer branches. It uses heuristics to do this check, and only when the heuristics fail, is the general check performed by introducing a new variable, because of the inefficiency of Rabinowitz's trick. Another major improvement of the KSW method is that along any other branch for which the specialized Gröbner basis is different from $\{1\}$, only one polynomial from $G \setminus K[\mathbf{u}]$ per minimal lpp is selected. The algorithm is based on the following definition:

Definition 3.13 (NonComparable) Given a set $G \subset K[\mathbf{u}][\mathbf{x}]$, we define $F = \text{NONCOMPARABLE}(G) \subseteq G$ to be a subset with the following properties:

(1) every polynomial $g \in G$ is such that $\text{lpp}_{\mathbf{x}}(g)$ is a multiple of $\text{lpp}_{\mathbf{x}}(f)$ for some $f \in F$,
(2) for any two distinct $f_1, f_2 \in F$, neither $\text{lpp}_{\mathbf{x}}(f_1)$ is a multiple of $\text{lpp}_{\mathbf{x}}(f_2)$ nor vice-versa.

Note that given a subset $G \subset K[\mathbf{u}][\mathbf{x}]$ different NONCOMPARABLE subsets of G can exist, but in any case $\langle \text{lpp}_{\mathbf{x}}(F)\rangle = \langle \text{lpp}_{\mathbf{x}}(G)\rangle$. The following lemma is a slight modification of Lemma 4.4 in [43].

Lemma 3.14 (Kapur-Sun-Wang) *Let G be a Gröbner basis of $\langle G\rangle \subset K[\mathbf{u}, \mathbf{x}]$ with respect to $\succ_{\mathbf{x},\mathbf{u}}$, $G_1 = \{g_1, \ldots, g_s\} \subseteq G$, and let $\sigma_{\mathfrak{a}} : K[\mathbf{u}] \to (K[\mathbf{u}])/\mathfrak{a}$ be a specialization such that $\sigma_{\mathfrak{a}}(\text{lc}_{\mathbf{x}}(g_i)) \neq 0$ for $1 \leq i \leq s$ and for each $\mathbf{u}^{(0)} \in \mathbf{V}(\mathfrak{a}) \setminus \mathbf{V}(\mathfrak{b}) \subset L^m$.*

If for each $p \in G \setminus G_1$, $\text{PDIVI}(\sigma_{\mathfrak{a}}(p), \sigma_{\mathfrak{a}}(G_1)) = 0$, then $\sigma(G_1)$ is a Gröbner basis of $\langle \sigma(G)\rangle$ in $L[\mathbf{x}]$ with respect to $\succ_{\mathbf{x}}$ for all $\mathbf{u}^{(0)} \in \mathbf{V}(\mathfrak{a}) \setminus \mathbf{V}(\mathfrak{b})$.

Proof Let us simplify the notation and write $\sigma = \sigma_{\mathbf{u}^{(0)}}$, where $\mathbf{u}^{(0)} \in \mathbf{V}(\mathfrak{a}) \setminus \mathbf{V}(\mathfrak{b})$. It is easy to check that, under the hypothesis, $\sigma(G) = \sigma(G_1)$, and hence $\sigma(G_1)$ is a basis of $\langle \sigma(G)\rangle$. So it remains to prove that $\sigma(G_1)$ is a Gröbner basis.

Theorem 3.7, and Formula (3.4) inside show that, under the hypothesis of the PDIVI reduction of p with respect to $\sigma_{\mathfrak{a}}(G_1)$, we have

$$\sigma_{\mathfrak{a}}(\mu)\sigma_{\mathfrak{a}}(p) = \sum_{i=1}^{s} \sigma_{\mathfrak{a}}(q_i)\sigma_{\mathfrak{a}}(g_i)$$

being $\mu \in K[\mathbf{u}]$ different from 0 for all $\mathbf{u}^{(0)} \in \mathbf{V}(\mathfrak{a}) \setminus \mathbf{V}(\mathfrak{b})$; this formula corresponds to the ordinary pseudo-division in $L[\mathbf{x}]$ of p by $\sigma(g_1), \ldots, \sigma(g_s)$ for every point in $\mathbf{V}(\mathfrak{a}) \setminus \mathbf{V}(\mathfrak{b})$, verifying that

$$\mu\sigma(p) = \sum_{i=1}^{s} \sigma(q_i)\sigma(g_i) \tag{3.6}$$

$$\mathrm{lpp}(\sigma(p)) \succeq \mathrm{lpp}(\sigma(q_i g_i)) \text{ for all } 1 \leq i \leq s. \tag{3.7}$$

For each $g_j, g_k \in G_1$, since $\sigma(\mathrm{lc}_{\mathbf{x}}(g_j)) \neq 0$ and $\sigma(\mathrm{lc}_{\mathbf{x}}(g_k)) \neq 0$, consider the S-polynomial of g_j and g_k. We have

$$S(\sigma(g_j), \sigma(g_k)) = c\sigma(S(g_j, g_k)), \tag{3.8}$$

where c is a nonzero constant in L.

Assume $G \setminus G_1 = \{g_{s+1}, \ldots, g_\ell\}$. Since G is a Gröbner basis of $\langle G \rangle \subset K[\mathbf{u}][\mathbf{x}]$, we have

$$S(g_j, g_k) = h_1 g_1 + \cdots + h_\ell g_\ell,$$

where $\mathrm{lcm}(\mathrm{lpp}_{\mathbf{x},\mathbf{u}}(g_j), \mathrm{lpp}_{\mathbf{x},\mathbf{u}}(g_k)) \succeq \mathrm{lpp}_{\mathbf{x},\mathbf{u}}(h_i g_i)$ for $1 \leq i \leq \ell$, and $\succ_{\mathbf{x},\mathbf{u}}$ being a product order, also $\mathrm{lcm}(\mathrm{lpp}_{\mathbf{x}}(g_j), \mathrm{lpp}_{\mathbf{x}}(g_k)) \succeq \mathrm{lpp}_{\mathbf{x}}(h_i g_i)$ for $1 \leq i \leq \ell$. Substituting above in Formula (3.8) we obtain

$$S(\sigma(g_j), \sigma(g_k)) = c(\sigma(h_1)\sigma(g_1) + \cdots + \sigma(h_\ell)\sigma(g_\ell)), \tag{3.9}$$

where $\mathrm{lcm}(\mathrm{lpp}_{\mathbf{x}}(\sigma(g_j)), \mathrm{lpp}_{\mathbf{x}}(\sigma(g_k))) = \mathrm{lcm}(\mathrm{lpp}_{\mathbf{x}}(g_j), \mathrm{lpp}_{\mathbf{x}}(g_k)) \succeq \mathrm{lpp}_{\mathbf{x}}(h_i g_i) \succeq \mathrm{lpp}_{\mathbf{x}}(\sigma(h_i), \mathrm{lpp}_{\mathbf{x}}(g_i)$ for $1 \leq i \leq \ell$.

Since by hypothesis $\textsc{Pdivi}(\sigma_{\mathfrak{a}}(g_i), \sigma_{\mathfrak{a}}(G_1)) = 0$, for all $s+1 \leq i \leq \ell$, Formula (3.9) reduces to

$$S(\sigma(g_j), \sigma(g_k)) = p'_1\sigma(g_1) + \cdots + p'_s\sigma(g_s), \tag{3.10}$$

where $p'_1, \ldots, p'_s \in L[\mathbf{x}]$ and $\mathrm{lcm}(\mathrm{lpp}_{\mathbf{x}}(\sigma(g_j), \mathrm{lpp}_{\mathbf{x}}(\sigma(g_k)))) \succeq \mathrm{lpp}_{\mathbf{x}}(p'_i\sigma(g_i))$ for $1 \leq i \leq s$. This shows that all the S-polynomials of $\sigma(G_1)$ reduce to zero with respect to $\sigma(G_1)$ (under PDIVI). Therefore, $\sigma(G_1)$ is a Gröbner basis, and the lemma is proved. □

Theorem 3.15 (Kapur-Sun-Wang) *Let $S = \mathbf{V}(\mathfrak{a}) \setminus \mathbf{V}(\mathfrak{b})$, G be a Gröbner basis of $\langle F \rangle \subset K[\mathbf{x}, \mathbf{u}]$ with respect to $\succ_{\mathbf{x},\mathbf{u}}$, $G_r = G \cap K[\mathbf{u}]$, $G_m =$ NONCOMPARABLE$(G \setminus G_r)$, $h = \prod_{g \in G_m} \mathrm{lc}_{\mathbf{x}}(g) \in K[\mathbf{u}]$.*

If the specialization $\sigma_{\mathbf{u}^{(0)}} = \sigma : K[\mathbf{u}] \to L$ is such that $\sigma(g) = 0$ for $g \in G_r$ and $\sigma(h) \neq 0$, then $\sigma(G_m)$ is a Gröbner basis of $\langle \sigma(F) \rangle$ in $L[\mathbf{x}]$ with respect to $\succ_{\mathbf{x}}$.

Proof We have to prove that the hypothesis of the theorem verifies the hypothesis of Lemma 3.14. Since $\sigma(h) \neq 0$, $\sigma(\mathrm{lc}_{\mathbf{x}}(g)) \neq 0$ for $g \in G_m$, and $\sigma(g) = 0$ for $g \in G_r$, it suffices to show that every $p \in G \setminus (G_r \cup G_m)$ verifies the hypothesis $\mathrm{PDIVI}_{\mathbf{x}}(p, G) = 0$ of the lemma.

But since $p, g_1, \ldots, g_s \in \mathrm{G} \subset \langle F \rangle$, and G is the Gröbner basis of $\langle F \rangle$ in $K[\mathbf{x}, \mathbf{u}]$, the result is that $\mathrm{RED}_{\mathbf{x}\cdot\mathbf{u}}(p, G) = 0$. Thus $\mathrm{PDIVI}_{\mathbf{x}}(p, G) = 0$ by Theorem 3.7. □

3.6 Kapur-Sun-Wang Algorithm for Computing a CGS

Using Theorem 3.15, Kapur-Sun-Wang [43] designed the algorithm KSW on Algorithm 3.4.

The KSW algorithm takes as input two subsets $E, N \subset K[\mathbf{u}]$ representing $\mathbf{V}(E) \setminus \mathbf{V}(N)$ and the parametric polynomial subset $F \subset K[\mathbf{u}][\mathbf{x}]$ generating the ideal $\langle F \rangle$.

$A' \leftarrow$ **KSW**(E, N, F)
Input:
E, N: finite subsets of $K[\mathbf{u}]$; F: finite subset of $K[\mathbf{u}][\mathbf{x}]$
where $\mathbf{V}(E) \setminus \mathbf{V}(N) \subset K[\mathbf{u}]$ and ideal $F \subset K[\mathbf{u}][\mathbf{x}]$ to be discussed
Output:
PGB: a CGS with disjoint segments and respective Gröbner bases

begin
1. **if** $\mathbf{V}(E) \setminus \mathbf{V}(N) = \emptyset$ **then return**$(\{\})$ **end if**
2. $G = \mathrm{REDUCEDGRÖBNERBASIS}(F \cup E, \succ_{\mathbf{x},\mathbf{u}})$
3. **if** $1 \in G$ then **return**$(\{(E, N, \{1\})\})$ **end if**
4. $G_r = G \cap K[\mathbf{u}]$
5. **if** $\mathbf{V}(E) \setminus \mathbf{V}(G_r \cdot N) = \emptyset$
6. **then** $PGB = \{\}$
7. **else** $PGB = \{(E, G_r \cdot N, \{1\})\}$
8. **end if**
9. **if** $\mathbf{V}(G_r) \setminus \mathbf{V}(N) = \emptyset$
10. **then return**(PGB)
11. **else** $G_m = \mathrm{NONCOMPARABLE}(G \setminus G_r)$
12. $\{h_1, h_2, \ldots, h_s\} = \{\mathrm{lc}_{\mathbf{x}}(g) : g \in G_m\}$
13. $h = \mathrm{lcm}(h_1, \ldots, h_s)$
14. **if** $\mathbf{V}(G_r) \setminus \mathbf{V}(N \cdot \{h\}) \neq \emptyset$
15. **then** $PGB = PGB \cup \{(G_r, N \cdot \{h\}, G_m)\}$
16. **end if**
17. **for** $i = 1$ **to** s **do**
18. $PGB = PGB \cup \mathrm{KSW}(G_r \cup \{h_i\}, N \cdot \{h_1 \cdots h_{i-1}\}, G \setminus G_r)$
19. **end do**
20. **return**(PGB)
21. **end if**
end

Algorithm 3.4: KSW

It is applied in its first call for $E = 0$ and $N = 1$, but the algorithm is recursive and is called inside for different values of E, N, F. The KSW procedure is as follows:

(1) First, tests whether $\mathbf{V}(E) \setminus \mathbf{V}(N)$ is empty or not. If is empty, then the algorithm finishes returning the empty set.
(2) Assuming now that $\mathbf{V}(E) \setminus \mathbf{V}(N) \neq \emptyset$, the algorithm computes

$$G = \text{REDUCEDGRÖBNEBASIS}(F \cup E, \succ_{\mathbf{x},\mathbf{u}}).$$

If $G = \{1\}$, then returns $\{(E, N, \{1\})\}$ and it is finished.
(3) If not, then considers another special case. Let $G_r = G \cap K[\mathbf{u}]$ be the part of G that contains only parameters. If $\mathbf{V}(E) \setminus \mathbf{V}(N \cdot G_r)$ is not empty, then gives a first element of PGB consisting of $\{(E, G_r \cdot N, \{1\})\}$, otherwise $PGB = \{\}$.
(4) If the remaining part $\mathbf{V}(G_r) \setminus \mathbf{V}(N)$ of $\mathbf{V}(E) \setminus \mathbf{V}(N)$ is empty, then returns PGB containing the computed special cases. Otherwise, proceeds applying Theorem 3.15 and determines

$$G_m = \text{NONCOMPARABLE}(G \setminus G_r)$$
$$\{h_1, \ldots, h_s\} = \{\text{lc}_{\mathbf{x}}(g) : g \in G_m\}$$
$$h = \text{lcm}\left(\prod_{i=1}^{s} h_i\right)$$

Thus, discarding $h = 0$, no $\text{lc}_{\mathbf{x}}(g)$ for $g \in G_m$ is zero, and thus by Theorem 3.15 the Gröbner Basis in the corresponding segment $\mathbf{V}(G_r) \setminus \mathbf{V}(N \cdot \{h\})$ is G_m, and the segment $\{(G_r, N \cdot \{h\}, G_m)\}$ is added to PGB.
(5) Finally, the procedure considers the remaining disjoint subsets of $\mathbf{V}(G_r) \setminus \mathbf{V}(N)$. First, where $h_1 = 0$; then the segment where $h_2 = 0$ but $h_1 \neq 0$, and so on for the segments where $h_i = 0$ and none of the previous h_i are zero. These form a disjoint set. For all of these segments, the set of polynomials to be discussed is $G \setminus G_r$. So the algorithm adds to PGB the result of the KSW algorithm applied to the ideal $G \setminus G_r$.

Summarizing, KSW algorithm decomposes $V(E) \setminus V(N)$ into the following disjoint segments:

Segment	Ideal
$S_0 = \mathbf{V}(E) \setminus \mathbf{V}(N \cdot G_r))$	$F_0 = \{1\}$
$S_T = \mathbf{V}(G_r) \setminus \mathbf{V}(N \cdot \{h_1 h_2 \cdots h_s\})$	$F_T = G_m$
$S_1 = \mathbf{V}(G_r \cup \{h_1\}) \setminus \mathbf{V}(N)$	$F_1 = G \setminus G_r$
$S_2 = \mathbf{V}(G_r \cup \{h_2\}) \setminus \mathbf{V}(N \cdot \{h_1\})$	$F_2 = G \setminus G_r$
$\cdots$	$\cdots$
$S_s = \mathbf{V}(G_r \cup \{h_s\}) \setminus \mathbf{V}(N \cdot \{h_1 h_2 \cdots h_{s-1}\})$	$F_s = G \setminus G_r.$

Theorem 3.15 is used for computing the output in S_T, the remaining segments being treated either as special cases with obvious basis $\{1\}$ or computed again using recursively KSW.

The algorithm does not necessarily use a canonical representation of the locally closed sets, and allows different treatments to accelerate or improve the output. Kapur-Sun-Wang propose a heuristic method for testing the compatibility of segments, and use Rabinowitz's trick only when the test fails. They are focused on the efficiency of the algorithm.

The result of applying the KSW algorithm is a disjoint CGS. It contains relatively few segments compared with other CGS algorithms and is largely effective.

3.7 Examples

Example 3.16 We give first an example to show the efficiency of the algorithm and the reduced number of segments that it generates. This example will be discussed after the introduction of the GROBCOV algorithm. The problem is the classification of the conics by their singular points. (See Chap. 5, Example 5.15). We consider the ideal

$$\langle x^2 + 2cxy + by^2 + 2dx + 2ey + f,\ 2x + 2cy + 2d,\ 2cx + 2by + 2e\rangle,$$

and compute a CGS using the command `cgsdr` (CGS disjoint and reduced) of the SINGULAR library "grobcov.lib" that implements Algorithm 3.4: KSW. The output is a set of segments, each one consisting of a list of three objects: (E_i, N_i, B_i), representing the segment $\mathbf{V}(E_i) \setminus \mathbf{V}(N_i)$ over which the specialized Gröbner basis is B_i. We proceed to solve the problem using the command `cgsdr` of the SINGULAR library "grobcov.lib".

```
> LIB "grobcov.lib";
> ring R=(0,b,c,d,e,f),(x,y),dp;
> short=0;
> ideal H=x^2+2*c*x*y+b*y^2+2*d*x+2*e*y+f, x+c*y+d, c*x+b*y+e;
> cgsdr(H);
[1]:
   [1]:
      _[1]=0
   [2]:
      _[1]=(b*d^2-b*f+c^2*f-2*c*d*e+e^2)
   [3]:
      _[1]=1
[2]:
   [1]:
      _[1]=(b*d^2-b*f+c^2*f-2*c*d*e+e^2)
   [2]:
      _[1]=(c*d-e)
   [3]:
```

```
      _[1]=(c*d-e)*y+(d^2-f)
      _[2]=x+(c)*y+(d)
[3]:
   [1]:
      _[1]=(b*d^2-b*f+c^2*f-2*c*d*e+e^2)
      _[2]=(c*d-e)
   [2]:
      _[1]=(d^2-f)
      _[2]=(c*f-d*e)
      _[3]=(c*d-e)
   [3]:
      _[1]=1
[4]:
   [1]:
      _[1]=(d^2-f)
      _[2]=(c*f-d*e)
      _[3]=(c*d-e)
   [2]:
      _[1]=(b-c^2)
   [3]:
      _[1]=(b-c^2)*y
      _[2]=x+(c)*y+(d)
[5]:
   [1]:
      _[1]=(d^2-f)
      _[2]=(c*f-d*e)
      _[3]=(c*d-e)
      _[4]=(b-c^2)
   [2]:
      _[1]=1
   [3]:
      _[1]=x+(c)*y+(d)
```

Observe that the algorithm produces only 5 segments. Using the canonical GROBCOV algorithm which we study in Chap. 5 the number of segments will be reduced to 4. So we can consider KSW to be an optimal algorithm for CGS computation.

Example 3.17 We will now consider an example for academic reasons, rather than for the physical interpretation of the output. Moreover, in this problem we left 3 degrees of freedom that can be restricted before or after the use of the algorithm, if we are interested in the discussion. In Part II we will present interesting problems and discuss them, after introducing the Gröbner Cover in Chap. 5. Here we insist in understanding the steps in Algorithm 3.4: KSW. We shall follow the recursive calls to the algorithm. Consider the parametric system corresponding to the system of the Rychlik robot [73], (see Fig. 3.1), where the problem is to determine the angles (θ_1, θ_2) and the lengths (ℓ_1, ℓ_2) to reach the target located at position (a, b). The system is

$$F = \{a - l_2c_2 - l_1c_1, b - l_2s_2 - l_1s_1, c_1^2 + s_1^2 - 1, c_2^2 + s_2^2 - 1\}$$

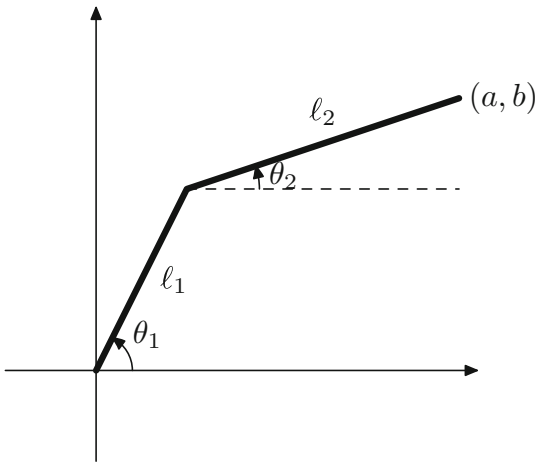


Fig. 3.1 The Rychlik robot with two arms

and we apply the KSW algorithm, using the implemented command `cgsdr` of the SINGULAR library "grobcov.lib":

```
> LIB "grobcov.lib";
> ring R=(0,a,b,l1,l2),(c2,s2,c1,s1),dp;
> short=0;
> cgsdr(S12);
[1]:
   [1]:
      _[1]=0
   [2]:
      _[1]=(a^3*l1*l2+a*b^2*l1*l2)
   [3]:
      _[1]=(2*a*l1)*c1+(2*b*l1)*s1+(-a^2-b^2-l1^2+l2^2)
      _[2]=(l2)*s2+(l1)*s1+(-b)
      _[3]=(l2)*c2+(l1)*c1+(-a)
      _[4]=(4*a^2*l1+4*b^2*l1)*s1^2+(2*a^3+2*a*b^2-2*a*l2^2)*c1
       +(-2*a^2*b-2*b^3-4*b*l1^2+2*b*l2^2)*s1+(-3*a^2*l1+b^2*l1
       +l1^3-l1*l2^2)
[2]:
   [1]:
      _[1]=(a)
   [2]:
      _[1]=(b*l1*l2)
   [3]:
      _[1]=(2*b*l1)*s1+(-b^2-l1^2+l2^2)
      _[2]=(l2)*s2+(l1)*s1+(-b)
      _[3]=(l2)*c2+(l1)*c1
      _[4]=c1^2+s1^2-1
[3]:
   [1]:
      _[1]=(a)
      _[2]=(b)
   [2]:
```

```
      _[1]=(l1^2-l2^2)
      _[2]=(b)
      _[3]=(a)
   [3]:
      _[1]=1
[4]:
   [1]:
      _[1]=(l1^2-l2^2)
      _[2]=(b)
      _[3]=(a)
   [2]:
      _[1]=(l2)
   [3]:
      _[1]=(l2)*s2+(l1)*s1
      _[2]=(l2)*c2+(l1)*c1
      _[3]=c1^2+s1^2-1
[5]:
   [1]:
      _[1]=(l2)
      _[2]=(l1)
      _[3]=(b)
      _[4]=(a)
   [2]:
      _[1]=1
   [3]:
      _[1]=c1^2+s1^2-1
      _[2]=c2^2+s2^2-1
[6]:
   [1]:
      _[1]=(a)
      _[2]=(l1)
   [2]:
      _[1]=(b*l1)
      _[2]=(b^3-b*l2^2)
      _[3]=(a*b)
   [3]:
      _[1]=1
[7]:
   [1]:
      _[1]=(l1)
      _[2]=(b^2-l2^2)
      _[3]=(a)
   [2]:
      _[1]=(b*l2)
   [3]:
      _[1]=(l2)*s2+(-b)
      _[2]=(l2)*c2
      _[3]=c1^2+s1^2-1
[8]:
   [1]:
      _[1]=(a)
      _[2]=(l2)
   [2]:
      _[1]=(b*l1*l2)
```

```
      _[2]=(b^3*l1-b*l1^3)
      _[3]=(a*b*l1)
   [3]:
      _[1]=1
[9]:
   [1]:
      _[1]=(l2)
      _[2]=(b^2-l1^2)
      _[3]=(a)
   [2]:
      _[1]=(b*l1)
   [3]:
      _[1]=(l1)*s1+(-b)
      _[2]=(l1)*c1
      _[3]=c2^2+s2^2-1
[10]:
   [1]:
      _[1]=(l1)
   [2]:
      _[1]=(a*l1)
      _[2]=(a^3+a*b^2-a*l2^2)
   [3]:
      _[1]=1
[11]:
   [1]:
      _[1]=(l1)
      _[2]=(a^2+b^2-l2^2)
   [2]:
      _[1]=(a*l2)
   [3]:
      _[1]=(l2)*s2+(-b)
      _[2]=(l2)*c2+(-a)
      _[3]=c1^2+s1^2-1
[12]:
   [1]:
      _[1]=(l1)
      _[2]=(a^2+b^2-l2^2)
      _[3]=(l2)
   [2]:
      _[1]=(a*l2)
      _[2]=(a*l1)
      _[3]=(a*b)
      _[4]=(a)
   [3]:
      _[1]=1
[13]:
   [1]:
      _[1]=(l2)
   [2]:
      _[1]=(a*l1*l2)
      _[2]=(a^3*l1+a*b^2*l1-a*l1^3)
   [3]:
      _[1]=1
[14]:
```

```
   [1]:
      _[1]=(l2)
      _[2]=(a^2+b^2-l1^2)
   [2]:
      _[1]=(a*l1)
   [3]:
      _[1]=(l1)*s1+(-b)
      _[2]=(l1)*c1+(-a)
      _[3]=c2^2+s2^2-1
[15]:
   [1]:
      _[1]=(a^2+b^2)
   [2]:
      _[1]=(a*b*l1^3*l2-a*b*l1*l2^3)
   [3]:
      _[1]=(4*b*l1^3-4*b*l1*l2^2)*s1+(-4*b^2*l1^2-l1^4
       +2*l1^2*l2^2-l2^4)
      _[2]=(b*l1^2-b*l2^2)*c1+(a*l1^2-a*l2^2)*s1+(-2*a*b*l1)
      _[3]=(l2)*s2+(l1)*s1+(-b)
      _[4]=(l2)*c2+(l1)*c1+(-a)
[16]:
   [1]:
      _[1]=(a^2+b^2)
      _[2]=(l1-l2)
   [2]:
      _[1]=(a*b*l1^2*l2-a*b*l1*l2^2)
      _[2]=(a*b*l1*l2)
      _[3]=(a*b*l1*l2)
      _[4]=(a*b*l1*l2)
   [3]:
      _[1]=1
[17]:
   [1]:
      _[1]=(a^2+b^2)
      _[2]=(l1+l2)
   [2]:
      _[1]=(a*b*l1^3*l2-a*b*l1*l2^3)
      _[2]=(a*b*l1^2*l2-a*b*l1*l2^2)
      _[3]=(a*b*l1^2*l2-a*b*l1*l2^2)
      _[4]=(a*b*l1^2*l2-a*b*l1*l2^2)
   [3]:
      _[1]=1
```

The output has 17 segments. The algorithm GROBCOV that we introduce in Chap. 5 reduces to 11 the number of segments in this example.

Figure 3.2 shows the scheme of the tree generated by the recursive algorithm. Black circles represent the recursive calls to the algorithm and red circles the computed cases of the CGS. We set a number corresponding to the order in which the recursive calls and cases are produced. At each call, the algorithm can finish at line 1 and at line 10 without any further calling. But if not, a Case can be produced at lines 7 and 15. Then at line 18, the algorithm calls himself recursively with increasing values of i. This produces a tree, which can be seen in Fig. 3.2. The

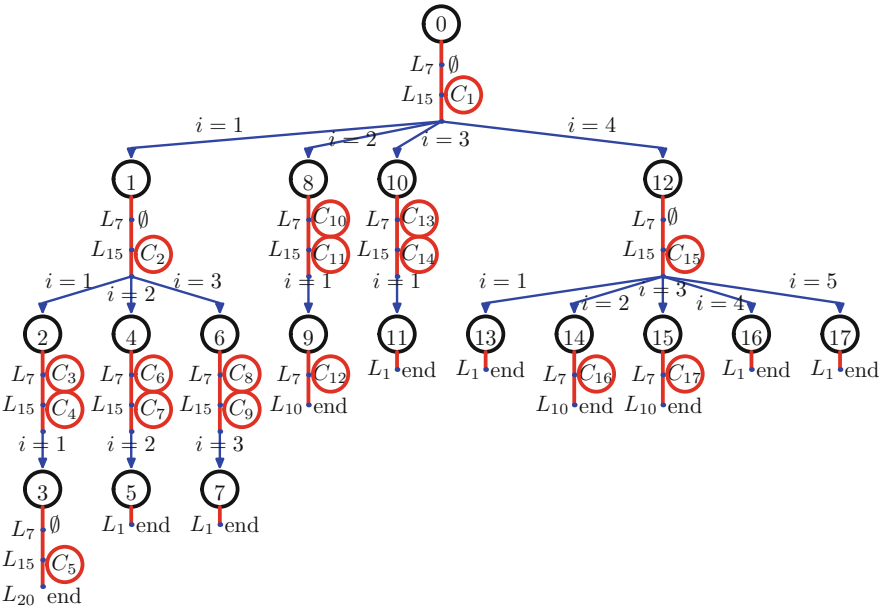


Fig. 3.2 Tree generated by KSW algorithm in Example 3.17

tree is followed in pre-order, i.e. always to a deeper level, and when finished, to the next value of i, as can be seen in Fig. 3.2.

The first Call, denoted 0, takes as input values $E = \langle 0 \rangle$, $N = \langle 1 \rangle$ and the ideal F. At this call, on line 7 it does not obtain any case but at line 15 it obtains Case 1 (encircled in red). After this, the variable PGB contains only Case 1.

Then, on line 18 it makes the first recursive call, for $i = 1$, denoted Call 1 (in black). At this call, it does not obtain any case at line 7, but it obtains Case 2 at line 15. At this stage, the variable PGB contains Case 2. Then at line 18, a new recursive Call 2 to KSW for $i = 1$ is produced. Now Cases 3 and 4 are respectively obtained at lines 7 and 15, and a new recursive Call 3 is produced at line 18. At line 7, no new case is produced but at line 15, Case 5 is obtained. Then the last call is finished at line 20 and C_5 is added to the PGB value of the previous Call 2, and PGB contains cases $\{3, 4, 5\}$. Then Call 2 is also finished at line 20 and PGB obtains cases $\{2, 3, 4, 5\}$, and the control returns to Call 1, where PGB contains cases $\{1, 2, 3, 4, 5\}$.

Then it follows by $i = 2$ with a new Call 4, and so on. Figure 3.2 shows the exact recursions of the algorithms for this example.

At the end, all the calls are finished and Call 0 returns PGB containing the cases $\{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17\}$.

Chapter 4
I-Regular Functions on a Locally Closed Set



Regular functions and specially I-regular functions defined on a locally closed set are necessary instruments for establishing Wibmer's Theorem [85], which is the basis of the Gröbner Cover. They are a generalization of polynomial functions, and allow grouping segments of a CGS having bases with the same set of lpp's. In this chapter we describe I-regular functions, and in Chap. 5 we use them to build the Gröbner Cover and the algorithms for obtaining them.

4.1 Regular and I-Regular Functions on a Locally Closed Set

Definition 4.1 (Regular function) A function $f : S = \mathbf{V}(\mathfrak{a}) \setminus \mathbf{V}(\mathfrak{b}) \to L$ is called regular if for every $a \in S$, there exists an open neighborhood $U \subset S$ of a (i.e., $U = S \setminus \mathbf{V}(M) = \mathbf{V}(\mathfrak{a}) \setminus (\mathbf{V}(\mathfrak{b}) \cup \mathbf{V}(M))$, with $a \in U$) such that

$$f(b) = \frac{p(b)}{q(b)} \text{ for all } b \in U,$$

where $p, q \in K[\mathbf{u}]$ and $q(b) \neq 0$ for all $b \in U$. We denote the ring of regular functions on S by $\mathcal{O}(S)$.

Definition 4.2 (I-regular function) Let S be a locally closed subset of L^m. We call a function $f : S \to L[\mathbf{x}]$ *regular with respect to* I (or simply I*-regular* for short) if the following holds:

for each $a \in S$, there exists an open subset U of S with $a \in U$ and

$$f(b) = \frac{p(b, \mathbf{x})}{q(b)} \in L[x] \text{ for all } b \in U, \tag{4.1}$$

where $p \in I$ and $q \in K[\mathbf{u}]$ such that $q(b) \neq 0$ for all $b \in U$.

© Springer Nature Switzerland AG 2018

A. Montes, *The Gröbner Cover*, Algorithms and Computation in Mathematics 27, https://doi.org/10.1007/978-3-030-03904-2_4

For an I-regular function on S to be defined, there must exist a finite set of pairs $\{(p_1, q_1), \ldots, (p_t, q_t)\}$ with $p_i \in I$ such that

(1) For each point $a \in S$, at least one of the $q_i(a) \neq 0$;
(2) For each point $b \in S$ if $q_i(b) \neq 0$ and $q_j(b) \neq 0$, then

$$p_i(b, \mathbf{x})q_j(b) - p_j(b, \mathbf{x})q_i(b) = 0, \quad \text{so that}$$
$$f(b, \mathbf{x}) = \frac{p_i(b, \mathbf{x})}{q_i(b)} = \frac{p_j(b, \mathbf{x})}{q_j(b)}.$$

(3) On the points $b \in S$ where $q_i(b) = 0$, and f cannot be computed using $(p_i; q_i)$ we must have $p_i(b, \mathbf{x}) = 0$.
(4) The $\mathrm{lpp}_{\mathbf{x}}(f)$ must be constant on S.

To represent f, we can substitute the defining functions by the reduced polynomials expressions modulo $\mathfrak{a}$, since they are identical on S, and so use non-faithful functions. In the algorithms we will always use reduced polynomials, since they preserve the lpp. Following Chap. 3, Sect. 3.2.1, in considering the specialization

$$\begin{array}{rccc} \sigma_{\mathfrak{a}} \colon & K[\mathbf{u}][\mathbf{x}] & \to & (K[\mathbf{u}]/\mathfrak{a})[\mathbf{x}] \\ & p & \mapsto & p^r_{\mathfrak{a}} \end{array}$$

of the representatives of the I-regular function $f \colon \{(p_1, q_1,), \ldots, (p_k, q_k)\}$, we can determine the image $f^r_{\mathfrak{a}}$ of f by their reduced polynomials:

$$f^r_{\mathfrak{a}} \colon \{(p^r_{\mathfrak{a},1}; q^r_{\mathfrak{a},1}), \ldots, (p^r_{\mathfrak{a},k}; q^r_{\mathfrak{a},k})\},$$
$$f(a, \mathbf{x}) = f^r_{\mathfrak{a}}(a, \mathbf{x}) \ \text{ for all } \ a \in S,$$
$$\mathrm{lpp}_{\mathbf{x}}(f_{\mathfrak{a}}) = \mathrm{lpp}_{\mathbf{x}}(f^r_{\mathfrak{a}}) \text{ being constant on } S.$$

The set of all I-regular functions on S is denoted by $\mathcal{I}(S)$. Obviously we can interpret $\mathcal{I}(S)$ as an ideal in the polynomial ring $\mathcal{O}(S)[\mathbf{x}]$. In particular, the leading power product and leading coefficient is defined for an element of $\mathcal{I}(S)$. Intuitively we may think of $\mathcal{I}(S)$ as being the restriction of $I = \mathcal{I}(L^m)$ to S.

Example 4.3 Consider the following parametric ring R, the ideal I, the locally closed set S, and the order $\mathrm{lex}(x, y)$:

$$R = \mathbb{Q}(a_0, b_0, c_0, a_1, b_1, c_1)[x],$$
$$I = \langle a_0x^2 + b_0x + c_0, a_1x^2 + b_1x + c_1 \rangle,$$
$$S = \mathbf{V}(a_0, b_0c_1 - c_0b_1, a_1) \setminus \mathbf{V}(a_1, b_1, b_0, a_0).$$

Define $F \colon S \to L$ by the representation $F : \{(b_1x + c_1; b_1), (b_0x + c_1; b_0)\}$.

$$F(a_0, b_0, c_0, a_1, b_1, c_1; x) = \frac{b_1 x + c_1}{b_1} = x + \frac{c_1}{b_1} \text{ for } b_1 \neq 0$$

$$F(a_0, b_0, c_0, a_1, b_1, c_1; x) = \frac{b_0 x + c_0}{b_0} = x + \frac{c_0}{b_0} \text{ for } b_0 \neq 0.$$

Let us prove that the function F defined above represents an I-regular function on S. We have to prove that

(1) p_1 and p_2 are functions of I specialized on S.
(2) Either p_1/q_1 or p_2/q_2 is defined for each point $a \in S$.
(3) $p_1 q_2 - p_2 q_1 = 0$ for every point $a \in S$ where both functions are defined.
(4) $\mathrm{lpp}_{\mathbf{x}}(F) = x$.

Proof

(1) On S we have $a_0 = a_1 = 0$ and thus $p_1 = b_0 x + c_0$ and $p_2 = b_1 x + c_1$ are the reduced form of the specialization on S of the functions $a_0 x^2 + b_0 x + c_0$ and $a_1 x^2 + b_1 x + c_1$ defining I.
(2) If for a point $a \in S$ both expressions are undefined, we have $b_0 = b_1 = 0$. Since $a_0 = a_1 = 0$ on S, and the point $a_0 = a_1 = b_0 = b_1 = 0$ is excluded from S, then either $b_1 \neq 0$ or $b_0 \neq 0$, and so F is defined on every point $a \in S$.
(3) We have $p_1 q_2 - p_2 q_1 = b_0 c_1 - b_1 c_0 = 0$ on S. Thus both representations give the same result when both are defined and where $q_i = 0$ also $p_i = 0$.
(4) For $b_0 \neq 0$ and for $b_1 \neq 0$, there is $\mathrm{lpp}_{\mathbf{x}}(F) = x$. □

In this example, we have $\mathcal{I}(S) = \langle F \rangle$. We can also affirm that F is the reduced Gröbner basis of I on S with $\mathrm{lpp}_{\mathbf{x}}(F) = x$.

4.2 Representations of Regular and I-Regular Functions

Definition 4.4 (Generic representation of regular functions) Let S be a locally closed set, $f : S \to L$ a regular function and $p, q \in K[\mathbf{u}]$. We say that the pair $(p; q)$ is a *generic representation of* f if

(1) $S \setminus \mathbf{V}(q)$ is dense in S and
(2) $f(a) = \dfrac{p(a)}{q(a)}$ for all $a \in S \setminus \mathbf{V}(q)$.

If $(p; q)$ is a generic representation of $f : S \to L[\mathbf{x}]$ and $a \in S$ with $q(a) = 0$, then also $p(a) = 0$. To see this we observe that by the definition of regular functions, we can find polynomials $p', q' \in L[\mathbf{x}]$ such that $q'(a) \neq 0$ and $p(a)q'(a) - p'(a)q(a) = 0$ on U, where U is dense on S. So the same must occur in all S. So if $q(a) = 0$ and $q'(a) \neq 0$, we must have $p(a) = 0$ on S.

A generic representation $(p; q)$ of a regular function $f : S \to L$ determines f univocaly. The reason is because a regular function f defined on a dense open

subset U of S can be extended to a larger open subset of S, and this extension is unique. (Short proof: Let g_1, g_2 be extensions of g to an open subset V of S. Since U is dense in S, the closure of U in V equals V. But U is contained in the closed subset $V' = \{a \in V : g_1(a) = g_2(a)\}$ of V so that $V = V'$, i.e., g_1 and g_2 agree on all of V.)

The advantage of the generic representation is that it is very convenient for computations, the disadvantage is that one cannot immediately determine the value of f at $a \in S$ if the denominator q vanishes at a.

For an I-regular function, we can give a similar definition.

Definition 4.5 (Generic representation of I-regular functions) Let $F\colon S \to L[\mathbf{x}]$ be a monic I-regular function on the locally closed set S. We say that $p \in K[\mathbf{u}][\mathbf{x}]$ is a *generic representation* of F if

(1) $S \setminus \mathbf{V}(q)$ is dense in S, where $q = \mathrm{lc}(p) \in K[\mathbf{u}]$,
(2) $F(a, \mathbf{x}) = \dfrac{p(a, \mathbf{x})}{q(a)}$ for all $a \in S \setminus \mathbf{V}(q)$,
(3) $p(a, \mathbf{x}) = 0$ for all $a \in \mathbf{V}(q) \cap S$.

Definition 4.6 (Full representation of regular functions) Let $f : S \to L$ be a regular function on the locally closed set S. Let $p_1, \ldots, p_r, q_1, \ldots, q_r \in K[\mathbf{u}]$. We say that $((p_1, q_1) \ldots, (p_r, q_r))$ *is a full representation of* f if the following conditions are satisfied.

(1) $f(a) = \dfrac{p_i(a)}{q_i(a)}$ for every $a \in S$ for which $q_i(a) \neq 0$,
(2) for every $a \in S$ there exists $j \in [r]$ such that $q_j(a) \neq 0$ and
(3) $p_i(a)q_j(a) = q_i(a)p_j(a)$ for all $a \in S$ and $i, j \in [r]$.

It follows from (2) and (3) that $p_i(a) = 0$ if $q_i(a) = 0$ for some $a \in S$. Note that it is not required that $S \setminus \mathbf{V}(q_i)$ be dense in S. We will see later in this section that every regular function admits a full representation as defined above. Conversely, it is obvious that if $p_1, \ldots, p_r, q_1, \ldots, q_r \in K[\mathbf{u}]$ satisfy conditions (2) and (3), then there exists a unique regular function $f : S \to L$ such that $((p_1, q_1), \ldots, (p_r, q_r))$ is a full representation of f.

Definition 4.7 (Full representation of I-regular functions) Let $S \subset L^m$ be a locally closed set and $F : S \to L[\mathbf{x}]$ an I-regular function. We say that a polynomial $\sum_\alpha c_\alpha \mathbf{x}^\alpha$ *is a full representation of* F if for every α the coefficient c_α, is a full representation of $\mathrm{coef}(f, \alpha) \in \mathcal{O}(S)$.

In practice we will only have to deal with monic I-regular functions with $c_{\alpha_0} = 1$. In this case we can represent the function by $(p_1, \ldots, p_s)$ being $q_i = \mathrm{lc}_{\mathbf{x}}(p_i)$ for $i \in [s]$. This simplifies the representation.

4.2.1 Combining Segments of an I-Regular Function

We now need algorithms to obtain generic and full representations of monic I-regular functions. Let $F : S \to L[\mathbf{x}]$ be a regular function, and let the prime decomposition of $\mathbf{I}(S)$ be $\{\mathfrak{p}_1, \cdots \mathfrak{p}_s\}$. Assume that we know generic representations of F on each subset $\mathbf{V}(\mathfrak{p}_i)$.

The purpose of Algorithm 4.2: COMBINE is to compute a generic representation, of F on S and the task of Algorithm 4.3: EXTEND is to compute a full representation from a generic representation.

Algorithm 4.2: COMBINE is nothing more than a Chinese remainder method [6], and uses a previous Algorithm 4.1: DELTA.

Lemma 4.8 (Delta) *Let* $[\mathfrak{p}_1, \ldots, \mathfrak{p}_s]$ *be the minimal prime decomposition of the radical ideal* $\mathfrak{a}$. *Then Algorithm 4.1:* DELTA *computes polynomials* $[\delta_1, \ldots, \delta_s; \delta] \subset K[\mathbf{u}]$ *such that*

(i) $\delta_i(a) \neq 0$ *for all* a *in an open subset* $a \in U_i \subset \mathbf{V}(\mathfrak{p}_i)$, *i.e.,* $\overline{U_i} = \mathbf{V}(\mathfrak{p}_i)$,
(ii) $\delta_i(a) = 0$ *for all* $a \in (\mathbf{V}(\mathfrak{p}_i) \setminus U_i) \bigcup \left(\bigcup_{j \neq i} \mathbf{V}(\mathfrak{p}_j)\right)$,

$[\delta_1, \ldots, \delta_s; \delta] \leftarrow$ **Delta**$(\mathfrak{p}_1, \ldots, \mathfrak{p}_s)$
Input:
$\mathfrak{p}_1, \ldots, \mathfrak{p}_s \subset K[\mathbf{u}]$ is the prime decomposition of an ideal $\mathfrak{a}$
Output:
$[\delta_1, \ldots, \delta_s; \delta] \subset K[\mathbf{u}]$ such that for all i
$\delta_i(a) = 0$ on $\bigcup_{j \neq i} \mathbf{V}(\mathfrak{p}_j)$,
$\delta(a) = \delta_i(a) \neq 0$ on $U_i \subset \mathbf{V}(\mathfrak{p}_i)$ with $\overline{U_i} = \mathbf{V}(\mathfrak{p}_i)$

begin
$L_1[1] = \mathfrak{p}_1$; $L_2[1] = \mathfrak{p}_s$
for $i = 2$ **to** $s-1$ **do** $L_1[i] = L_1[i-1] \cap \mathfrak{p}_i$ **end for**
for $i = 2$ **to** $s-1$ **do** $L_2[i] = L_1[i-1] \cap \mathfrak{p}_{s-i+1}$ **end for**
$L_3[1] = L_2[s-1]$; $L_3[s] = L_1[s-1]$
for $i = 2$ **to** $s-1$ **do** $L_3[i] = L_1[i-1] \cap L_2[s-i]$ **end for**
for $i \in [s]$ **do**
$\mathfrak{h}_i = L_3[i]$; # resulting in $\left(\mathfrak{h}_i = \bigcap_{j \in [s] \setminus i} \mathfrak{p}_j\right)$
$j = 1$; $t =$true
while t **and** $j <= s$ **do**
if REDUCE$(\mathfrak{h}_i[j], \mathfrak{p}_i) \neq 0$ **then** t=false; $\delta_i = \mathfrak{h}_i[j]$ **end if**
$j = j + 1$
end while
end for
$\delta = \sum_{i=1}^{s} \delta_i$
rerturn$([\delta_1, \ldots, \delta_s; \delta])$
end

Algorithm 4.1: DELTA

$B \leftarrow$ **Combine**$((\mathfrak{p}_1, f_1), \ldots, (\mathfrak{p}_s, f_s))$
`Input:`
$(\mathfrak{p}_1, f_1), \ldots, (\mathfrak{p}_s, f_s)$ where $[\mathfrak{p}_i, \ldots, \mathfrak{p}_s]$ is the prime decomposition of the radical ideal $\mathfrak{a} = \mathbf{I}(S)$ and f_i is a generic representation of F on $\mathbf{V}(\mathfrak{p}_i)$
`Output:`
$f \in K[\mathbf{u}][\mathbf{x}]$ a generic representation of F

begin
$[\delta_1, \ldots, \delta_s; \delta] = \text{Delta}(\mathfrak{p}_1, \ldots, \mathfrak{p}_s)$
$f = \sum_{i=1}^{s} \delta_i f_i$ # We can normalize dividing by δ.
return(f)
end

Algorithm 4.2: COMBINE

(iii) $\delta(a) \neq 0$ *for all* a *in an open and dense subset* $a \in U = \bigcup_j U_j \subset \bigcup_j \mathbf{V}(\mathfrak{p}_j)$, *and* $\delta(a) = \delta_i(a)$ *for all* $a \in U_i$,
(iv) $\delta(a) = 0$ *for all* $a \in \left(\bigcup_j \mathbf{V}(\mathfrak{p}_j)\right) \setminus U$.

Proof The number of intersections carried out on the first 5 lines of the code is $(s-2)+(s-2)+(s-2) = 3s-6$, and the result is the list $L_3 = [\mathfrak{h}_1, \ldots, \mathfrak{h}_s]$ of ideals, where $\mathfrak{h}_i = \bigcap_{j \neq i} \mathfrak{p}_j$.

Thus if $h \in \mathfrak{h}_i$, then $h(a) = 0$ for all $a \in \bigcup_{j \neq i} \mathbf{V}(\mathfrak{p}_j)$. Then the algorithm chooses a δ_i of $\text{GB}(\mathfrak{h}_i)$ that does not lie in $\mathfrak{p}_i$, so that we have $\delta_i(a) \neq 0$ on an open subset $U_i \subset \mathbf{V}(\mathfrak{p}_i)$, and $\delta_i(a) = 0$ for all $a \in (\mathbf{V}(\mathfrak{p}_i) \setminus U_i) \bigcup \left(\bigcup_{j \neq i} \mathbf{V}(\mathfrak{p}_j)\right)$.

Finally, δ is computed as the sum of all δ_i that will have the desired properties, and can be used to normalize. □

Now it is easy to obtain a generic representation of an I-regular function $F : S \to L[\mathbf{x}]$.

Lemma 4.9 (COMBINE) *Let* $((\mathfrak{p}_1, f_1), \ldots, (\mathfrak{p}_s, f_s))$ *be the set of pairs* $(\mathfrak{p}_i, f_i)$ *where the* $\mathfrak{p}_i$ *are the prime ideals, top of the components of the locally closed set* S, *and the* f_i *are generic representations of the restriction of* F *on* $\mathbf{V}(\mathfrak{p}_i)$, *i.e.* $f_i : \mathbf{V}(\mathfrak{p}_i) \to L[\mathbf{x}]$, *then Algorithm 4.2:* COMBINE *computes a generic representation of* F *on* S.

Proof The proof is obvious. □

Example 4.10 Let $\mathfrak{a} = \mathfrak{p}_1 \cap \mathfrak{p}_2 \subset K[a_1, a_2]$ with $\mathfrak{p}_1 = \langle a_1 \rangle$, $\mathfrak{p}_2 = \langle a_2 \rangle$ and

$$S = \mathbf{V}(a_1 a_2) \setminus (\mathbf{V}(a_1, a_2 - 1) \cup \mathbf{V}(a_1 - 4, a_2) \cup \mathbf{V}(a_1, a_2)).$$

Define a monic I-regular function $F : S \to L[x]$ by $p_1 = (a_2 - 1)x + (a_2^2 - 4)$ on $\mathbf{V}(a_1) \setminus (\mathbf{V}(a_1, a_2 - 1) \cup \mathbf{V}(a_1, a_2))$ and $p_2 = (a_1 - 4)x + (a_1^3 - 16)$ on $\mathbf{V}(a_2) \setminus (\mathbf{V}(a_1 - 4, a_2) \cup \mathbf{V}(a_1, a_2))$. We compute first DELTA and obtain $\delta_1 = a_2$, $\delta_2 = a_1$, $\delta = a_1 + a_2$. Then we apply COMBINE and obtain:

$$
\begin{aligned}
p &= a_2 p_1 + a_1 p_2 \\
&= a_2((a_1 - 1)x + (a_2^2 - 4)) + a_1((a_1 - 4)x + (a_1^3 - 16)) = \\
&= (a_1^2 - 4a_1 + a_2^2 - a_2)x + a_1^4 - 16a_1 + a_2^3 - 4a_2 \\
F &= x + \frac{a_1^4 - 16a_1 + a_2^3 - 4a_2}{a_1^2 - 4a_1 + a_2^2 - a_2} \text{ for } a \neq (0,0) \text{ and } a \neq (0,1) \text{ and } a \neq (4,0).
\end{aligned}
$$

The generic representation is already a full representation on S, because in S, either $a_1 = 0$ or $a_2 = 0$ and the unique points where the denominator $a_1^2 - 4a_1 + a_2^2 - a_2$ become zero inside are the points $(0, 0)$, $(0, 1)$ and $(4, 0)$ which are excluded from the segment S.

4.2.2 Computing a Full Representation from a Generic One

If the generic representation p of a monic I-regular function $F : S \to L[\mathbf{x}]$ obtained by the COMBINE algorithm does not satisfy $q(a) = \mathrm{lc}_{\mathbf{x}}(p)(a) \neq 0$ for all $a \in S$, then we can use the following Algorithm EXTEND to compute a full representation $((p_1, q_1), \ldots, (p_s, q_s))$ of F. This is not the only reason for using EXTEND. We shall see in Example 4.12 that extending the representation can simplify the result and also define the function in some greater region than S.

We note that the $K[\mathbf{u}]$-module defined in Algorithm 4.3: EXTEND can be computed using standard Gröbner bases techniques.

Proposition 4.11 (Extend algorithm) *Let $S \subset L^m$ be locally closed and let $f : S \to L$ be a regular function. Let $p, q \in K[\mathbf{u}]$ be such that $S \setminus \mathbf{V}(q)$ is dense in S and $f(a) = \dfrac{p(a)}{q(a)}$ for all $a \in S \setminus \mathbf{V}(q)$. Then Algorithm 4.3: EXTEND computes a full representation of f.*

Proof To prove that $((p_1, q_1) \ldots, (p_s, q_s)) = \text{EXTEND}(S, p, q)$ of Proposition 4.11 is a full representation of f, we have to show that (1), (2) and (3) of Definition 4.6 are verified.

(1) By construction $p_i q - q_i p \in \mathfrak{a}$ so that $\dfrac{p_i(a)}{q_i(a)} = \dfrac{p(a)}{q(a)}$ for all $a \in S \setminus \mathbf{V}(q q_i)$. Since $S \setminus \mathbf{V}(q)$ is dense in S, we see that $(S \setminus \mathbf{V}(q)) \cap (S \setminus \mathbf{V}(q_i)) = S \setminus \mathbf{V}(q q_i)$ is dense in $S \setminus \mathbf{V}(q_i)$. Therefore, if the regular function defined by $\dfrac{p}{q}$ on $S \setminus \mathbf{V}(q q_i)$ can

$((p_1, q_1) \dots, (p_s, q_s)) \leftarrow$ **Extend**(S, p, q)
Input:
$S \subset L^m$ locally closed
$p, q \in K[\mathbf{u}]$ such that
$S \setminus \mathbf{V}(q)$ is dense in S,
there exists a regular function $f : S \to L$ such that
$f(a) = \dfrac{p(a)}{q(a)}$ for every $a \in S \setminus \mathbf{V}(q)$.
Output: A full representation of f

begin
Let $\mathfrak{a} \subset K[\mathbf{u}]$ be the radical ideal such that $\mathbf{V}(\mathfrak{a}) = \overline{S}$, and $\begin{pmatrix} p_1 \\ q_1 \end{pmatrix}, \dots, \begin{pmatrix} p_s \\ q_s \end{pmatrix}$ a generating set of the $K[\mathbf{u}]$-module

$$\left\{ \begin{pmatrix} g \\ h \end{pmatrix} \in K[\mathbf{u}]^2 : \ gq + h(-p) \in \mathfrak{a} \right\}$$

which describes the syzygies of $(q, -p)$ modulo $\mathfrak{a}$.
return$((p_1, q_1) \dots, (p_s, q_s))$
end

Algorithm 4.3: EXTEND

be extended to $S \setminus \mathbf{V}(q_i)$, then this extension is unique. But both f and $\dfrac{p_i}{q_i}$ define such extensions. Consequently, $f(a) = \dfrac{p_i(a)}{q_i(a)}$ for all $a \in S \setminus \mathbf{V}(q_i)$ and (i) is proved.

(2) Fix $a \in S$. The open subsets of S of the form $S \setminus \mathbf{V}(q')$ with $q' \in K[\mathbf{u}]$ are a basis of the topology of S. Thus there exists polynomials $q', \widetilde{p}, \widetilde{q} \in K[\mathbf{u}]$ such that $a \in S \setminus \mathbf{V}(q')$ and $f(b) = \dfrac{\widetilde{p}(b)}{\widetilde{q}(b)}$ for all $b \in S \setminus \mathbf{V}(q')$. In particular, $\widetilde{q}(b) \neq 0$ for all $b \in S \setminus \mathbf{V}(q')$. This means that $\mathbf{V}(\widetilde{q}) \cap S \subset \mathbf{V}(q') \cap S$. If $\mathfrak{a} \subset K[\mathbf{u}]$ is the radical ideal with $\mathbf{V}(\mathfrak{a}) = \overline{S}$, then $\mathbf{V}(\mathfrak{a} + \langle \widetilde{q} \rangle) \subset \mathbf{V}(\mathfrak{a} + \langle q' \rangle)$. Therefore $q' \in \sqrt{\mathfrak{a} + \langle \widetilde{q} \rangle}$. So we can find $n \geq 1$, $h \in K[\mathbf{u}]$ and $g \in \mathfrak{a}$ such that $q'^n = h\widetilde{q} + g$. Then we have for $b \in S \setminus \mathbf{V}(q')$

$$f(b) = \frac{\widetilde{p}(b)}{\widetilde{q}(b)} = \frac{\widetilde{p}(b)h(b)}{\widetilde{q}(b)h(b)} = \frac{p'(b)}{q'(b)^n}$$

with $p' = \widetilde{p}h$.

We claim that $(p'q - q'^n p)q'$ lies in $\mathfrak{a}$. Because $S \setminus \mathbf{V}(q)$ is dense in $\mathbf{V}(\mathfrak{a})$, it suffices to see that $(p'(b)q(b) - q'^n(b)\,p(b))q'(b) = 0$ for all $b \in S \setminus \mathbf{V}(q)$. But if $b \in S \cap \mathbf{V}(q')$ this is trivial and if $b \in S \setminus \mathbf{V}(q'q)$, then $p'(b)q(b) - q'(b)^n p(b) = 0$ because

$$\frac{p'(b)}{q'(b)^n} = f(b) = \frac{p(b)}{q(b)}.$$

Consequently there exist $h_1, \dots, h_s \in K[\mathbf{u}]$ such that

$$\begin{pmatrix} p'q' \\ q'^{m+1} \end{pmatrix} = h_1 \begin{pmatrix} p_1 \\ q_1 \end{pmatrix} + \cdots + h_s \begin{pmatrix} p_s \\ q_s \end{pmatrix}.$$

Now if $q_i(a)$ were equal to zero for every $i \in [r]$ then also $q'(a)$ would be equal to zero, which is not the case since $a \in S \setminus \mathbf{V}(q')$. Therefore (ii) is proved.

(3) Finally, let $i, j \in [s]$. Multiplying the equation $p_i q - q_i p \in \mathfrak{a}$ by q_j and replacing $q_j p$ with $p_j q$, we obtain $q_j p_i q - q_i p_j q \in \mathfrak{a}$, so that

$$q(p_i q_j - q_i p_j) \in \mathfrak{a} = \mathfrak{p}_1 \cap \cdots \cap \mathfrak{p}_r.$$

Since $q \notin \mathfrak{p}_1, \dots, \mathfrak{p}_r$, we see that $p_i q_j - q_i p_j \in \mathfrak{a}$ and (iii) is proved. □

Example 4.12 To understand the power of the EXTEND algorithm, we apply it to the result of Example 4.10, even if it was not necessary there, because it was already a full representation. We had $S = \mathbf{V}(a_1, a_2) \setminus (\mathbf{V}(a_1, a_2) \cup \mathbf{V}(a_1 - 4, a_2) \cup \mathbf{V}(a_1, a_2 - 1))$, and the generic (and also full) representation of $f : S \to L$

$$F = x + \frac{a_1^4 - 16a_1 + a_2^3 - 4a_2}{a_1^2 - 4a_1 + a_2^2 - a_2}.$$

Applying EXTEND to F, we obtain a pair of syzygies:

$$\left((a_2^3 - 4a_2, a_2^2 - a_2), (a_1^3 + 4a_2^2 - 16, a_1 + 4a_2 - 4)\right).$$

Observe that whether the first pair is zero on $\mathbf{V}(\mathfrak{p}_1)$, the second one is non-null in all points of S and it extends f to $S' = S \cup \{(0,0)\}$, as it assigns $f(0,0)$ to the value $x + 4$, as expected by the initial data.[1] So we only need the second syzygy

$$\frac{p}{q} = \frac{a_1^3 + 4a_2^2 - 16}{a_1 + 4a_2 - 4}.$$

This provides the full representation of the I-regular function

$$P = (a_1 + 4a_2 - 4)x + (a_1^3 + 4a_2^2 - 16),$$

showing that f can be defined as an I-regular function in the larger set $S' = S \cup \{(0,0)\}$. Moreover, the new expression is simpler.

[1] Consider $g = \dfrac{a_1^4 - 16a_1 + a_2^3 - 4a_2}{a_1^2 - 4a_1 + a_2^2 - a_2}$. We have $\lim_{(a_1,a_2) \to (0,0)} g(a_1, a_2) = 4$.

Let us show the answer of the "grobcov.lib" library for this problem

```
> ring R=(0,a1,a2),(x),lp;
> poly F=(a1^2-4*a1+a2^2-a2)*x+(a1^4-16*a1+a2^3-4*a2);
> short=0;
> ideal p=a1*a2;
> ideal q=a2^2-a2,a1*a2,a1^2-4*a1;
> extendpoly(F,p,q);
(a1+4*a2-4)*x+(a1^3+4*a2^2-16)
```

Observe the exact matching with the result described above. The command used is `extendpoly`, which is the name of the implementation in the library of the Algorithm 4.3: EXTEND. The library also contains the command `extendGC` for extending the whole output of the `grobcov` command when it has been evaluated with special options. This will be commented on the description of the `grobcov` command in Chap. 5, Sect. 5.3.

4.2.3 Computations with I-Regular Functions

For computating with I-regular functions, the generic representation is quit practical, since the addition and multiplication of generic representations of I-regular functions can be performed by the usual computations in $K(\mathbf{u})$.

If P is a generic representation of $f : S \to L[\mathbf{x}]$ and $a \in S$ with $\mathrm{lc}_{\mathbf{x}}(P)(a) = 0$, then $P(a, \mathbf{x}) = 0$. This follows immediately from Definition 4.5.

Using generic representations it is very easy to perform computations like reduction with monic I-regular functions. The disadvantage of the generic representation is that the value of f at $a \in S$ cannot immediately be determined from p if $\mathrm{lc}(p)(a) = 0$. However, we only need to apply EXTEND to the coefficients to convert a generic representation into a full representation. Assume that the generic representation of an I-regular function $F : S \to L[\mathbf{x}]$ is $p = \Sigma_\alpha c_\alpha \mathbf{x}^\alpha$, where $p \in K[\mathbf{u}][\mathbf{x}]$ and the coefficients $c_\alpha \in K[\mathbf{u}]$. We can extend F carrying out the extension of all the coefficients, i.e.,

$$F = \sum_\alpha \mathrm{EXTEND}(S, c_\alpha, \mathrm{lc}_{\mathbf{x}}(p))\mathbf{x}^\alpha$$

is a full representation of f.

Computations (like reduction) with monic I-regular functions are most easily performed using the generic representation. The computations in the library "grobcov.lib" are carried out in $K[\mathbf{u}, \mathbf{x}]$ and the operations with the I-regular functions simply correspond to the usual operations in $K[\mathbf{u}, \mathbf{x}]$, except that we occasionally have to multiply by the leading coefficient of a generic representation to avoid denominators.

Example 4.13 Consider the monic I-regular function defined by the generic representation

$$\begin{aligned} f\colon S &\to \mathbb{C}[x, y] \\ (a, b, c, d) &\mapsto 2(ab + cd)x + 2(b^2 + d^2)y - b(a^2 + b^2) - d(c^2 + d^2), \end{aligned}$$

where $S = \mathbf{V}(bd, ad, bc, ac) \setminus \mathbf{V}(bd, ad, bc, ac, c^3 + cd^2, a^3 + ab^2)$. We compute

$$\operatorname{syz}(-\operatorname{coef}(f, y), \operatorname{coef}(f, x)) = \operatorname{syz}(-2(b^2 - 2d^2), 2(ab + cd))$$

$$\operatorname{syz}(-\operatorname{coef}(f, 1), \operatorname{coef}(f, x)) = \operatorname{syz}(b(a^2 + b^2) + d(c^2 + d^2), 2(ab + cd)),$$

obtaining

$$\operatorname{syz}(-\operatorname{coef}(f, y), \operatorname{coef}(f, x)) = ((d, c)(b, a))$$

$$\operatorname{syz}(-\operatorname{coef}(f, 1), \operatorname{coef}(f, x)) = ((c^2 + d^2, -2c), (a^2 + b^2, -2a)).$$

Multiplying the first set of syzygies by 2 and the second by -1, we have corresponding syzygies with the same value for the $\operatorname{lc}_{x,y}(f)$, namely $2c$ and $2a$ respectively. This provides two polynomials that extend the given generic representation to a full representation, which is given by

$$(2c)x + (2d)y + (-c^2 - d^2) \to x + \frac{d}{c} - \frac{c^2 + d^2}{2c} \quad \text{for } c \neq 0$$

$$(2a)x + (2b)y + (-a^2 - b^2) \to x + \frac{b}{a} - \frac{a^2 + b^2}{2a} \quad \text{for } a \neq 0.$$

Let us use the command `extendpoly` of the library to answer this question

```
> ring R=(0,a,b,c,d),(x,y),lp;
> short=0;
> poly F=2*(a*b+c*d)*x+2*(b^2+d^2)*y-b*(a^2+b^2)-d*(c^2+d^2);
> ideal p=b*d,a*d,b*c,a*c;
> ideal q=b*d,a*d,b*c,a*c,c^3+c*d^2,a^3+a*b^2;
> extendpoly(F,p,q);
_[1]=(2*c)*x+(2*d)*y+(-c^2-d^2)
_[2]=(2*a)*x+(2*b)*y+(-a^2-b^2).
```

We can see that the command corresponds adequately to the expected result.

Chapter 5
The Canonical Gröbner Cover



5.1 Introduction

In the previous chapters we have developed all the necessary tools needed for the Gröbner Cover, which is the objective of the book, and we are now ready to present our aims. As commented in previous chapters, the Gröbner Cover was introduced in [63]. Let us give some definitions.

Given a parametric ideal $I \subset K[\mathbf{u}][\mathbf{x}]$, we define:

Definition 5.1 (Parametric set) A locally closed subset S of L^m is called *parametric for* I (*with respect to* $\succ_{\mathbf{x}}$) if there exist monic I-regular functions $g_1, \ldots, g_r \in \mathcal{I}(S)$ such that $\{g_1(a, \mathbf{x}), \ldots, g_r(a, \mathbf{x})\}$ is the *Reduced Gröbner Basis* (RGB) of I_a for every $a \in S$.

From the uniqueness of the RGB, it follows immediately that if $S \subset L^m$ is parametric, then the monic I-regular functions $g_1, \ldots, g_m \in \mathcal{I}(S)$ of Definition 5.1 are uniquely determined. We call them the *reduced Gröbner basis of* I *over* S (*with respect to* $\succ_{\mathbf{x}}$). Also the definition immediately implies that if a, b lie in a parametric set S, then $\mathrm{lpp}_{\mathbf{x}}(I_a) = \mathrm{lpp}_{\mathbf{x}}(I_b)$. So we may define $\mathrm{lpp}_{\mathbf{x}}(S) = \mathrm{lpp}_{\mathbf{x}}(I_a)$ and call $\mathrm{lpp}_{\mathbf{x}}(S)$ the (set of) *leading power products of* I *over* S.

Remark 5.2 Let $S = \mathbf{V}(\mathfrak{a}) \setminus \mathbf{V}(\mathfrak{b})$, (where $\mathfrak{a} = \mathbf{I}(V(\mathfrak{a}))$) be a locally closed subset of L^m, such that $\mathrm{lpp}_{\mathbf{x}}(I_a) = \mathrm{lpp}_{\mathbf{x}}(I_b)$ for all $a, b \in S$ and let $\{\mathbf{x}^{\alpha_1}, \ldots, \mathbf{x}^{\alpha_r}\}$ be the minimal generating set of $\mathrm{lpp}_{\mathbf{x}}(I_a) = \mathrm{lpp}_{\mathbf{x}}(I_b)$. For each $i \in [r]$ consider the I-regular function $g_i : S \to L[\mathbf{x}]$ which sends $a \in S$ to the unique element of the reduced Gröbner basis of I_a with $\mathrm{lpp}_{\mathbf{x}}(g_i)$ equal to $\mathbf{x}^{\alpha_i}$. Then S is parametric if and only if for each $i \in [r]$, the function g_i has the following natural property: for every $a \in S$ there exists an open neighborhood U of a in S and a polynomial $p \in I$ such

© Springer Nature Switzerland AG 2018

A. Montes, *The Gröbner Cover*, Algorithms and Computation in Mathematics 27, https://doi.org/10.1007/978-3-030-03904-2_5

that $\mathrm{coef}_{\mathbf{x}}(p, \mathbf{x}^{\alpha_i})(b) \neq 0$ for all $b \in U$ and

$$g_i(b, \mathbf{x}) = \frac{p(b, \mathbf{x})}{\mathrm{coef}_{\mathbf{x}}(p, \mathbf{x}^{\alpha_i})(b)} \in L[\mathbf{x}]$$

for all $b \in U$.

To represent the function $g_i(b, \mathbf{x})$, we can substitute $p(b, \mathbf{x})$ with its reduced expression modulo $\mathfrak{a}$, i.e.,

$$p^r_{\mathfrak{a}}(b, \mathbf{x}) = \mathtt{pnormalf}(p(\mathbf{u}, \mathbf{x}), \mathfrak{a}, \mathfrak{b})_{\mathbf{u}=b},$$

considering a non-faithful representation, as p and $p^r_{\mathfrak{a}}$ are equivalent on S.

We generalize the concept of a Comprehensive Gröbner System (CGS) accepting bases formed by I-regular functions and defining the following.

Definition 5.3 (Gröbner Cover) By a *Gröbner Cover of L^m with respect to I and $\succ_{\mathbf{x}}$*, we mean a finite set of pairs $\{(S_1, B_1), \ldots, (S_r, B_r)\}$ such that

(1) the S_i are parametric and B_i is the reduced Gröbner basis of I over S_i for $i = 1, \ldots, r$ and
(2) the union of all S_i equals L^m.

The S_i are called the *segments* of the Gröbner Cover. A Gröbner Cover is called disjoint if the S_i are pairwise disjoint.

5.2 Canonical Partition of the Parameter Space

We deal now with the fundamental lemma that allow us to prove the existence of the Canonical Gröbner Cover.

Lemma 5.4 (M. Wibmer) *Let $\mathfrak{a} \subset K[\mathbf{u}]$ be a radical ideal and let $I_{\mathfrak{a}}$ denote the image of I under the canonical map $K[\mathbf{u}][x] \to (K[\mathbf{u}]/\mathfrak{a})[\mathbf{x}]$. Furthermore let $\mathbf{x}^{\alpha_1}, \ldots, \mathbf{x}^{\alpha_s}$ be the minimal generating set of $\mathrm{lpp}_{\mathbf{x}}(I_{\mathfrak{a}})$. We define ideals of $K[\mathbf{u}]/\mathfrak{a}$ by setting*

$$\mathrm{lc}_{\mathbf{x}}(I_{\mathfrak{a}}, \mathbf{x}^{\alpha_i}) = \langle \mathrm{lc}_{\mathbf{x}}(p) : \ p \in I_{\mathfrak{a}} \textit{with } \mathrm{lpp}_{\mathbf{x}}(p) = \mathbf{x}^{\alpha_i} \rangle.$$

Since the ideal is finitely generated, define

$$J = \sqrt{\prod_{i=1}^{s} \mathrm{lc}_{\mathbf{x}}(I_{\mathfrak{a}}, \mathbf{x}^{\alpha_i})}.$$

Let $\mathfrak{b} \subset K[\mathbf{u}]$ *denote the inverse image of* J *under the canonical map* $K[\mathbf{u}] \to K[\mathbf{u}]/\mathfrak{a}$. *Then* $S = \mathbf{V}(\mathfrak{a}) \setminus \mathbf{V}(\mathfrak{b})$ *is parametric and* $\mathrm{lpp}_{\mathbf{x}}(\sigma_a(I)) = \mathrm{lpp}_{\mathbf{x}}(I_{\mathfrak{a}})$ *for all* $a \in S$.

By the definition of $\mathfrak{b}$, we have $[\mathfrak{a}, \mathfrak{b}] = \textsc{Crep}(S)$ and so $\mathfrak{a} \subset \mathfrak{b}$. Lemma 5.4 is central in constructing the Gröbner Cover, and so it is interesting to summarize the construction that it provides in Algorithm 5.1: WLemma.

WLemma$(F, \mathfrak{a},)$
Input:
$F \subset K[\mathbf{u}][\mathbf{x}]$
$\mathfrak{a} \subset K[\mathbf{u}]$ radical ideal
Output:
$(\mathrm{lpp}_{\mathbf{x}}, G, S)$, where $S = \textsc{Crep}(\mathfrak{a}, \mathfrak{b})$, is the Crep of $\mathbf{V}(\mathfrak{a}) \setminus \mathbf{V}(\mathfrak{b})$
over which $G = \{g_1, \ldots, g_s\}$ is the Gröbner basis of F and has fixed $\mathrm{lpp}_{\mathbf{x}}$. (The g_i are a representation of an I-regular function)

begin
$B = \mathrm{GB}(F + \mathfrak{a}, \succ_{\mathbf{x},\mathbf{u}})$
$T = (t_1, \ldots, t_s) = \mathrm{GB}(\mathrm{lpp}_{\mathbf{x}}(B), \succ_x)$
$G_0 = \{p \in B : \mathrm{lpp}_{\mathbf{x}}(p) \in T\}$
$G_0 =$ reduce all polynomials in G_0 wrt $\mathfrak{a}$
for $i = 1, \ldots, s$ **do**
$g_i = \{p \in G_0 : \mathrm{lpp}_{\mathbf{x}}(p) = t_i\}$
$J_i = \langle \mathrm{lc}_{\mathbf{x}}(p) : p \in g_i \rangle$
end for
$\mathfrak{b} = \prod_{i=1}^{s} J_i$
$[\mathfrak{a}, \mathfrak{b}] = \textsc{Crep}(\mathfrak{a}, \mathfrak{b})$
return$(T, G_0, (\mathfrak{a}, \mathfrak{b}))$
end

Algorithm 5.1: WLemma (Lemma 5.4)

Proof Given I, compute the reduced Gröbner basis $G = \{p_1, \ldots, p_s\}$ of I with respect to $\succ_{\mathbf{x}} \cdot \succ_{\mathbf{u}}$. Determining now their reduction with respect to $\mathfrak{a}$, we obtain the polynomials $G_{\mathfrak{a}} = \{\overline{p_1}^{\mathfrak{a}}, \ldots, \overline{p_s}^{\mathfrak{a}}\}$.

To say that a lies in $S = \mathbf{V}(\mathfrak{a}) \setminus \mathbf{V}(\mathfrak{b})$ simply means that for every $i = 1, \ldots, s$, there exists a polynomial $q_i \in I$ such that $\mathrm{lpp}_{\mathbf{x}}(\overline{q_i}^{\mathfrak{a}}) = \mathbf{x}^{\alpha_i}$ and $\mathrm{coef}_{\mathbf{x}}(q_i, \mathbf{x}^{\alpha_i})(a) \neq 0$. But the polynomials q_i can be expressed in terms of the p_i.

We can compute the reduced Gröbner basis of I wrt $\succ_{\mathbf{x}} \cdot \succ_{\mathbf{u}}$, say G. So we obtain polynomials $q_i \in I$ and then we can reduce them wrt to $\mathfrak{a}$. In this way we obtain reduced polynomials $p_1, \ldots, p_s \in I_{\mathfrak{a}}$ such that $\mathrm{lpp}_{\mathbf{x}}(p_i) = \mathbf{x}^{\alpha_i}$, $\mathrm{coef}_{\mathbf{x}}(p_i, \mathbf{x}^{\alpha_i})$ is a power product in $\mathrm{coef}_{\mathbf{x}}(q_1, \mathbf{x}^{\alpha_1}), \ldots, \mathrm{coef}_{\mathbf{x}}(q_s, \mathbf{x}^{\alpha_s})$ and no monomial of p_i is divisible by $\mathbf{x}^{\alpha_j}$ for $j \neq i$. Note that

$$U = \mathbf{V}(\mathfrak{a}) \setminus \mathbf{V}(\mathrm{coef}(q_1, \mathbf{x}^{\alpha_1}) \cdots \mathrm{coef}(q_s, \mathbf{x}^{\alpha_s})) \subset \mathbf{V}(\mathfrak{a}) \setminus \mathbf{V}(b) = S$$

is an open neighborhood of a in S. For $i = 1, \ldots, s$, let $g_i : U \to L[\mathbf{x}]$ be the I-regular function with

$$g_i(b) = \frac{p_i(b, \mathbf{x})}{\operatorname{coef}(p_i, \mathbf{x}^{\alpha_i})(b)}.$$

We want to show that $g_1(b), \ldots, g_s(b) \in L[\mathbf{x}]$ is the reduced Gröbner basis of $\sigma_b(I)$ for all $b \in U$. We need to show that for $p \in I$, we can reduce $\sigma_b(p) \in \sigma_b(I) \subset L[x]$ modulo $g_1(b), \ldots, g_s(b)$. PDIVI-reduction of p modulo $p_1, \ldots, p_s$ over $\mathfrak{a}$ yields a representation

$$hp = r_1 p_1 + \cdots + r_s p_s + r,$$

where $r_1, \ldots, r_s \in K[\mathbf{u}, \mathbf{x}]$, $h \in K[\mathbf{u}]$ is a power product in $\operatorname{coef}(q_1, \mathbf{x}^{\alpha_1}), \ldots, \operatorname{coef}(q_s, \mathbf{x}^{\alpha_s})$ and $\operatorname{lpp}_{\mathbf{x}}(r)$ is not divisible by any $\mathbf{x}^{\alpha_i}$ for $i = 1, \ldots, s$. But as the $\mathbf{x}^{\alpha_i}$ are the minimal generating set of $\operatorname{lpp}_{\mathbf{x}}(I_{\mathfrak{a}})$ and $\overline{r} \in I_{\mathfrak{a}}$, we must have $\overline{r}^{\mathfrak{a}} = 0$. In particular $\sigma_b(q) = 0$ for all $b \in U \subset \mathbf{V}(\mathfrak{a})$. For $b \in U$ we have $h(b) \neq 0$ and therefore we obtain

$$\sigma_b(p) = h(b)^{-1}\sigma_b(r_1)\sigma_b(p_1) + \cdots + h(b)^{-1}\sigma_b(r_s)\sigma_b(p_s)$$

with $\operatorname{lpp}_{\mathbf{x}}(\sigma_b(r_i)\sigma_b(p_i)) \leq \operatorname{lpp}_{\mathbf{x}}(\sigma_b(p))$. This shows that $g_1(b), \ldots, g_s(b)$ is a Gröbner basis of $\sigma_b(I)$ and from the construction it is clear that it is the unique reduced Gröbner basis of $\sigma_b(I)$.

We recall what we have done so far: For an $a \in S$ we found an open neighborhood U of a in S and I-regular functions $g_1, \ldots, g_s \in \mathcal{I}(U)$ such that $\operatorname{lpp}_{\mathbf{x}}(g_i) = \mathbf{x}^{\alpha_i}$ for $i = 1, \ldots, s$ and $g_1(b), \ldots, g_s(b)$ is the reduced Gröbner basis of $\sigma_b(I)$ for all $b \in U$. If we start with a different element $a' \in S$ and similarly obtain U' and $g'_1, \ldots, g'_s \in \mathcal{I}(U')$, then we must have $g_i(b) = g'_i(b)$ for $i = 1, \ldots, m$, and all $b \in U \cap U'$ because $g_i(b)$ as well as $g'_i(b)$ is the unique element of the reduced Gröbner basis of $\sigma_b(I)$ with $\operatorname{lpp}_{\mathbf{x}}$ equal to $\mathbf{x}^{\alpha_i}$. This shows that we can glue together g_i and g'_i and the claim follows. □

Example 5.5 Let us illustrate Lemma 5.4 with a simple example. Consider the parametric ideal $I = \langle ax^2 + bxy + cy^2, dx^2 + exy + fy^2 \rangle$ in the ring $R = \mathbb{Q}[a, b, c, d, e, f][x, y]$, and the radical ideal $\mathfrak{a} = \langle ae - bd \rangle$.

Computing the Gröbner basis of $I + \mathfrak{a}$ with respect to $\succ_{\mathbf{x}\cdot\mathbf{u}}$, reducing modulo $\mathfrak{a}$ and grouping as polynomials in R results in the specialized basis

$$G_{\mathfrak{a}} = \{(bf - ce)y^2, (af - cd)y^2, dx^2 + exy + fy^2, ax^2 + bxy + cy^2\}.$$

The minimal generating set of its lpp is $T = \langle y^2, x^2 \rangle$. Now we compute J. The coefficient ideals are

$$\operatorname{lc}_{y^2} = \langle bf - ce, af - cd \rangle,$$

$$\operatorname{lc}_{x^2} = \langle d, a \rangle,$$

and the product $J = \langle (bf - ce), (af - cd) \rangle \cdot \langle a, d \rangle$. Since either $bf - ce \neq 0$ or $af - cd \neq 0$ on S, the first and second elements of $G_{\mathfrak{a}}$ reduce to y^2. We can further simplify the basis on S by reducing the remaining elements of $G_{\mathfrak{a}}$ with respect to y^2. Using PDIVI, the basis becomes $B_S = y^2, dx^2 + exy, ax^2 + bxy$. After computing PREP($\mathfrak{a}$, J), the result is

$$\begin{aligned} T =& \{y^2, x^2\}, \\ S =& \mathbf{V}(ae - bd) \setminus (\mathbf{V}(a, d) \cup \mathbf{V}(ae - bd, bf - ce, af - cd)), \\ B_S =& \{\{y^2\}, \{dx^2 + exy, ax^2 + bxy\}\}. \end{aligned}$$

We can see that the function determining the element of the Gröbner basis corresponding to $\mathrm{lpp}_{\mathbf{x}}(g_2) = x^2$ is not a simple polynomial, but an I-regular function g_2 determined by two polynomials:

$$g_2 : \quad S \to R$$
$$(a, b, c, d, e) \mapsto \begin{cases} x^2 + \dfrac{e}{d}xy \text{ if } d \neq 0 \\ x^2 + \dfrac{b}{a}xy \text{ if } a \neq 0. \end{cases}$$

The call to the command `WLemma` of the "grobcov.lib" library outputs three objects: (lpp, B, S), where B is the Gröbner basis expressed in a full representation, and S is the P-representation of the segment. Let us apply this::

```
> LIB "grobcov.lib";
> ring R=(0,a,b,c,d,e,f),(x,y),lp;
> short=0;
> ideal F=a*x^2+b*x*y+c*y^2,d*x^2+e*x*y+f*y^2;
> ideal A=a*e-b*d;
> WLemma(F,A);
[1]:
   _[1]=y^2
   _[2]=x^2
[2]:
   [1]:
      _[1]=y^2
   [2]:
      _[1]=(d)*x^2+(e)*x*y
      _[2]=(a)*x^2+(b)*x*y
[3]:
   [1]:
      [1]:
         _[1]=(a*e-b*d)
      [2]:
         [1]:
            _[1]=(d)
            _[2]=(a)
         [2]:
            _[1]=(b*f-c*e)
```

```
_[2]=(a*f-c*d)
_[3]=(a*e-b*d)
```

Corollary 5.6 *Applying Algorithm 5.1:* WLEMMA *for a prime ideal* $\mathfrak{a}$ *(different from* $\langle 1\rangle$*) produces an open locally closed set that is open in* $\mathbf{V}(\mathfrak{a})$.

Proof The Gröbner basis of $F+\mathfrak{a}$ remains stable in reducing with respect to $\mathfrak{a}$, and so, none of the elements defining J_i reduces modulo $\mathfrak{a}$. Thus $J_i \not\subset \mathfrak{a}$. Since $\mathfrak{b}$ is the product of J_i and $\mathfrak{a}$ is prime, in order to have $\mathfrak{b} \subseteq \mathfrak{a}$ there should exist some $J_i \subseteq \mathfrak{a}$, which is impossible. Thus $\mathfrak{b} \not\subseteq \mathfrak{a}$ and the corollary follows. □

Remark 5.7 The above result is not true when $\mathfrak{a}$ is not prime, and the resulting locally closed set can be empty. To show this, we continue with the previous example taking the output $\mathfrak{b}$ of the previous example as input $\mathfrak{a} = \langle bdf-cde, af-cd, ae-bd\rangle = \langle bf-ce, af-cd, ae-bd\rangle\langle d, a\rangle$:

```
 > LIB "grobcov.lib";
> ring R=(0,a,b,c,d,e,f),(x,y),lp;
> short=0;
> ideal F=a*x^2+b*x*y+c*y^2,d*x^2+e*x*y+f*y^2;
> ideal A1=b*d*f-c*d*e,a*f-c*d,a*e-b*d;
> WLemma(F,A1);
[1]:
   _[1]=y3
   _[2]=x2
[2]:
   [1]:
      _[1]=(bef-ce2)*y3
      _[2]=(bf-ce)*y3
   [2]:
      _[1]=(d)*x2+(e)*xy+(f)*y2
      _[2]=(a)*x2+(b)*xy+(c)*y2
[3]:
   empty list
```

5.2.1 The Case of Homogeneous Ideals

We can now prove the fundamental Wibmer Theorem [85] on the existence of the Canonical Gröbner Cover.

Theorem 5.8 (M. Wibmer) *Let* $I \subset K[\mathbf{u}][\mathbf{x}]$ *be a homogeneous ideal with respect to the variables and* $a \in L^m$. *Then the set*

$$S = \{b \in L^m : \operatorname{lpp}_{\mathbf{x}}(I_b) = \operatorname{lpp}_{\mathbf{x}}(I_a)\}$$

is parametric. In particular S is locally closed.

Proof Let $\mathfrak{a} = \mathbf{I}(S)$, let $I_\mathfrak{a}$ denote the image of I under the canonical map $K[\mathbf{u}][x] \to (K[\mathbf{u}]/\mathfrak{a})[\mathbf{x}]$, and define $\mathfrak{b}$ as in Lemma 5.4. Since $S' = \mathbf{V}(\mathfrak{a}) \setminus \mathbf{V}(\mathfrak{b})$ is parametric, it suffices to prove that $S = S'$.

For proving the inclusion $S' \subseteq S$, it suffices to prove that S' is non-empty and also $S \cap S'$ is non-empty, because then, having a common point a, since S' is parametric and has fixed set $\mathrm{lpp}(I_\mathfrak{a})$ of lpp, and S is defined as the set of all points having the same set $\mathrm{lpp}(\sigma_a(I))$ of lpp, both will have the same set of lpp and necessarily $S' \subseteq S$. Let

$$\mathfrak{a} = \mathfrak{a}_1 \cap \cdots \cap \mathfrak{a}_m$$

be the prime decomposition of $\mathfrak{a}$, and $Y_i = \mathbf{V}(\mathfrak{a}_i)$. Let $I_{\mathfrak{a}_i}$ denote the image of I in $(K[\mathbf{u}]/\mathfrak{a}_i)[x]$. From Lemma 5.4 and Corollary 5.6 it follows that there exists a non-empty open parametric locally closed subset U_i of Y_i with $\mathrm{lpp}_\mathbf{x}(\sigma_b(I)) = \mathrm{lpp}_\mathbf{x}(I_{\mathfrak{a}_i})$ for all $b \in U_i$. Since S is a constructible set, it is open in its closure $\overline{S} = \bigcup_{i=1}^m Y_i$. Thus the intersection of S with U_i is non-empty for $i \in [m]$. Consequently $\mathrm{lpp}_\mathbf{x}(I_{\mathfrak{a}_i}) = \mathrm{lpp}_\mathbf{x}(\sigma_a(I))$ for $i \in [m]$ and $S' \subseteq S$.

For the reverse inclusion $S \subseteq S'$, it remains to prove that $\mathrm{lpp}_\mathbf{x}(\sigma_b(I)) \neq \mathrm{lpp}_\mathbf{x}(\sigma_a(I)) = \mathrm{lpp}_\mathbf{x}(I_\mathfrak{a})$ for $b \in \mathbf{V}(\mathfrak{a}) \setminus S' = \mathbf{V}(\mathfrak{b}) \subset \mathbf{V}(\mathfrak{a})$. Suppose that $\mathrm{lpp}_\mathbf{x}(\sigma_b(I)) = \mathrm{lpp}_\mathbf{x}(I_\mathfrak{a})$ for some $b \in \mathbf{V}(\mathfrak{b})$. Let $p \in I$ and denote $\mathbf{x}^\alpha = \mathrm{lpp}_\mathbf{x}(p^r_\mathfrak{a})$. From the definition of $\mathfrak{b}$, it follows that there exists an $i \in [m]$ such that $\mathrm{coef}(p, \mathbf{x}^\alpha)(b) = 0$. In particular there exists a power product $\mathbf{x}^\alpha \in \mathrm{lpp}_\mathbf{x}(I_\mathfrak{a})$ such that $\mathrm{coef}(p, \mathbf{x}^\alpha)(b) = 0$ for every $p \in I$ with $\mathrm{lpp}_\mathbf{x}(p^r_\mathfrak{a}) = \mathbf{x}^\alpha$. We may assume that $\mathbf{x}^\alpha$ is maximal in its degree, i.e., for every power product $\mathbf{x}^{\alpha'} \in \mathrm{lpp}_\mathbf{x}(I_\mathfrak{a})$ with $\deg(\mathbf{x}^{\alpha'}) = \deg(\mathbf{x}^\alpha)$ and $\mathrm{coef}(p, \mathbf{x}^{\alpha'})(b) = 0$ for every $p \in I$ with $\mathrm{lpp}_\mathbf{x}(p^r_\mathfrak{a}) = \mathbf{x}^{\alpha'}$, we have $\mathbf{x}^\alpha \geq \mathbf{x}^{\alpha'}$.

Since $\mathbf{x}^\alpha \in \mathrm{lpp}_\mathbf{x}(I_\mathfrak{a}) = \mathrm{lpp}_\mathbf{x}(\sigma_b(I))$ there exists $p \in I$ with $\mathrm{lpp}_\mathbf{x}(\sigma_b(p)) = \mathbf{x}^\alpha$. Because I is homogeneous we can assume that p is homogeneous and thus $\deg(p) = \deg(\mathbf{x}^\alpha)$. We can also assume that $\mathrm{lpp}_\mathbf{x}(p^r_\mathfrak{a})$ is minimal, i.e., for $p' \in I$ homogeneous with $\mathrm{lpp}_\mathbf{x}(\sigma_b(p')) = \mathbf{x}^\alpha$, we have $\mathrm{lpp}_\mathbf{x}(p^r_\mathfrak{a}) \leq \mathrm{lpp}_\mathbf{x}(p'^r_\mathfrak{a})$. Since $\mathrm{coef}(p, \mathbf{x}^\alpha)(b) \neq 0$ we must have $\mathrm{lpp}_\mathbf{x}(p^r_\mathfrak{a}) > \mathbf{x}^\alpha$. By the maximality of $\mathbf{x}^\alpha$, there exists $q \in I$ with $\mathrm{lpp}_\mathbf{x}(q^r_\mathfrak{a}) = \mathrm{lpp}_\mathbf{x}(p^r_\mathfrak{a})$ and $\mathrm{coef}(q, \mathrm{lpp}_\mathbf{x}(q^r_\mathfrak{a}))(b) \neq 0$. Set

$$p' = \mathrm{coef}(q, \mathrm{lpp}_\mathbf{x}(q^r_\mathfrak{a}))p - \mathrm{coef}(p, \mathrm{lpp}_\mathbf{x}(p^r_\mathfrak{a}))q \in I.$$

Then $\mathrm{coef}(p', \mathbf{x}^{\alpha'})(b) = 0$ for $\mathbf{x}^{\alpha'} > \mathbf{x}^\alpha$ because $\mathrm{coef}(p, t')(b) = 0$ for $\mathbf{x}^{\alpha'} > \mathbf{x}^\alpha$. On the other hand, $\mathrm{coef}(p', \mathbf{x}^\alpha)(b) = \mathrm{coef}(q, \mathrm{lpp}_\mathbf{x}(q^r_\mathfrak{a}))(b)\,\mathrm{coef}(p, \mathbf{x}^\alpha)(b) \neq 0$. Therefore $\mathrm{lpp}_\mathbf{x}(\sigma_b(p')) = \mathbf{x}^\alpha$, but since $\mathrm{lpp}_\mathbf{x}(p'^r_\mathfrak{a}) < \mathrm{lpp}_\mathbf{x}(p^r_\mathfrak{a})$, this contradicts the minimality of $\mathrm{lpp}_\mathbf{x}(p^r_\mathfrak{a})$. □

From Theorem 5.8 the definition of the Canonical Gröbner Cover is obvious if I is a homogeneous ideal:

Theorem 5.9 (Canonical Gröbner Cover) *If $I \subset K[\mathbf{u}][\mathbf{x}]$ is homogeneous (with respect to the variables), then there exists a unique Gröbner Cover of L^m with minimal cardinality which we call the* Canonical Gröbner Cover of L^m (with respect

to I and $\succ_{\mathbf{x}}$). *It is disjoint and two points* $a, b \in L^m$ *lie in the same segment if and only if* $\mathrm{lpp}_{\mathbf{x}}(I_a) = \mathrm{lpp}_{\mathbf{x}}(I_b)$. *The segments of this Gröbner Cover will be called* $\mathrm{lpp}_{\mathbf{x}}$-segments.

5.2.2 The Case of Arbitrary Ideals

For non-homogeneous ideals the situation is somewhat more complicated. But as we will see below, we can use the method of homogenization to look into Theorem 5.8 also in this case. For the rest of this section, we focus on the case where I is a non-homogeneous ideal. Our aim is to generalize the definition of the Canonical Gröbner Cover to arbitrary ideals.

For homogenization, we introduce a new variable x_0 and extend $\succ_{\mathbf{x}}$ to the monomials in $x_0, x_1, \ldots, x_n$ by setting

$$\mathbf{x}^{\alpha} x_0^i \succ_{\mathbf{x},x_0} \mathbf{x}^{\beta} x_0^j$$

if $\mathbf{x}^{\alpha} \succ_{\mathbf{x}} \mathbf{x}^{\beta}$ or $\mathbf{x}^{\alpha} = \mathbf{x}^{\beta}$ and $i > j$. If p is a homogeneous polynomial in the variables $\mathbf{x}, x_0$ with coefficients in some ring, then the *dehomogenization of* p is denoted with $\tau(p)$, i.e., $\tau(p) = p(\mathbf{x}, 1)$.

It is immediately seen that $\succ_{\mathbf{x},x_0}$ is a monomial order with the property that $\tau(\mathrm{lpp}_{\mathbf{x}}(p)) = \mathrm{lpp}_{\mathbf{x}}(\tau(p))$ and $\mathrm{lc}(\tau(p)) = \mathrm{lc}(p)$ for every *homogeneous* polynomial p.

Lemma 5.10 (cf. [30], Exercise 15.39, page 375) *Let* $I' \subset K[\mathbf{x}]$ *be an ideal and* $J' \subset K[\mathbf{x}, x_0]$ *an homogeneous ideal such that* $\tau(J') = I'$. *If* $\{g_1, \ldots, g_r\}$ *is a Gröbner basis of* J' *with respect to* $\succ_{\mathbf{x},x_0}$ *and the* g_i *are homogeneous then* $\{\tau(g_1), \ldots, \tau(g_r)\}$ *is a Gröbner basis of* I' *with respect to* $\succ_{\mathbf{x}}$.

Proof Let $p \in I'$. Then there exists $q \in J'$ homogeneous such that $\tau(q) = p$. Since $q \in J'$ there exists an i such that $\mathrm{lpp}_{\mathbf{x}}(g_i)$ divides $\mathrm{lpp}_{\mathbf{x}}(q)$, say $\mathrm{lpp}_{\mathbf{x}}(q) = \mathbf{x}^{\alpha}\,\mathrm{lpp}_{\mathbf{x}}(g_i)$. But then

$$\mathrm{lpp}_{\mathbf{x}}(p) = \mathrm{lpp}_{\mathbf{x}}(\tau(q)) = \tau(\mathrm{lpp}_{\mathbf{x}}(q)) = \tau(\mathbf{x}^{\alpha}\,\mathrm{lpp}_{\mathbf{x}}(g_i)) = \tau(\mathbf{x}^{\alpha})\,\mathrm{lpp}_{\mathbf{x}}(\tau(g_i)),$$

so that $\mathrm{lpp}_{\mathbf{x}}(p)$ is divisible by $\mathrm{lpp}_{\mathbf{x}}(\tau(g_i))$ and $\tau(g_1), \ldots, \tau(g_r)$ is a Gröbner basis of I'. □

Nevertheless, it is not true that τ preserves reduced Gröbner bases. Consider, for example, the homogeneous ideal $F = \langle x^2y - yt^2 + t^3, x^2 - t^2\rangle$. Its Reduced Gröbner basis with respect to $\mathrm{grevlex}(x, y) \cdot \mathrm{lex}(t)$ is $G = \{t^3, x^2 - t^2\}$, which specializes to $G = \{1, x^2 - 1\}$ for $t = 1$. This is really a Gröbner basis of $F_{t=1}$ but not the reduced one which is $G_0 = \{1\}$.

Proposition 5.11 *Let $J \subset K[\mathbf{u}][\mathbf{x}, x_0]$ be a homogeneous ideal such that $\tau(J) = I$ and $S \subset L^m$ is parametric with respect to J and $\succ_{\mathbf{x},x_0}$. Then S is parametric with respect to I and $\succ_{\mathbf{x}}$.*

Proof Let $h_1, \ldots, h_r \in \mathcal{O}(S)[\mathbf{x}, x_0]$ be the reduced Gröbner basis of J over S. We note that since J is homogeneous, $J_a \subset L[\mathbf{x}, x_0]$ is also homogeneous for every $a \in S$. The reduced Gröbner basis of a homogeneous ideal is homogeneous and so also the $h_1, \ldots, h_r$ are homogeneous. Since $\mathrm{lc}(\tau(p)) = \mathrm{lc}(p)$ for homogeneous polynomials p, we see that $\tau(h_1), \ldots, \tau(h_r) \in \mathcal{O}(S)[\mathbf{x}]$ are monic polynomials. Since $h_1, \ldots, h_r$ are J-regular, $\tau(h_1), \ldots, \tau(h_r)$ are also I-regular. Let $f_1, \ldots, f_s$ denote the monic I-regular functions obtained from $\tau(h_1), \ldots, \tau(h_r)$ by discarding those $\tau(h_i)$ whose leading power product is divisible by some $\mathrm{lpp}_{\mathbf{x}}(\tau(h_j))$ for $i \neq j$. Further let $g_1, \ldots, g_s \in \mathcal{O}(S)[\mathbf{x}]$ be the monic I-regular functions obtained by reducing f_i modulo $\{f_1, \ldots, f_s\} \setminus \{f_i\}$. To conclude the proof, we will show that $g_1(a), \ldots, g_s(a)$ is the reduced Gröbner basis of I_a for every $a \in S$. So choose $a \in S$. Since $h_1(a), \ldots, h_r(a)$ is a Gröbner basis of J_a it follows from Lemma 5.10 that $\tau(h_1(a)), \ldots, \tau(h_r(a))$ is a Gröbner basis of $\tau(J_a) = I_a$. Therefore

$$\begin{aligned}\langle \mathrm{lpp}_{\mathbf{x}}(I_a)\rangle &= \langle \mathrm{lpp}_{\mathbf{x}}(\tau(h_1)(a)), \ldots, \mathrm{lpp}_{\mathbf{x}}(\tau(h_r)(a))\rangle \\ &= \langle \mathrm{lpp}_{\mathbf{x}}(\tau(h_1)), \ldots, \mathrm{lpp}_{\mathbf{x}}(\tau(h_r))\rangle \\ &= \langle \mathrm{lpp}_{\mathbf{x}}(f_1), \ldots, \mathrm{lpp}_{\mathbf{x}}(f_s)\rangle = \langle \mathrm{lpp}_{\mathbf{x}}(g_1), \ldots, \mathrm{lpp}_{\mathbf{x}}(g_s)\rangle.\end{aligned}$$

This shows that $g_1(a), \ldots, g_s(a)$ is a Gröbner basis of I_a and since the g_i are mutually reduced, the $g_i(a)$ are also mutually reduced. Consequently $g_1(a), \ldots, g_s(a)$ is the reduced Gröbner basis of I_a. □

Definition 5.12 (Canonical Gröbner Cover) Let $I \subset K[\mathbf{u}][\mathbf{x}]$ be an arbitrary ideal and let $J \subset K[\mathbf{u}][\mathbf{x}, x_0]$ denote its homogenization. By Proposition 5.11, the segments of the Canonical Gröbner Cover of L^m with respect to J and $\succ_{\mathbf{x},x_0}$ are parametric with respect to I and $\succ_{\mathbf{x}}$. The disjoint Gröbner Cover of L^m with respect to I and $\succ_{\mathbf{x}}$ thus obtained will be called the *Canonical Gröbner Cover* of L^m with respect to I and $\succ_{\mathbf{x}}$.

From now on, we shall refer to the Canonical Gröbner Cover simply as the Gröbner Cover (GC).

In general, homogenization does not commute with specialization. For example if we homogenize the polynomial a_1x_1+1 and then evaluate at $a_1 = 0$ we get x_0, but if we first evaluate and then homogenize, we get 1. However, since the homogenization of a homogeneous polynomial is of course just the polynomial itself, there is no such problem if we only have to deal with homogeneous polynomials. So if $I \subset K[\mathbf{u}][\mathbf{x}]$ was already homogeneous, we immediately see that for $a \in L^m$, the reduced Gröbner basis of J_a with respect to $\succ_{\mathbf{x},x_0}$ equals the reduced Gröbner basis of I_a with respect to $\succ_{\mathbf{x}}$. From this observation it follows that the definition of the Canonical Gröbner Cover is unambiguous. That is, if the ideal I in Definition 5.12 is already homogeneous, then Definition 5.12 agrees with the definition in Theorem 5.9.

$G = \{[\text{lpp}_i, B_i, S_i] : i = 1, \ldots, s\} \leftarrow \mathbf{GCover}(F, \succ_{\mathbf{x}})$.
Input: $F \subset K[\mathbf{u}][\mathbf{x}]$: finite set of homogeneous polynomials
generating the ideal I
Output: $G = \{[\text{lpp}_i, B_i, S_i] : i = 1, \ldots, s\}$: the Canonical
Gröbner Cover of I.
The basis elements are given in generic representation
The segments are given in P-representation

begin
$C = \text{CGSDR}(F)$ # Disjoint, reduced CGS (Use KSW)
Transform the representations of the segments of C into PREP
$L =$ the list of all lpp occurring in C
$G = \emptyset$
for each $\mathbf{x}^\alpha \in L$ **do**
$M =$ the list of all segments $((\mathfrak{a}, \ell_\mathfrak{a}), B)$ given in C with
lpp equal $\mathbf{x}^\alpha$ #($\ell_\mathfrak{a}$ are the holes of $\mathfrak{a}$)
$H = M$ (without bases, conserving only its index position)
$S = \text{LCUNION}(H)$ # (see Chap. 2, Sect. 2.2)
Let $(\mathfrak{p}_1, k_1), \ldots, (\mathfrak{p}_r, k_r)$ denote the (component, index) of S,
where k_i denotes the index of the segment of M
having $\mathfrak{p}_i$ as component.
$B = \emptyset$
for $i \in [r]$ **do**
for $j \in [\#B_1]$ **do** # All bases have same number of elements
$p = \text{COMBINE}((\mathfrak{p}_1, B_{k_1}[j]), \ldots, (\mathfrak{p}_r, B_{k_r}[j]))$
$B = B \cup p$
end for
end for
$G = G \cup \{[\text{lpp}(B), B, S]\}$
end for
return(G)
end

Algorithm 5.2: GCOVER

Remark 5.13 If I is not homogeneous, the Canonical Gröbner Cover, can have segments with the same set of lpp. This is obvious, since two different monomials of the homogenized ideal, say $\mathbf{x}^\alpha x_0^i$ and $\mathbf{x}^\alpha x_0^j$, reduce to the same monomial $\mathbf{x}^\alpha$ by dehomogenizing. So two lpp segments of the homogeneous ideal with different lpp sets can correspond to two segments of the associated non-homogeneous ideal having the same lpp.

5.3 Algorithms for Computing the Canonical Gröbner Cover

In this section we describe two algorithms: GCOVER and GROBCOV. The first is a consequence of Wibmer's Theorem 5.8. GCOVER considers a homogeneous parametric ideal and computes its Canonical Gröbner Cover. GROBCOV (the basic

algorithm) takes into account Lemma 5.10 and Proposition 5.11 to compute the Canoncal Gröbner Cover of an arbitrary parametric ideal. GCover is the heart of the main algorithm GrobCov.

Throughout this section we consider a fixed term-order $\succ_{\mathbf{x}}$ on the variables that is not explicit in the algorithm's description.

GCover takes as input a finite set of homogeneous polynomials (with respect to the variables) which generates the ideal $I \subset K[\mathbf{u}][\mathbf{x}]$. In the output, the bases of the lpp-segments are I-regular functions given in generic representation.

GCover has three main steps: CGSDR, LCUnion and Combine, already described in Chaps. 3, 2 and 4 respectively. The algorithm starts with the call to the CGSDR algorithm, i.e., the algorithm to compute a disjoint reduced CGS (Comprehensive Gröbner System). In our implementation, CGSDR is the Kapur-Sun-Wang algorithm KSW, described in Chap. 3, Sect. 3.5, Algorithm 3.4. But it can, eventually, be replaced by another CGS algorithm. We recall that originally we used BuildTree, described in the Appendix, which has been replaced by KSW for efficiency reasons.

We assume that the CGSDR algorithm produces a disjoint and reduced CGS, as is the case for KSW and BuildTree. The CGS obtained can contain different segments with the same lpp, and the bases are formed by regular functions (simple polynomials). To continue the algorithm we need the segments to be given in P-representation, so if the CGSDR does not already return them so, we must transform them using Prep.

With the CGS expressed in P-representation, GCover groups the segments with the same lpp, and it first adds them together assuming that its addition forms a locally closed set, as certified by Wibmer's Theorem 5.8 to obtain the unique lpp-segment. For this addition, GCover uses LCUnion algorithm, described in Chap. 2, Sect. 2.4, Algorithm 2.6, which is very efficient under the thesis of Wibmer's Theorem, and provides the P-representation (Prep) of the lpp segment.

Then, GCover has to compute the generic representation of the basis on each lpp segment. LCUnion also provides the index k_i to find the basis B_{k_i} on the CGS that corresponds to the component of the lpp segment with top $\mathfrak{p}_i$. Now GCover is ready to apply Algorithm 4.2: Combine, (Chap. 4, Sect. 4.2), with $((\mathfrak{p}_1, B_{k_1}[j]), \ldots, (\mathfrak{p}_r, B_r[j]))$, where $B_{k_i}[j]$ represents the jth polynomial of the basis B_{k_i}, and obtain the generic representation of the I-regular function representing them on the whole lpp segment.

Finally GCover collects all the pairs $(\text{Prep}(S_i), B_i)$ of the lpp segments. The result is the Gröbner Cover of the homogeneous ideal I. It is described in Algorithm 5.2: GCover.

Now we have to compute the Gröbner Cover of a non-homogeneous ideal, following Sect. 5.2.2. We must first homogenize with respect to the variables, then compute the GCover and then deshomogenize the result, and reduce the bases. The algorithm is called GrobCov, and is the main object of this book. It is described in Algorithm 5.3: GrobCov.

$G = \{[\text{lpp}_i, B_i, S_i) : i = 1, \ldots, s\} \leftarrow \textbf{grobcov}(F, \succ_{\mathbf{x}})$.
`Input:` $F \subset K[\mathbf{u}][\mathbf{x}]$: finite set of polynomials
generating the arbitrary parametric ideal I
`Output:` $G = \{[\text{lpp}_i, B_i, S_i] : i = 1, \ldots, s\}$: the Canonical
Gröbner Cover of I.
The basis elements are given in generic representation

begin
$H = \text{HOMOGENIZE}(F)$ using the extra variable x_0.
$G_0 = \text{GCOVER}(H)$
$G = \text{DE-HOMOGENIZE}$ the generic representation of the bases
and lpp in G_0 substituting $x_0 = 1$ inside and reduce them.
return(G)
end

Algorithm 5.3: GROBCOV

5.4 The SINGULAR Library and the `grobcov` Command

The `grobcov` command is the fundamental command of the SINGULAR library "grobcov.lib". We show here to some extent how to use it and give some simple examples.

There are many interesting applications in mathematics, engineering, physics, biology, etc. but we shall not develop all of them in the book. In Part II: Applications, we describe special kind of interesting applications in Geometry. Also in the Appendix, Sect. A.4, we give a list of ideals that the reader can test, and for which a comparison of efficiency between using KSW versus BUILDTREE is provided.

Example 5.14 (Casas-Alvero conjecture) The Casas-Alvero conjecture states that on a field of characteristic 0, if a polynomial of degree n in x has a common root with each of its $n - 1$ derivatives (not assumed to be the same), then it is of the form $P(x) = k(x + a)^n$, i.e., the common roots must all be the same. Recently Hans-Christian Graf von Bothmer, Oliver Labs, Joseph Schicho, and Christiaan van de Woestijne proved that it is true whenever n is a prime power or twice a prime power. Except for the fact that this shows that it is true for infinitely many n, the conjecture remains wide open. For fixed n, we can verify it by solving a system of equations. Considering monic polynomials ($a_n = 1$) and setting

$$f = \sum_{i=0}^{n} \binom{n}{i} a_i x^i,$$

the system of equations becomes

$$\left\{ \sum_{i=0}^{n} \binom{n}{i} a_i x_k^i,\ \sum_{i=k}^{n} \binom{n-k}{i-k} a_i x_k^{i-k} \ :\ k = 1 \ldots n-1 \right\}.$$

Using the SINGULAR grobcov command, the output is a list of segments, each consisting of three elements: (lpp_i, B_i, S_i), where lpp_i is the set of leading power products of the elements of the basis, B_i is the reduced Gröbner basis, and S_i represents the segment. By default, the basis is given in generic representation, and the segment in P-representation. Using the options we can obtain the C-representation ("rep",1) or even both ("rep",2). We can also obtain the set of lpp of the homogenized system ("showhom",1), and also the full representation of the basis ("ext",1) (see the help of the command grobcov in the library). Let us apply the command for $n = 5$. In all examples of the book, lines beginning with > are inputs, being the remaining lines output:

```
> LIB "grobcov.lib";
> ring R=(0,a0,a1,a2,a3,a4),(x1,x2,x3,x4),dp
> short=0;
> ideal S92=
  x1^5+(5*a4)*x1^4+(10*a3)*x1^3+(10*a2)*x1^2+(5*a1)*x1+(a0),
  x1^4+(4*a4)*x1^3+(6*a3)*x1^2+(4*a2)*x1+(a1),
  x2^5+(5*a4)*x2^4+(10*a3)*x2^3+(10*a2)*x2^2+(5*a1)*x2+(a0),
  x2^3+(3*a4)*x2^2+(3*a3)*x2+(a2),
  x3^5+(5*a4)*x3^4+(10*a3)*x3^3+(10*a2)*x3^2+(5*a1)*x3+(a0),
  x3^2+(2*a4)*x3+(a3),
  x4^5+(5*a4)*x4^4+(10*a3)*x4^3+(10*a2)*x4^2+(5*a1)*x4+(a0)
  x4+(a4);
 > grobcov(S92);
[1]:
   [1]:
      _[1]=1
   [2]:
      _[1]=1
   [3]:
      [1]:
         [1]:
            _[1]=0
         [2]:
            [1]:
               _[1]=(a3-a4^2)
               _[2]=(a2-a4^3)
               _[3]=(a1-a4^4)
               _[4]=(a0-a4^5)
[2]:
   [1]:
      _[1]=x4
      _[2]=x3^2
      _[3]=x2^3
      _[4]=x1^4
   [2]:
      _[1]=x4+(a4)
      _[2]=x3^2+(2*a4)*x3+(a4^2)
      _[3]=x2^3+(3*a4)*x2^2+(3*a4^2)*x2+(a4^3)
      _[4]=x1^4+(4*a4)*x1^3+(6*a4^2)*x1^2+(4*a4^3)*x1+(a4^4)
   [3]:
      [1]:
```

```
[1]:
   _[1]=(a3-a4^2)
   _[2]=(a2-a4^3)
   _[3]=(a1-a4^4)
   _[4]=(a0-a4^5)
[2]:
   [1]:
      _[1]=1
```

Observe the compact answer. The generic segment has lpp and basis equal to $\{1\}$, and states that there are no solutions except when $a_3 = a_4^2, a_2 = a_4^3, a_1 = a_4^4, a_0 = a_4^5$. The second segment determines the basis for the above values of $\mathbf{a}$, being the polynomials in the basis

$$x_4 + a_4, (x_3 + a_4)^2, (x_2 + a_4)^3, (x_1 + a_4)^4,$$

and so the polynomial f becomes

$$f = \sum_{i=1}^{5} \binom{5}{i} a_4^{n-i} x^i = (x_1 + a_4)^5$$

thus having a unique multiple root. This proves the conjecture for $n = 5$.

We can try to obtain the same result considering the elimination of $\mathbf{x}$ by computing the Gröbner basis of the system, considering $\mathbf{x}$ and $\mathbf{a}$ as variables setting $\mathbf{x} \succ \mathbf{a}$ and taking lex order. Nevertheless, the result is much more costly, the basis contains 56 polynomials, and moreover, obtaining the desired result from the polynomials in the $\mathbf{a}$ is not immediate, because the resulting ideal over $\mathbf{a}$ is not radical and contains 7 big polynomials.

Example 5.15 As a second example, we consider the generic conic

$$x^2 + 2cxy + by^2 + 2dx + 2ey + f = 0,$$

and discuss for which values of the parameters b, c, d, e, f the conic has singular points, and which are their nature. So, we have to add the derivatives with respect to the variables. To classify the conics by their singular points, we have to find the different types of solutions for the following system.

$$\begin{cases} x^2 + 2cxy + by^2 + 2dx + 2ey + f = 0 \\ 2x + 2cy + 2d = 0 \\ 2cx + 2by + 2e = 0. \end{cases}$$

Applying the `grobcov` command of the SINGULAR "grobcov.lib" library and using the option `"showhom",1` in order to obtain information about the behaviour

of the conic at infinity, which as we shall see, is significant for this problem, we have

```
> LIB "grobcov.lib";
> ring R=(0,b,c,d,e,f),(x,y),dp;
> ideal S=x^2+2*c*x*y+2*d*x+b*y^2+2*e*y+f, x+c*y+d, c*x+b*y+e;
> short=0;
> grobcov(S,"showhom",1);
[1]:
   [1]:
      _[1]=1
   [2]:
      _[1]=1
   [3]:
      [1]:
         [1]:
            _[1]=0
         [2]:
            [1]:
               _[1]=(b*d^2-b*f+c^2*f-2*c*d*e+e^2)
   [4]:
      1
[2]:
   [1]:
      _[1]=y
      _[2]=x
   [2]:
      _[1]=(c*d-e)*y+(d^2-f)
      _[2]=(c*d-e)*x+(c*f-d*e)
   [3]:
      [1]:
         [1]:
            _[1]=(b*d^2-b*f+c^2*f-2*c*d*e+e^2)
         [2]:
            [1]:
               _[1]=(c*d-e)
               _[2]=(b-c^2)
   [4]:
      y,x
[3]:
   [1]:
      _[1]=x
   [2]:
      _[1]=x+(c)*y+(d)
   [3]:
      [1]:
         [1]:
            _[1]=(d^2-f)
            _[2]=(c*f-d*e)
            _[3]=(c*d-e)
            _[4]=(b-c^2)
         [2]:
            [1]:
               _[1]=1
```

```
    [4]:
       x
[4]:
    [1]:
       _[1]=1
    [2]:
       _[1]=1
    [3]:
       [1]:
          [1]:
             _[1]=(c*d-e)
             _[2]=(b-c^2)
          [2]:
             [1]:
                _[1]=(d^2-f)
                _[2]=(c*f-d*e)
                _[3]=(c*d-e)
                _[4]=(b-c^2)
    [4]:
       @t,x
```

The Gröbner Cover determines four distinct segments, depending on the parameter values. We also obtain information about the lpp of the homogenized system (due to the option "showhom",1). The result can be summarized in the following table:

1. Generic case: lpp $= \{1\}$ Basis: $\{1\}$ Segment: $\mathbb{C}^5 \setminus \mathbf{V}(bd^2 - bf + c^2 f - 2cde + e^2)$ Description: Non-degenerate conic without singular points. lpph=1
2. lpp $= \{y, x\}$ Basis: $\left\{ y - \dfrac{d^2 - f}{cd - e},\ x - \dfrac{cf - de}{cd - e} \right\}$ Segment: $\mathbf{V}(bd^2 - bf + c^2 f - 2cde + e^2) \setminus \mathbf{V}(cd - e, b - c^2)$ Description: A unique singular point: two intersecting lines. lpph=y,x
3. lpp $= \{x\}$ Basis: $\{x + cy + d\}$ Segment: $\mathbf{V}(d^2 - f, cf - de, cd - e, b - c^2)$ Description: Singular double line. lpph=x

4. lpp = {1}
   Basis: {1}.
   Segment: $= \mathbf{V}(cd - e, b - c^2) \setminus \mathbf{V}(d^2 - f, cf - de, cd - e, b - c^2)$
   Description: Special case without singular points: two parallel lines.
   lpph=@t,x

The above table gives a precise and complete discussion, classifying the conics by their singular points into 4 segments given as the difference between two varieties. The default option determines the P-representation (PREP) of the segment, which in this example is almost the same as the C-representation, because the segments have only one single component with one single hole. (Using option `"rep",1` we would obtain the CREP). The lpph represents the set of lpp of the homogenized ideal that uses the homogenizing variable @t.

Even if segments 1 and 4 have the same basis {1}, showing that the system has no solution, they correspond to two different types of conics. In segment 1 there is no singular point even considering the points at infinity, as shown by lpph=1, so that the homogenized ideal does not have any solution either. Thus the conic has no singular point at infinity.

For segment 4 the basis is also 1, showing that the conic has no singular points. But lpph=@t,x shows that at infinity it has a solution. Substituting the values of $b = c^2$ and $e = cd$, determined by the segment, the variety becomes

$$\begin{aligned} V &= \mathbf{V}(x^2 + 2cxy + c^2y^2 + 2dx + 2cdy + f) \\ &= \mathbf{V}\left((x + cy + d + \sqrt{d^2 - f})(x + cy + d - \sqrt{d^2 - f})\right) \\ &= \mathbf{V}\left(x + cy + d + \sqrt{d^2 - f}\right) \cup \mathbf{V}\left(x + cy + d - \sqrt{d^2 - f}\right), \end{aligned}$$

showing that, for $d^2 - f \geq 0$, it consists of two real parallel lines. The two lines intersect at infinity in the projective space, thus the conic has a singular point at infinity. This is the reason why segments 1 and 4 are separated in the Gröbner Cover, for which the different segments correspond to different lpph. (The segments are lpph-segments).

Remark 5.16 (Generic versus full representation of the bases) The `grobcov` command uses as the default option the generic representation of the bases. The reasons for this instead of using the full representation are the following:

(1) In the majority of the examples studied, the full representation is unnecessary because the generic representation is already a full one.
(2) The computing time when using option `"ext",1` for obtaining the extended representation of the bases, is in many cases much longer than with the default option, and in some special complex cases, the default option finishes in a reasonable time, whereas the full option does not finish at all.

(3) There is a counterpart. Namely, in general, using option (`"ext",1`) introduces simplifications in the bases, which in some problems can be important. For example in Chap. 6, Sect. 6.1, we give an interesting application, where the bases have been obtained with option (`"ext",1`), the output being much simpler than with the default option, and the time is reasonable. But even so, we recommend first using the default option, and only trying the extended option if the result is not sufficiently simple or needs proper extension.

Example 5.17 We now give a special problem especially designed to give advantage to the full representation. The computing times in both options are similar (10% higher for the full representation).

Consider the followings ring and system:

```
> ring R=(0,u1,u2,u3,u4,u5,u6),(x,y,z),dp
> ideal S41=(u1)*x+(u2)*y+(u3)*z, (u4)*x+(u5)*y+(u6)*z.
```

We leave the computations to the reader, and we emphasize here only the differences between using (or not) the option `"ext",1`.

The times are 480 milliseconds using the default option and 800 milliseconds using option `"ext",1`. Thus there is not much time consumed using the option in this example.

Now we provide the bases of some segments in the generic and in the full representation (the reader can do the computations and verify the assertions).

	Generic representation	Full representation
1.	$(u_1u_5 - u_2u_4)y + (u_1u_6 - u_3u_4)z$, $(u_1u_5 - u_2u_4)x + (-u_2u_6u_3u_5)z$	$(u_1u_5 - u_2u_4)y + (u_1u_6 - u_3u_4)z$, $(u_1u_5 - u_2u_4)x + (-u_2u_6u_3u_5)z$
2.	z, $u_4x + u_5y$	z, $\{u_4x + u_5y,\ u_1x + u_2y\}$
3.	$u_4x + u_5y + u_6z$	$\{u_4x + u_5y + u_6z$, $u_1x + u_2y + u_3z\}$
4.	$u_5y + u_6z$	$\{u_5y + u_6z, u_2y + u_3z\}$
5.	z	z
6.	0	0
7.	z, x	z, x

We observe that in this specially designed example, in segments 2, 3 and 4, option (`"ext",1`) for obtaining the full representation is really useful, as some of the I-regular functions need to be represented by more than 1 polynomial. But this is strong enough exceptional, and in the majority of cases this option only introduces simplifications. This is the reason for using as default the generic representation for the bases.

We recommend using the default option first, and only when the results make it convenient to repeat the computation using the options.

Part II
Applications

Chapter 6
Automatic Deduction of Geometric Theorems



A first natural and interesting application of the Gröbner Cover is to Automatic Deduction of Geometric Theorems (ADGT).

The classical approach to ADGT stems from [71, 82], and has been further extended in [27] and, particularly, in [51, 52, 72]. We refer the interested reader to the above mentioned papers for details and for references to previous and related work, as we do not discuss these approaches here.

The classical approaches to ADGT problems can be improved by using the Comprehensive Gröbner Systems (CGS) and particularly the canonical Gröbner Cover. In [60] Tomás Recio and the author already introduced and developed the foundations for applying algorithmic methods to discuss parametric polynomial systems of equations to the field of ADGT using the old algorithm MCCGS [56] there.

The analysis using the Gröbner Cover was presented in [61] to obtain the generalized Steiner-Lehmus Theorem. We recover it in Sect. 6.1, to show the power of GC for ADGT problems.

Then, in Sect. 6.2, we give a new automatic algorithm ADGT which given a proposition, obtains automatically supplementary conditions for converting the given proposition into a theorem.

In Sect. 6.3, we apply the new command `ADGT` to a classical problem, and consider different related formulations for the proposition to become a theorem, including negative hypotheses and theses.

Finally, in Sect. 6.4, we apply the new command `ADGT` to Automatic Demonstration of Theorems and prove automatically the classical Euler-Feuerbach Nine-Points Circle Theorem.

© Springer Nature Switzerland AG 2018

A. Montes, *The Gröbner Cover*, Algorithms and Computation in Mathematics 27, https://doi.org/10.1007/978-3-030-03904-2_6

6.1 Steiner-Lehmus Theorem

As a first example of a problem where we are interested in determining supplementary conditions for a proposition to become true, we consider the generalization of the classical Steiner-Lehmus Theorem. The classical Steiner-Lehmus Theorem was proposed by the well known nineeteenth century geometer Steiner to Sturm, and it was proved by Lehmus for the first time in 1848. It states as follows:

Theorem 6.1 (Classical Steiner-Lehmus) *The inner bisectors of angles A and B of a triangle ABC ($\alpha = \beta$ and $\gamma = \delta$) are of equal length ($\overline{AA'} = \overline{BB'}$) if and only if the triangle is isosceles with $\overline{AC} = \overline{BC}$ (see Fig. 6.1).*

This is an issue which has attracted considerable interest over the years, and we refer to [76] for an extensive collection of references and comments on this classical statement and its proof.

More recently, its generalization regarding internal as well as external angle bisectors, has been approached through automatic tools, cf. [7, 11, 27, 52, 53, 82, 86]. All of these results were obtained by using ideal-theoretical elimination methods, which do not allow for a fine grain analysis of the involved situation, in particular, with respect to the different behavior of some real-geometry objects, indistinguishable from a complex-geometry point of view, such as the internal/external bisectors at a vertex.

The Gröbner Cover approach is particularly well suited in this context, bringing out, in its output, the possibility of a detailed case analysis that significantly extends our knowledge of the generalized Steiner-Lehmus Theorem [61]. Let us apply `grobcov` to obtain its generalization.

In order to automatically discover the Steiner-Lehmus theorem using the Gröbner Cover, let ABC be the given triangle and consider the bisectors at angle A. To construct the bisectors (see Fig. 6.2), we consider the circle with center A and radius $\overline{AC}$. There are two intersection points P and P' of the circle with side AB, and thus two middle points Q and Q' of $\overline{CP}$ and $\overline{CP'}$ determining the bisectors $\overline{AM}$ and $\overline{AM'}$ whose length we are interested in. So, if we only use the equations determining

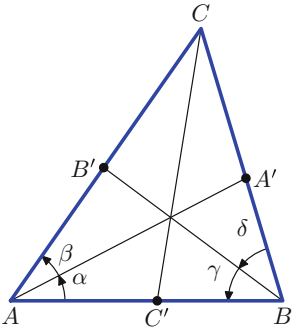


Fig. 6.1 Triangle and inner bisectors

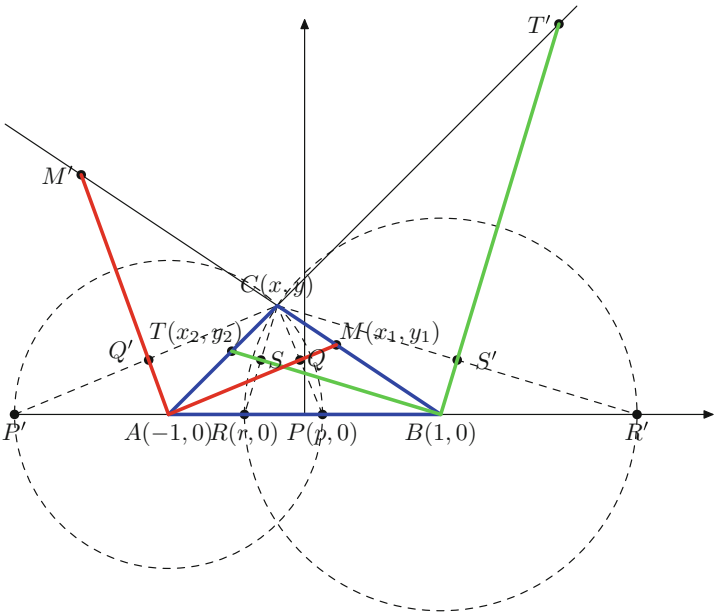


Fig. 6.2 The bisectors of A and B are equal

M and M', we would not distinguish between the inner and the outer bisector. It is precisely this difficulty that will allow us to generalize the theorem.

Without loss of generality, we set coordinates $A(-1, 0)$, $B(1, 0)$, $C(x, y)$. Then let $(p, 0)$ be the intersection of the circle centred at A passing through C, (i.e., points P or P'), and let (x_1, y_1) stand for the feet of the bisectors, (i.e., points M or M'). The equation of the circle is $(x + 1)^2 + y^2 - (p + 1)^2$. The middle point between $(0, p)$ and C is $Q = \left(\frac{x+p}{2}, \frac{y}{2}\right)$. Point (x_1, y_1) is on the line AQ and so $x_1 y - 2y_1 + y - y_1(x + p)$ shows that (x_1, y_1) is on the bisector line. Finally, the equation stating that (x_1, y_1) lies on side BC is $xy_1 + y - y_1 - x_1 y$. Thus, the equations determining (x_1, y_1) in terms of (x, y) are

$$\begin{cases} (x + 1)^2 + y^2 - (p + 1)^2, \\ x_1 y - 2y_1 + y - y_1(x + p), \\ xy_1 + y - y_1 - x_1 y. \end{cases} \tag{6.1}$$

Notice that the sign of p discriminates which bisector of A is of concern. If a solution of our problem has $p > -1$ it will correspond to the inner bisector of A, whereas a solution with $p < -1$ will correspond to the outer bisector of A. But the sign is not algebraically (from the complex point of view) relevant, so both points M and M' are solutions to the same set of equations. The length of the bisector is $l_A^2 = (x_1 + 1)^2 + y_1^2$.

Consider now the bisectors of B (see Fig. 6.2). Denoting $(r, 0)$ the intersection point of the circle centred in B with radius $\overline{BC}$ (points R or R') and (x_2, y_2) the coordinates of the foot of the bisector of B (points T or T'), the corresponding equations for them are

$$\begin{cases} (x-1)^2 + y^2 - (r-1)^2, \\ x_2 y + 2y_2 - y - y_2(x+r), \\ x y_2 - y + y_2 - x_2 y. \end{cases} \tag{6.2}$$

In this case, a discriminant between the inner and outer bisectors of B is $1 - r$. A solution with $1 - r > 0$ will correspond to the inner bisector, whereas a solution with $1 - r < 0$ will correspond to the outer bisector. The length of the bisector of the angle B is $l_B^2 = (x_2 - 1)^2 + y_2^2$. Equations (6.1) and (6.2) correspond to the hypothesis H of the proposition.

The thesis T of our proposition is that the length of one bisector of the angle A is equal to the length of one bisector of angle B. In terms of equations we have

$$T = (x_1 + 1)^2 + y_1^2 - (x_2 - 1)^2 - y_2^2.$$

Now, using the set of all the above equations, we turn to searching for the necessary and also sufficient conditions to convert our Proposition 6.1

$$H \Rightarrow T$$

into a theorem.

We can moreover impose that the given triangle be non-degenerate. For this it suffices to add the condition $y \neq 0$.

Remark that in Proposition 6.1 we are not distinguishing between which inner or outer bisector is concerned. It can happen that the two equal bisectors are the two inner bisectors ($i_A = i_B$), or the two outer bisectors ($e_A = e_B$), or one inner and one outer bisector (cases $i_A = e_B$ and $e_A = i_B$). There are, thus, four possibilities.

In order to compute the Gröbner Cover, we include the set of Eqs. (6.1) and (6.2) of the hypothesis H, and the thesis T. Thus the complete set of equations $H + T$ is

$$\begin{cases} (x+1)^2 + y^2 - (p+1)^2, \ x_1 y - 2y_1 + y - y_1(x+p), \\ x y_1 + y - y_1 - x_1 y, \ (x-1)^2 + y^2 - (r-1)^2, \\ x_2 y + 2y_2 - y - y_2(x+r), \ x y_2 - y + y_2 - x_2 y, \\ (x_1+1)^2 + y_1^2 - (x_2-1)^2 - y_2^2. \end{cases} \tag{6.3}$$

Now, we take (x, y), the free position of C, as parameters and x_1, y_1, x_2, y_2, p, r as variables. The reason is that we are interested in analyzing the character of the solutions of the system (6.3) depending on point $C(x, y)$. These solutions will correspond to one bisector of A being equal to one bisector of B, but the conditions

over x, y will not distinguish between internal and external bisectors. When $1+p$ is positive, the bisector of A will be internal and it will be external if $1+p$ is negative. The same happens considering the sign of $1-r$ for the bisector of B.

The GC algorithm is used here taking the lex$(p, r, x_2, y_2, x_1, y_1)$ order for the variables. The call in SINGULAR is

```
> LIB "grobcov.lib";
> ring R=(0,x,y),(p,r,x2,y2,x1,y1),lp;
> ideal S93=(x+1)^2+y^2-(p+1)^2, x1*y-2*y1+y-y1*(x+p),
          x*y1+y-y1-x1*y, (x-1)^2+y^2-(r-1)^2,
          x2*y+2*y2-y-y2*(x+r), x*y2-y+y2-x2*y,
          (x1+1)^2+y1^2-(x2-1)^2-y2^2;
> short=0;
> grobcov(S93,"nonnull",y,"ext",1,"showhom",1);
```

The resulting GC contains 6 segments (verify the computation), and the following irreducible curves $\mathcal{C}_1, \mathcal{C}_2, \mathcal{C}_3$ and varieties (see Table 6.1) appear:

$$\begin{aligned}\mathcal{C}_1 = \mathbf{V}(&8x^{10} + 41x^8y^2 - 56x^8 + 84x^6y^4 - 164x^6y^2 + 144x^6 + 86x^4y^6\\ &- 148x^4y^4 + 86x^4y^2 - 176x^4 + 44x^2y^8 - 28x^2y^6 - 148x^2y^4\\ &+ 156x^2y^2 + 104x^2 + 9y^10 + 12y^8 - 74y^6 - 172y^4 + 137y^2 - 24)\\ \mathcal{C}_2 = \mathbf{V}(&x).\\ \mathcal{C}_3 = \mathbf{V}(&y).\end{aligned}$$

We use option (`"nonnull",y`), to avoid degenerate triangles. We also use option (`"ext",1`) for obtaining the full representation of the bases. But in fact, the true reason for using it is that, as in many cases, its use simplifies the expressions of the bases. Finally we use option (`"showhom",1`) in order to obtain the set of lpp's of the homogeneous system. The curves are represented in Fig. 6.3. Special real points are either singular points of $\mathcal{C}_1$ or intersection points between the three curves:

(1) V_0, V_1, V_2, V_3 correspond to degenerate triangles, and their points P_0, P_1, P_2, P_{31} and P_{32} appear only as holes of the interesting segments.
(2) V_4 and V_5 are singular points of $\mathcal{C}_1$, and their real points are P_{41}, P_{42}, P_{51}, P_{52}. The varieties also contain complex points in which we are not interested.
(3) V_6 and V_7 correspond to intersection points of $\mathcal{C}_1$ and $\mathcal{C}_2$. They contain the real points P_{61}, P_{62}, P_{71}, P_{72}.
(4) Finally V_8 does not contain any real point.

Table 6.1 Real special points of the Steiner-Lehmus output

Varieties	Real points
$V_0 = \mathbf{V}(y, x)$	$P_0 = (0, 0)$
$V_1 = \mathbf{V}(y, x-1)$	$P_1 = (1, 0)$
$V_2 = \mathbf{V}(y, x+1)$	$P_2 = (-1, 0)$
$V_3 = \mathbf{V}(y, x^2-3)$	$P_{31} = \left(-\sqrt{3}, 0\right)$
	$= (-1.732050808, 0)$
	$P_{32} = \left(\sqrt{3}, 0\right)$
	$= (1.732050808, 0)$
$V_4 = \mathbf{V}(y^4 + 44y^2 - 16,$	$P_{41} \quad \left(3 - 2\sqrt{5}, -\sqrt{-22 + 10\sqrt{5}}\right)$
$5x + y^2 + 7)$	$(-1.472135954, -.6005662078)$
	$P_{42} = \left(3 - 2\sqrt{5}, \sqrt{-22 + 10\sqrt{5}}\right)$
	$(-1.472135954, -.6005662078)$
$V_5 = \mathbf{V}(y^4 + 44y^2 - 16,$	$P_{51} = \left(-3 + 2\sqrt{5}, -\sqrt{-22 + 10\sqrt{5}}\right)$
$5x - y^2 - 7)$	$= (1.472135954, -.6005662078)$
	$P_{52} = \left(-3 + 2\sqrt{5}, \sqrt{-22 + 10\sqrt{5}}\right)$
	$= (1.472135954, .6005662078)$
$V_6 = \mathbf{V}(y^2 - 3, x)$	$P_{61} = \left(0, -\sqrt{3}\right)$
	$= (0, -1.732050808)$
	$P_{62} = \left(0, \sqrt{3}\right)$
	$= (0, 1.732050808)$
$V_7 = \mathbf{V}(3y^2 - 1, x)$	$P_{71} = \left(0, -\dfrac{\sqrt{3}}{3}\right)$
	$= (0, -.5773502693)$
	$P_{72} = \left(0, \dfrac{\sqrt{3}}{3}\right)$
	$= (0, .5773502693)$
$V_8 = \mathbf{V}(y^4 + 5y^2 + 8, x)$	

Let us now describe the output of the Gröbner Cover. We obtain the following description with 6 segments:

1. Segment with lpp = lpph = {1} Generic segment
   No solution on the segment
   Segment: $\mathbb{C}^2 \setminus (\mathcal{C}_1 \cup \mathcal{C}_2 \cup \mathcal{C}_3)$
   Description: The whole parameter space except for the curves $(\mathcal{C}_1 \cup \mathcal{C}_2 \cup \mathcal{C}_3)$.
   Basis: $B_1 = \{1\}$.

2. Segment with lpp = lpph = $\{y_1, x_1, y_2, x_2, r, p\}$
Single solution on each point of the segment
Segment: $\mathcal{C}_1 \setminus (V_1 \cup V_2 \cup V_3 \cup V_4 \cup V_5 \cup V_6 \cup V_7 \cup V_8)$
Description of the real points: The curve $\mathcal{C}_1$ except for the points P_1, P_2, P_{31}, P_{32}, P_{41}, P_{42}, P_{51}, P_{52}, P_{61}, P_{62},P_{71}, P_{72}.
Basis B_2:

$$
\begin{aligned}
B_2 = \{&(x^5 + 3x^4 + 2x^3y^2 - 2x^3 + 6x^2y^2 - 22x^2 + xy^4 + 2xy^2 + 17x + 3y^4 - 10y^2 + 3)y_1 \\
&+ (6x^4y + 12x^2y^3 + 4x^2y - 32xy + 6y^5 + 28y^3 - 10y), \\
&(x^5 + 3x^4 + 2x^3y^2 - 2x^3 + 6x^2y^2 - 22x^2 + xy^4 + 2xy^2 + 17x + 3y^4 - 10y^2 + 3)x_1 \\
&+ (5x^5 - 9x^4 + 10x^3y^2 + 6x^3 - 18x^2y^2 - 14x^2 + 5xy^4 + 26xy^2 + 5x - 9y^4 - 18y^2 + 7), \\
&(x^5 - 3x^4 + 2x^3y^2 - 2x^3 - 6x^2y^2 + 22x^2 + xy^4 + 2xy^2 + 17x - 3y^4 + 10y^2 - 3)y_2 \\
&+ (-6x^4y - 12x^2y^3 - 4x^2y - 32xy - 6y^5 - 28y^3 + 10y), \\
&(x^5 - 3x^4 + 2x^3y^2 - 2x^3 - 6x^2y^2 + 22x^2 + xy^4 + 2xy^2 + 17x - 3y^4 + 10y^2 - 3)x_2 \\
&+ (-5x^5 - 9x^4 - 10x^3y^2 - 6x^3 - 18x^2y^2 - 14x^2 - 5xy^4 - 26xy^2 - 5x - 9y^4 - 18y^2 + 7), \\
&(3x^4 + 6x^2y^2 + 2x^2 + 16x + 3y^4 + 14y^2 - 5)r \\
&+ (x^5 - 12x^4 + 2x^3y^2 - 2x^3 - 24x^2y^2 + 16x^2 + xy^4 + 2xy^2 - 31x - 12y^4 - 32y^2 + 12), \\
&(3x^4 + 6x^2y^2 + 2x^2 - 16x + 3y^4 + 14y^2 - 5)p \\
&+ (x^5 + 12x^4 + 2x^3y^2 - 2x^3 + 24x^2y^2 - 16x^2 + xy^4 + 2xy^2 - 31x + 12y^4 + 32y^2 - 12)\}.
\end{aligned}
$$

3. Segment with lpp = lpph = $\{y_1^2, x_1, y_2, x_2, r, p\}$
2 solutions on each point of the segment.
Segment: $(\mathcal{C}_2 \setminus (V_0 \cup V_6 \cup V_7)) \cup V_5$
Description: ($\mathcal{C}_2$ minus intersecting points with $\mathcal{C}_1$ and $\mathcal{C}_2$) plus V_5
Basis:

$$
\begin{aligned}
B_3 = \{&(87xy^2 - 91x - 15y^4 + 90y^2 - 135)y_1^2 + (-16xy^3 - 632xy - 120y^3 + 360y)y_1 \\
&+ (-36xy^2 + 128x + 60y^4 - 180y^2), \\
&(y)x_1 + (-x + 1)y_1 + (-y), \\
&(25x - 55)y_2 + (-xy^2 - 37x + 55)y_1 + (20xy), \\
&(55y)x_2 + (xy^2 + 37x - 55)y_1 + (-4xy^3 - 148xy + 55y), \\
&(14xy + 22y)r + (2xy^2 + 19x + 11y^2 - 33)y_1 + (-66xy + 22y), \\
&(2xy - 2y)p + (x + y^2 - 3)y1 + (-2xy + 2y)\}.
\end{aligned}
$$

4. Segment with lpp = $\{y_1, x_1, y_2, x_2, r, p\}$.
and with lpph = $\{y_1t, x_1, y_2t, y_2^2, x_2, r, p\}$.
Single solution on each point of the segment.
Segment: V_6
Description: Vertex of equilateral triangle
Basis:
$B_4 = \{2y_1 + (-y), 2x_1 - 1, 2y_2 + (-y), 2x_2 + 1, r + 1, p - 1\}$.

5. Segment with lpp = lpph = $\{y_1^2, x_1, y_2^2, x_2, r, p\}$.
4 solutions on both points of the segment.
Segment: V_7
Description: Vertex of an isosceles triangle with acute angles of 30°.
Basis:

$$B_5 = \{2y_1^2 + (-6y)y_1 + 1, x_1 + (3y)y_1 - 1, 2y_2^2 + (-6y)y_2 + 1, x_2 + (-3y)y_2 + 1,$$
$$r + (-4y)y_2 + 1, p + (4y)y_1 - 1\}.$$

6. Segment with lpp = lpph = $\{y_1, x_1, y_2^2, x_2, r, p\}$.
2 solutions on both points of the segment.
Segment: V_4
Description: Singular points of $\mathcal{C}_2$.
Basis:

$$B_6 = \{(5y)y_1 + (3y^2 - 4), x_1 + 3, (5y)y_2^2 + (-6y^2 + 8)y_2 + (3y^3 - 4y),$$
$$(5y)x_2 + (y^2 + 2)y2 + (5y), (10y)r + (y^2 + 12)y_2 + (10y), 10p + (-y^2 + 18)\}.$$

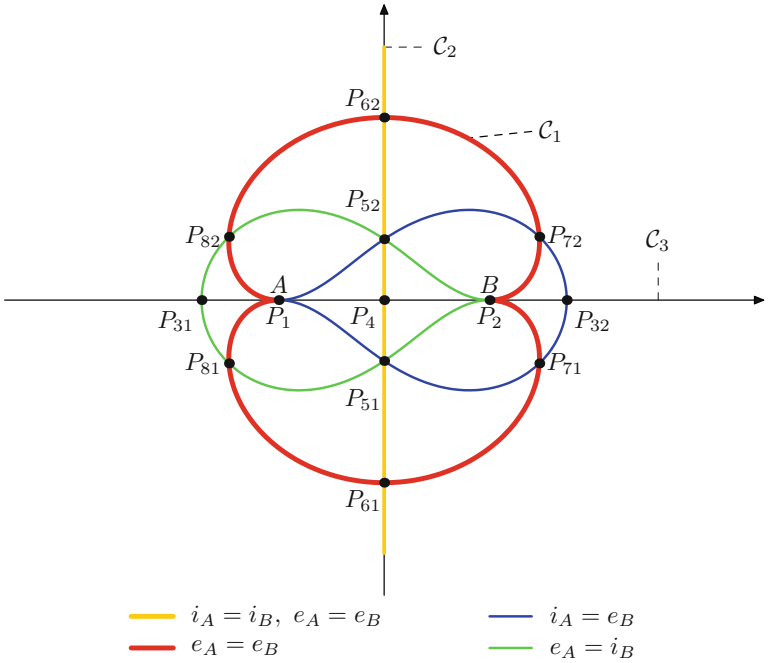


Fig. 6.3 Generalized Steiner-Lehmus locus. Curves $\mathcal{C}_1, \mathcal{C}_2, \mathcal{C}_3$ and special points

6.1.1 Discussion and Formulation of the Generalized Theorem

– Segment 1 of the Gröbner Cover proves that the thesis of the equality of bisectors at A and B does not hold in general, except for triangles with vertex C placed on the three curves $\mathcal{C}_1$, $\mathcal{C}_2$ and $\mathcal{C}_3$ (see Fig. 6.3). For the points inside these curves, system (6.3) always provides some solution. We have already discarded $\mathcal{C}_3$, because it represents degenerate triangles. Let us discuss which different types of solutions exist on $\mathcal{C}_1$ and $\mathcal{C}_2$.

– Segment 2: This segment contains all the points of the curve $\mathcal{C}_1$ except for the special points P_1, P_2, P_{31}, P_{32}, P_{41}, P_{42}, P_{51}, P_{52}, P_{61}, P_{62},P_{71}, P_{72}. There exists a unique solution on each point of segment 2, and so only one equality between one bisector of A and one bisector of B can happen. The type of solution cannot change, by continuity, on the curve except when the curve reaches a special point. The reason is that in the changing points one needs to have equality of more bisectors and this can only occur in some special segment. So we only need to determine the color (i.e., the type of solution) – in Fig. 2.2 – of a single point of the curve between special points.

We choose two vertical lines, $x = -1$ and $x = 1$, which intersect almost all sectors of the curve, and determine their intersection with the curve. To do this we substitute the value of x in $\mathcal{C}_1$ and use the command `fsolve` of Maple to determine all the intersections. These form the first column of Table 6.2.

Then, substituting the values of (x, y) in the last two equations of basis B_2, we isolate the pair $(1 + p, 1 - r)$ for each of these points, thus determining which bisectors are equal at the corresponding point. This allows Table 6.2 to be completed and the application of the corresponding colors to the curve $\mathcal{C}_1$.

We set red if $e_A = e_B$, blue if $i_A = e_B$, and green if $e_A = i_B$. The possibility $i_A = i_B$ never occurs on $\mathcal{C}_1$. (Remember the classical theorem).

– Segment 3: It has two components:

(1) Component $(\mathcal{C}_2 \setminus (V_0 \cup V_6 \cup V_7))$.

For $x = 0$, vertex C is on the bisector of side AB (so that the triangle ABC is isosceles), leaving out the points P_{61}, P_{62}, P_{71}, P_{72}, P_0. Specializing the basis on

Table 6.2 Real points on $\mathcal{C}_1$ and its color

Point		In branch	$(1 + p, 1 - r)$	Bisectors
$(-1,$	$1.402734397)$	P_{42}–P_{62}	$(-1.402734397, -2.442880224)$	$e_A = e_B$
$(-1,$	$.8380575637)$	P_{42}–P_{72}	$(-.838057564, 2.168488052)$	$e_A = i_B$
$(-1,$	$-.8380575637)$	P_{41}–P_{71}	$(-.838057564, 2.168488052)$	$e_A = i_B$
$(-1,$	$-1.402734397)$	P_{41}–P_{61}	$(-1.402734397, -2.442880224)$	$e_A = e_B$
$(1,$	$1.402734397)$	P_{52}–P_{62}	$(-2.442880224, -1.402734397)$	$e_A = e_B$
$(1,$	$.8380575637)$	P_{52}–P_{72}	$(2.168488052, -0.838057564)$	$i_A = e_B$
$(1,$	$-.8380575637)$	P_{51}–P_{71}	$(2.168488052, -0.838057564)$	$i_A = e_B$
$(1,$	$-1.402734397)$	P_{51}–P_{61}	$(-2.442880224, -1.402734397)$	$e_A = e_B$

this branch (setting $x = 0$) results in

$$B_{31} = \{(-3y^4 + 18y^2 - 27)y_1^2 + (-24y^3 + 72y)y_1 + (12y^4 - 36y^2),$$
$$(y)x_1 + y_1 + (-y), -y_2 + y_1, (y)x_2 - y_1 + (y),$$
$$(2y)r + (y^2 - 3)y_1 + (2y), (-2y)p + (y^2 - 3)y_1 + (2y)\}.$$

Adding together the last two equations, we obtain $p + r = 0$, as $y \neq 0$. Computing the standard basis of B_{31} in lex$(x_1, y_1, x_2, y_2, r, p, x, y)$ order, from its last polynomial we obtain $p^2 + 2p - y^2 = 0$. From both it results:

$$1 + p = 1 - r = \pm\sqrt{1 + y^2}.$$

Thus there are two solutions: In one case $1 + p = 1 - r > 0$, so that the equality of both internal bisectors (i.e., $i_A = i_B$) holds, and in the other solution it holds that $1 + p = 1 - r < 0$, corresponding to the case $e_A = e_B$.

Thus, on this part of the segment the two inner bisectors as well as the two outer ones are equal. This corresponds to the classical Steiner-Lehmus Theorem, enlarging it with the coincidence of the outer bisectors as well.

(2) Component V_5: Contains the pair of points P_{51} and P_{52} (also two solutions). We have

Point	$(1 + p, 1 - r)$	Bisectors
P_{51}, P_{52}	(−0.7639319052, −2.544039300)	$e_A = e_B$
	(−0.7639319052, 2.544039300)	$e_A = i_B$

– Segment 4: Component V_6: Contains the two real points P_{61}, P_{62}, and for each there is a unique solution corresponding to $i_A = i_B$ (which can be checked by actually solving the system associated to segment 6). Nevertheless, regarding the lpph's of the homogenized system, there are two solutions to the homogeneous system. This is because the solution with $e_A = e_B$ of the homogeneous system corresponds to infinite values that disappear in the inhomogeneous system.

Even if the lpp of this segment 6 is equal to the lpp of segment 2, the lpph's are different and so, they appear in different segments. We will see below that in the curve described in segment 2, in the neighborhood of P_{61} and P_{62}, we have the equality $e_A = e_B$ instead of $i_A = i_B$.

Point	$(1 + p, 1 - r)$	Bisectors
P_{61}, P_{62}	(2,2)	$i_A = i_B$

– Segment 5: Component V_7: Contains the two real points P_{71}, P_{72}, and for each there are four solutions, as is clear by observing the values of $1 + p$ and $1 - r$ in each of the solutions. All four bisectors are equal.

Point	$(1 + p, 1 - r)$	Bisectors
P_{71}, P_{72}	(0.5773502693, 0.5773502693)	$i_A = i_B$
	(1.15471938, −1.15471938)	$i_A = e_B$
	(−1.15471938, 1.15471938)	$e_A = i_B$
	(−1.15471938, −1.15471938)	$e_A = e_B$

– Segment 6: Component V_4: Contains the real points P_{41} and P_{42}.

Point	$(1 + p, 1 - r)$	Bisectors
P_{41}, P_{42}	(−2.54403930, −0.763931952)	$e_A = e_B$
	(2.54403930, −0.763931952)	$i_A = e_B$

Obviously all of the properties concerning the shape of the curve and the special points and colors are easily transformed in scaling the curve changing the distance $\overline{AB}$.

In summary, we have thus proved the following

Theorem 6.2 (Generalized Steiner-Lehmus) *Let ABC be a non-degenerate triangle and i_A, e_A, i_B, e_B, the lengths of the inner and outer bisectors of the angles A and B. Then, considering the conditions for the* equality of some bisector of A and some bisector of B, *the following excluding situations occur:*

(i) ABC *is* equilateral *and then $i_A = i_B$, whereas e_A and e_B become infinite,* (P_{61}, P_{62}).

(ii) *Point C is in the* center of an equilateral triangle, *and then $i_A = i_B = e_A = e_B$,* (P_{71}, P_{72}).

(iii) *The triangle is* isosceles, *but not of the special form of cases* (i) *or* (ii), *and then $i_A = i_B \neq e_A = e_B$,* (*ordinary theorem*).

(iv) $\dfrac{\overline{AC}}{\overline{AB}} = \sqrt{\dfrac{1+\sqrt{5}}{2}}$, $\dfrac{\overline{BC}}{\overline{AB}} = \dfrac{3-\sqrt{5}}{2}$, *and then $e_A = e_B = i_A$,* (P_{51}, P_{52}).

(v) $\dfrac{\overline{AC}}{\overline{AB}} = \dfrac{3-\sqrt{5}}{2}$, $\dfrac{\overline{BC}}{\overline{AB}} = \sqrt{\dfrac{1+\sqrt{5}}{2}}$, *and then $e_A = e_B = i_B$,* (P_{41}, P_{42}).

(vi) C lies in the curve of degree 10 *relative to points A and B (case of curve $\mathcal{C}_1$) passing through all of the special points above but is none of these points, and then only one of the following possibilities happen: either $e_A = e_B$ or $i_A = e_B$ or $e_A = i_B$, depending on the piece of the curve* (*see Fig.* 6.3, *the color representing which of these situations occur*).

(vii) None of the above cases *occur, and then no bisector of A is equal to any bisector of B.*

Remark 6.3 (Concluding) As we have seen, the GC is very useful for automatic discovery of geometric theorems, and can highly improve the results. The very relevant case distinction of Sect. 6.1.1 (describing the pieces of the involved curves where equality holds for internal/internal, internal/external or external/external bisectors), as graphically displayed in Fig. 6.3, is a natural output when using the Gröbner cover method.

6.2 Automatic Discovery Using the Gröbner Cover

In Sect. 6.1 we have already shown all the details that can be obtained, using GC for studying a proposition and converting it into a theorem.

We are now concerned with an automatic algorithm for obtaining the *supplementary conditions* for a *geometric proposition* to become *a theorem*, not focusing on the details, but only on the resulting theorem.

Our starting point is a geometric statement of the form

$$H \Rightarrow T \tag{6.4}$$

such as: *Given a triangle, if we construct the orthic triangle formed by the feet of the heights with respect to the vertices then ... it is isosceles, where H stands for the equations describing the construction (orthic triangle) and T describes the desired property (equality of two sides, etc.).*

We consider a class of geometric statements like the previous one, in which there is given some initial figure (a triangle, a rectangle, two circles and a line, etc.), which, when setting coordinates and distances, has some free variables, say $\mathbf{u} = u_1, \ldots, u_n$, constrained possibly to vary on a hyper-surface. The proposition considers then a construction depending on the figure (heights, bisectors, parallel lines, etc.) generating new variables $\mathbf{v} = v_1, \ldots, v_m$ depending on the $\mathbf{u}$'s. The initial figure and the construction are the hypothesis of the proposition, that can be translated into equations H in the free variables $\mathbf{u}$ and the dependent variables $\mathbf{v}$. Finally the proposition affirms some thesis which can be translated into a new equation or set of equations T in $\mathbf{u}$, $\mathbf{v}$. We are interested in obtaining *supplementary conditions* for the statement $H \Rightarrow T$ to be true.

This Automatic Deduction of Geometric Theorem (ADGT) problem can be formulated taking the free variables $\mathbf{u}$ as parameters and the dependent variables $\mathbf{v}$ as variables of the ring $R = \mathbb{C}[\mathbf{u}][\mathbf{v}]$, and asking which *restrictions on the parameters* $\mathbf{u}$ ensure that the set of equations given by the ideal $H + T$ has solutions. It is a special kind of geometric locus problem and can be solved using the Gröbner Cover. In Chap. 7 general loci problems will be considered.

With this objective, we compute

$$G = \texttt{grobcov}(H + T). \tag{6.5}$$

Let $J = \{i_1, \ldots, i_t\}$ be the set of indices of the segments S_i of G with lpp $\neq \{1\}$, i.e., the indices of the segments where the system $H + T$ has solutions. Then, the necessary and sufficient complementary hypothesis for statement (6.4) to hold is the set of the parameter values in $S = \bigcup_{i \in J} S_i$. We can read the conditions directly in G. In that case, we can obtain further information, and distinguish between points with a single solution, with a finite number of solutions or with infinitely many solutions. We have already considered these differences in Sect. 6.1. But if we only want to determine all the values of the parameters for which the system has solutions, we can automatize and extend the procedure to more complex propositions.

We can summarize the conditions S, representing a constructible set, and determine the levels of S. We show here the method for determining the canonical form of the constructible set S. Computing

$$G = \texttt{grobcov}(H + T, \texttt{"rep"}, 1)$$

with option `"rep",1`, will return the GC with the segments in C-representation. Then, we can proceed applying Algorithm 6.1: CompLev.

The procedure CompLev will first select the segments S_i of G for which lpp $\neq \{1\}$, and will return a list of pairs $[E_i, N_i]$ representing (in C-representation) the sets $\mathbf{V}(E_i) \setminus \mathbf{V}(N_i)$ where the system defined in G has solutions. We will have $S = (\mathbf{V}(E_1) \setminus \mathbf{V}(N_1)) \cup \cdots \cup (\mathbf{V}(E_r) \setminus \mathbf{V}(N_r))$. Then ConsLevels constructs the canonical levels

$$I_1 \subset I_2 \subset \cdots \subset I_k$$

of the constructible set S (see Chap. 2). Denoting $V_i = \mathbf{V}(I_i)$, we have

$$\overline{S} = V_1 \supset V_2 \supset \cdots \supset V_k = \emptyset,$$
$$\dim(S) = \dim(V_1) > \dim(V_2) > \ \ldots \ > \dim(V_k) = -1.$$
$$S = \bigcup_{i=1}^{k \text{ div } 2} V_{2i-1} \setminus V_{2i}.$$

$L \leftarrow$ **CompLev**(G)
`Input:`
$G =$ the `grobcov` of a proposition in C-representation
`Output:`
$L = [I_1, I_2, \ldots, I_k]$: the levels of the constructible set S
of supplementary conditions.
begin
$S =$ Select the segments S_i of G for which lpp $\neq \{1\}$
$L =$ ConsLevels(S)
return(L)
end

Algorithm 6.1: CompLev

In many problems, S is simply a variety V_1 or at most a locally closed set $V_1 \setminus V_2$, because the set S is not only constructible.

We consider now a more general geometrical statement or proposition containing also negative hypothesis H_1, (for example *the given triangle should be non-degenerate*), where H_1 is an ideal in the parameters only. We are concerned with the statement

$$(H \wedge \neg H_1) \Rightarrow T. \tag{6.6}$$

Translating this statement to GC is also automatic. The `grobcov` command accepts two options: `("null",E,"nonnull",N),`, that limits the scope of the parameter space to $\mathbf{V}(E) \setminus \mathbf{V}(N)$.

For obtaining the supplementary conditions for Proposition (6.6), it suffices to restrict the parameter space to $\mathbb{C}^n \setminus \mathbf{V}(H_1)$ by considering the option `("nonnull", `H_1`)`, call

$$G = \texttt{grobcov}(H + T, \texttt{"nonnull"}, H_1, \texttt{"rep"}, 1), \tag{6.7}$$

and repeat the computation of S as before if desired.

Finally, consider a proposition containing also the negative thesis T_1 (for example *the orthic triangle should be non-degenerate*). Here T_1 can contain both parameters and variables. Now we are concerned with obtaining the supplementary conditions for the statement

$$(H \wedge \neg H_1) \Rightarrow (T \wedge \neg T_1). \tag{6.8}$$

To tackle Problem (6.8) consider first a previous Problem (1): Determine the complementary conditions for the statement

$$H \Rightarrow T_1. \tag{6.9}$$

To solve Problem (1) using GC, we compute, as before

$$G_1 = \texttt{grobcov}(H + T_1, \texttt{"rep"}, 1),$$

and proceed to compute S_1 as in Eq. (6.7). In this case it is useful the procedure described below and obtain the levels $I_1, \ldots, I_k$ of S_1, using `CompLev`.

For obtaining the negative conclusion of Proposition (6.9), to be considered for Proposition (6.8), we have to restrict the parameter space to the complement of S_1, namely to

$$(C^n \setminus V_1) \cup (V_2 \setminus V_3) \cup \ldots .$$

So, we begin with the first restriction $\mathbb{C}^n \setminus V_1$, and call

$$G_2 = \texttt{grobcov}(H + T, \texttt{"nonnull"}, I_1 \cap H_1, \texttt{"rep"}, 1).$$

Then, select the set of segments S_0 with lpp $\neq \langle 1 \rangle$. These conditions ensure the theorem, but can be excessively restrictive, as the remaining sets $V_2 \setminus V_3$ and so on also ensure the proposition. We must add the resulting segments. So we determine all the conditions S_i for

$$G_i = \texttt{grobcov}(H + T, \texttt{"null"}, I_{2i}, \texttt{"nonnull"}, I_{2i+1} \cap H_1),$$

and adjoin them together. We have $S = \bigcup_i S_i$. Finally, we can apply `CompLev` to obtain the canonical levels, and then transform them into the true levels set using `Levels`.

The result will be obtained in C-representation. But by default, the algorithm transforms it into P-representation, as it is more useful. If we set the option `"rep",1`, then the C-representation is maintained.

All the actions described below are summarized in Algorithm 6.2: ADGT, that is implemented in the Singular library *grobcov.lib*.

The important fact about the use of Gröbner Cover in this context is that it provides – in a compact and concise way – all the essential pieces (a finite number of them) on the parameter space, allowing us to determine those that correspond to the validity of the given statement. Moreover the Gröbner Cover also provides rich information about the solutions, as it classifies the solutions by the lpp and this will correspond to the unique solution, multiple solutions, or even, infinite number of solutions (these last segments will correspond to degenerate situations). We remark that the Algorithm 6.2: ADGT can be improved in order to make it distinguish between conditions with a single solution, with multiple solutions, or with infinitely many solutions.

Let us consider some examples of the use of the ADGT Algorithm.

6.3 Orthic Triangle

With the objective of testing the power of Algorithm 6.2: ADGT and its corresponding Singular command for obtaining the supplementary hypothesis for converting a given proposition into a theorem, we consider now a set of different propositions related to the classical problem of the orthic triangle. This problem has been studied, for example in [71], using the classical methods, and its results can be useful for comparing with ours.

Let $A(-1, 0)$, $B(1, 0)$, $C(x, y)$ be the coordinates of the triangle ABC (see Fig. 6.4). We want to determine the allowed positions of C in order to obtain the necessary and sufficient conditions for complementing a set of different "propositions" related to the orthic triangle to convert them into "theorems". Let $A'(x_1, y_1)$, $B'(x_2, y_2)$, $C'(x, 0)$ be the feet of the heights.

$L \leftarrow$ **ADGT**$(H, T, H_1, T_1,$ `"options"`$)$
`Input:`
  Ideals H, T, H_1, T_1
`Output:`
  $L = [[1, [A_1, A_2]], [2, [A_3, A_4]]..]$: the complementary conditions
    over the parameters, that must be added to H in order for
    the proposition $(H \wedge \neg H_1) \Rightarrow (T \wedge \neg T_1)$ to be true.
begin
  $rep = 0$
    Default option to present the results in P-representation
    It can optionally be set to $rep = 1$ if the C-representation of the
    result is sufficient.
  if $T_1 = \langle 1 \rangle$ **then**
    $G_2 =$ `grobcov`$(H + T,$ `"nonnull"`$, H_1,$ `"rep"`$, 1)$
  else
    $G_1 =$ `grobcov`$(H + T_1,$ `"rep"`$, 1)$
    $L_0 =$ `CompLev`(G_1)
    if $H_1 \neq \langle 1 \rangle$ **then**
      for all even i of L_0 **do** $L_0[i] = L_0[i] \cap H_1$ **end for**
    end if
    $G_2 =$ `grobcov`$(H + T,$ `"nonnull"`$, L_0[1],$ `"rep"`$, 1)$
    $m = \#L_0$; $r =$ quot$(m, 2)$; **if** m is even **then** $r = r - 1$ **end if**
    for $i = 1..r$ **do**
      $G_3 =$ `grobcov`$(H + T,$ `"null"`$, L_0[2i],$ `"nonnull"`$, L_0[2i + 1],$ `"rep"`$, 1)$
      $G_2 = G_2 \cup G_3$
    end for
  end if
  $L =$ `Levels(CompLev`$(G_2))$;
  if $rep = 0$ **then**
    for $i = 1..\#L$ **do**
      $L[i][2] =$ `Prep`$(L[i][2][1], L[i][2][2])$
    end for
  end if
  return(L)
end

Algorithm 6.2: ADGT

We consider different hypotheses and theses. All the commands in this section can be executed by the reader or be included in a file that is to be read in Singular. We begin by defining rings that are largely used in the library and in many applications. Setting $\mathbf{u} = x, y$ and $\mathbf{v} = x_2, y_2, x_1, y_1$, the following ideals $R = \mathbb{C}[\mathbf{u}][\mathbf{v}]$, $P = \mathbb{C}[\mathbf{u}]$ and $RP = \mathbb{C}[\mathbf{v}, \mathbf{u}]$ can be useful. After defining the rings, we define the ideals of the different hypotheses and theses.

```
> LIB "grobcov.lib";
> ring R=(0,x,y),(x2,y2,x1,y1),lp;
> def Rx=ringlist(R);
> def P=ring(Rx[1]);
> Rx[1]=0;
> def D=ring(Rx);
> def RP=D+P;
> short=0;
```

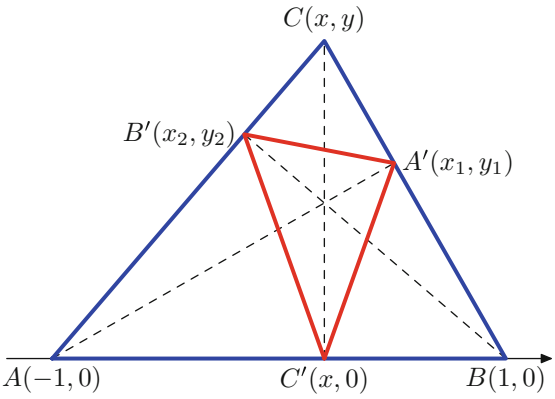


Fig. 6.4 Orthic triangle

Hypothesis H: The triangle $A'B'C'$ is the orthic triangle of ABC (we shall use it in all of our propositions):

```
> ideal H=-y*x1+(x-1)*y1+y,
        (x-1)*(x1+1)+y*y1,
        -y*x2+(x+1)*y2-y,
        (x+1)*(x2-1)+y*y2;
```

Hypothesis H_1. The triangle ABC is degenerate (we shall deny it):

```
> ideal H1=y;
```

Thesis T. The orthic triangle $A'B'C'$ is isosceles ($\overline{A'C'} = \overline{B'C'}$):

```
> ideal T=(x1-x)^2+y1^2-(x2-x)^2-y2^2;
```

Thesis T_1. The orthic triangle $A'B'C'$ is equilateral (we shall consider examples with T_1 and with $\neg T_1$):

```
> ideal T1=(x1-x)^2+y1^2-(x2-x)^2-y2^2,
        (x1-x)^2+y1^2-(x2-x1)^2-(y2-y1)^2;
```

Thesis T_2. The orthic triangle $A'B'C'$ is degenerate (points aligned, we shall deny it):

```
> ideal T2=x*(y1-y2)-y*(x1-x2)+x1*y2-x2*y1;
```

Thesis $T_{12} = T_1 \cap T_2$ (we shall deny it).

```
> ideal T12=intersectpar(T1,T2);
```

Considering now the above defined $H, T, H_1, T_1, T_2, T_{12}$, we analyze now different ADGT propositions and use the `ADGT` command of the *grobcov.lib* library for complementing them:

Proposition 6.1 $(H \wedge \neg H_1) \Rightarrow T$. *Calling sequence:*

```
> ADGT(H,T,H1,1);
```

obtaining the complementary condition (we translate the result in current mathematical typos):

$$(x, y) \in (\mathbf{V}(x^2 - y^2 - 1) \setminus (\mathbf{V}(y, x - 1) \cup \mathbf{V}(y, x + 1) \cup \mathbf{V}(y^2 + 1, x)))$$
$$\cup (\mathbf{V}(x^2 + y^2 - 1) \setminus (\mathbf{V}(y, x - 1) \cup \mathbf{V}(y, x + 1)))$$
$$\cup (\mathbf{V}(x) \setminus (\mathbf{V}(x, y) \cup \mathbf{V}(y^2 + 1, x)));$$

thus $(H \wedge \neg H_1) \Rightarrow T$ is true if and only if vertex C is

(a) either on the mediatrix of BC (i.e., ABC is isosceles with $\overline{AC} = \overline{BC}$),
(b) either on the hyperbola $x^2 - y^2 - 1$,
(c) either on the circle x^2+y^2-1 (i.e. ABC has a right angle, in which case $A'B'C'$ is degenerate). See Fig. 6.5.

The points of the varieties that are eliminated in the supplementary conditions obtained all correspond to points in the x-axis that have already been excluded by the hypothesis $\neg H_1$, and appear to be reflected in the result of applying `ADGT`. We can observe that the circle is eliminated in the next proposition, where we impose the non-degenerate orthic triangle.

Points C_{11}, C_{12}, C_{21}, C_{22}, C_{31}, C_{32}, C_{41}, C_{42} of Fig. 6.5 are not relevant for Propositions 6.1 and 6.2 or for Theorem 6.4 but they become relevant for Proposition 6.5.

Proposition 6.2 $(H \wedge \neg H_1) \Rightarrow (T \wedge \neg T_2)$. *Calling sequence:*

```
> ADGT(H,T1,H1,T2);
```

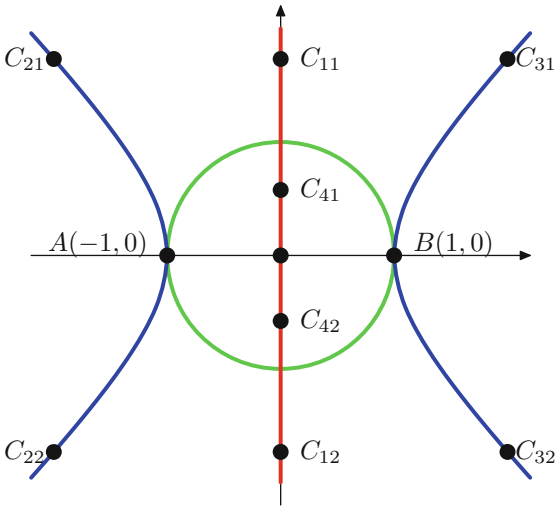


Fig. 6.5 Triangles having an isosceles orthic triangle. Different cases

obtaining the complementary condition:

$$(x, y) \in (\mathbf{V}(x^2 - y^2 - 1) \setminus (\mathbf{V}(y, x - 1) \cup \mathbf{V}(y, x + 1) \cup \mathbf{V}(y^2 + 1, x))) \\ \cup (\mathbf{V}(x) \setminus (\mathbf{V}(y - 1, x) \cup \mathbf{V}(x, y) \cup \mathbf{V}(y^2 + 1, x) \cup \mathbf{V}(y + 1, x))).$$

We see that the circle appearing in Proposition 6.1 is eliminated, because for C in the circle, the orthic triangle $A'B'C'$ is degenerate.

We can generalize the condition of C being on the hyperbola, independent of the coordinates considering that the sides a, b can be expressed in terms of (x, y) as

$$a^2 = y^2 + (x - 1)^2,$$
$$b^2 = y^2 + (x + 1)^2$$

and imposing that (x, y) lies on the hyperbola $x^2 - y^2 - 1 = 0$. We can take the two first equations as the hypothesis and the hyperbola as the thesis and compute

```
> ring R2=(0,a,b,c),(x,y),lp;
> short=0;
> ideal H4=(x-1)^2+y^2-a^2,
           (x+1)^2+y^2-b^2;
> ideal T4=x^2-y^2-1;
> def L=ADGT(H4,T4,a*b*c,1,"rep",1);
> L;
[1]:
   [1]:
      1
   [2]:
      [1]:
         _[1]=(a^4-2*a^2*b^2-4*a^2+b^4-4*b^2)
      [2]:
         _[1]=1
> poly h=L[1][2][1][1][1];
> h;
(a^4-2*a^2*2^2-4*a^2+b^4-4*b^2)
> def hh=c^4*(subst(h,a,2*a/c,b,2*b/c))/16;
> hh;
(a^4-2*a^2*b^2-a^2*c^2+b^4-b^2*c^2)
> setring R;
```

h represents the relation between the sides $a, b, c = 2$ of the given triangle ABC. Multiplying by $c/2$, the sides become $a' = ac/2, b' = bc/2, c' = c$. So the condition over the sides becomes

$$hh = (a^2 - b^2)^2 - (a^2 + b^2)c^2.$$

The points retrieved in the obtained supplementary conditions correspond all to a degenerate triangle ABC, already eliminated by assumption. Thus, we obtain the following.

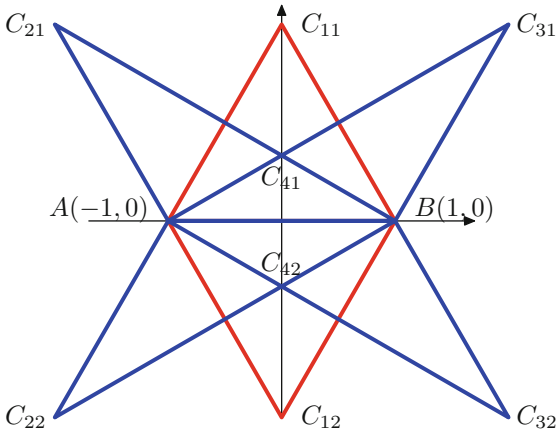


Fig. 6.6 Non-degenerate triangles having an equilateral orthic triangle

Theorem 6.4 *Given a non-degenerate triangle $ABC$with sides a, b, c, its orthic triangle $A'B'C'$ is isosceles ($\overline{A'C'} = \overline{B'C'}$) and non-degenerate if and only if*

(a) *either $\overline{AC} = \overline{BC}$, i.e., ABC is isosceles*
(b) *or the sides a, b, c verify the relation $(a^2 - b^2)^2 - (a^2 + b^2)c^2 = 0$.*

Proposition 6.3 $(H \wedge \neg H_1) \Rightarrow T_1$. *Calling sequence:*

```
> ADGT(H,T1,H1,1);
```

obtaining the complementary condition:

$$(x, y) \in \mathbf{V}(y^2 - 3, x) \cup \mathbf{V}(y^2 - 3, x + 2) \cup \mathbf{V}(y^2 - 3, x - 2) \cup \mathbf{V}(3y^2 - 1, x).$$

We obtain that the vertex C can be one of the points C_{11}, C_{12}, C_{21}, C_{22}, C_{31}, C_{32}, of Figs. 6.6 and 6.5. We obtain the following:

Theorem 6.5 *The orthic triangle of a given non-degenerate triangle is equilateral, if and only if*

(a) *ABC is equilateral: $a = b = c$; (points C_{11}, C_{12});*
(b) *or ABC is isosceles with two equal sides, say $a = c$ and the third side $b = \sqrt{3}c$ (points C_{21}, C_{22}), or $b = c$ and the third side $a = \sqrt{3}c$ (points C_{31}, C_{32}), or $a = b$ and the third side $c = \sqrt{3}a$ (points C_{41}, C_{42}).*

Proposition 6.4 $H \Rightarrow T_2$. *Calling sequence:*

```
> ADGT(H,T2,1,1);
```

obtaining the complementary condition:

$$(x, y) \in \mathbf{V}(x^2 + y^2 - 1) \cup \mathbf{V}(y) \cup (\mathbf{V}(x + 1) \setminus \mathbf{V}(y^2 + 4, x + 1)) \cup (\mathbf{V}(x - 1) \setminus \mathbf{V}(y^2 + 4, x - 1)).$$

The supplementary condition for Proposition 6.4 is that (x, y) lies on the circle, or on the line $x + 1$, or the line $x - 1$, or the line $y = 0$ (excluding uninteresting complex points). Except for the condition of being on the y-axis, where ABC is degenerate, the condition corresponds to ABC being rectangular, in which case it is obvious, from a geometrical point of view, that $A'B'C'$ is degenerate, having two coincidental vertices at the normal angle.

Proposition 6.5 $(H \wedge \neg H_1) \Rightarrow (T \wedge \neg T_{12})$. *Calling sequence:*

```
> ADGT(H,T, H1,T12,"rep",0);
```

obtaining the complementary condition:

$$(x, y) \in (\mathbf{V}(x^2 - y^2 - 1) \setminus (\mathbf{V}(y, x - 1) \cup \mathbf{V}(y, x + 1) \cup \mathbf{V}(y^2 - 3, x - 2) \cup \mathbf{V}(y^2 - 3, x + 2) \cup \mathbf{V}(y^2 + 1, x))) \cup (\mathbf{V}(x) \setminus (\mathbf{V}(y - 1, x) \cup \mathbf{V}(y, x) \cup \mathbf{V}(y^2 + 1, x) \cup \mathbf{V}(3y^2 - 1, x) \cup \mathbf{V}(y^2 - 3, x) \cup \mathbf{V}(y + 1, x))).$$

We obtain

Theorem 6.6 (Isosceles but non-equilateral nor degenerate orthic triangle) *The orthic triangle $A'B'C'$ of a given non degenerate triangle ABC with sides a, b, c is isosceles with $\overline{A'C'} = \overline{B'C'}$ but not equilateral nor degenerate if and only if*

(a) *either sides a, b, c verify $(a^2 - b^2)^2 - c^2(a^2 + b^2) = 0$, except if the triangle is isosceles with two equal sides, say $a = c$, and the third side being $b = \sqrt{3}c$ or $b = c$, and the third side being $a = \sqrt{3}c$;*

(b) *or ABC is isosceles with $\overline{AC}$ and $\overline{BC}$ except if it is equilateral or has angles in A and B of $30°$ and the angle in C of $120°$, (i.e., except if it has two equal sides, say $a = b$, and the third side is $c = \sqrt{3}a$).*

We verify the coherence of the different results.

6.4 Automatic Demonstration of Geometric Theorems

Algorithm 6.2: ADGT can obviously also be applied to the Automatic Demonstration of Theorems. If the given proposition is true, then the generic segment of `grobcov`$(H + T)$ will have a basis different from $\{1\}$, as the thesis must be

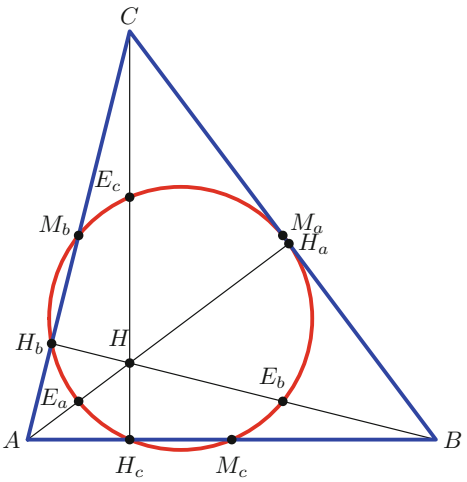


Fig. 6.7 Nine-Points Circle

true in at least an open set of the parameter space. The ADGT algorithm will return $\mathbf{V}(0) \setminus \mathbf{V}(N)$. That is, the proposition is true (theorem) for all values of the parameters, except for some special configurations (degenerate, or similar), that will be detected by the algorithm. We illustrate the method proving the *Nine-Points Circle Theorem*.

The *Nine-Points Circle*, also called Euler's circle or Feuerbach's circle (see Fig. 6.7) is the circle that passes through the feet of the heights H_a, H_b, and H_c traced from the vertices of a triangle ABC perpendicular to their opposite sides. Euler showed in 1765 that the circle also passes through the midpoints M_a, M_b, M_c of the sides of ABC. By Feuerbach's theorem, the Nine-Points Circle also passes through the midpoints E_a, E_b, and E_c of the segments that join the vertices and the orthocenter H. These points are commonly referred to as the Euler points.

We want to prove the theorem using the `ADGT` command. Consider a triangle with vertices $A = (-1, 0)$, $B = (1, 0)$ and $C = (x, y)$. Assign the following coordinates to the feet of the heights:

$$H_a = (x_1, y_1),\ H_b = (x_2, y_2)\ \text{ and }\ H_c = (x, 0),$$

as obviously, by construction, $H_c = (x, 0)$. To determine (x_1, y_1), consider the equations of the side BC and its perpendicular passing through A. Its intersection is (x_1, y_1), and similarly for H_b. The equations are

$$\begin{aligned} H_a =& yx_1 + (1 - x)y_1 - y, \\ & (x - 1)(x_1 + 1) + yy_1; \\ H_b =& - yx_2 + (1 + x)y_2 - y, \\ & (x + 1)(x_2 - 1) + yy_2. \end{aligned}$$

The middle points of the sides are

$$M_a = \left(\frac{x+1}{2}, \frac{y}{2}\right),\ M_b = \left(\frac{x-1}{2}, \frac{y}{2}\right),\ \text{and}\ M_c = (0,0).$$

Let (x, y_h) be the intersection point between h_a and h_c. Then, the coordinates of the Euler's points E_a and E_c are

$$E_a = \left(\frac{x-1}{2}, \frac{y_h}{2}\right),\ E_c = \left(x, \frac{y+y_h}{2}\right).$$

Let (a, b) be the center of the circle passing through the three feet of the heights and radius r. Let (X, Y) be a generic point on the circle. Then its equation is

$$c : (X-a)^2 + (Y-b)^2 - r^2$$

and by hypothesis it passes through H_a, H_b, and H_c. Thus we have the hypothesis

$$\begin{aligned} H_1 =& (x_1-a)^2 + (y_1-b)^2 - r^2, \\ & (x_2-a)^2 + (y_2-b)^2 - r^2, \\ & (x-a)^2 + b^2 - r^2. \end{aligned}$$

Let (x, y_h) be the intersection point of h_a and h_c. To determine the equation of the high h_a, consider that the points A, H_a and (x, y_h) are aligned. We have

$$h_a = -y_h + xy_1 - x_1y_h + y_1.$$

The set of equations H_a, H_b, H_1 and h_a are the hypothesis of the proposition.

Now the theses are the following:

(1) height h_b also intersects (x, y_h) (the tree heights intersect at the same point: orthocenter);
(2) circle c passes through the three middle points of the sides (Euler's theorem);
(3) circle c passes through the three Euler's points (Feuerbach's theorem).

The corresponding equations for the theses T_1, T_2 and T_3 are:

$$\begin{aligned} T_1 =& y_h + xy_2 - x_2y_h - y_2; \\ T_2 =& \left(\frac{x+1}{2} - a\right)^2 + \left(\frac{y}{2} - b\right)^2 - r^2, \\ & \left(\frac{x-1}{2} - a\right)^2 + \left(\frac{y}{2} - b\right)^2 - r^2, \end{aligned}$$

$$a^2+b^2-r^2;$$

$$T_3=\left(\frac{x-1}{2}-a\right)^2+\left(\frac{y_h}{2}-b\right)^2-r^2,$$

$$\left(\frac{x+1}{2}-a\right)^2+\left(\frac{y_h}{2}-b\right)^2-r^2,$$

$$(x-a)^2+\left(\frac{y_h+y}{2}-b\right)^2-r^2.$$

The hypotheses are $H=H_a, H_b, H_1, h_a$; and the theses $T=T_1, T_2, T_3$.

We obtain 14 equations in the 10 unknowns $(x, y, x_1, y_1, x_2, y_2, y_h, a, b, r)$. The Nine-Points Circle theorem states that they must be compatible. We can use the `grobcov` command, considering (x, y) as parameters and leave the remaining unknowns as variables, since we want to determine for which values of (x, y) the system $H+T$ is compatible. Moreover, we can also discard degenerate triangles corresponding to the three vertices being aligned, assuming $y\neq 0$ (considering the option `("nonnull",y)`).

Nevertheless, we can also use simply the `ADGT` command that will give an automatic answer. We proceed as follows:

```
> LIB "grobcov.lib";
> ring R=(0,x,y),(x1,y1,x2,y2,yh,a,b,r),dp;
> short=0;
> ideal H=y*x1+y1-x*y1-y,
          (x-1)*(x1+1)+y*y1,
          -y*x2+y2+x*y2-y,
          (x+1)*(x2-1)+y*y2,
          (x1-a)^2+(y1-b)^2-r^2,
          (x2-a)^2+(y2-b)^2-r^2,
          (x-a)^2+b^2-r^2,
          -yh+x*y1-x1*yh+y1;
> ideal T=yh+x*y2-x2*yh-y2,
          (x-1-2*a)^2+(yh-2*b)^2-4*r^2,
          (x+1-2*a)^2+(yh-2*b)^2-4*r^2,
          4*(x-a)^2+(y+yh-2*b)^2-4*r^2,
          (x-1-2*a)^2+(y-2*b)^2-4*r^2,
          (x+1-2*a)^2+(y-2*b)^2-4*r^2,
          4*(x-a)^2+(yh+y-2*b)^2-4*r^2;
> ideal NH=y;
> ADGT(H,T,NH,1);
[1]:
   [1]:
      1
   [2]:
      [1]:
         [1]:
            _[1]=0
```

```
          [2]:
             [1]:
                _[1]=(x^2+2*x+y^2+1)
             [2]:
                _[1]=(x^2-2*x+y^2+1)
             [3]:
                _[1]=(y)
[2]:
   [1]:
      Generic segment
   [2]:
      [1]:
         _[1]=b
         _[2]=a
         _[3]=yh
         _[4]=y2
         _[5]=x2
         _[6]=y1
         _[7]=x1
         _[8]=r^2
      [2]:
         _[1]=(4*y)*b+(x^2-y^2-1)
         _[2]=2*a+(-x)
         _[3]=(y)*yh+(x^2-1)
         _[4]=(x^2+2*x+y^2+1)*y2+(-2*x*y-2*y)
         _[5]=(x^2+2*x+y^2+1)*x2+(-x^2-2*x+y^2-1)
         _[6]=(x^2-2*x+y^2+1)*y1+(2*x*y-2*y)
         _[7]=(x^2-2*x+y^2+1)*x1+(x^2-2*x-y^2+1)
         _[8]=(16*y^2)*r^2+(-x^4-2*x^2*y^2+2*x^2-y^4-2*y^2-1)
      [3]:
         [1]:
            [1]:
               _[1]=0
            [2]:
               [1]:
                  _[1]=(x^2-2*x+y^2+1)
               [2]:
                  _[1]=(x^2+2*x+y^2+1)
               [3]:
                  _[1]=(y)
```

When the Algorithm ADGT detects that the supplementary conditions are of the form $\mathbf{V}(0) \setminus \mathbf{V}(N)$, (i.e., the theorem is generally true), then it automatically returns a second element: the generic segment of the GC used inside. The reason for this is to provide the values of all the variables as function of the free parameters.

In our example, the application of ADGT shows that the theorem is true for every point C except when it verifies $y = 0$, which has already been discarded, or when it verifies $(x+1)^2 + y^2 = 0$ or $(x-1)^2 + y^2 = 0$, whose unique real points are A and B respectively. Thus the theorem is true for every non-degenerate triangle ABC.

From the basis of the generic segment, we can determine the values of $b, a, y_h, y_2, x_2, y_1, x_1, r$, that peovide the coordinates of H, H_a, H_b, a, b, r for all values of the parameters x, y. To complete the information, we add also the values of H_c,

$M_a, M_b, M_c, E_a, E_b, E_c$, which are determined by the hypothesis. We obtain:

$$H = \left(x, \frac{-x^2+1}{y}\right),\ H_c = (x, 0),$$

$$H_a = \left(\frac{-x^2+y^2+2x-1}{x^2+y^2-2x+1}, \frac{-2y(x-1)}{x^2+y^2-2x+1}\right),$$

$$H_b = \left(\frac{x^2-y^2+2x+1}{x^2+y^2+2x+1}, \frac{2y(x+1)}{x^2+y^2+2x+1}\right),$$

$$M_a = \left(\frac{x+1}{2}, \frac{y}{2}\right),\ M_b = \left(\frac{x-1}{2}, \frac{y}{2}\right),\ M_c = (0, 0)\,,$$

$$E_a = \left(\frac{x-1}{2}, \frac{-x^2+1}{y}\right),\ E_b = \left(\frac{x+1}{2}, \frac{-x^2+1}{y}\right),$$

$$E_c = \left(x, \frac{-x^2+y^2+1}{y}\right),$$

$$a = \frac{x}{2},\ b = \frac{-x^2+y^2+1}{4y},\ r = \frac{(x^2+y^2-1)^2}{4y}.$$

Obviously a change of coordinates can send points A, B, C, and consequently all the associated points to any non-aligned points in the plane.

We conclude the effectiveness of `AGDT` and `grobcov` also for Automatic Demonstration of Geometric Theorems.

Chapter 7
Geometric Loci



7.1 Elementary Loci

A geometric locus is the set of points (usually considered in $2d$-space or in $3d$-space) verifying some conditions. Loci problems involve distances between two points. We shall refer to Euclidean space, and so the distance d between two points $P_1(x_1, \ldots, x_n)$ and $P_2(y_1, \ldots, y_n)$ is the positive value given by

$$d^2 = (x_1 - y_1)^2 + \cdots + (x_n - y_n)^2.$$

Classical elementary two-dimensional loci are well known. For example:

(1) The set of points equidistant from two fixed points P_1 and P_2. Take as points $P_1(a, 0)$ and $P_2(b, 0)$. We have $(x-a)^2 + y^2 = (x-b)^2 + y^2$, from which we obtain $2(b-a)x + (a+b)(a-b)$, and as $b \neq a$, this results in $x = \frac{a+b}{2}$, which is the equation of the perpendicular bisector line to the segment $\overline{P_1P_2}$.

(2) The set of points at the fixed distance r from a point $O(a, b)$, which is the circle $(x-a)^2 + (y-b)^2 = r^2$,

(3) The set of points whose sum of distances to two fixed foci points is given. Taking as focus $(-f, 0)$ and $(f, 0)$ and as sum of distances $2a$, we have $\sqrt{(x+f)^2 + y^2} + \sqrt{(x-f)^2 + y^2} = 2a$. Eliminating square roots and denoting $b = \sqrt{f^2 - a^2}$, we obtain

$$\frac{x^2}{a^2} + \frac{y^2}{b^2} = 1,$$

which is the classical formula of the ellipse of semi-axes a and b.

(4) The set of points whose difference of distances to two foci is fixed is a hyperbola, and the set of points whose distances to a focus point and a line are equal is a parabola. All conics are elementary loci.

© Springer Nature Switzerland AG 2018

A. Montes, *The Gröbner Cover*, Algorithms and Computation in Mathematics 27, https://doi.org/10.1007/978-3-030-03904-2_7

Quadrics are elementary loci in $3d$, and we can also consider such kinds of loci in higher dimensional spaces. In fact every polynomial equation in the variables of the space can be considered as a locus.

7.2 Locus of a Geometric Construction

The ideas developed in this chapter were partially be exposited by Abánades, Botana, Montes, Recio in [1]. Here, we try to give a wider perspective, providing complete algorithms, and generalizing the kind of problems in the study. Loci problems can be of different types, and it is not easy to classify them. We center our study on a wide class of problems that includes all the classical loci problems, and fix their characteristics.

We shall analyze some particular loci problems in the examples that cannot be included and treated in the class that we study now.

Even if usually loci problems considered in Dynamical Geometry (D.G.) are $2d$-loci or at most $3d$-loci, we can generalize to problems in n-dimensional space. Every locus problem in n-dimensional space must contain, as variables, the coordinates of a *tracer point* $T(x_1, \ldots, x_n)$ corresponding to the points of the locus. Loci problems also contain another set of *auxiliary* variables $\mathbf{u} = u_1, \ldots, u_m$ and a polynomial system of equations $F(\mathbf{x}, \mathbf{u})$ relating all the variables. We expect that the projection of the solution over the tracer variables space to be formed by $n-1$ dimensional sets, i.e., hypersurfaces perhaps excluding some included smaller varieties. So we shall define class A of loci problems for which its translation to equations consists of a polynomial system $F[\mathbf{x}, \mathbf{u}]$ with $m + 1$ independent polynomials, in order to expect solutions of dimension $n - 1$. For this class of loci problems, we can expect that the projection of the solutions of the system F over the $\mathbf{x}$ variables, which consists of the locus points, to also have dimension $n - 1$.

Moreover, in many classical loci problems, part of the $\mathbf{u}$ variables are the coordinates of a *mover point* of the n-dimensional space, and the equations in F translate the fact that the mover point glides on an $n - 1$ dimensional hypersurface, in such a form that every position of the mover generates one or a finite set of values of the tracer variables. There also exist loci problems in which an explicit mover point does not exist. For reasons that will become clear later, in this case it is convenient to appropriately assign n of the $\mathbf{u}$ variables to be the $\mathbf{w}$ coordinates of the mover point. In case that the number m of auxiliary variables $\mathbf{u}$ is smaller than n, then we shall consider as mover variables all of the $\mathbf{u}$ variables. To fix ideas, and in order to make the algorithms easier to eliminate the remaining $\mathbf{u}$ variables, we shall consider as mover variables $\mathbf{w}$, the latest n' variables $\mathbf{u}$, where $n' = n$ if $m \geq n$ and $n' = m$ if $m < n$.

In consequence, we define the following class of loci problems, which includes the current loci studied in Dynamical Geometry (D.G.).

Definition 7.1 (Class A loci problems) We say that a locus problem of the n-dimensional space belongs to *class* A of loci problems, if it contains the following objects:

(1) A *tracer point* $T(x_1, \ldots, x_n)$ in the n tracer variables $\mathbf{x} = x_1, \ldots, x_n$, corresponding to the coordinates of the locus points.
(2) A set of *auxiliary variables* $\mathbf{u} = u_1, \ldots, u_m$, for the geometric construction. These variables can contain or not an explicit subset of variables $\mathbf{w} = w_1, \ldots, w_n$ considered as the coordinates of a *mover point* generating the locus. If the mover point is not explicitly given in the problem, we have to decide in a reasonable form, which n' variables are better adapted to play the role of the mover variables. Its number is $n' = n$ if $m \geq n$ and $n' = m$ if $m < n$, and in the algorithms, we consider the last n' variables $\mathbf{u}$ as the mover variables.
(3) A *polynomial system of equations* $F \subset \mathbb{Q}[\mathbf{x}][\mathbf{u}]$, relating the tracer variables $\mathbf{x}$ and the auxiliary variables $\mathbf{u}$, that describe the locus problem. The system F must contain $m + 1$ independent equations, in order to expect an $n - 1$-dimensional set of solutions in the $\mathbf{x}$ variables.

We shall consider the system F in the ring $\mathbb{Q}[\mathbf{x}][\mathbf{u}]$, taking the tracer variables as parameters and the remaining $\mathbf{u}$ as variables. Moreover, we take the mover variables $\mathbf{w}$ as the last n' variables $\mathbf{u}$, in order to make it easier to eliminate the remaining variables. It is also preferable to use lex order for the variables $\mathbf{u}$ as well as for the parameters $\mathbf{x}$.

The simplest locus of this kind is a hypersurface, i.e., a simple polynomial equation in the tracer variables $\mathbf{x} = x_1 \ldots, x_n$ of the n-dimensional space ($m = 0$), like a circle, a sphere or any polynomial equation. But the problems we are interested in are more general, and are derived from elaborate geometric constructions.

To deal with class A loci problems, we have to eliminate the variables $\mathbf{u}$. For this objective, we can use the standard Gröbner lexicographical basis. But, as we shall see, we can obtain a more detailed description using the Gröbner Cover.

The Gröbner Cover is especially indicated for solving these types of problems. Moreover, the `grobcov` command gives the simplest canonical form of the discussion, and allow us to go into the problem in depth, define and determine a taxonomy of the locus components in an automatic way. The *grobcov.lib* library is to be used in the ring $\mathbb{Q}[\mathbf{x}][\mathbf{u}]$.

The question is to obtain the projection into the tracer space of the solution of the system F. This question can be formulated in the *grobcov* philosophy as follows: for which values of $\mathbf{x}$ do there exist solutions to F, and which ones are the corresponding values of the remaining variables.

Even if locus problems concern real solutions, in order to use automatic computations we will use the $\mathbb{Q}$-Zariski topology of $\mathbb{C}^{n+m}$, as in the rest of the book. In practice we are interested in the projection of the locus L over the reals $\mathbb{R}$. This is not always straightforward and can cause problems in the computation. We discuss this question and give some partial solutions in the examples (see, for example, Sect. 7.5.3). This question leads to the following challenging problem:

how to transform the *Complex Gröbner Cover* (GC) into the *Real Gröbner Cover* (RGC). We shall refer to the *complex locus* and to the *real locus* in which we are properly interested. It is not sufficient to consider $L_{\mathbb{R}} = L_{\mathbb{C}} \cap \mathbb{R}^n$, because this can produce some undesired components that can be avoided if we use the real GC.

We have tried to elaborate on a didactical approach that includes the study of a large number of problems. In order to introduce the subject, let us consider a very simple problem where a tracer and a mover point clearly exist and has an elementary solution.

Example 7.2 (Mover and tracer points in locus) Consider a fixed point A in the plane and a fixed circle c of radius r. Consider the locus of midpoints T of the segment AM, where the mover point M glides on the circle c. The result is obviously a circle of radius $r/2$ with center in the midpoint of AC, where C is the center of the given circle c.

In the given example, let (x_1, y_1) be the coordinates of the mover point $M(x_1, y_1)$, the circle having the equation $c : x_1^2 + y_1^2 - 4 = 0$ and the fixed point is $A = (6, 2)$. The tracer $T(x, y)$ is the midpoint of AM, whose coordinates are $x = (x_1 + 6)/2$ and $y = (y_1 + 2)/2$. To compute the locus of points, we have to consider the polynomial system

$$S = [x_1^2 + y_1^2 - 4, x = (x_1 + 6)/2, y = (y_1 + 2)/2]$$

and eliminate the coordinates (x_1, y_1) of the mover point. The result is obviously (see Fig. 7.1)

$$(x - 3)^2 + (y - 1)^2 - 1.$$

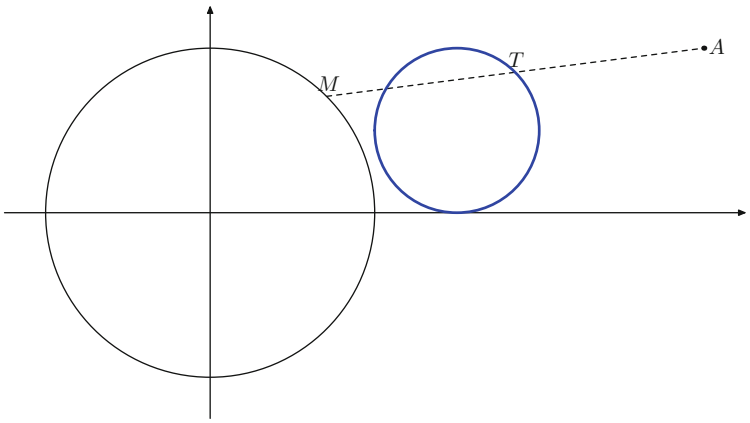


Fig. 7.1 Simple locus of Example 7.2

$L \leftarrow$ **stdlocus**(F)
`Input:` F is the system describing the locus
`Output:` L, the prime components of the elimination ideal.
begin
  $B = \text{GB}(F)$ in lex$(\mathbf{u}, \mathbf{x})$ # in ring $\mathbb{Q}[\mathbf{u}, \mathbf{x}]$
  $B = B \cap \mathbb{Q}[\mathbf{x}]$
  $L =$ prime decomposition(B)
  return(L)
end

Algorithm 7.1: STDLOCUS

Recall that the system verifies the general conditions of the Definition 7.1 of class A loci problems. We have $m = 2$, the locus is a 1-dimensional set as well as the mover positions, and the system has $m + 1 = 3$ equations.

Let us now consider an automatic approach. We start with the locus ideal:

$$F = \langle x_1^2 + y_1^2 - 4, 2x - x_1 - 6, 2y - y_1 - 2 \rangle$$

For eliminating automatically (x_1, y_1) and obtaining the projection on the $\mathbf{x}$ variables (x, y), we can simply use lex GB. The Elimination Theorem (see [25], page 114) guarantees that we obtain the elimination ideal $\langle F \rangle \cap \mathbb{C}[x, y]$. In this example, the elimination ideal has only one irreducible polynomial but, as we shall see, in other cases it can contain more than one polynomial or non-irreducible polynomials. So we can do better and use the Gianni-Trager-Zacharias Algorithm [36] to obtain the prime components of the elimination ideal. Algorithm 7.1: STDLOCUS includes also the prime decomposition.

Using `stdlocus` we have:

```
> LIB "grobcov.lib";
> ring R=(0,x,y),(x1,y1),lp;
> short=0;
> ideal F=x1^2+y1^2-4,2*x-(x1+6),2*y-(y1+2);
> stdlocus(F);
[1]:
   x^2-6*x+y^2-2*y+9
```

Observe that all the commands of the *grobcov.lib* library assume that the base ring is $\mathbb{Q}[\mathbf{x}][\mathbf{u}]$, even if inside the routines, as in `stdlocus`, other rings can be used.

For the alternative method, let us start using the `grobcov` command. We will see that it can provide more varied information about the locus components. Using `grobcov` the question is determining for which values of the parameters (x, y) there exist points in $\mathbf{V}(F)$.

Coming back to the example, using `grobcov`, we consider x, y as parameters and x_1, y_1 as variables, and the Gröbner Cover provides an automatic answer.

```
> LIB "grobcov.lib";
> ring R=(0,x,y),(x1,y1),lp;
> short=0;
> ideal F=x1^2+y1^2-4,2*x-(x1+6),2*y-(y1+2);
> grobcov(F);
[1]:
   [1]:
      _[1]=1
   [2]:
      _[1]=1
   [3]:
      [1]:
         [1]:
            _[1]=0
         [2]:
            [1]:
               _[1]=(x^2-6*x+y^2-2*y+9)
[2]:
   [1]:
      _[1]=y1
      _[2]=x1
   [2]:
      _[1]=y1+(-2*y+2)
      _[2]=x1+(-2*x+6)
   [3]:
      [1]:
         [1]:
            _[1]=(x^2-6*x+y^2-2*y+9)
         [2]:
            [1]:
               _[1]=1
```

The `grobcov` command obtains two segments. The first is the generic segment and shows that there does not exist any solution in $\mathbb{C}^2 \setminus \mathbf{V}(x^2 - 6x + y^2 - 2y + 9)$. The second segment is $\mathbf{V}(x^2 - 6x + y^2 - 2y + 9) = \mathbf{V}((x-3)^2 + (y-1)^2 - 1)$, where for each point (x, y) of the locus, there exists a unique solution $(x_1, y_1) = (2x - 6, 2y - 2)$.

We see in this simple example that both approaches provide the same result. But the `grobcov` approach provides the information about which is the position of the mover that generates a given point of the tracer. We shall see in the development of the algorithms that this information will be useful for classifying the locus components.

As noted above, in Dynamic Geometry (DG), the term "locus" generally refers to loci of this kind: i.e., to the trajectory determined by the different positions of a *tracer* point, (as point T above), corresponding to the different configuration of the remaining variables. These can be, in simple problems, uniquely the set of coordinates of a mover point M restricted to move on some hypersurface, as in

this problem. This is the kind of problems considered by the first standard DG systems developed in the late 1980s (such as in Cabri [16] and The Geometer's Sketchpad [77]), but it is also true for more recent ones, such as GeoGebra [35] or Java Geometry Expert [40].

There is a wide consensus among DG developers to consider locus computation as one of the five basic properties in the DG paradigm (together with dynamic transformation, measurement, free dragging and animation, see for instance [33]).

In this study, we consider the larger class A of loci problems already defined in Definition 7.1, for which the number of $\mathbf{u}$ variables need not be the same as the number of tracer variables $\mathbf{x}$, which is equal to n, the dimension of the space of the problem.

7.3 Taxonomy of Loci

For class A loci problems (Definition 7.1), we define a taxonomy for the components. Let $\mathbf{x} = x_1, \ldots, x_n$ be the set of n tracer variables, and $\mathbf{u} = u_1, \ldots, u_m$ the remaining variables including at the end the mover coordinates $\mathbf{w}$. Let $nv =$ number of variables and $np =$ number of parameters. Then the number of mover variables is taken by default to be the $\min(np, nv)$ but it can be assigned by an option to a greater value smaller or equal to nv. Let $F(\mathbf{x}, \mathbf{u})$ be the $m + 1$ set of independent equations defining the locus problem.

Consider the solution of the system:

$$\mathbf{V}(F) = \{(\mathbf{x}, \mathbf{u}) \in \mathbb{C}^{n+m} : \forall f \in F, f(\mathbf{x}, \mathbf{u}) = 0\}.$$

Denote π_1, π_2 and π_3 the projections onto the $\mathbf{x}$ the $\mathbf{u}$ and the $\mathbf{w}$ spaces respectively:

$$\begin{array}{ccc} \pi_1 : \mathbb{C}^{n+m} \to \mathbb{C}^n & \pi_2 : \mathbb{C}^{n+m} \to \mathbb{C}^m & \pi_3 : \mathbb{C}^m \to \mathbb{C}^{n'} \\ (\mathbf{x}, \mathbf{u}) \mapsto \mathbf{x} & (\mathbf{x}, \mathbf{u}) \mapsto \mathbf{u} & \mathbf{u} \mapsto \mathbf{w} \end{array}$$

and the anti-images $\mathcal{A}(\mathbf{x})$ and $\mathcal{A}_m(\mathbf{x})$ of a locus point on the $\mathbf{u}$ and $\mathbf{w}$ spaces:

$$\begin{array}{ll} \mathcal{A} : \mathbb{C}^n \to \mathbb{C}^m & \mathcal{A}_m : \mathbb{C}^n \to \mathbb{C}^{n'} \\ \mathbf{x} \mapsto \pi_2\left(\mathbf{V}(F) \cap \pi_1^{-1}(\mathbf{x})\right) & \mathbf{x} \mapsto \pi_3(\mathcal{A}(\mathbf{x})) \end{array}$$

For class A locus problems, we can give a formal generic definition of locus in algebraic terms.

Definition 7.3 (Algebraic locus) The locus L associated to the parametric polynomial system $F(\mathbf{x}, \mathbf{u})$ is the set

$$L = \pi_1(\mathbf{V}(F)) \subset \mathbb{C}^n.$$

Definition 7.4 (Normal and non-normal locus) A point $\mathbf{x} \in \mathbb{C}^n$ of the locus is *normal* if $\dim(\mathcal{A}(\mathbf{x})) = 0$. The points $\mathbf{x} \in \mathbb{C}^n$ of the locus for which $\dim(\mathcal{A}(\mathbf{x})) > 0$ are called *non-normal*. The set of all normal points is called the *normal locus* and the set of all non-normal points is called the *non-normal locus*.

Thus, for normal points, there exist a finite number of anti-images of each point of the tracer, whereas for the non-normal points, there exist an anti-image set of dimension greater than 0.

With this definition we see that the locus points of Example 7.2 are normal locus points, as each point has a single anti-image.

Proposition 7.5 *The* normal *locus* L_{nor} *and the* non-normal *locus* L_{nonor} *are disjoint constructible sets. We have*

$$L = L_{\text{nor}} \oplus L_{\text{nonor}},$$

and each of both subsets can be decomposed into irreducible components

$$C_i = \mathbf{V}(\mathfrak{p}_i) \setminus \left(\bigcup_j \mathbf{V}(\mathfrak{p}_{ij}) \right), \tag{7.1}$$

where all the $\mathfrak{p}_i$ *and all the* $\mathfrak{p}_{ij}$ *are prime ideals, being*

$$L_{\text{nor}} = \bigcup_j C_{\text{nor},j}, \qquad L_{\text{nonor}} = \bigcup_j C_{\text{nonor},j}.$$

Proof Consider the ring $R = \mathbb{Q}[\mathbf{x}][\mathbf{u}]$ and determine

$$G = \texttt{grobcov}(F) = \left\{ G_i = (\text{lpp}_i, B_i, S_i) : 1 \leq i \leq s \right\}$$

The segments S_i are disjoint locally closed sets, and for every point $\mathbf{x} \in S_i$ is $\dim(\mathcal{A}(\mathbf{x})) = \dim(\text{lpp}_i)$. Thus all the segments for which $\dim(\text{lpp}_i) = 0$ contain only normal locus points, and all the segments for which $\dim(\text{lpp}_i) > 0$ contain only non-normal points. Thus, the union of all normal point segments is constructible and is equal to L_{nor}. As well, the union of all non-normal point segments is equal to L_{nonor}. By construction, both sets are disjoint. We can use Algorithm 2.3: ConsLevels of Chap. 2 for obtaining their canonical forms. But we can even use Algorithm 2.6: LCUnion, which is more effective and will also obtain the result, because the union is in general locally closed and not only constructible. The result can be decomposed into components of the form (7.1), and the proposition is proved. □

The proof of Proposition 7.5 is constructive. We can follow it to obtain the proper locus components of the normal and non-normal loci.

We can refine the taxonomy classifying into subclasses the normal and non-normal components, analyzing their anti-images. For a normal component C,

we can consider the whole anti-image $\mathcal{A}(C)$ over the **u** variables and the anti-image over the mover variables $\mathcal{A}_m(C)$. Learning from certain loci problems (see Example 7.10), we observe that the second anti-image gives a more precise classification in the case of normal components. So we will use the anti-image $\mathcal{A}_m(C)$ over the mover variables, even if in some problems the concept of mover is not apparent and we must force appropriate variables to play the role of mover coordinates. S a consequence, we give the following definitions for classifying the normal point components.

Definition 7.6 ("Normal" and "Special" components) Let C be a component of the normal locus and let **w** be the subset of the auxiliary variables **u** representing the mover coordinates (if they exist), or the last n' auxiliary variables **u** conveniently chosen. A component C of the normal locus is "Normal" if $\dim(C) = \dim(\mathcal{A}_m(C))$. But it can happen that $\dim(C) > \dim(\mathcal{A}_m(C))$, and then the component is "Special".

Thus, for a "Special" component, its anti-image over the mover variables is of a dimension smaller than the dimension of the component itself, whereas for a "Normal" component, its anti-image over the mover variables is of the same dimension. Of course, the dimension of the anti-image cannot be higher than the dimension of the component, as each normal point has only a finite number of anti-images. We also distinguish two kinds of components of the non-normal locus.

Definition 7.7 ("Degenerate" and "Accumulation" components) A component of the non-normal locus is "Degenerate" if its dimension is $n - 1$. If its dimension is smaller than $n - 1$, then it is an "Accumulation" component.

The given taxonomy is motivated by the interpretation of the solutions in a lot of loci problems of different types. In this chapter we have restricted the study to loci problems that do not fill the whole parameter space $\mathbb{C}^n$ or an n-dimensional set. In the next chapter we will analyze some problems giving rise to n-dimensional sets.

We now develop the algorithms that using the GC allow the computation of the locus components and their taxonomy, following Definitions 7.6 and 7.7. First, we consider the usual loci problems for which the generic GC segment has no solution. For these problems we design the algorithm LOCUS2 that follows the definitions given in Sect. 7.2. Then we consider a problem (Example 7.13) where the generic GC segment has solutions that need to be analyzed in order to prevent some points of the mover from obtaining a correct geometric locus. This will justify the design of LOCUS algorithm before calling on the LOCUS2 algorithm.

Let us begin with an example using the `grobcov` command for the automatic locus computation to illustrate the design of the LOCUS2 algorithm.

7.3.1 Illustrative Example

Example 7.8 (Pascal's Limaçon: "Normal" and "Special" components) Pascal's Limaçon is described by Sutherland [79].

Let O be a fixed point on a circle c of radius r. Let ℓ be a line passing through O and the mover point $M(x_1, y_1)$ that glides on c. Let $T(x, y)$ be the tracer point located on ℓ at a fixed distance k from M. Pascal's Limaçon is the locus of points T as M moves on the circle c.

To fix ideas let $O = (0, 2)$, $r = 2$ and $k = 1$. The equations become

$$c : x_1^2 + y_1^2 - 4 = 0$$
$$\ell : xy_1 - yx_1 - 2x + 2x_1 = 0$$
$$c_1 : (x - x_1)^2 + (y - y_1)^2 - 1 = 0.$$

We have three equations and $m = 2$ variables as expected for a class A locus problem. We also have a mover point $M(x_1, y_1)$.

Let us first use standard lex Gröbner basis elimination Algorithm 7.1: STDLOCUS (Fig. 7.2).

```
> LIB "grobcov.lib";
> ring R=(0,x,y),(x1,y1),lp;
> short=0;
> ideal F96=x1^2+y1^2-4,x*y1-y*x1-2*x+2*x1,(x-x1)^2+(y-y1)^2-1;
> stdlocus(F96);
[1]:
   _[1]=(x^4+2*x^2*y^2-9*x^2+y^4-9*y^2+4*y+12)
[2]:
   _[1]=(x^2+y^2-4*y+3)
```

We see how, ordinary Gröbner basis, joint with prime decomposition, allows us to determine two components. Nevertheless, the ordinary Gröbner basis is not able

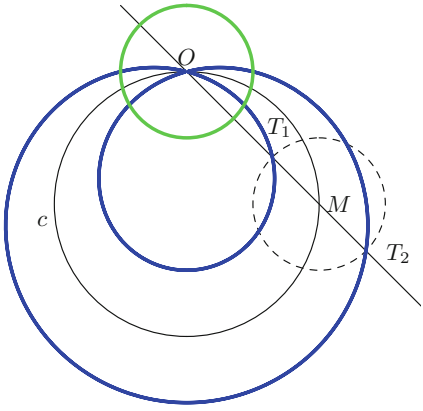


Fig. 7.2 "Normal" and "Special" components of Pascal's Limaçon

to discern their taxonomies. We can instead use `grobcov`:

```
> LIB "grobcov.lib";
> ring R=(0,x,y),(x1,y1),lp;
> short=0;
> ideal F96=x1^2+y1^2-4, x*y1-y*x1-2*x+2*x1, (x-x1)^2+(y-y1)^2-1;
> def G96=grobcov(F96);
> G96;
G96=
[1]:
   [1]:
      _[1]=1
   [2]:
      _[1]=1
   [3]:
      [1]:
         [1]:
            _[1]=0
         [2]:
            [1]:
               _[1]=(x^4+2*x^2*y^2-9*x^2+y^4-9*y^2+4*y+12)
            [2]:
               _[1]=(x^2+y^2-4*y+3)
[2]:
   [1]:
      _[1]=y1
      _[2]=x1
   [2]:
      _[1]=(8*y-12)*y1+(x^4+2*x^2*y^2-2*x^2*y-6*x^2
           +y^4-2*y^3-14*y^2+26*y-3)
      _[2]=(8*y^2-28*y+24)*x1+(x^5+2*x^3*y^2-2*x^3*y
          -6*x^3+x*y^4-2*x*y^3-14*x*y^2+42*x*y-27*x)
   [3]:
      [1]:
         [1]:
            _[1]=(x^4+2*x^2*y^2-9*x^2+y^4-9*y^2+4*y+12)
         [2]:
            [1]:
               _[1]=(2*y-3)
               _[2]=(4*x^2-3)
            [2]:
               _[1]=(y-2)
               _[2]=(x)
      [2]:
         [1]:
            _[1]=(x^2+y^2-4*y+3)
         [2]:
            [1]:
               _[1]=(2*y-3)
               _[2]=(4*x^2-3)
[3]:
   [1]:
```

```
      _[1]=y1^2
      _[2]=x1
   [2]:
      _[1]=y1^2-3*y1+2
      _[2]=x1+(2*x)*y1+(-4*x)
   [3]:
      [1]:
         [1]:
            _[1]=(2*y-3)
            _[2]=(4*x^2-3)
         [2]:
            [1]:
               _[1]=1
[4]:
   [1]:
      _[1]=y1
      _[2]=x1^2
   [2]:
      _[1]=4*y1-7
      _[2]=16*x1^2-15
   [3]:
      [1]:
         [1]:
            _[1]=(y-2)
            _[2]=(x)
         [2]:
            [1]:
               _[1]=1
```

The generic first segment establishes that there are no solutions except when $x^4 + 2x^2y^2 - 9x^2 + y^4 - 9y^2 + 4y + 12 = 0$ or $x^2 + y^2 - 4y + 3 = 0$. The interesting second segment shows that there exist solutions on

$$S_{21} = \mathbf{V}(x^4 + 2x^2y^2 - 9x^2 + y^4 - 9y^2 + 4y + 12) \setminus (\mathbf{V}(2y - 3, 4x^2 - 3) \cup \mathbf{V}(y - 2, x))$$

and

$$S_{22} = \mathbf{V}(x^2 + y^2 - 4y + 3) \setminus \mathbf{V}(2y - 3, 4x^2 - 3).$$

The set of leading power products in segment 2 is $\mathrm{lpp}_2 = \langle y_1, x_1 \rangle$, showing that for each point (x, y) in the subsets S_{21} and S_{22}, the anti-image is zero-dimensional (in fact unique for the points in them). Thus, the points of both subsets are *normal* points following our Definition 7.4. The holes of segment 2, $\mathbf{V}(2y - 3, 4x^2 - 3)$ and $\mathbf{V}(y - 2, x)$ are described in segments 3 and 4 respectively and also have zero-dimensional anti-image, their lpp being respectively $\mathrm{lpp}_3 = \langle y_1^2, x_1 \rangle$ and $\mathrm{lpp}_4 = \langle y_1, x_1^2 \rangle$ (having thus two anti-images). Thus all the points in $C_1 = \mathbf{V}(x^4 + 2x^2y^2 - 9x^2 + y^4 - 9y^2 + 4y + 12)$ and $C_2 = \mathbf{V}(x^2 + y^2 - 4y + 3)$ are *normal* points. We can say that the set of complete components of normal

points is $L_{\text{nor}} = \{C_1, C_2\}$. We see that the GC, even without more elaboration, provides directly the classification of the components into normal and non-normal components, and can provide components with holes (i.e. locally closed sets) instead of simple hypersurfaces. From now on, we develop more algorithms to obtain the proper locus components and their taxonomy.

The next step is deciding, for each normal point component in L_{nor}, if it is a "Normal" or a "Special" component. Before discussing how to do it, let us denote NS to be the algorithm for deciding this and for obtaining the anti-image of the component. We shall come back to its design. In this example we do not have non-normal point components, and L_{nonor} is empty. Nevertheless, in other loci computations L_{nonor} is not empty, and we also have to decide if the components inside are "Accumulation" or "Degenerate". Let us denote AD to be the routine to detect this. Before describing how to proceed for building effective routines NS and AD, we describe LOCUS2 Algorithm 7.2 (which call inside the routines NS and AD).

7.4 Algorithm LOCUS2

Algorithm 7.2: LOCUS2 begins setting $L_{\text{nor}} = L_{\text{nonor}} = \emptyset$. Then, for every element G_i of the Gröbner Cover G, if the dimension of the lpp_i is 0, all the segments S_{ij} in G_i are formed by normal points, and thus it adjoins them to L_{nor}. If not, it adjoins them to L_{nonor}, as they are formed by non-normal points.

After having joined all the segments formed by normal points in L_{nor} and all the segments formed by non-normal points in L_{nonor}, LOCUS2 determines the P-canonical form of the constructible sets formed by them. They will consist of complete components $L_{\text{nor}} = \bigcup_k C_{\text{nor},k}$ and $L_{\text{nonor}} = \bigcup_k C_{\text{nonor},k}$. For this purpose we can use either Algorithm 2.6: LCUNION or Algorithm 2.3: CONSLEVELS of Chap. 2. As we expect the union to be locally closed, it is preferable to use the former which is more efficient.

GROBCOV classifies the parameter space into three sets: (1) the set of points without solution, (2) the set of normal points, and (3) the set of non-normal points. Each of these sets are given as a disjoint union of locally closed sets. The union is constructible, but each component is the union of disjoint locally closed sets. Usually, its union will be locally closed, even if in higher dimensions it could exist also some normal point component inside a hole of a normal component. But even so, Algorithm 2.6: LCUNION will obtain these components, and so we can still use it in Algorithm 7.2: LOCUS2 for obtaining the true components.

In our Example 7.8, at this step, the result of using Algorithm LCUNION will obtain the two components $L_{\text{nor}} = [C_1, C_2]$ of normal points.

$L \leftarrow$ **Locus2**(G, F)
Central sub-algorithm of LOCUS used after the reduction
of the generic segment by LOCUS.
`Input`: $G = \{(\text{lpp}_i, B_i, S_i) : i \le i \le s\}$ the Gröbner Cover of
the locus ideal (with the generic segment) with basis $\{1\}$,
F is the system describing the locus
`Output`: $L = \{(C_k, H_k) : 1 \le k \le t\}$, the components of the locus
in P-representation, and their taxonomies, where
$C_k = \{\mathfrak{q}_k, \{\mathfrak{q}_{k\ell} : 1 \le \ell \le t_k\}\}$, and
$H_k = (d_k, \text{taxonomy}_k, A_k)$, being $A_k = \mathcal{A}_m(C_k)$,
$d_k = \dim(A_k)$, and taxonomy ="Normal" if $d_k = \dim(C_k)$
and "Special" if $d_k < \dim(C_k)$ for normal point components;
for non-normal point components, $H_k = (d_k, \text{taxonomy}_k)$,
being $d_k = \dim(C_k)$ and
if $d_k = n - 1$ then taxonomy= "Degenerate" and
if $d_k < n - 1$ then taxonomy= "Accumulation".
begin
$L_{\text{nor}} = L_{\text{nonor}} = \emptyset$
for all i in G
if $\dim(\text{lpp}_i) = 0$ **then**
for all components S_{ij} in G_i **do**
$L_{\text{nor}} = L_{\text{nor}} \cup S_{ij}$ # normal segments
end for
else # $\dim(\text{lpp}_i) > 0$
for all components S_{ij} in G_i **do**
$L_{\text{nonor}} = L_{\text{nonor}} \cup S_{ij}$ # non-normal segments
end for
end if
end for
$S_{\text{nor}} = \text{LCUNION}(L_{\text{nor}}) = \cup_k C_{\text{nor},k}$ # components
$S_{\text{nonor}} = \text{LCUNION}(L_{\text{nonor}}) = \cup_k C_{\text{nonor},k}$ # components
for all $C_k \in L_{\text{nor}}$ **do**
substitute C_k by $(C_k, H_k) = \text{NS}(F, G, C_k)$
end for
for all $C_k \in L_{\text{nonor}}$ **do**
substitute C_k by $(C_k, H_k) = \text{AD}(C_k)$
end for
return $[L_{\text{nor}}, L_{\text{nonor}}]$
end

Algorithm 7.2: LOCUS2

Now LOCUS2 has to analyze each $C_{\text{nor},k}$ component in L_{nor} to determine if it is "Normal" or "Special", and each $C_{\text{nonor},k}$ component in L_{nonor} to determine if it is "Accumulation" or "Degenerate". For these purposes LOCUS2 uses specific algorithms NS and AD.

Separating "Accumulation" and "Degenerate" components of the non-normal locus is very simple. We only need to determine the dimension of the component. If the dimension is equal to $n - 1$, then the component is "Degenerate" and if it is smaller, then it is "Accumulation". (See Algorithm 7.3: AD).

$L \leftarrow$ **AD**(C)
Subalgorithm of LOCUS2 to determine the
taxonomy of a component C of the non-normal locus
and its taxonomy "Degenerate" or "Accumulation".
Input: $C = \{\mathfrak{p}, \{\mathfrak{p}_j : 1 \leq j \leq r\}$ the component to be analyzed,
Output: (C, H)
where $H = (d,$ taxonomy), being $d = \dim(C)$ and
taxonomy ="Degenerate" if $d = n - 1$ and
taxonomy ="Accumulation" if $d < n - 1$.
begin
$d = \dim(C)$
if $d = n - 1$ **then** $H = (d,$ "Degenerate")
else $H = (d,$ "Accumulation")
end if
return (C, H)
end

Algorithm 7.3: AD: determines the taxonomy of a non-normal locus component

7.4.1 Detecting the Taxonomy of a Normal Point Component

Determining the taxonomy "Normal" or "Special" of a component of the normal locus is more expensive and elaborate. We describe here Algorithm 7.4: NS, for obtaining the anti-image of the top of the component on the mover variables. In the following, $C = (\mathfrak{p}, \{\mathfrak{p}_j : 1 \leq j \leq r\})$ will be a component of the normal locus with top $\mathfrak{p}$, and holes $\{\mathfrak{p}_j : 1 \leq j \leq r\}$. (Usually it will have dimension $n - 1$). NS determines $\mathcal{A}_m(C)$, as well as its dimension, and as a consequence, its "Normal" or "Special" character. The design of NS follows Definition 7.6.

Algorithm 7.4: NS takes as input the ideal F describing the locus problem, its grobcov G (previously computed inside Algorithm 7.5: LOCUS), and the component C of normal points. From the GC, NS selects the basis B, corresponding to the given component C.

Then NS determines the set N of polynomials in $\mathbb{Q}[\mathbf{u}]$ that are null in some hole of C, and consequently cannot be zero on an open subset of $\mathcal{A}(C)$. This will be used later to reduce the scope of the computed basis to the strict points of the anti-image of $\mathcal{A}(C)$.

In the next step NS adds F to B and specializes the result on C, as the initial basis B can also be a basis specializing to the GB of F for other components of the same segment. This basis specializes to the reduced GB of F for every point of the component. So the basis B represents all the points of $\mathcal{A}(C)$.

Then NS determines the lex$(\mathbf{u}, \mathbf{x})$ GB of B, in order to be able to use Extension Theorem (see [25], page 114) for eliminating the variables $\mathbf{x}$ and obtain the basis of $\mathcal{A}(C)$, or more precisely a super-set of $\mathcal{A}(C)$, as we comment below, because some polynomials can contain extraneous factors. For this, NS selects only the polynomials in the obtained basis depending only on the $\mathbf{u}$ variables. This corresponds to the projection π_2 over the $\mathbf{u}$ variables.

$(d, \text{taxonomy}, \mathcal{A}_m) \leftarrow \mathbf{NS}(F, G, C)$
Subalgorithm of LOCUS2. Base ring = $\mathbb{Q}[\mathbf{x}][\mathbf{u}]$
Input: F=ideal defining the locus,
G=grobcov of F already computed by LOCUS
$C = (\mathfrak{p}, \{\mathfrak{p}_j : 1 \le j \le r\})$, the component
Output: $(d, \text{taxonomy}, \mathcal{A}_m(C))$, where $d = \dim(\mathcal{A}_m(C))$ and
taxonomy="Normal" if $d = \dim(C)$ and
taxonomy="Special" if $d < \dim(C)$
begin
nv = number of variables; np = number of parameters
$nw = \min(nv; np)$ by default
but can be set optionally to a higher value
$\mathbf{w}$ = last nw variables
B = basis of the segment of G corresponding to the top of C
$N = \emptyset$.
for each hole of C, search the segment i of G where
it is included, and let B_i be the basis of segment i
$N = N \cup (B_i \cap \mathbb{Q}[\mathbf{u}])$
end for
$B = B \cup F$
$B = \text{REDUCE}(B, C)$ in lex($\mathbf{x}$, $\mathbf{u}$)
$B = \text{GB}(B)$ in lex($\mathbf{x}$, $\mathbf{u}$)
$B = B \cap \mathbb{Q}(\mathbf{u})$
$A = \emptyset$
for all $f \in B$ **do**
$f = 1$
$H = \text{factors}(f)$
for all $g \in H$ **do**
if $g \notin N$ **then** $f = fg$ **end if**
end for
$A = A \cup f$
end for
$A = \text{GB}(A)$ in lex($\mathbf{u}$)
$\mathcal{A}_m = A \cap \mathbb{Q}[\mathbf{w}]$
$d_0 = \dim(C)$
$d = \dim(\mathcal{A}_m)$ (in $\mathbb{Q}[\mathbf{w}]$)
if $d = d_0$ **then** taxonomy="Normal"
else taxonomy="Special"
end if
return $(d, \text{taxonomy}, \mathcal{A}_m)$
end

Algorithm 7.4: NS: determines the anti-image of the normal point component C, its dimension and taxonomy

Then NS factorizes every polynomial in A and eliminates all factors that are in N as they are non-null on an open set of A. Finally NS eliminates the polynomials containing variables not in $\mathbf{w}$ to obtain the projection $\mathcal{A}_m(C)$ on the mover space, and computes its GB in lex($\mathbf{u}$) order to present the result.

Then NS computes the dimension of $\mathcal{A}_m(C)$, compares this dimension with the dimension of C, and consequently determines the taxonomy of C.

$L \leftarrow$ **Locus**(F)
`Input:` F= ideal defining the locus problem.
The algorithm considers $\mathbf{w}$ = the last n variables.
(even if they are not properly mover variables.)
`Output:` L, the components of the locus in P-representation.
$L = \{\{\mathfrak{q}_k\{\mathfrak{q}_{k\ell} : 1 \le \ell \le t_k\}, H_k\} : 1 \le k \le r\}$, where
$H_k = (d_k, \text{taxonomy}_k, A_k)$, where A_k is the anti-image of C,
d_k its dimension, and taxonomy ="Normal" if $d_k = n - 1$
and "Special" if $d_k < n - 1$ for normal point components,
and $H_k = (d_k, \text{taxonomy}_k)$, where d_k is the dimension of the
component and taxonomy= "Accumulation" if $d_k = n - 1$ and
taxonomy="Degenerate" if $d_k < n - 1$, for non-normal
point components.
begin
$G =$ GROBCOV(F)
if lpp$(G[1]) = 1$ **then return**(LOCUS2)(G, F))
else
if exists $B_0 \subset B_1$ of the basis of the generic segment only
dependent on the variables $\mathbf{w}$, with $\dim(B_0) < \dim(C)$,
then
set basis and lpp of $G[1] = \{1\}$
for all segments G_i of G **do**
$t = 0$
for all polynomials $p \in B_i$ **do** $p =$ radical(p) **end for**
for all polynomials $q \in B_0$ **do**
if PDIV$(q, B_i) \neq 0$ **then** $t = 1$ **end if**
end for
if $t = 0$ eliminate G_i from G
end for
return (LOCUS2(G, F)) (the reduced G)
else return$(\emptyset)$
end if
end

Algorithm 7.5: LOCUS

We remark that in the case of the space of dimension $n = 2$, the component C has dimension 1, and the anti-image will have either dimension 1 for "Normal" components or 0 for "Special" ones.

In problems where a proper mover point gliding on a hypersurface H exists, the anti-image of a "Normal" component will be H. When a proper mover point does not exist, it can be interesting to analyze the anti-image and study the trajectories of the $\mathbf{w}$ variables.

The SINGULAR library has incorporated all the described algorithms. At his time, we have designed a more general LOCUS algorithm that calls LOCUS2 after a previous analysis of the GC when its generic segment does not have basis 1. We describe it in Sect. 7.5. But for usual problems where the generic segment has basis {1}, LOCUS calls directly the LOCUS2 algorithm, so that in the present problem Example 7.8 we can use simply `locus`. It will internally call `locus2` that uses

Algorithm 7.4: NS to determine the anti-image of the normal point components. The result is:

```
> LIB "grobcov.lib";
> ring R=(0,x,y),(x1,y1),lp;
> short=0;
> ideal F96=x1^2+y1^2-4,
                (y-2)*x1-x*y1+2*x,
                (x-x1)^2+(y-y1)^2-1;
> locus(F96);
[1]:
   [1]:
      _[1]=(x^4+2*x^2*y^2-9*x^2+y^4-9*y^2+4*y+12)
   [2]:
      [1]:
         _[1]=1
   [3]:
      [1]:
         1
      [2]:
         Normal
      [3]:
         _[1]=x1^2+y1^2-4
[2]:
   [1]:
      _[1]=(x^2+y^2-4*y+3)
   [2]:
      [1]:
         _[1]=1
   [3]:
      [1]:
         0
      [2]:
         Special
      [3]:
         _[1]=y1^2-3*y1+2
         _[2]=x1^2+3*y1-6
```

We obtain, as expected, the respective anti-images of both components. The first component C_1 has anti-image $A_1 = \mathbf{V}(x_1^2 + y_1^2 - 4)$, which has dimension 1, and represents all the positions of the mover. Thus C_1 is declared "Normal".

On the contrary, component C_2 has anti-image $A_2 = \mathbf{V}(y_1^2 - 3y_1 + 2, x_1^2 + 3y_1 - 6)$, which contains only the points $P_0 = (0, 2)$, $P_1 = (\sqrt{3}, 1)$ and $P_2 = (-\sqrt{3}, 1)$, and has dimension 0. It is thus declared "Special". The anti-image of the circle is really the unique point P_0, but the positions P_1 and P_2 of the mover generate the points of C_1 that intersect the circle C_2. Thus they belong also to the anti-image of C_2.

So we see the precise information provided by LOCUS2 algorithm. Using instead the simple algorithm STDLOCUS the result is

```
> LIB "gobcov.lib";
> ring R=(0,x,y),(x1,y1),lp;
> ideal F96=x1^2+y1^2-4,
                (y-2)*x1-x*y1+2*x,
                (x-x1)^2+(y-y1)^2-1;
> stdlocus(F96);
[1]:
   _[1]=(x^4+2*x^2*y^2-9*x^2+y^4-9*y^2+4*y+12)
[2]:
   _[1]=(x^2+y^2-4*y+3)
```

providing both components, but not giving information about their taxonomies, or their anti-images.

The software accepts the option ("anti-image",0) to avoid making public the anti-image of "Normal" components and can be set when the user is not interested in them. The anti-image of "Special" components continues to be printed with this option.

Example 7.9 (Simple locus with "Special" and "Accumulation" components) We give a new very elementary example with a simple geometric interpretation of a locus with a "Special" and an "Accumulation" component.

Consider the following locus problem. Let the mover point $M(x_1, y_1)$ be on the circle $x_1^2 + y_1^2 - 1$, and the tracer point $T(x, y)$ be in the intersection of line $x = c$ and the line passing through the origin and the tracer point.

There are two distinct locus cases depending on the constant c. If $c \neq 0$ one obtains the single "Normal" locus component $x = c$. But let us consider the case $c = 0$.

The locus system ideal is S122b $= \langle x_1^2 + y_1^2 - 1,\ x,\ xy_1 - yx_1 \rangle$. See Fig. 7.3.[1] Computing its `locus` using the *grobcov.lib* library, we have:

```
> LIB "grobcov.lib";
> ring R=(0,x,y),(x1,y1),lp;
> ideal S122=x1^2+y1^2-1,x,x*y1-y*x1;
> locus(S122);
[1]:
   [1]:
      _[1]=(x)
   [2]:
      [1]:
         _[1]=(y)
         _[2]=(x)
   [3]:
```

[1]We shall use the public command `locus` instead of the internal `locus2`. For locus problems for which the generic first segment of `grobcov(S122)` has basis {1}, `locus` is equivalent to `locus2`, the first one being accessible to the user.

```
      [1]:
         0
      [2]:
         Special
      [3]:
         _[1]=y1^2-1
         _[2]=x1
[2]:
   [1]:
      _[1]=(y)
      _[2]=(x)
   [2]:
      [1]:
         _[1]=1
   [3]:
      [1]:
         0
      [2]:
         Accumulation
```

We obtain two components: $C_1 = \mathbf{V}(x) \setminus \mathbf{V}(x, y)$ which is "Special" with anti-image $\mathbf{V}(y_1^2 - 1, x_1)$ and $C_2 = \mathbf{V}(x, y)$, whose taxonomy is "Accumulation". The geometric interpretation is the followingr. For all positions of the mover in the circle, the line from the origin to the mover intersects the line $x = 0$ at the origin, becoming therefore an "Accumulation" point. But when the mover reaches point $(0, 1)$ or point $(0, -1)$ the line intersects $x = 0$ along the whole line, and thus generates the "Special" $x = 0$ component, whose anti-image are points $(0, 1)$ and $(0, -1)$. If we have computed the anti-image of the "Accumulation" component C_2, we have obtained the circle $\mathbf{V}(x_1^2 + y_1^2 - 1)$.

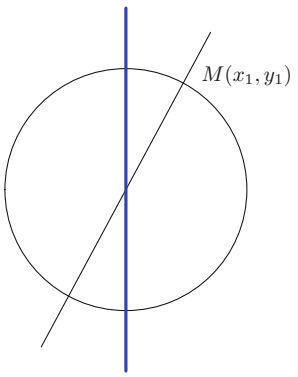


Fig. 7.3 Simple locus with a "Special" and an "Accumulation" components

In this example `stdlocus` only provides the "Special" component and does not distinguish it from the "Accumulation" point.:

```
> LIB "grobcov.lib";
> ring R=0,(x,y,x1,y1),lp;
> ideal S122=x1^2+y1^2-1,x,x*y1-y*x1;
> stdlocus(S122);
[1]:
   _[1]=(x)
```

Example 7.10 Consider now the Russell Linkage problem [46, 47], where the distinction between "Normal" and "Special" components depends on the consideration of the projection of the anti-image over the mover coordinates. This example, after discussions with F. Botana and the authors of [47], has lead us to define the taxonomy of normal components depending on the projection of the anti-image over the mover variables instead of considering the anti-image over the whole **u** variables. Let

(1) $M = (x_1, y_1)$ be the mover point gliding on the line $x_1 = 3$.
(2) $A(3, 5)$ and $B(3, 2)$ be fixed points.
(3) $C =$ midpoint of $AB = (3, 7/2)$.
(4) $r = \overline{AC} = 3/2$.
(5) $c =$circle of center M and radius r: $(x_2 - x_1)^2 + (y_2 - y_1)^2 - (3/2)^2$.
(6) $d =$circle of center B: $(x_2 - 3)^2 + (y_2 - 2)^2 - (3/2)^2$.
(7) $E(x_2, y_2)$ intersection of circles c and d.
(8) $T(x, y)$ the tracer of the locus is the symmetrical point of M with respect to E.
(9) Find the locus of $T(x, y)$.

Computing the locus we have:

```
> ring R=(0,x,y),(x2,y2,x1,y1),lp;
> ideal F=4*(x2-x1)^2+4*(y2-y1)^2-9,
              4*(x2-3)^2+4*(y2-2)^2-9,
              x1-3,
              x+x1-2*x2,
              y+y1-2*y2;
> locus(F);
[1]:
   [1]:
      _[1]=(x^2-6*x+y^2-4*y+4)
   [2]:
      [1]:
         _[1]=1
      [2]:
         _[1]=1
   [3]:
      [1]:
         0
      [2]:
         Special
      [3]:
         _[1]=y1-2
```

```
          _[2]=x1-3
[2]:
   [1]:
      _[1]=(y-2)
   [2]:
      [1]:
         _[1]=1
   [3]:
      [1]:
         1
      [2]:
         Normal
      [3]:
         _[1]=x1-3
```

For the component $C_1 = (x-3)^2+(y-2)^2-9$, `locus` shows that the projection of its anti-image over the mover variables is $(y_1 - 3, x_1 - 2)$, i.e., the mover on position B. Thus the set is zero-dimensional and the component is "Special". But if instead we consider the whole anti-image, we must add also $4(x_2-3)^2+4(y_2-2)^2-9$ and then the whole anti-image is of dimension 1. If we used the whole anti-image, we should declare the component as being "Normal", which contradicts the fact that it is produced by a single position of the mover. Here we have a natural mover point, and there is no doubt about the nature of the component to be "Special". If instead we consider as mover the non-natural variables (x_2, y_2) we obtain that the component is "Normal", which contradicts our intuition. The point $x_1 = 2$, $y_1 = 3$ generates the circle $4(x_2 - 3)^2 + 4(y_2 - 2)^2 - 9$ which generates the "Special" component C_1 (Fig. 7.4).

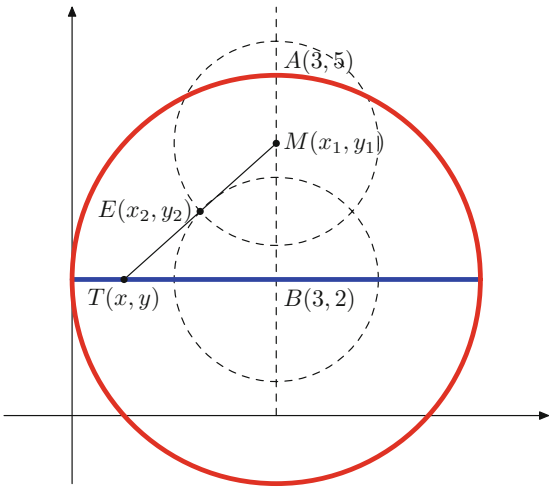


Fig. 7.4 Simple locus with "Special" component

In this example we also want to consider the projection of the locus over the reals. We shall not discuss this problem here, but considering the geometric construction, it is obvious that not the whole line $y = 2$ forms part of the real locus, but only the segment $0 \leq x \leq 6$. For the external values of x, the circles centred at B and M do not intersect, and (x_2, y_2) becomes complex.

Example 7.11 Consider now the simple problem of a family of curves depending on m parameters that are constricted by m constraints. Consider, for example the following family of circles:

$$F_1 = (x-a)^2 + (y-b)^2 - r^2$$

restricted by the conditions

$$(a-1)(a-2)(a-3), b-a^2, r-b$$

Using `locus` we have:

```
> LIB "grobcov.lib";
> ring R=(0,x,y),(r,b,a),lp;
> short=0;
> ideal F1c=(x-a)^2+(y-b)^2-r^2,
          (a-1)*(a-2)*(a-3),
          b-a^2,
          r-b;
> locus(F1c);
[1]:
   [1]:
      _[1]=(x^2-4*x+y^2-8*y+4)
   [2]:
      [1]:
         _[1]=1
   [3]:
      [1]:
         0
      [2]:
         Special
      [3]:
         _[1]=a^3-6*a^2+11*a-6
         _[2]=b-a^2
[2]:
   [1]:
      _[1]=(x^2-2*x+y^2-2*y+1)
   [2]:
      [1]:
         _[1]=1
   [3]:
      [1]:
         0
      [2]:
         Special
```

```
      [3]:
         _[1]=a^3-6*a^2+11*a-6
         _[2]=b-a^2
[3]:
   [1]:
      _[1]=(x^2-6*x+y^2-18*y+9)
   [2]:
      [1]:
         _[1]=1
   [3]:
      [1]:
         0
      [2]:
         Special
      [3]:
         _[1]=a^3-6*a^2+11*a-6
         _[2]=b-a^2
```

We obtain the three circles, which considered as a locus, are "Special" components, as expected, because they all correspond to fixed values of (a, b, r). Observe that the anti-image determined by the NS algorithm is in fact larger than the proper anti-image for each particular circle. The NS algorithm is not able to separate the concrete values of (a, b, r) for each circle. This will happen for every locus of this type.

In this example chose (a, b) as mover variables. But the declaration of the components as "Special" is independent of the choice of the mover, even if we use the projection over all the three variables $\mathbf{u}$ that in this case are (a, b, r).

Example 7.12 (Locus of the Steiner-Lehmus Theorem: "Degenerate" component) To illustrate "Degenerate" components, let us now come back to the example already discussed regarding the Steiner-Lehmus Theorem (see Chap. 6). Now we consider it only as a locus problem, i.e., we are interested only in finding the locus components and their taxonomies. We do not focus our interest in all the details provided by `grobcov`.

In this example, (see Fig. 6.2 of Chap. 6) we have two tracer variables corresponding to the coordinates of vertex $C(x, y)$, and $m = 6$ variables $\mathbf{u}$. But instead, there does not exist an explicit mover point. We have to choose two mover variables that seem appropriate, and observe if the result, after this choice, is reasonable.

We have $6 + 1 = 7$ equations. So we expect one degree of freedom, and the problem corresponds well to Definition 7.1 loci class A. A possible choice of mover variables are the coordinates of point $M(x_1, y_1)$, the end of the bisector of A. We observe that the classification into "Normal" and "Special" components can depend on this choice, and it is convenient to analyze the obtained result using the `locus` command to verify if it is reasonable.

The system defining the Steiner-Lehmus locus problem is detailed in Chap. 6, Sect. 6.1. We apply first the command `stdlocus`, and then `locus` command of the library *grobcov.lib*, and compare both. For the second, having chosen (x_1, y_1) as mover, we place them as the latests $\mathbf{u}$ variables in the definition of the ring (Fig. 7.5).

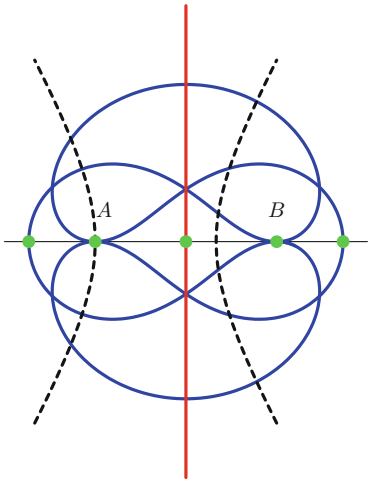


Fig. 7.5 Components of the Steiner-Lehmus locus, and the anti-image of C_2

```
> LIB "grobcov.lib";
> ring R=(0,x,y),(p,r,y2,x2,y1,x1),lp;
> ideal S93=(x+1)^2+y^2-(p+1)^2,
                (x-1)^2+y^2-(1-r)^2,
                x*y1-y*x1-y1+y,
                x*y2-y*x2+y2-y,
                -2*y1+y*x1-(x+p)*y1+y,
                2*y2+y*x2-(x+r)*y2-y,
                (x1+1)^2+y1^2-(x2-1)^2-y2^2;
> short=0;
> stdlocus(S93);
[1]:
_[1]=(8*x^10+41*x^8*y^2-56*x^8+84*x^6*y^4-164*x^6*y^2
      +144*x^6+86*x^4*y^6-148*x^4*y^4+86*x^4*y^2-176*x^4
      +44*x^2*y^8-28*x^2*y^6-148*x^2*y^4+156*x^2*y^2
      +104*x^2+9*y^10+12*y^8-74*y^6-172*y^4+137*y^2-24)
[2]:
   _[1]=(y)
[3]:
   _[1]=(x)
> locus(S93);
[1]:
   [1]:
      _[1]=(8*x^10+41*x^8*y^2-56*x^8+84*x^6*y^4-164*x^6*y^2
         +144*x^6+86*x^4*y^6-148*x^4*y^4+86*x^4*y^2-
         176*x^4+44*x^2*y^8-28*x^2*y^6-148*x^2*y^4+156*x^2*y^2
         +104*x^2+9*y^10+12*y^8-74*y^6-172*y^4+137*y^2-24)
   [2]:
      [1]:
         _[1]=(y)
         _[2]=(x-1)
      [2]:
         _[1]=(y)
```

```
            _[2]=(x+1)
         [3]:
            _[1]=(y)
            _[2]=(x^2-3)
      [3]:
         [1]:
            2
         [2]:
            normal
         [3]:
            _[1]=0
[2]:
      [1]:
         _[1]=(x)
      [2]:
         [1]:
            _[1]=(y)
            _[2]=(x)
      [3]:
         [1]:
            1
         [2]:
            Normal
         [3]:
            _[1]=y1^2-3*x1^2-2*x1+1
[3]:
      [1]:
         _[1]=(y)
      [2]:
         [1]:
            _[1]=1
      [3]:
         [1]:
            1
         [2]:
            Degenerate
```

We obtain two "Normal" components, having holes (the points on the x-axis), and one "Degenerate" component corresponding to the x axis $y = 0$, where the triangle becomes really degenerate.

Observe that the intersections between two normal components are included in both intersecting locus components, because these points are "normal" from the locus point of view.[2] On the contrary, the intersecting points between normal and non-normal ("Degenerate" and "Accumulation") components are excluded from the "Normal " components, as these points are non-normal from the locus point of view. This is carefully dealt with by Algorithm 7.2: LOCUS2 in adding together all normal points, separately from the non-normal points.

[2]The singular points of C_1 are also included in C_1, as they are locus normal-points.

We verify that the simple `stdlocus` command can obtain the three components (without the holes) and without determining their taxonomies.

Let us discuss the result regarding components (1) and (2) using `locus`.

(1) Component C_1. We observe that the taxonomy of C_1 returns only "normal" without specification of the anti-image that is set to 0. This is due to the complexity of the component (degree 10, 6 variables **u**) that allows the `locus` command to avoid the analysis of the anti-image in order to be able to finish the computations. We know that it is formed by normal points, but the character "Normal" or "Special" of the component is not determined due to the complexity of the computations.

Nevertheless, the component is clearly "Normal", as can be verified by other simple computations. We can use, for example, numerical computations and observe that for different points of the component, the values of the corresponding anti-images are different, suggesting that they are not due to a 0-dimensional set, which would be the case for a "Special" component. For example, we can do the following numerical computations in Maple:

```
> g93:=8*x^10+41*x^8*y^2-56*x^8+84*x^6*y^4-164*x^6*y^2
  +144*x^6+86*x^4*y^6-148*x^4*y^4+86*x^4*y^2-176*x^4
  +44*x^2*y^8-28*x^2*y^6-148*x^2*y^4+156*x^2*y^2+104*x^2
  +9*y^10+12*y^8-74*y^6-172*y^4+137*y^2-24:
> s1:=[fsolve(subs(x=0.1,g93),{y})];
  [{y = -1.729160967}, {y = -0.6404851638}, {y = -0.5075302918},
   {y = 0.5075302918}, {y = 0.6404851638}, {y = 1.729160967}]
> F:=[x1*y-2*y1+y-y1*(x+p),
          x2*y+2*y2-y-y2*(x+r),
          x*y2-y+y2-x2*y,
          x*y1+y-y1-x1*y,
          (x+1)^2+y^2-(p+1)^2,
          (x-1)^2+y^2-(r-1)^2,
          (x1+1)^2+y1^2-(x2-1)^2-y2^2]:
> for i to 6 do P:=s1[i];fsolve(subs(P,F),{x,r,p,x1,y1,x2,y2});od;
   {y=-1.729160967}
   {p=-2.949358266, r=3.049389580, x=-0.1000000006,
     x1=-42.44243025, x2=-37.44493417, y1=-68.28995878,
     y2=70.02128628}
   {y=-0.6404851638}
   {p=-1.793603190, r=3.550333504, x=-1.468598740,
     x1=-3.092515365, x2=0.7029627894, y1=-1.061815082,
     y2=2.327625554}
   {y =-0.5075302918}
   {p=-1.693470841, r=3.524113640, x=-1.472561965,
     x1=-2.784931929, x2=0.8032805440, y1=-0.7769138383,
     y2=1.936718503}
   {y=0.5075302918}
   {p=-1.693470841, r=3.524113640, x=-1.472561965,
     x1=-2.784931929, x2=0.8032805440, y1=0.7769138383,
     y2=-1.936718503}
   {y=0.6404851638}
   {p=-3.550333504, r=1.793603190, x=1.468598740,
     x1= 0.7029627894, x2=3.092515365, y1=-2.327625554,
     y2=1.061815082}
```

```
{y=1.729160967}
{p=-3.049389580, r=2.949358266, x=0.1000000006,
   x1=37.44493417, x2=42.44243025, y1=-70.02128628,
   y2=68.28995878}
```

and observe that the anti-images of six points of the tracer are different, and so we can determine that the component is, with high probability, "Normal".

(2) It is interesting to study the anti-image of the second "Normal" component $x = 0$. `locus` shows that the mover point $M(x_1, y_1)$ describes a hyperbola $y_1^2 - 3x_1^2 - 2x_1 + 1 = 0$. We have already studied in Chap. 6 that each point of the component $x = 0$ corresponds to an isosceles triangle, and there are two solutions concerning the equality of the bisectors of A and B: one corresponds to the equality of the internal bisectors, and another to the equality of the external bisectors.

From the equations of the locus system, and taking into account that $x = 0$ on the component, we obtain

$$y_1 = \pm\sqrt{3x_1^2 + 2x_1 - 1}$$

$$y = \frac{y_1}{1 - x_1}.$$

Using Maple, we can plot the bisector $AM = (-1, 0) - (x_1, y_1)$ and the triangle $ABC = (-1, 0), (0, 1), (0, y)$, (see Fig. 7.6) in the three different regions $x_1 < -1$,

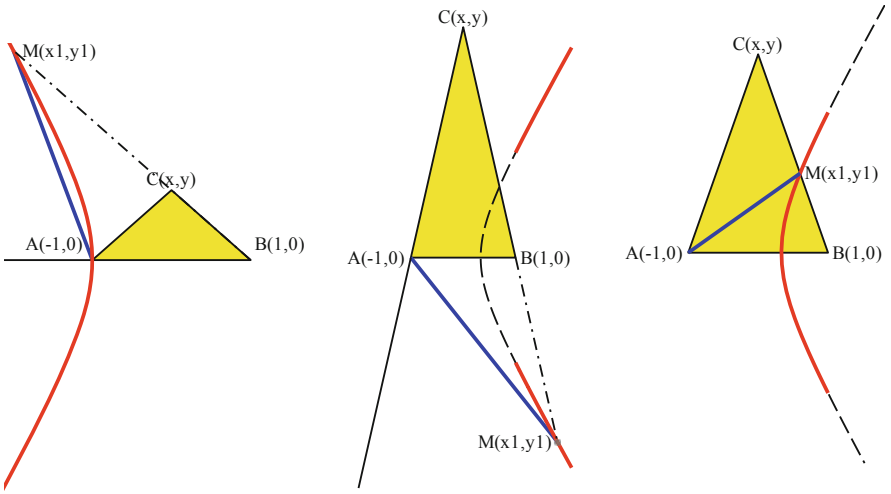


Fig. 7.6 Anti-images of component $x = 0$ of Steiner-Lehmus locus

$1 < x_1$ and $1/3 < x_1 < 1$ where y_1 is defined. The anti-image of C_2 is divided into the three following regions:

$x_1 < -1$	External bisectors	$\|A\| < \pi/4$
$1 < x_1$	External bisectors	$\|A\| > \pi/4$
$1/3 < x_1 < 1$	Internal bisectors	$\|A\| < \pi/2$

The value $x_1 = 1$ corresponds to vertex C at infinity and should be discarded. The value $x_1 = -1$ corresponds to vertex C at the origin, where the triangle is degenerate and is also to be discarded. Thus the correct anti-image of C_2 should be $V(y_1^2 - 3x_1^2 - 2x_1 + 1) \setminus (\mathbf{V}(x_1 - 1) \cup \mathbf{V}(x_1 + 1))$.

(3) Based on experience, we tend to discard "Degenerate" components as geometrically irrelevant, as they usually correspond to degenerate instances of a construction, such as the three vertices in a triangle being aligned, as in the present locus. Component C_3 produces holes in components (1) and (2) on the intersection points.

Instead, "Accumulation" points, when they occur, should be considered as forming part of the (geometric) locus, since they represent special points that are determined by infinite values of the variables.

7.5 LOCUS Algorithm

We now show an example in which the generic GC segment of the system has solutions, but nevertheless the geometric locus makes sense and will justify the design of the LOCUS algorithm.

7.5.1 Illustrative Example

Example 7.13 (Detecting bad mover positions. Problem proposed by M.A. Abánades and F. Botana [1]) There is another special situation that is to be taken into account for certain loci. Namely, problems, for which the mover point has to avoid certain points in order to obtain a proper locus.

Here is an example (see Fig. 7.7).

(1) Consider a mover point $M(x_1, y_1)$ moving on the circle $x_1^2 + y_1^2 - 9 = 0$ with center $O(0, 0)$ and radius $r = 3$.
(2) Consider fixed points $A(0, 3)$ and $B(0, -3)$.
(3) Let ℓ be the line parallel to line AM passing through O and the line m perpendicular to ℓ passing through B.
(4) Let $P(x_2, y_2)$ be the intersection of lines ℓ and m.
(5) The tracer point $T(x, y)$ is the intersection of line AP and the circle with center M and radius MP.

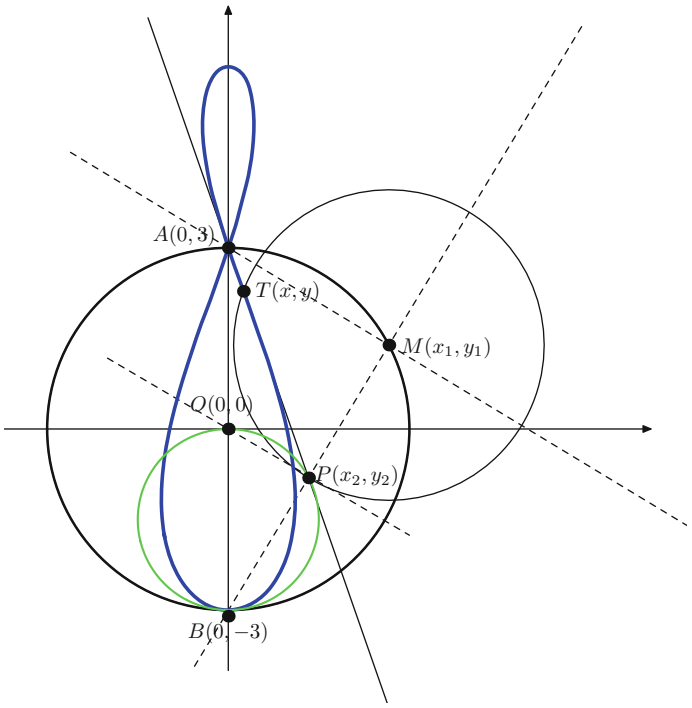


Fig. 7.7 "Normal" and "Accumulation" components of Example 7.13

The system is

$$\begin{aligned} S = {} & x_1^2 + y_1^2 - 9, \\ & - x_1 y_2 + (y_1 - 3)x_2, \\ & x_1(x_2 - x_1) + (y_1 - 3)(y_2 - y1), \\ & (y_2 - 3)x - x_2(y - 3), \\ & (x - x_1)^2 + (y - y_1)^2 - (x_1 - x_2)^2 - (y_1 - y_2)^2. \end{aligned}$$

In this problem we have $m = 4$ auxiliary variables, from which, two variables correspond properly to the mover, and $m + 1 = 5$ equations. Thus the problem corresponds well to our Definition 7.1 loci class A. Let us apply the command `grobcov` to the corresponding system:

```
> LIB "grobcov.lib";
> ring R=(0,x,y),(y2,x2,y1,x1),dp;
> ideal S=x1^2+y1^2-9,
          -x1*y2+(y1-3)*x2,
          x1*(x2-x1)+(y1-3)*(y2-y1),
          (y2-3)*x-x2*(y-3),
```

```
          (x-x1)^2+(y-y1)^2-(x1-x2)^2-(y1-y2)^2;
> short=0;
> def G=grobcov(S);
> G;
[1]:
   [1]:
      _[1]=x1
      _[2]=y1
      _[3]=y2
      _[4]=x2^2
   [2]:
      _[1]=x1
      _[2]=y1-3
      _[3]=(x)*y2+(-y+3)*x2+(-3*x)
      _[4]=x2^2+(-x^2)
   [3]:
      [1]:
         [1]:
            _[1]=0
         [2]:
            [1]:
               _[1]=(x^4+2*x^2*y^2-9*x^2*y+171*x^2+y^4-9*y^3+9*y^2
               +81*y-162)
            [2]:
               _[1]=(x^2+y^2-6*y+9)
            [3]:
               _[1]=(x^2+y^2+3*y)
            [4]:
               _[1]=(x)
[2]:
   [1]:
      _[1]=y1
      _[2]=y2
      _[3]=x1^2
      _[4]=x2*x1
      _[5]=x2^2
   [2]:
      _[1]=(x^4+2*x^2*y^2+9*x^2*y+117*x^2+y^4+9*y^3+9*y^2
      -81*y-162)*y1+
      (-3*x^3-3*x*y^2-135*x*y-54*x)*x1+(-3*x^4-6*x^2*y^2-27*x^2*y-351*x^2
      -3*y^4-27*y^3-27*y^2+243*y+486)
      _[2]=(x^4*y^2+3*x^4*y+2*x^2*y^4-2*x^2*y^3+138*x^2*y^2+504*x^2*y
      +54*x^2+y^6-5*y^5-21*y^4+81*y^3+108*y^2-324*y)*y2+
      (x^5*y-2*x^5+2*x^3*y^3-12*x^3*y^2+192*x^3*y-342*x^3+x*y^5
      -10*x*y^4+33*x*y^3+198*x*y^2+108*x*y+486*x)*x2+
      (-3*x^4*y^2-9*x^4*y-6*x^2*y^4+6*x^2*y^3-414*x^2*y^2
      -1512*x^2*y-162*x^2-3*y^6+15*y^5+63*y^4-243*y^3-324*y^2+972*y)
      _[3]=(x^4+2*x^2*y^2+36*x^2*y+36*x^2+y^4+36*y^3+9*y^2
      -324*y-162)*x1^2
      +(-12*x^3*y+72*x^3-12*x*y^3+288*x*y^2+1080*x*y+324*x)*x1
      _[4]=(x^4+2*x^2*y^2+36*x^2*y+36*x^2+y^4+36*y^3+9*y^2-324*y
      -162)*x2*x1
      +(-6*x^3*y+36*x^3-6*x*y^3+144*x*y^2+540*x*y+162*x)*x1
      _[5]=(x^4+2*x^2*y^2+531*x^2*y-1449*x^2+y^4+531*y^3+9*y^2
      -4779*y-162)*x2^2
      +(-5*x^5*y+21*x^5-10*x^3*y^3-18*x^3*y^2+270*x^3*y
      +702*x^3-5*x*y^5-39*x*y^4+135*x*y^3+1323*x*y^2+2106*x*y)*x1
      +(x^4*y^2-537*x^4*y+1620*x^4+2*x^2*y^4-543*x^2*y^3
```

```
      +144*x^2*y^2+5373*x^2*y
      +y^6-6*y^5-18*y^4+108*y^3+81*y^2-486*y)
   [3]:
      [1]:
         [1]:
            _[1]=(x^2+y^2+3*y)
         [2]:
            [1]:
               _[1]=(y-1)
               _[2]=(x^2+4)
            [2]:
               _[1]=(y)
               _[2]=(x)
            [3]:
               _[1]=(y+3)
               _[2]=(x)
            [4]:
               _[1]=(7*y+3)
               _[2]=(49*x^2-54)
      [2]:
         [1]:
            _[1]=(x^4+2*x^2*y^2-9*x^2*y+171*x^2+y^4-9*y^3+9*y^2
            +81*y-162)
         [2]:
            [1]:
               _[1]=(y+3)
               _[2]=(x)
            [2]:
               _[1]=(y-6)
               _[2]=(x)
            [3]:
               _[1]=(7*y+3)
               _[2]=(49*x^2-54)
            [4]:
               _[1]=(y-3)
               _[2]=(x)
[3]:
   [1]:
      _[1]=x1
      _[2]=x2
      _[3]=y1^2
      _[4]=y2*y1
      _[5]=y2^2
   [2]:
      _[1]=x1
      _[2]=x2
      _[3]=y1^2-9
      _[4]=y2*y1-3*y2+3*y1-9
      _[5]=y2^2-6*y2-27
   [3]:
      [1]:
         [1]:
            _[1]=(y+3)
            _[2]=(x)
         [2]:
            [1]:
               _[1]=1
[4]:
```

```
   [1]:
      _[1]=y2
      _[2]=x1^2
      _[3]=y1*x1
      _[4]=x2*x1
      _[5]=y1^2
      _[6]=x2*y1
      _[7]=x2^2
   [2]:
      _[1]=9*y2+(28*x)*x2-27
      _[2]=35*x1^2+36*y1+(-56*x)*x1-108
      _[3]=35*y1*x1+(-112*x)*y1-123*x1+(336*x)
      _[4]=35*x2*x1+18*y1+(-28*x)*x1-54
      _[5]=35*y1^2-36*y1+(56*x)*x1-207
      _[6]=35*x2*y1-105*x2+(-56*x)*y1-9*x1+(168*x)
      _[7]=245*x2^2+108*y1+(42*x)*x1-594
   [3]:
      [1]:
         [1]:
            _[1]=(7*y+3)
            _[2]=(49*x^2-54)
         [2]:
            [1]:
               _[1]=1
[5]:
   [1]:
      _[1]=y1
      _[2]=x2
      _[3]=x1^2
      _[4]=y2*x1
      _[5]=y2^2
   [2]:
      _[1]=y1-3
      _[2]=x2
      _[3]=x1^2
      _[4]=y2*x1
      _[5]=y2^2-6*y2
   [3]:
      [1]:
         [1]:
            _[1]=(y)
            _[2]=(x)
         [2]:
            [1]:
               _[1]=1
      [2]:
         [1]:
            _[1]=(y-6)
            _[2]=(x)
         [2]:
            [1]:
               _[1]=1
[6]:
   [1]:
      _[1]=y1
      _[2]=y2
      _[3]=x1^2
      _[4]=x2*x1
```

```
   [2]:
      _[1]=4*y1+(x)*x1-12
      _[2]=2*y2+(-x)*x2-6
      _[3]=x1^2+(-2*x)*x1
      _[4]=x2*x1+(-x)*x1
   [3]:
      [1]:
         [1]:
            _[1]=(y-1)
            _[2]=(x^2+4)
         [2]:
            [1]:
               _[1]=1
[7]:
   [1]:
      _[1]=x1^2
      _[2]=y1*x1
      _[3]=x2*x1
      _[4]=y2*x1
      _[5]=y1^2
      _[6]=x2*y1
      _[7]=y2*y1
      _[8]=y2^2
   [2]:
      _[1]=5*x1^2+24*y1-72
      _[2]=5*y1*x1-9*x1
      _[3]=5*x2*x1+12*y1-36
      _[4]=5*y2*x1+3*x1
      _[5]=5*y1^2-24*y1+27
      _[6]=5*x2*y1-15*x2+3*x1
      _[7]=5*y2*y1-15*y2+3*y1-9
      _[8]=y2^2+x2^2-6*y2+12*y1-27
   [3]:
      [1]:
         [1]:
            _[1]=(y-3)
            _[2]=(x)
         [2]:
            [1]:
               _[1]=1
[8]:
   [1]:
      _[1]=x1
      _[2]=y1
      _[3]=x2
      _[4]=y2^2
   [2]:
      _[1]=x1
      _[2]=y1-3
      _[3]=x2
      _[4]=y2^2-6*y2+(-y^2+6*y)
   [3]:
      [1]:
         [1]:
            _[1]=(x)
         [2]:
            [1]:
               _[1]=(y+3)
```

```
                _[2]=(x)
            [2]:
                _[1]=(y-3)
                _[2]=(x)
            [3]:
                _[1]=(y)
                _[2]=(x)
            [4]:
                _[1]=(y-6)
                _[2]=(x)
[9]:
   [1]:
      _[1]=x1
      _[2]=y1
      _[3]=y2
   [2]:
      _[1]=x1
      _[2]=y1-3
      _[3]=(y-3)*y2+(x)*x2+(-3*y+9)
   [3]:
      [1]:
         [1]:
            _[1]=(x^2+y^2-6*y+9)
         [2]:
            [1]:
               _[1]=(y-3)
               _[2]=(x)
            [2]:
               _[1]=(y-1)
               _[2]=(x^2+4)
```

In this problem, the generic segment 1 has solutions, and if we do not pay attention to the structure of the solutions of these segments, we would say that the locus is the whole plane. But we observe that in this problem, we have a proper mover point M and the solutions for this first segment give fixed values for the mover coordinates, namely $x_1 = 0$ and $y_1 = 3$ (see segment 1), i.e., the mover in point A.

So, if the mover avoids the special point $M = A$, then there are no solutions in the first segment. If we want the mover to avoid this point, we also have to eliminate the points (x, y) of all the segments where the mover takes the values $(x_1, y_1) = (0, 3)$.

This can be done applying `pdiv` to the polynomials x_1 and $y_1 - 3$ with respect to the basis of each segment. If they reduce to 0, then the segment is to be excluded from the output. In fact we can do better, eliminating multiple factors in the polynomials of the basis before dividing. Doing so, we eliminate segments 8, 9 and 5 that contain x_1 and $y_1 - 3$. For segment 5 we have to previously eliminate multiple factors of the polynomials in the basis. (Its basis contains x_1^2 and $y_1 - 3$.)

We proceed as follows:

(1) Modify the first segment substituting the lpp and basis by $\langle 1 \rangle$ setting

```
> G[1][1]=ideal(1);
> G[1][2]=ideal(1);
```

(2) Define

```
> list Gm;
> Gm=G[1],G[2],G[3],G[4],G[6],G[7].
```

We can now compute the locus calling `locus`(S) with the option `"grobcov"`, `Gm` that forces `locus` to use Gm as the true GC of S, instead of computing it inside. The result is:

```
> locus(S,"grobcov",Gm);
[1]:
   [1]:
      _[1]=(x^2+y^2+3*y)
   [2]:
      [1]:
         _[1]=(y-1)
         _[2]=(x^2+4)
   [3]:
      [1]:
         1
      [2]:
         Normal
      [3]:
         _[1]=y1^2+x1^2-9
[2]:
   [1]:
      _[1]=(x^4+2*x^2*y^2-9*x^2*y+171*x^2+y^4-9*y^3+9*y^2+81*y-162)
   [2]:
      [1]:
         _[1]=(y-6)
         _[2]=(x)
      [2]:
         _[1]=(y-3)
         _[2]=(x)
   [3]:
      [1]:
         1
      [2]:
         Normal
      [3]:
         _[1]=y1^2+x1^2-9
[3]:
   [1]:
      _[1]=(y-1)
      _[2]=(x^2+4)
   [2]:
      [1]:
         _[1]=1
   [3]:
      [1]:
         0
      [2]:
         Accumulation
[4]:
```

```
[1]:
   _[1]=(y-3)
   _[2]=(x)
[2]:
   [1]:
      _[1]=1
[3]:
   [1]:
      0
   [2]:
      Accumulation
```

which is the "complex" locus, and corresponds only partially to the expected locus, as we shall see in Sect. 7.5.3 when analyzing the real locus.

In this example, the simple command `stdlocus` cannot provide any answer, because it is not able to eliminate the point generated by the bad mover position $(x_1, y_1) = (0, 3)$.

7.5.2 The Algorithm

Algorithm 7.5: LOCUS is designed to avoid situations like that in Example 7.13. LOCUS begins computing $G = \texttt{grobcov}(F)$ and then tests first if the generic segment has solutions. If not, $(B_1 = 1)$, then it passes the control to LOCUS2(G, F).

If $B_1 \neq 1$, then it tests if the basis B_1 contains a subset $B_0 \subset B_1$ only in the mover variables of dimension less than the dimension of the component. If not, no locus can be computed and it returns the empty list.

If such a B_0 exists, then it sets 1 in the basis and lpp of the generic segment and eliminates all the segments G_i containing B_0. With this objective, for all G_i in G, LOCUS tests if all the polynomials in B_0 reduce to zero modulo the basis B_i of the segment G_i. If they do, then it eliminates G_i from G. After this, it passes the control to LOCUS2(G, F) with the remaining elements of G.

From now on we wlll always use `locus` command instead of `locus2` in order to prevent situations like that in Example 7.13. Thus, routine `locus2` is considered internal in the library and is not accessible to the users.

7.5.3 Real Gröbner Cover and Real Locus

Nevertheless, we are not satisfied with the complex locus and we now try to obtain the *real grobcov* and locus. Let us continue with Example 7.13.

First we can delete the actual 5th element of Gm because it defines a segment without real points. Doing so, we also have to delete the hole in Gm[2] that is equal

to the top of the deleted segment Gm[5]. We show these objects:

```
> Gm[5];
   [1]:
      _[1]=y1
      _[2]=y2
      _[3]=x1^2
      _[4]=x2*x1
   [2]:
      _[1]=4*y1+(x)*x1-12
      _[2]=2*y2+(-x)*x2-6
      _[3]=x1^2+(-2*x)*x1
      _[4]=x2*x1+(-x)*x1
   [3]:
      [1]:
         [1]:
            _[1]=(y-1)
            _[2]=(x^2+4)
         [2]:
            [1]:
               _[1]=1
> Gm[2][3][1][2];
[1]:
   _[1]=(y-1)
   _[2]=(x^2+4)
[2]:
   _[1]=(y)
   _[2]=(x)
[3]:
   _[1]=(y+3)
   _[2]=(x)
[4]:
   _[1]=(7*y+3)
   _[2]=(49*x^2-54)
```

And proceed to do the modifications in the grobcov:

```
> list J=Gm[2][3][1][2];
> J=delete(J,1);
> Gm[2][3][1][2]=J;
> Gm[2][3][1][2];
[1]:
   _[1]=(y)
   _[2]=(x)
[2]:
   _[1]=(y+3)
   _[2]=(x)
[3]:
   _[1]=(7*y+3)
   _[2]=(49*x^2-54)
> Gm=delete(Gm,5);
```

Finally, there is a more subtle problem. In the last segment (now Gm[5]) there are complex points in the basis.

Decomposing the basis B_5 of Gm[5] into prime ideals we have:

```
> ideal B=Gm[5][2];
> ring RR=0,(y2,x2,y1,x1),lp;
> ideal B5=imap(R,B);
> B5;
B5[1]=24*y1+5*x1^2-72
B5[2]=5*y1*x1-9*x1
B5[3]=5*x2*x1+12*y1-36
B5[4]=5*y2*x1+3*x1
B5[5]=5*y1^2-24*y1+27
B5[6]=5*x2*y1-15*x2+3*x1
B5[7]=5*y2*y1-15*y2+3*y1-9
B5[8]=y2^2-6*y2+x2^2+12*y1-27
> def BB=minAssGTZ(B5);  //  prime decomposition
> BB;
[1]:
   _[1]=y2^2-6*y2+x2^2+9
   _[2]=x1
   _[3]=y1+5/24*x1^2-3
[2]:
   _[1]=5*x1+12
   _[2]=y1+5/24*x1^2-3
   _[3]=5*x2+6
   _[4]=5*y2+3
[3]:
   _[1]=5*x1-12
   _[2]=y1+5/24*x1^2-3
   _[3]=5*x2-6
   _[4]=5*y2+3
```

that can be simplified to

```
> BB[1][1]=y2^2-6*y2+x2^2+9;
> BB[1][3]=y1-3;
> BB[2][2]=5*y1+21;
> BB[3][2]=5*y1+21;
> BB;
[1]:
   _[1]=y2^2-6*y2+x2^2+9
   _[2]=x1
   _[3]=y1-3
[2]:
   _[1]=5*x1+12
   _[2]=5*y1+21
   _[3]=5*x2+6
   _[4]=5*y2+3
[3]:
   _[1]=5*x1-12
   _[2]=5*y1+21
   _[3]=5*x2-6
   _[4]=5*y2+3
```

We observe that the first prime ideal in the basis determines a one-dimensional set of complex points $\{y_2 = 3 \pm i x_2, x_1 = 0, y_1 = 3\}$. From this set of points, there is

only one real point, namely $\{x_1 = 0, y_1 = 3, x_2 = 0, y_2 = 3\}$. Thus, for the real GC, by avoiding the point $x_1 = 0$, $y_1 = 3$, we can delete the first component.

```
> BB=delete(BB,1);
BB;
[1]:
   _[1]=5*x1+12
   _[2]=5*y1+21
   _[3]=5*x2+6
   _[4]=5*y2+3
[2]:
   _[1]=5*x1-12
   _[2]=5*y1+21
   _[3]=5*x2-6
   _[4]=5*y2+3
```

We can finally compute the intersection of the ideals to form the real basis of segment Gm[6] and substitute it in the segment. We also have to determine the set of lpp, which now after having eliminated the one dimensional complex component of the basis becomes 0-dimensional.

```
> ideal BR5=intersect(BB[1],BB[2]);
> BR5;
BR5[1]=5*y1+21
BR5[2]=2*x2-x1
BR5[3]=5*y2+3
BR5[4]=25*x1^2-144
```

whose set of lpp is y_1, x_2, y_2, x_1^2, representing a zero-dimensional real set. So for the RGC, we make the following transformation of Segment 6:

```
> setring R;
> def Br5=imap(RR,BR5);
> Gm[5][1]=ideal(y1,x2,y2,x1^2);
> Gm[5][2]=Br5;
```

With these operations we have obtained the *real grobcov* (and eliminating the segments containing $x_1 = 0$, $y_1 = 3$). We have:

```
> Gm;
[1]:
   [1]:
      _[1]=1
   [2]:
      _[1]=1
   [3]:
      [1]:
         [1]:
            _[1]=0
         [2]:
            [1]:
               _[1]=(x^4+2*x^2*y^2-9*x^2*y+171*x^2+y^4-9*y^3
               +9*y^2+81*y-162)
            [2]:
```

```
                _[1]=(x^2+y^2-6*y+9)
             [3]:
                _[1]=(x^2+y^2+3*y)
             [4]:
                _[1]=(x)
[2]:
   [1]:
      _[1]=y1
      _[2]=y2
      _[3]=x1^2
      _[4]=x2*x1
      _[5]=x2^2
   [2]:
      _[1]=(x^4+2*x^2*y^2+9*x^2*y+117*x^2+y^4+9*y^3+9*y^2
      -81*y-162)*y1+
      (-3*x^3-3*x*y^2-135*x*y-54*x)*x1+(-3*x^4-6*x^2*y^2-27*x^2*y
      -351*x^2-3*y^4-27*y^3-27*y^2+243*y+486)
      _[2]=(x^4*y^2+3*x^4*y+2*x^2*y^4-2*x^2*y^3+138*x^2*y^2
      +504*x^2*y+54*x^2+y^6-5*y^5-21*y^4+81*y^3+108*y^2-324*y)*y2+
       (x^5*y-2*x^5+2*x^3*y^3-12*x^3*y^2+192*x^3*y-342*x^3
      +x*y^5-10*x*y^4+33*x*y^3+198*x*y^2+108*x*y+486*x)*x2+
       (-3*x^4*y^2-9*x^4*y-6*x^2*y^4+6*x^2*y^3-414*x^2*y^2
      -1512*x^2*y-162*x^2-3*y^6+15*y^5+63*y^4-243*y^3-324*y^2+972*y)
      _[3]=(x^4+2*x^2*y^2+36*x^2*y+36*x^2+y^4+36*y^3+9*y^2
      -324*y-162)*x1^2
      +(-12*x^3*y+72*x^3-12*x*y^3+288*x*y^2+1080*x*y+324*x)*x1
      _[4]=(x^4+2*x^2*y^2+36*x^2*y+36*x^2+y^4+36*y^3+9*y^2
      -324*y-162)*x2*x1
      +(-6*x^3*y+36*x^3-6*x*y^3+144*x*y^2+540*x*y+162*x)*x1
      _[5]=(x^4+2*x^2*y^2+531*x^2*y-1449*x^2+y^4+531*y^3+9*y^2
      -4779*y-162)*x2^2
      +(-5*x^5*y+21*x^5-10*x^3*y^3-18*x^3*y^2+270*x^3*y+702*x^3
      -5*x*y^5-39*x*y^4+135*x*y^3+1323*x*y^2+2106*x*y)*x1
      +(x^4*y^2-537*x^4*y+1620*x^4+2*x^2*y^4-543*x^2*y^3
      +144*x^2*y^2+5373*x^2*y+y^6-6*y^5-18*y^4+108*y^3
      +81*y^2-486*y)
   [3]:
      [1]:
         [1]:
            _[1]=(x^2+y^2+3*y)
         [2]:
            [1]:
               _[1]=(y)
               _[2]=(x)
            [2]:
               _[1]=(y+3)
               _[2]=(x)
            [3]:
               _[1]=(7*y+3)
               _[2]=(49*x^2-54)
      [2]:
         [1]:
            _[1]=(x^4+2*x^2*y^2-9*x^2*y+171*x^2+y^4-9*y^3
            +9*y^2+81*y-162)
```

```
         [2]:
            [1]:
               _[1]=(y+3)
               _[2]=(x)
            [2]:
               _[1]=(y-6)
               _[2]=(x)
            [3]:
               _[1]=(7*y+3)
               _[2]=(49*x^2-54)
            [4]:
               _[1]=(y-3)
               _[2]=(x)
[3]:
   [1]:
      _[1]=x1
      _[2]=x2
      _[3]=y1^2
      _[4]=y2*y1
      _[5]=y2^2
   [2]:
      _[1]=x1
      _[2]=x2
      _[3]=y1^2-9
      _[4]=y2*y1-3*y2+3*y1-9
      _[5]=y2^2-6*y2-27
   [3]:
      [1]:
         [1]:
            _[1]=(y+3)
            _[2]=(x)
         [2]:
            [1]:
               _[1]=1
[4]:
   [1]:
      _[1]=y2
      _[2]=x1^2
      _[3]=y1*x1
      _[4]=x2*x1
      _[5]=y1^2
      _[6]=x2*y1
      _[7]=x2^2
   [2]:
      _[1]=9*y2+(28*x)*x2-27
      _[2]=35*x1^2+36*y1+(-56*x)*x1-108
      _[3]=35*y1*x1+(-112*x)*y1-123*x1+(336*x)
      _[4]=35*x2*x1+18*y1+(-28*x)*x1-54
      _[5]=35*y1^2-36*y1+(56*x)*x1-207
      _[6]=35*x2*y1-105*x2+(-56*x)*y1-9*x1+(168*x)
      _[7]=245*x2^2+108*y1+(42*x)*x1-594
   [3]:
      [1]:
         [1]:
```

```
            _[1]=(7*y+3)
            _[2]=(49*x^2-54)
         [2]:
            [1]:
               _[1]=1
[5]:
   [1]:
      _[1]=y1
      _[2]=x2
      _[3]=y2
      _[4]=x1^2
   [2]:
      _[1]=5*y1+21
      _[2]=2*x2-x1
      _[3]=5*y2+3
      _[4]=25*x1^2-144
   [3]:
      [1]:
         [1]:
            _[1]=(y-3)
            _[2]=(x)
         [2]:
            [1]:
               _[1]=1
```

From the above transformation we can obtain the *real locus*, using the options (`"grobcov"`,`Gm`) to force `locus` to use `Gm` instead of computing the GC inside:

```
> locus(S,"grobcov",Gm,"anti-image",0);
[1]:
   [1]:
      _[1]=(x^2+y^2+3*y)
   [2]:
      [1]:
         _[1]=(y)
         _[2]=(x)
   [3]:
      [1]:
         1
      [2]:
         Normal
[2]:
   [1]:
      _[1]=(x^4+2*x^2*y^2-9*x^2*y+171*x^2+y^4-9*y^3+9*y^2
      +81*y-162)
   [2]:
      [1]:
         _[1]=(y-6)
         _[2]=(x)
   [3]:
      [1]:
         1
      [2]:
         Normal
```

which is the "real" locus, represented in Fig. 7.7. We see that the "Accumulation" components obtained with the complex GC are included as real normal points and added to the "Normal" components. The holes of the real locus are $\mathbf{V}(y, x)$ for the first component and $\mathbf{V}(y-6, x)$ for the second component. Both holes correspond to the mover in position $x_1 = 0$, $y_1 = 3$ that we have avoided.

We expect that in loci class A problems deriving from a real geometric locus problem, the major difficulties that will occur for projecting the "complex" GC into the "reals" will be of the kind that we have seen here:

(1) Segments without real solutions.
(2) Segments with non-normal points (lpp of dimension non zero) due to complex components of the basis which do not have any real solution and having some smaller component with real solutions.
(3) Segments for which one must add inequalities (as in Example 7.10).

The reason is that the segments of the "complex" GC, when projecting them over the reals, can only have less solutions.

We propose a challenging objective for the future: that is to obtain an algorithm for reducing the complex GC to the real RGC.

Example 7.13 has been useful for developing new methods for obtaining locus: eliminating bad mover positions and obtaining the real locus. Nevertheless these problems can be avoided using a better formulation. From the geometric construction it is obvious that $P(x_2, y_2)$ is the middle point between $M(x_1, y_1)$ and point $(0, -3)$. Using this, the system is

$$\begin{aligned} S = & x_1^2 + y_1^2 - 9, \\ & 9x + (y-3)x_1 - xy_1, \\ & x_1^2 + (y_1+3)^2 - 4(x-x_1)^2 - 4(y-y_1), \end{aligned}$$

and the computation becomes:

```
> ring R=(0,x,y),(x1,y1),lp;
> ideal S=x1^2+y1^2-9,
             9*x+(y-3)*x1-x*y1,
             x1^2+(y1+3)^2-4*(x-x1)^2-4*(y-y1)^2;
> short=0;
> locus(S);
[1]:
   [1]:
      _[1]=(x^4+2*x^2*y^2-9*x^2*y+171*x^2+y^4-9*y^3
      +9*y^2+81*y-162)
   [2]:
      [1]:
         _[1]=1
   [3]:
      [1]:
         1
      [2]:
```

```
         Normal
      [3]:
         _[1]=x1^2+y1^2-9
[2]:
   [1]:
      _[1]=(x^2+y^2+3*y)
   [2]:
      [1]:
         _[1]=1
   [3]:
      [1]:
         1
      [2]:
         Normal
      [3]:
         _[1]=x1^2+y1^2-9
```

Doing so, there is no need to eliminate bad mover positions or projecting over the reals for obtaining the same right result.

This shows that when formulating loci problems, it is convenient to search a simple formulation avoiding unnecessary difficulties, even if some of them can be automatically eliminated using `grobcov`.

7.6 Some Other Examples

Example 7.14 (Pascal's Limaçon in 3d: "Accumulation" point) Consider now a locus problem in $3d$: namely the generalization of Pascal's Limaçon in Example 7.8 to $3d$. The problem reads as follows.

Let O be a fixed point on a sphere c of radius r. Let ℓ be a line passing through O and a general point $M(x_1, y_1, z_1)$ on c (the mover). Let T be a point on ℓ at a fixed distance k from M. Pascal's Limaçon $3d$ is the locus of points T as M moves on the sphere c.

As in Example 7.8, let $O = (0, 0, 2)$, $r = 2$ and $k = 1$. We have $m = 3$ variables and $m + 1 = 4$ equations, corresponding well to the Definition 7.1 locus class A problems.

The equations and call are:

```
> LIB "grobcov.lib";
> ring R=(0,x,y,z),(x1,y1,z1),dp;
> short=0;
> ideal S=x1^2+y1^2+z1^2-4,
            x*y1-x1*y,
            x1*(z-2)-x*(z1-2),
            y1*(z-2)-y*(z1-2),
            (x-x1)^2+(y-y1)^2+(z-z1)^2-1;
> locus(S);
[1]:
   [1]:
```

```
      _[1]=(x^4+2*x^2*y^2+2*x^2*z^2-9*x^2+y^4+2*y^2*z^2
           -9*y^2+z^4-9*z^2+4*z+12)
   [2]:
      [1]:
         _[1]=(z-2)
         _[2]=(x^2+y^2)
   [3]:
      [1]:
         2
      [2]:
         Normal
      [3]:
         _[1]=x1^2+y1^2+z1^2-4
[2]:
   [1]:
      _[1]=(x^2+y^2+z^2-4*z+3)
   [2]:
      [1]:
         _[1]=1
   [3]:
      [1]:
         1
      [2]:
         Special
      [3]:
         _[1]=z1^2-3*z1+2
         _[2]=x1^2+y1^2+3*z1-6
[3]:
   [1]:
      _[1]=(z-2)
      _[2]=(y)
      _[3]=(x)
   [2]:
      [1]:
         _[1]=1
   [3]:
      [1]:
         0
      [2]:w
         Accumulation
```

The result is the following three components:

$$
\begin{aligned}
C_1 =& \mathbf{V}(x^4 + 2x^2y^2 + 2x^2z^2 - 9x^2 + y^4 + 2y^2z^2 - 9y^2 + z^4 - 9z^2 + 4z + 12) \\
& \setminus \mathbf{V}(z-2, x^2 + y^2), && \text{Normal} \\
C_2 =& \mathbf{V}(x^2 + y^2 + z^2 - 4z + 3), && \text{Special} \\
C_3 =& \mathbf{V}(z-2, y, x), && \text{Accumulation},
\end{aligned}
$$

that generalizes ordinary $2d$-Pascal Limaçon (Component C_1) well. If we determine first the real GC and compute the locus with it, then the component C_1 is reduced to

$$C_1 = \mathbf{V}(x^4 + 2x^2y^2 + 2x^2z^2 - 9x^2 + y^4 + 2y^2z^2 - 9y^2 + z^4 - 9z^2 + 4z + 12) \setminus \mathbf{V}(z - 2, x, y), \quad \text{Normal.}$$

Regarding the anti-image of component C_2, we see that C_2 is generated by the real mover positions $M = (0, 0, 2) = O$, and M on the circle $c_0 = \mathbf{V}(x_1^2 + y_1^2 - 3, z_1 - 1)$. It is thus generated by an anti-image of dimension 1, smaller than the dimension of C_1 and therefore the component is declared "Special". Properly speaking, the mover on position $(0, 0, 2)$ is the generator of the "Special" component. But the points in the circle c_0 also generate points of the component.

If we want to know which points generate the "Accumulation" point $O = (0, 0, 2)$ of the tracer, we can look at the output of `grobcov`. We see that the basis of the sixth segment $\mathbf{V}(z - 2, y, x)$ is $B_6 = \{4z_1 - 7, 16x_1^2 + 16y_1^2 - 15\}$. Thus the points of the mover on the circle $\mathbf{V}(4z_1 - 7, 16x_1^2 + 16y_1^2 - 15)$ all generate the same point $O = (0, 0, 2)$.

The graphic can be seen in Fig. 7.8. Projecting on the plane $x = 0$ the result is the classic Pascal Limaçon already described in Example 7.8 and whose graphic can be seen in Fig. 7.2.

Example 7.15 (3d-Pedal of an ellipsoid) We describe now the Pedal of an ellipsoid. The construction is the following.

(1) Consider an ellipsoid $x_1^2 + \frac{y_1^2}{4} + \frac{z_1^2}{9} = 1$;
(2) Take a mover point $M(x_1, y_1, z_1)$ on it;
(3) Take the tangent plane t through M to the ellipsoid;

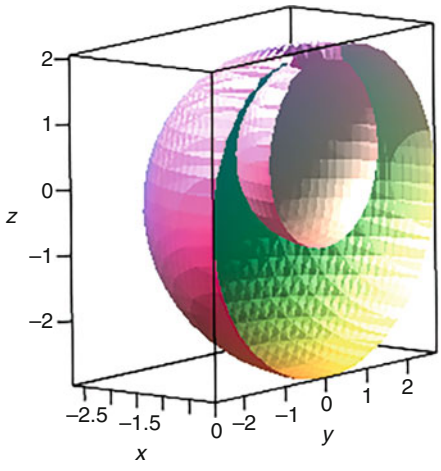


Fig. 7.8 Normal component of $3d$-Pascal's Limaçon

(4) Construct the perpendicular line p to t through the origin (center of the ellipsoid);
(5) The tracer $T(x, y, z)$ is the intersection point of p and t, when M moves on the ellipsoid;

The system becomes:

```
> LIB "grobcov.lib";
> ring R=(0,x,y,z),(x1,y1,z1),lp;
> short=0;
> poly f=4*x1^2+9*y1^2+36*z1^2-36;
> poly t=diff(f,x1)*(x-x1)+diff(f,y1)*(y-y1)+diff(f,z1)*(z-z1);
> poly p1=x*diff(f,y1)-y*diff(f,x1);
> poly p2=x*diff(f,z1)-z*diff(f,x1);
> poly p3=y*diff(f,z1)-z*diff(f,y1);
> ideal S=f,t,p1,p2,p3;
> locus(S);
[1]:
   [1]:
      _[1]=(x^4+2*x^2*y^2+2*x^2*z^2-9*x^2+y^4+2*y^2*z^2-4*y^2+z^4-z^2)
   [2]:
      [1]:
         _[1]=(5*y^2+8*z^2)
         _[2]=(5*x^2-3*z^2)
   [3]:
      [1]:
         2
      [2]:
         Normal
      [3]:
         _[1]=4*x1^2+9*y1^2+36*z1^2-36
```

The result gives one "Normal" component. It is obvious that the hole of the component contains the unique real point $\mathbf{V}(x, y, z)$. Thus for the real locus the unique component of the locus is

$$C = \mathbf{V}(x^4 + 2x^2y^2 + 2x^2z^2 - 9x^2 + y^4 + 2y^2z^2 - 4y^2 + z^4 - z^2) \setminus \mathbf{V}(x, y, z),$$

shown in Fig. 7.9.

In this problem there are five equations and only three auxiliary variables $\mathbf{u}$. But equations p_1, p_2, p_3 representing the line from the origin perpendicular to the tangent plane are not independent, and so there are only four independent equations. Thus the locus is also of class A.

We propose now two examples, suggested by J.M. Brunat and J.C. Lario, inspired on the nice Richard Serrra surfaces exposed in Bilbao's Guggenheim Museum and described in R. Zalaya thesis [87].

Example 7.16 Consider the planes $z_1 = -1$ and $z_2 = 1$ and two circles on them, namely $x_1^2 + y_1^2 - 1$ and $x_2^2 + y_2^2 - 1$. Consider the lines relaying points of one circle with points of the second circle when the tangent lines on both points are parallel.

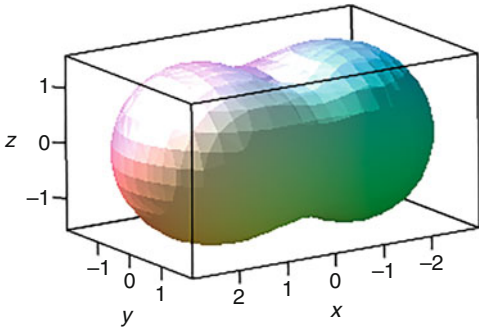


Fig. 7.9 Pedal of an ellipsoid

The tangent vectors to both circles are $(2x_1, 2y_1, 0)$ and $(2x_2, 2y_2, 0)$. They must be proportional so that we have $x_1y_2 - y_1x_2 = 0$. Moreover, the equations of the line joining the points of both circles are

$$x = x_1 + \lambda(x_2 - x_1),\ y = y_1 + \lambda(y_2 - y_1),\ z = z_1 + \lambda(z_2 - z_1).$$

We have eight equations and the seven variables $x_1, y_1, z_1, x_2, y_2, z_2, \lambda$. Thus we have a class A locus problem:

$$x_1^2 + y_1^2 - 1,\ x_2^2 + y_2^2 - 1$$
$$x_1y_2 - y_1x_2$$
$$z_1 + 1,\ z_2 - 1,$$
$$x - x_1 - \lambda(x_2 - x_1),\ y - y_1 - \lambda(y_2 - y_1),\ z - z_1 - \lambda(z_2 - z_1).$$

We consider as mover variables x_1, y_1, z_1, and so we place them as the latest variables in the definition of the ring. We can alternatively consider instead x_2, y_2, z_2 because of the symmetry. Applying `locus` as well as `stdlocus`, we obtain:

```
> LIB "grobcov.lib";
> ring R=(0,x,y,z),(lam,x2,y2,z2,x1,y1,z1),lp;
> short=0;
> ideal S141=x1^2+y1^2-1,x2^2+y2^2-1,
        x1*y2-y1*x2,
        z1+1,z2-1,
        x-x1-lam*(x2-x1), y-y1-lam*(y2-y1), z-z1-lam*(z2-z1);
locus(S141);
[1]:
   [1]:
      _[1]=(x^2+y^2-z^2)
   [2]:
      [1]:
         _[1]=(z)
         _[2]=(x^2+y^2)
   [3]:
```

```
      [1]:
         1
      [2]:
         Special
      [3]:
         _[1]=z1+1
         _[2]=x1^2+y1^2-1
[2]:
   [1]:
      _[1]=(x^2+y^2-1)
   [2]:
      [1]:
         _[1]=1
   [3]:
      [1]:
         1
      [2]:
         Special
      [3]:
         _[1]=z1+1
         _[2]=x1^2+y1^2-1
[3]:
   [1]:
      _[1]=(z)
      _[2]=(y)
      _[3]=(x)
   [2]:
      [1]:
         _[1]=1
   [3]:
      [1]:
         0
      [2]:
         Accumulation
> stdlocus(S141);
[1]:
   _[1]=(x^2+y^2-z^2)
[2]:
   _[1]=(x^2+y^2-1)
```

The hole of the first component only contains the real point $(0, 0, 0)$. The components are

$$C_1 = \mathbf{V}(x^2 + y^2 - z^2) \setminus \mathbf{V}(x, y, z), \qquad \text{“Special”}$$

$$C_2 = \mathbf{V}(x^2 + y^2 - 1), \qquad \text{“Special”}$$

$$C_3 = \mathbf{V}(x, y, z), \qquad \text{“Accumulation”}$$

We obtain two “Special” components: a cylinder and a cone minus its vertex, plus an “Accumulation” point representing the vertex of the cone. Observe the anti-image A_i of both components. We have $A_1 \cap \mathbb{Q}[x_1, y_1, z_1] = A_2 \cap \mathbb{Q}[x_1, y_1, z_1] = \langle z_1 + 1, x_1^2 + y_1^2 - 1 \rangle$, which represents the circle where the mover glides and has

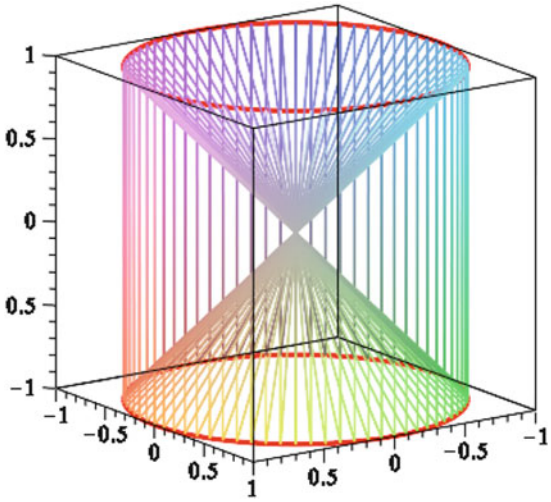


Fig. 7.10 Brunat-Lario's locus 1

dimension 1, which is smaller than the dimension 2 of the components. Thus the components are "Special" following our definition (Fig. 7.10).

`stdlocus` does not separate the "Accumulation" point $(0, 0, 0)$, or find the taxonomy of the components.

Observe the basis of the anti-image. Of course, it depends on the order in which the variables are given, but in any case, it is useful to determine the parametric representation of the lines forming the locus components.

Example 7.17 (Proposed by J.M. Brunat and J.C. Lario) A similar problem to the previous one is obtained substituting the circles with orthogonal parabolas. Now we have a parabola $y_1 = x_1^2$ on the plane $z_1 = -1$ and a parabola $x_2 = y_2^2$ on the plane $z_2 = 1$. As before, the locus is formed by the lines relying on the points of both parabolas with parallel tangents. The tangent vectors are now $(2x_1, -1, 0)$ and $(1, -2y_2, 0)$. The condition of parallels becomes $4x_1y_2 - 1 = 0$.

The system and its solution are:

```
> LIB "grobcov.lib";
> ring R=(0,x,y,z),(lam,x2,y2,z2,x1,y1,z1), lp;
> short=0;
> ideal S142=x1^2-y1, y2^2-x2,
             4*x1*y2-1,
             z1+1, z2-1,
             x-x1-lam*(x2-x1), y-y1-lam*(y2-y1), z-z1-lam*(z2-z1);
> stdlocus(S142); " ";
[1]:
   _[1]=(2048*x^3*z+2048*x^3-4096*x^2*y^2+1152*x*y*z^2
   -1152*x*y-2048*y^3*z+2048*y^3+27*z^4-54*z^2+27)
> locus(S142);
[1]:
   [1]:
```

```
    _[1]=(2048*x^3*z+2048*x^3-4096*x^2*y^2+1152*x*y*z^2
    -1152*x*y-2048*y^3*z+2048*y^3+27*z^4-54*z^2+27)
[2]:
    [1]:
        _[1]=(z+1)
        _[2]=(y)
    [2]:
        _[1]=(z-1)
        _[2]=(x)
[3]:
    [1]:
        1
    [2]:
        Special
    [3]:
        _[1]=z1+1
        _[2]=x1^2-y1
```

We obtain a single "Special" component that intersects the plane $z = -1$ on the parabola $\mathbf{V}(y - x^2, z + 1)$ and the plane $z = 1$ on the parabola $\mathbf{V}(x - y^2, z - 1)$ (see Fig. 7.11). We note that there are two lines of the component which do not form part of the locus, namely the lines $\mathbf{V}(z - 1, x)$ and $\mathbf{V}(z + 1, y)$. Observe that the first one is generated by the vertex of the parabola at $z = 1$, which would be connected with the point at infinity in the x-direction of the plane $z = 1$. But in the non-homogeneous system it does not form part of the locus. Similarly for the second line on the plane $z = -1$. As in previous example, the component is "Special" as the mover glides on the parabola $\mathbf{V}(z_1 + 1, x_1^2 - y_1)$ of dimension 1, whereas the component has dimension 2.

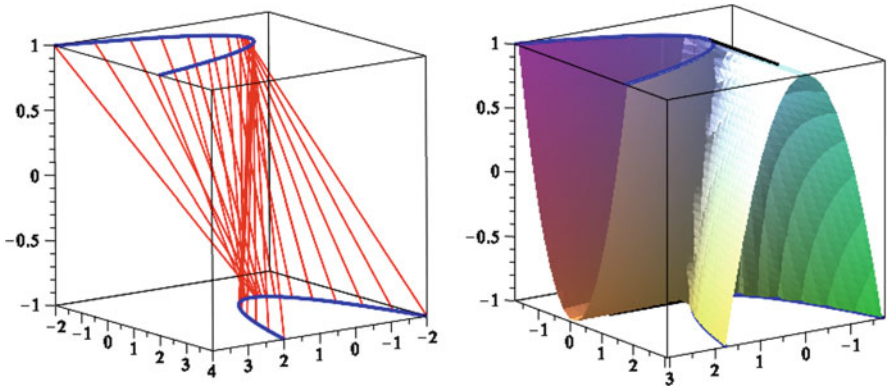


Fig. 7.11 Brunat-Lario's locus 2

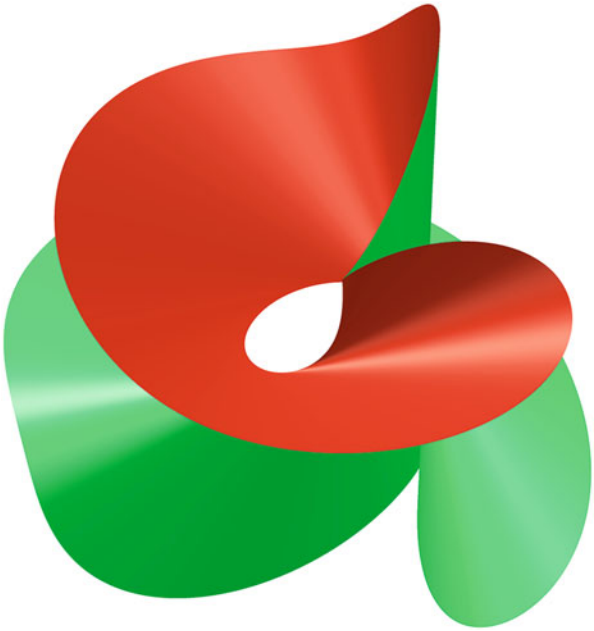


Fig. 7.12 Artistic vision of locus 7.16 using SURFER-IMAGINARY software

In Fig. 7.12 we show an artistic vision of the "Special" component using the nice software SURFER-IMAGINARY.

7.7 Web Implementation in GeoGebra

Francisco Botana developed a web prototype that includes `grobcov` and `locus` algorithms for computing and representing loci in the Dynamical Geometry free software GeoGebra [35]. The library also contains two other commands for this purpose, namely `locusdg`, which selects from the locus output only the DG "Relevant" components, and `locusto` that transforms the output of `locus`, `locusdg` and `envelop` into a string.

The web prototype is based on [31]. It is based on the free DG system GeoGebra[3] and the open source CAS Sage.[4]

LOCUS algorithm described in previous sections provides four different types of components for a locus set, namely, "Normal", "Special", "Degenerate" and "Accumulation" components. Although all of them are algebraically meaningful, only the "Normal" and "Accumulation" components of a locus have been considered *true geometric parts* of a dynamic geometry locus in the web prototype.

The system consists of a drawing canvas, where the computed locus is displayed together with the initial elements. More precisely, to obtain the algebraic description of a given locus, the algebraic knowledge obtained from a construction introduced through a GeoGebra applet is automatically encoded and sent to a Sage server, where it is processed remotely by the SINGULAR [62] "grobcov.lib" library.

Despite the technicalities of the remote interconnection of GeoGebra and Sage, the web application is presented as a simple web page with a GeoGebra applet in which to construct/upload a locus. Given a locus construction (specified using a predetermined set of GeoGebra commands), the prototype provides the algebraic description of the locus set by just pressing one button. The process goes roughly as follows.

First, the XML description of the GeoGebra construction is sent to a sagecell server [74]. On the server, the construction follows an algebraization process as specified by a special library [8]. The obtained parametric polynomial system is then fed into SINGULAR. The results are finally returned to the user in text form as well as graphically in the applet. Other applications are described in [9].

A screen capture of the web page with an accurate description of Pascal Limaçon discussed in Example 7.8 is shown in Fig. 7.13, (first part). It provides its graph (thick dotted circles) together with its description as an algebraic set. Note that no extra special component is included in the description, unlike the description provided by standard algebraic methods.

Although only the normal and accumulation components of a locus, as provided by the algorithm, are used by the system when describing the locus, all four sets of components are provided when pressing the *Show components (from the GC algorithm)* button. Figure 7.13 (bottom part) shows the different components for the limaçon of Pascal, where the extra circle mentioned in Example 7.8 is identified as "Special".

The goal of this web implementation is not to provide a system for complete general use, but to show proof of the concept of the feasibility of using the Gröbner Cover to supplement the symbolic capabilities of existing dynamic geometry systems, as well as to show the advantage of connecting different systems by using web services.

[3] http://www.geogebra.org

[4] http://www.sagemath.org

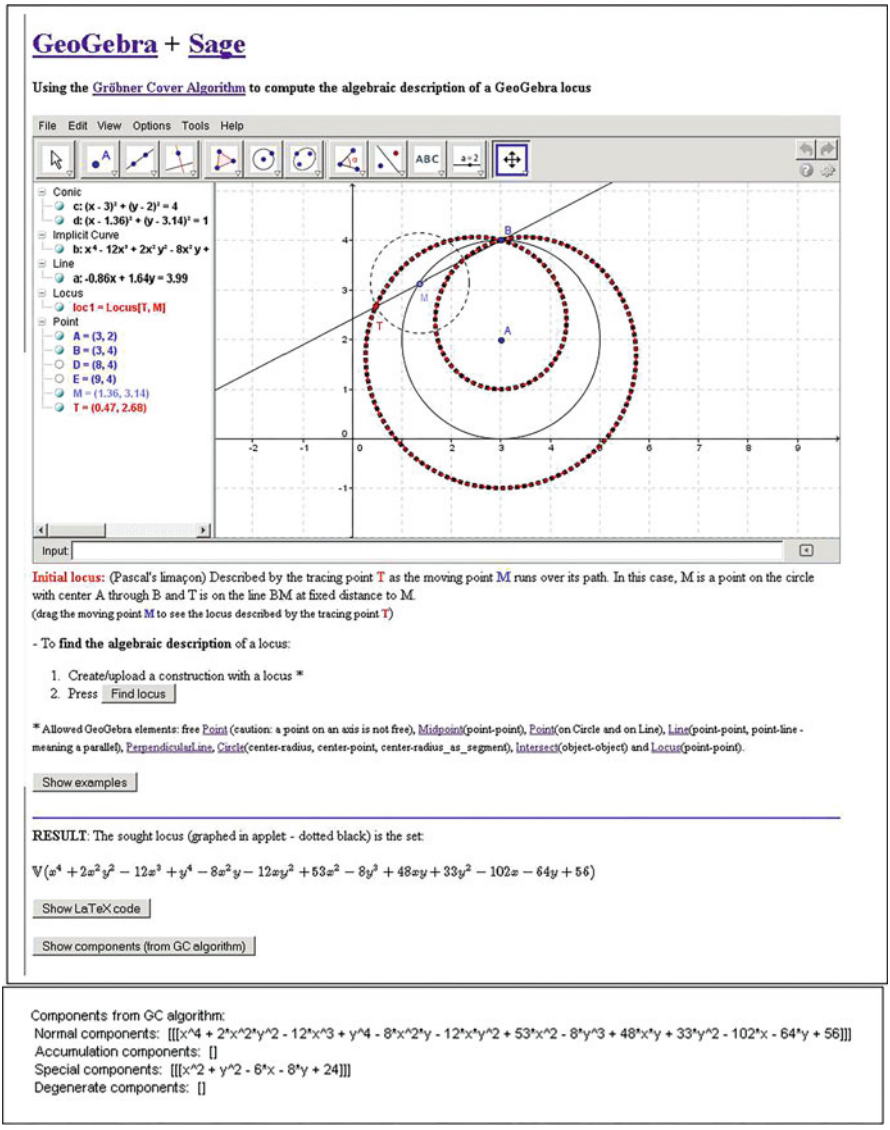


Fig. 7.13 All types of components of the limaçon of Pascal as provided by the system when pressing the *Show components (from GC algorithm)* button

7.8 Exercises

We propose some exercises for familiarizing the reader with the SINGULAR *grobcov.lib* library and its commands for computing the locus.

(1) Consider the line $r : x_1 + y_1 - 1$ and a mover point $M(x_1, y_1)$ on r. Let ℓ be the line from the origin $O(0, 0)$ to point M. Consider the projections of M over the axes and the line m passing through these projections. Find the locus of the intersection point P of lines ℓ and m. (Formulated by Miguel de Guzman).
Hint: You should obtain a "Normal" and a "Special" component.

(2) Find the locus of the points $T(x, y)$ which are equidistant from the parabola $x_1 - 1 - y_1^2$ and the line $x = -1$.
Answer: A "Normal" component with equation:

$$L = \mathbf{V}(64x^5 - 16x^4y^2 + 128x^4 - 168x^3y^2 + 352x^3 + 32x^2y^4 - 468x^2y^2 + 288x^2 + 168xy^4 + 18xy^2 + 324x - 16y^6 - 9y^4 - 54y^2).$$

(3) Consider the circle $c : x_1^2 + y_1^2 - 4$ and the mover point $M(x_1, y_1)$ gliding on it. Determine the locus of the intersection of the lines $y = y_1$ and $x = 4x_1/(y_1 + 2)$. Formulated by Zoltan Kovacs.
Hint: You should obtain a "Normal" component and a "Special" one. Plot the result.

(4) Find the locus of points in $\mathbb{R}^3$ that are equidistant from the plane $z = 0$ and the sphere$(z_1 - 3)^2 + x_1^2 + y_1^2 - 1$.
Hint: You will obtain two "Normal" components and a "Special" one. Does the "Special" component correspond to the real locus? Give a geometric interpretation of the result.

(5) Let $A(-4, -3, 0)$, $B(4, -3, 0)$, $C(0, 5, 0)$, $D = (x, y, z)$ be the vertices of a tetrahedron. Find the locus of points D for which the distance from A to its opposite side BCD are equal to the distance from B to its opposite side ACD.
Hint: Compute the complex locus, and try to obtain the *real grobcov* of the problem and obtain from it the real locus.

Chapter 8
Geometric Envelopes



8.1 Envelopes

The concept of envelope of a family of curves or surfaces is a well known idea in Differential Geometry. But, in general, it is used only for families with low degrees of freedom, in whose context it has full sense. Even if for families with higher degrees of freedom, the concept of envelope becomes less evident, we can generalize it for families of hypersurfaces with arbitrary constraints and higher degrees of freedom. We provide theorems showing the scope of the new definition and how it generalizes the known envelope definitions.

The first idea of the envelope of a family of hypersurfaces consist of finding a new hypersurface that, on each point of it, is tangent to an element of the family. The objective is obtaining a hypersurface that separates two regions: one containing all the elements of the family and the second not containing any hypersurface of the family. The first objective can be solved using only algebraic methods. But the second one, i.e., determining which points of the algebraic envelope is also part of the real envelope, is not trivial, and requires methods of real geometry. In this chapter, we provide algorithms for obtaining the algebraic envelope in an automatic way, and will partially solve the second problem in some easy cases.

In the algebraic approach, envelopes are considered as a special kind of locus, and using the Gröbner Cover we consider the problem over $\mathbb{C}^n$. The projection over $\mathbb{R}^n$ is not always elementary. In particular, the projection over the reals of the complex solution does not always "envelop" the hypersurfaces of the family. We give particular methods to restrict the scope of the algebraic envelope for the parts which really "envelop" the hypersurfaces.

© Springer Nature Switzerland AG 2018

A. Montes, *The Gröbner Cover*, Algorithms and Computation
in Mathematics 27, https://doi.org/10.1007/978-3-030-03904-2_8

Considered as a special kind of locus, the algebraic envelope inherits the taxonomy of loci, even if it is convenient to consider "Degenerate" components as non-relevant.

8.2 Definitions and Theorems

Definition 8.1 (Family of hypersurfaces) We say that

$$F(x_1, \ldots, x_n; u_1, \ldots, u_m) = 0$$

and the independent *constraints* $C = \{g_1, \ldots, g_s\}$ (with $s < m$)

$$\begin{cases} g_1(u_1, \ldots, u_m) = 0 \\ \qquad \cdots \\ g_s(u_1, \ldots, u_m) = 0 \end{cases}$$

represent a *family of hypersurfaces* of $\mathbb{C}^n$ if F depends at least on 1 *parameter* u and $\dim(C) = m - s \geq 1$. As always, we denote $\mathbf{x} = x_1, \ldots, x_n$, $\mathbf{u} = u_1, \ldots, u_m$.

Definition 8.2 (Algebraic Envelope) Consider a family of hypersurfaces

$$F(x_1, \ldots, x_n; u_1, \ldots, u_m) = 0 \tag{8.1}$$

with m parameters $\mathbf{u}$, constrained by the $s < m$ independent equations

$$C = \langle g_1(\mathbf{u}), \ldots, g_s(\mathbf{u}) \rangle. \tag{8.2}$$

Let $J = \dfrac{\partial(F, g_1, \ldots, g_s)}{\partial(u_1, \ldots, u_m)}$ be the Jacobian and let J_c be the set of $\binom{m}{s+1}$ equations of the minors of J implying that the rank of the matrix J is less than or equal to s:

$$J_c = \{\text{minors}(J) \ \text{ of order } (s+1) \times (s+1)\}. \tag{8.3}$$

Consider the ideal S

$$S = \langle F, \ C, \ J_c \rangle. \tag{8.4}$$

From the $\binom{m}{s+1}$ equations of J_c, there are only $m - s$ linearly independent, so that the total number of independent equations for the envelope system is $m + 1$. Thus the problem is a class A locus problem (cf. Chap. 7, Definition 7.1).

The envelope of F, if it exists, is

$$L = \texttt{locus}(S). \tag{8.5}$$

Remembering the definition of locus, the envelope is $E = \pi_2(V)$, where

$$V = \mathbf{V}(F, C, J_c),$$
$$\pi_2 : \mathbb{C}^{n+m} \to \mathbb{C}^n$$
$$(\mathbf{x}, \mathbf{u}) \mapsto \mathbf{x}$$

being π_2 the projection of the variety V over $\mathbb{C}^n$.

If E is open in $\mathbb{C}^n$ we consider that the problem has no envelope. Otherwise, being defined as a locus, it will consist of "Normal", "Special", "Accumulation", and "Degenerate" components.

Remark 8.3 We concentrate our study in obtaining and classifying the components of the algebraic envelope, and then, try to determine for which points of a component of the algebraic envelope there exist a unique real intersecting hypersurface of the family, namely the associated tangent one.

Remark 8.4 The usual known examples of envelope in the literature consider $n = 2$ or 3 and $s = m - 1$, [10, 45]. In this case our definition will use the equations

$$F(x_1, \ldots, x_n; u_1, \ldots, u_m) = 0,$$
$$g_1(u_1, \ldots, u_m) = 0,$$
$$\ldots$$
$$g_{m-1}(u_1, \ldots, u_m) = 0,$$

$$\left| \frac{\partial(F, g_1, \ldots, g_{m-1})}{\partial(u_1, \ldots, u_m)} \right| = 0,$$

which corresponds exactly to the classical set of equations used in these cases.

There is another classical example (see [50, 70]) with $n > m$ and $s = 0$. In that case the Jacobian conditions become simply the set of of equations $\left\{ \frac{\partial F}{\partial u_j} = 0 : j \in [m] \right\}$, which corresponds well to what is used in the literature.

Remark 8.5 In most envelope problems the components are in general $(n-1)$-dimensional "Normal" components, i.e., hypersurfaces minus perhaps some smaller variety inside. There can also exist "Accumulation" components of smaller dimension. Example 8.17 shows one such unusual case. In Examples 8.18 and 8.19, we show how part of a "Special" component is useful for obtaining the real envelope. "Degenerate" components are usually not considered as envelopes.

In the next theorem, we consider only *standard* $n-1$ dimensional *components* of the form $E_i = \mathbf{V}(p_i(\mathbf{x})) \setminus \mathbf{V}(\mathrm{q}_i(\mathbf{x}))$ where $p_i(\mathbf{x})$ is an irreducible polynomial, and $\mathbf{V}(\mathrm{q}_i(\mathbf{x}))$ is a variety inside $\mathbf{V}(p_i(\mathbf{x}))$.

Theorem 8.6 *Given the family* (8.1) *of irreducible hypersurfaces of* $\mathbb{C}^n$ *restricted by conditions* (8.2)*, if* E *is the envelope, and* $E_i = \mathbf{V}(p_i(\mathbf{x})) \setminus \mathbf{V}(\mathrm{q}_i(\mathbf{x}))$ *is the* C*-representation of a normal ("Normal" or "Special") standard* $(n-1)$*-dimensional component of* E *corresponding to a hypersurface and* $\mathbf{x}^{(0)} \in E_i$ *is a regular point of* $p_i(\mathbf{x}) = 0$*, then there exists at least one* associated hypersurface $F(\mathbf{x}, \mathbf{u}^{(0)})$ *of the family* F *that passes at point* $\mathbf{x}^{(0)}$ *(i.e.,* $F(\mathbf{x}^{(0)}, \mathbf{u}^{(0)}) = 0$*) and is tangent to* $p_i(\mathbf{x}) = 0$ *at point* $\mathbf{x}^{(0)}$*.*

Proof If there exists a component $E_i = \mathbf{V}(p_i) \setminus \mathbf{V}(\mathrm{q}_i)$ of the envelope of F, then the set of points of E_i are in the projection $\pi_s(V)$, V being given in Eqs. (8.4).

Let $\mathbf{x}^{(0)} \in E_i$. Being E_i a "Normal" (or a "Special") component, there exist at least one value $\mathbf{u}^{(0)}$ such that $F(\mathbf{x}^{(0)}, \mathbf{u}^{(0)}) = 0$. Thus $F(\mathbf{x}, \mathbf{u}^{(0)})$ verifies Eqs. (8.1) and (8.2), and is, in consequence, a hypersurface of the family F. Moreover, we consider the values $\mathbf{u}^{(0)}$ for which $F(\mathbf{x}, \mathbf{u}^{(0)})$ also verifies all Eqs. (8.3) for $(\mathbf{x}, \mathbf{u}) = (\mathbf{x}^{(0)}, \mathbf{u}^{(0)})$.

Let us prove that, for these values of $\mathbf{u}^{(0)}$ (possibly a single one), $F(\mathbf{x}, \mathbf{u}^{(0)})$ is tangent to $\mathbf{V}(p_i)$ at point $\mathbf{x}^{(0)}$.

The normal to the hypersurface $F(\mathbf{x}, \mathbf{u}^{(0)})$ at point $\mathbf{x}^{(0)}$ is

$$\mathbf{n}_F^{(0)} = \left(\frac{\partial F}{\partial x_1}, \ldots, \frac{\partial F}{\partial x_n} \right) (\mathbf{x}^{(0)}, \mathbf{u}^{(0)}). \tag{8.6}$$

By the Implicit Function Theorem, (see [78, pg. 41]), if $(\mathbf{x}^{(0)}, \mathbf{u}^{(0)})$ is a regular point of V, this implies the existence of locally defined implicit functions $\mathbf{u}(\mathbf{x})$ in the neighborhood of $(\mathbf{x}^{(0)}, \mathbf{u}^{(0)})$ that make Eqs. (8.1), (8.2), and (8.3) be identically verified on it. Particularly, it will happen that $g_i(u_1(\mathbf{x}), \ldots, u_m(\mathbf{x})) \equiv 0$ for all i. We can differentiate with respect to x_j and so we have

$$\sum_{k=1}^{m} \frac{\partial g_i}{\partial u_k} \frac{\partial u_k}{\partial x_j} (\mathbf{x}^{(0)}, \mathbf{u}^{(0)}) = 0, \quad \text{for } i \in [s] \text{ and } j \in [n]. \tag{8.7}$$

The normal to $p_i(\mathbf{x}) = 0$ at point $\mathbf{x}^{(0)}$ will be

$$\mathbf{n}_E(\mathbf{x}^{(0)}, \mathbf{u}^{(0)}) = \left(\frac{\partial F}{\partial x_1} + \sum_{j=1}^{m} \frac{\partial F}{\partial u_j} \frac{\partial u_j}{\partial x_1}, \ldots, \frac{\partial F}{\partial x_n} + \sum_{j=1}^{m} \frac{\partial F}{\partial u_j} \frac{\partial u_j}{\partial x_n} \right) (\mathbf{x}^{(0)}, \mathbf{u}^{(0)}).$$

To prove the equality of the normal directions $\mathbf{n}_F^{(0)}$ and $\mathbf{n}_E(\mathbf{x}^0, \mathbf{u}^{(0)})$, it suffices to prove that

$$\sum_{j=1}^{m} \frac{\partial F}{\partial u_j} \frac{\partial u_j}{\partial x_1} (\mathbf{x}^{(0)}, \mathbf{u}^{(0)}) = 0, \cdots \sum_{j=1}^{m} \frac{\partial F}{\partial u_j} \frac{\partial u_j}{\partial x_n} (\mathbf{x}^{(0)}, \mathbf{u}^{(0)}) = 0. \tag{8.8}$$

Consider the matrix product

$$M = \begin{pmatrix} \frac{\partial F}{\partial u_1} & \cdots & \frac{\partial F}{\partial u_m} \\ \frac{\partial g_1}{\partial u_1} & \cdots & \frac{\partial g_1}{\partial u_m} \\ \cdots & \cdots & \cdots \\ \frac{\partial g_s}{\partial u_1} & \cdots & \frac{\partial g_s}{\partial u_m} \end{pmatrix} \begin{pmatrix} \frac{\partial u_1}{\partial x_1} & \cdots & \frac{\partial u_1}{\partial x_n} \\ \cdots & \cdots & \cdots \\ \frac{\partial u_m}{\partial x_1} & \cdots & \frac{\partial u_m}{\partial x_n} \end{pmatrix} (\mathbf{x}^{(0)}, \mathbf{u}^{(0)}).$$

Taking into account Eqs. (8.7), lines $2, \ldots, m$ of M are identically 0. Now, the conditions (8.3) of J to have rank $\leq s$ implies that the first line of M is a linear combination of lines $2, \ldots, m$, and thus its elements are also 0. So M is the zero vector, and we have proved Eq. (8.8), and consequently the theorem. □

We consider now non-normal components of a dimension less than $n - 1$.

Theorem 8.7 *Given the family (8.1) of irreducible hypersurfaces of $\mathbb{C}^n$ restricted by conditions (8.2), if E is the envelope, and $H = \mathbf{V}(\mathfrak{p}(\mathbf{x})) \setminus \bigcup_j \mathbf{V}(\mathfrak{q}_j(\mathbf{x}))$ is the P-representation of an "Accumulation" component of E of dimension less than $n - 1$ and $\mathbf{x}^{(0)} \in H$ is a regular point of $\mathbf{V}(\mathfrak{p}(\mathbf{x}))$, then there exist infinitely many hypersurfaces $F(\mathbf{x}, \mathbf{u}_\lambda^{(0)})$ of the family F for some values $\mathbf{u}_\lambda^{(0)}$, varying λ, passing throug a point $\mathbf{x}^{(0)}$ (i.e., $F(\mathbf{x}^{(0)}, \mathbf{u}_\lambda) = 0$) and being tangent to $V(\mathfrak{p}(\mathbf{x}))$ at point $\mathbf{x}^{(0)}$.*

Remark 8.8 If the "Accumulation" component has dimension 0, consisting of a single point, then the concept of tangent hypersurface to the component simply mines that the hypersurface passes through the point-component.

Proof Assume that there exists a component $H = \mathbf{V}(\mathfrak{p}) \setminus \bigcup_j \mathbf{V}(\mathfrak{q}_j)$ of the envelope and let $\mathbf{x}^{(0)} \in H$ be a regular point on it. Every $f_i \in \langle \mathfrak{p} \rangle$ represents a hypersurface of the family, and by Theorem 8.6, there also exists a hypersurface of the family which is tangent to it passing through $\mathbf{x}^{(0)}$ for some value $\mathbf{u}_i^{(0)}$. This hypersurface is thus also tangent to the component H at $\mathbf{x}^{(0)}$. The same happens for every i. Moreover, for every linear combination $\sum_i \lambda_i f_i \in \langle \mathfrak{p} \rangle$ there also exists a hypersurface of the family for some value $\mathbf{u}_{\lambda_1, \ldots, \lambda_t}^{(0)}$ that will be tangent to the variety $\mathbf{V}(\mathfrak{p})$ at $\mathbf{x}^{(0)}$, and therefore the theorem is proved. □

8.3 Algorithms

It is obvious how to design the algorithm ENVELOP in correspondence with Theorem 8.6. The reason for denoting the command `envelop`, instead of `envelope`, comes from the fact that the SINGULAR library "grobcov.lib", where it is implemented, cannot use the reserved word "envelope", which has another meaning there.

$E \leftarrow$ **Envelop**(F, C)
`Input:` $F(x_1, \ldots, x_n; u_1, \ldots, u_m)$ a family of hypersurfaces,
(n variables, m parameters).
$C = \{g_1(u_1, \ldots, u_m), \ldots, g_s(u_1, \ldots, u_m)\}$: ideal of constraints,
with $s < m$.
Consider the ring $K[x_1, \ldots, x_n][u_1, \ldots, u_m]$,
taking $\mathbf{x}$ as parameters and $\mathbf{u}$ as variables.
`Output:` L, the components of the envelope (including its taxonomy)
$L = \{\{\mathfrak{q}_i, \{\mathfrak{q}_{ij} : 1 \le j \le r_i\}, \text{type}_i\} : 1 \le i \le r\}$
begin
$J = \dfrac{\partial(F, g_1, \ldots, g_s)}{\partial(u_1, \ldots, u_m)}$
$J_c = \{\text{all minors of } J \text{ of order } (s+1) \times (s+1)\}$
$S = \langle F, C, J_c \rangle$
$L = \text{LOCUS}(S)$
$L =$ Delete "Degenerate" components of L
return (L)
end

Algorithm 8.1: ENVELOP

The input is the family $F(\mathbf{x}, \mathbf{u})$ of hypersurfaces, in the $\mathbf{x}$ space depending on the parameters $\mathbf{u}$, and the constraints $C = \langle g_1(\mathbf{u}), \ldots, g_s(\mathbf{u}) \rangle$ over the parameters $\mathbf{u}$. It is necessary that $s < m$, m being the number of parameters $\mathbf{u}$. As for the `locus` command, here also the roles of $\mathbf{x}$ and $\mathbf{u}$ are inverted: the $\mathbf{x}$ are taken as parameters and the $\mathbf{u}$ as variables.

The algorithm adds the Jacobian conditions J_c to the ideal generated by F and C and computes the LOCUS, for which we have already given the algorithm and defined the taxonomy. See Algorithm 8.1: ENVELOP.

As a result, the envelope will consist of "Normal", "Special" and "Accumulation" components, as we shall discard "Degenerate" components. The algorithm is implemented in the SINGULAR "grobcov.lib" library [62] as command `envelop`, and is used in the examples.

As proved by Theorem 8.6, at each regular point of a normal component of the envelope, there exists an *associated tangent hypersurface of the family*. Given a component E_i of the envelope, the algorithm ASSOCTANTOENV determines the values of the parameters $\mathbf{u}$ of the associated tangent hypersurface at a given point of E_i.

The ASSOCTANTOENV algorithm is implemented in the SINGULAR library as command `AssocTanToEnv`, and is used in the examples. It consists of adding the top p_i of the component E_i to the ideal S used in the previous algorithm. Let $S_E = \langle F, C, J_c, p_i \rangle$, and compute its `grobcov`$(S_E)$. The result will have a generic segment $\mathbb{C}^m \setminus \mathbf{V}(p_i)$ without solutions, which can be discarded; then a segment $\mathbf{V}(p_i) \setminus \mathbf{V}(M)$ where $\mathbf{V}(M)$ are the possible singular points of the component E_i, that inform us about the values of the parameters of the associated tangent hypersurface on the regular points; and then possible segments determining the non-regular points of E_i with possibly more than one associated tangent hypersurfaces. See Algorithm 8.2: ASSOCTANTOENV.

$E \leftarrow$ **AssocTanToEnv**(F, C, p_i)
Input: $F(x_1, \ldots, x_n; u_1, \ldots, u_m)$ a family of hypersurfaces,
(n variables, m parameters).
$C = \{g_1(u_1, \ldots, u_m), \ldots, g_s(u_1, \ldots, u_m)\}$: ideal of constraints,
with $s < m$.
$p_i =$ the top of the envelope component (computed by ENVELOP).
Consider the ring $K[x_1, \ldots, x_n][u_1, \ldots, u_m]$,
taking **x** as parameters and **u** as variables.
Output: $G = ((S_i, B_i) : i = 1, \ldots, s)$, the basis giving the parameters
of the associated tangent hypersurface of (F, C).
In a similar description as the GROBCOV algorithm
begin
$J = \dfrac{\partial(F, g_1, \ldots, g_s)}{\partial(u_1, \ldots, u_m)}$
$J_c = \{$all minors of J of order $(s+1) \times (s+1)\}$
$S = \langle F, C, J_c, p_i \rangle$
$G =$ GROBCOV(S)
$G =$ Delete the first segment of G
return (G)
end

Algorithm 8.2: ASSOCTANTOENV

Another question of interest that can be determined is the set of family elements that pass through a point of an envelope component E_i. If at a point of E_i the unique surface of the family passing through it is the associated tangent one, then the point really separates internal and external part of the family, and can be considered part of the **real** envelope.

For this purpose we have designed the algorithm FAMELEMSATENVCOMPPOINTS. It suffices to consider the ideal $S = \langle F, C, p_i \rangle$ and compute its `grobcov`. We eliminate the generic segment that will be $\mathbb{C}^m \setminus \mathbf{V}(p_i)$, because it cannot contain any solution. This algorithm is also included in the SINGULAR "grobcov.lib" library in command `FamElemsAtEnvCompPoints` and is used in the examples. It can be useful for determining the true real parts of the envelope, as can be seen in the examples. Nevertheless the conclusions are not obvious and need discussion., (see Algorithm 8.3: FAMELEMSATENVCOMPPOINTS).

8.4 Simple Examples

We distinguish classic envelope problems in n-dimensional space, which have less than n degrees of freedom, from the new type of envelope problems that are proposed and discussed in the next section.

In this section, we give some classical examples of the first type. Even if it is not necessary to use the new algorithms, we use them in order to show their performance and the new results that can be obtained. So the reader can become familiar with the output of the algorithms and their usefulness.

$E \leftarrow$ **FamElemsAtEnvCompPoints**(F, C, p_i)
`Input:` $F(x_1, \ldots, x_n; u_1, \ldots, u_m)$ a family of hypersurfaces,
(n variables, m parameters).
$C = \{g_1(u_1, \ldots, u_m), \ldots, g_s(u_1, \ldots, u_m)\}$: ideal of constraints.
It must be $s < m$.
$p_i =$ the top of the envelope component (computed by ENVELOP)
The ring must be $K[x_1, \ldots, x_n][u_1, \ldots, u_m]$
and consider the **x** as parameters and the **u** as variables.
`Output:` $G = ((S_i, B_i) : i = 1, \ldots, s)$, the basis giving the parameter
values of the hypersurfaces of (F, C) passing through a point of E_i.
In a similar description for the GROBCOV algorithm
begin
$S = \langle F, C, p_i \rangle$
$G =$ GROBCOV(S)
$G =$ Delete the first segment of G
return (G)
end

Algorithm 8.3: FAMELEMSATENVCOMPPOINTS.

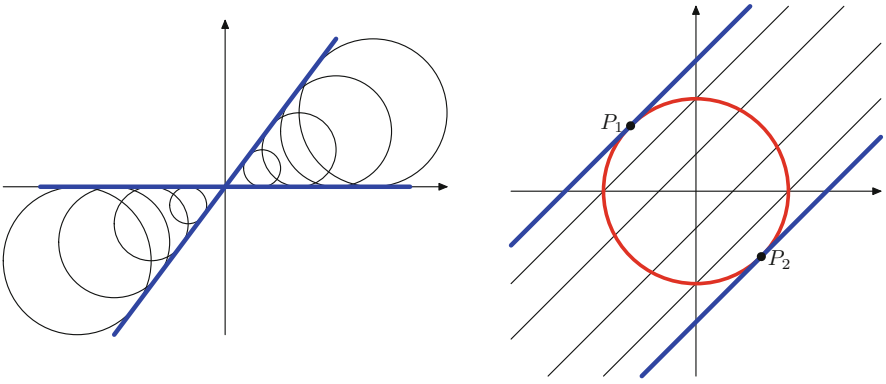


Fig. 8.1 Family of circles of Examples 8.9 and 8.10 and their envelopes

As already commented, Theorem 8.6 is not sufficient for obtaining the real envelope. We show in the examples how the new algorithms can help to give partial solutions to this question.

Example 8.9 Consider the family of circles centred at point $(2t, t)$ with radius t. The equation of the family is

$$F = (x - 2t)^2 + (y - t)^2 - t^2.$$

Let us apply the command `envelop`. (See Fig. 8.1, left).

```
> LIB "grobcov.lib";
> ring R=(0,x,y),(t),lp;
> short=0;
```

```
> poly F=(x-2*t)^2+(y-t)^2-t^2;
> ideal C;
> envelop(F,C);
[1]:
   [1]:
      _[1]=(y)
   [2]:
      [1]:
         _[1]=1
   [3]:
      [1]:
         1
      [2]:
         Normal
      [3]:
         _[1]=0
[2]:
   [1]:
      _[1]=(4*x-3*y)
   [2]:
      [1]:
         _[1]=1
   [3]:
      [1]:
         1
      [2]:
         Normal
      [3]:
         _[1]=0
```

The output is of the same form as the output of the `locus` command already described in Chap. 7, Sect. 7.5.

We see, as expected, that we obtain two "Normal" components consisting of the two external straight lines enveloping all the circles (see Fig. 8.1, left). These are $E_1 = y$ and $E_2 = 4x - 3y$. In this example, there is a single auxiliary variable t and this is the reason for obtaining a null anti-image: all values of t contribute to the envelope.

We can now determine the associated tangent circle at a component, say $E_1 = y$ at a point $(x_0, 0) \in E_1$. We use the `AssocTanToEnv` command. After the previous computation we continue:

```
> ideal E=Env[1][1];  E;
E[1]=y
> def A=AssocTanToEnv(F,C,E); A;
[1]:
   [1]:
      _[1]=t
   [2]:
      _[1]=2*t+(-x)
   [3]:
      [1]:
```

```
          [1]:
             _[1]=(y)
          [2]:
             [1]:
                _[1]=1
> def A1 =A[1][2][1]; A1;
2*t+(-x)
```

The output of `AssocTanToEnv` is identical to the output of the `grobcov` command already described in Chap. 7, Sect. 5.4, with the restriction `("null",E)`.

From the output we see that the value of the parameter t of the associated tangent circle to E_1 at a point $(x_0, 0) \in E_1$ is $t_1 = \frac{x_0}{2}$ and thus, the center of the associated circle is $(x_1, y_1) = (2t_1, t_1) = (x_0, \frac{x_0}{2})$, being the associated tangent circle

$$F_0 = (x - x_0)^2 + \left(y - \frac{x_0}{2}\right)^2 - \frac{x_0^2}{4}.$$

We can finish the analysis determining the circles of the family that pass at a point of E_1. As we shall see, this information can be useful to decide which points really form part of a separating envelope or are included in the internal region where there exist infinitely many family elements passing through the point. We will see this in next section. For this purpose we use the algorithm FAMELEMSATENVCOMPPOINTS implemented as command `FamElemsAtEnvCompPoints`.

We continue with the previous computation:

```
> FamElemsAtEnvCompPoints(F,C,E);
[1]:
   [1]:
      _[1]=t^2
   [2]:
      _[1]=4*t^2+(-4*x)*t+(x^2)
   [3]:
      [1]:
         [1]:
            _[1]=(y)
         [2]:
            [1]:
               _[1]=1
```

We obtain now the basis $4t^2 - 4x + x^2 = (x - 2t)^2$, showing that the solution is the double point corresponding to the tangent circle already found, being, in this example, the unique circle of the family passing through $(x_0, 0) \in E$ as expected.

Example 8.10 We present this simple example to show the presence of a "Special" envelope component. Let $M = (x_1, y_1)$ be a a moving point on the circle $C = x_1^2 + y_1^2 - 25$, and consider the family of lines F parallel to the line $y = x$ passing trough M. We expect to obtain the two limit lines of the family tangent to the circle.

The family is now $F = (y - y_1) - (x - x_1)$ and the constraint $C = x_1^2 + y_1^2 - 25$. (See Fig. 8.1, (right)). Let us apply the command `envelop`:

```
> LIB "grobcov.lib";
> ring R=(0,x,y),(x1,y1),dp;
> def F=(y-y1)-(x-x1);
> def C=x1^2+y1^2-25;
> short=0;
> envelop(F,C)
[1]:
   [1]:
      _[1]=(x^2-2*x*y+y^2-50)
   [2]:
      [1]:
         _[1]=1
   [3]:
      [1]:
         0
      [2]:
         Special
      [3]:
         _[1]=2*y1^2-25
         _[2]=x1+y1
```

We obtain a single "Special" component, namely $E = x^2 - 2xy + y^2 - 50$, that is irreducible over $\mathbb{Q}$, but can be reduced over $\mathbb{R}$ to

$$E = (x - y - \sqrt{50})(x - y + \sqrt{50}),$$

representing the two lines of Fig. 8.1, (right). The component is "Special", because the mover point M, the anti-image of a point $(x_0, y_0) \in E$ does not depend on the point $(x_0, y_0) \in E$ and consists of the two tangency points to the initial circle. The basis of the anti-image given by the `envelop` command is $B = [x_1 + y_1, 2y_1^2 - 25]$, which represents the pair of tangency points to the circle

$$P_1 = (-5/\sqrt{2}, 5/\sqrt{2}) \text{ and } P_2 = (5/\sqrt{2}, -5/\sqrt{2}),$$

that are also represented in Fig. 8.1, (right). If we use the commands `AssocTanToEnv` and `FamElemsAtEnvCompPoints`, the result is obviously the unique pair of tangent points and the envelope itself as the unique pair of tangent lines of the family tangent to the envelope. We leave the verification as an exercise.

Example 8.11 The next example is taken from Problem 5.5.2 in [12]. Let us analyze it from our perspective.

Consider the parabola $y = x^2$ and a point $M = (t, t^2)$ on the it. The equation of the tangent line at M is $F = y - 2tx + t^2$. Taking the set of tangent lines F as a new family and determining its envelope, we expect to obtain the parabola itself as the unique "Normal" component. We also expect that there is a unique family line intersecting the curve at a fixed point (x_0, y_0), namely the tangent line at this

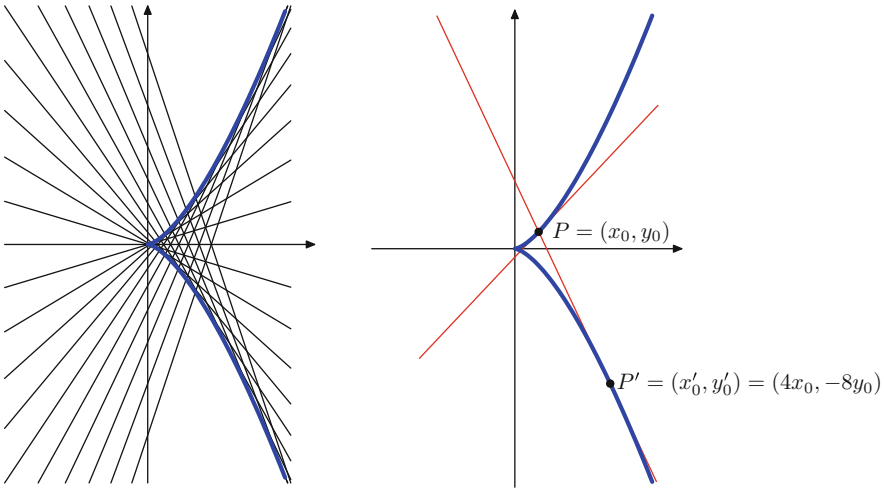


Fig. 8.2 Example 8.11

point for which the value of t is $t_0 = \dfrac{y_0}{x_0}$. We leave as an exercise to prove these assumptions using the algorithms.

Consider now the same problem for the cubic $y^2 - 2x^3 = 0$, that can be parametrized by $P = (x, y) = (2t^2, 4t^3)$. The tangent line at P is now

$$F \equiv y + 2t^3 - 3tx,$$

and we take this equation as our new family. Let us apply our three algorithms to the new family F. (See Fig. 8.2, (left)). We begin determining the envelope

```
> LIB "grobcov.lib";
> ring R=(0,x,y),(t),dp;
> short=0;
> poly F=2*t^3-3*t*x+y;
> ideal C;
> def Env=envelop(F,C);
> Env;
[1]:
   [1]:
      _[1]=(2*x^3-y^2)
   [2]:
      [1]:
         _[1]=1
   [3]:
      [1]:
         1
      [2]:
         Normal
      [3]:
```

```
          _[1]=0
> ideal E=Env[1][1]; E;
E[1]=(2*x^3-y^2)
```

As expected, the envelope has the unique "Normal" component

$$E = 2x^3 - y^2.$$

Now we determine the associated tangent line at point $P(2t^2, 4t^3)$.

```
> AssocTanToEnv(F,C,E);
[1]:
   [1]:
      _[1]=t
   [2]:
      _[1]=(y)*t+(-x^2)
   [3]:
      [1]:
         [1]:
            _[1]=(2*x^3-y^2)
         [2]:
            [1]:
               _[1]=(y)
               _[2]=(x)
[2]:
   [1]:
      _[1]=t^2
   [2]:
      _[1]=t^2
   [3]:
      [1]:
         [1]:
            _[1]=(y)
            _[2]=(x)
         [2]:
            [1]:
               _[1]=1
```

The result is that the value of t corresponding to the associated tangent line at point $P(x_0, y_0)$ different from $(0, 0)$ is

$$t_0 = \frac{x_0^2}{y_0},$$

as expected.

Finally we determine the set of all family lines passing through P:

```
> FamElemsAtEnvCompPoints(F,C,E);
[1]:
   [1]:
      _[1]=t^3
   [2]:
      _[1]=2*t^3+(-3*x)*t+(y)
```

```
[3]:
   [1]:
      [1]:
         _[1]=(2*x^3-y^2)
      [2]:
         [1]:
            _[1]=1
```

Observing the output of `FamElemsAtEnvCompPoints`, the set of family lines F passing through the point $P = (x_0, y_0)$ of E verify

$$2t^3 - 3x_0 t + y_0 = 0, \tag{8.9}$$

which is an equation of degree 3 in t, and will have three solutions for t.

We can verify that the value $t_0 = \dfrac{x_0^2}{y_0}$ corresponding to the tangent is a double solution of equation of (8.9).

Dividing Eq. (8.9) by $\left(t - \dfrac{x_0^2}{y_0}\right)^2$, quotient and remainder are

$$q = 2t + \frac{4x_0^2}{y_0}$$

$$r = \left(-3x_0 + \frac{6x_0^4}{y_0^2}\right) t + \left(-\frac{4x_0^6}{y_0^3} + y_0\right) = (-2x_0^3 + y_0^2)\left(-\frac{3x_0}{y_0^2} t + \frac{2x_0^3 + y_0^2}{y_0^3}\right).$$

Taking into account that $E(x_0, y_0) = 0$, the remainder r vanishes, proving that t_0 is a double solution of (8.9). The zero of q determines a second value of t of a family line passing through M, namely

$$t = t_0' = -\frac{2x_0^2}{y_0} = -2t_0.$$

The second family line will be therefore $F_{t_0'} = F(x, y, -\dfrac{2x_0^2}{y_0})$.

$F_{t_0'}$ is tangent to the envelope at another point $P' = (x_0', y_0')$ of the envelope for $t = t_0' = \dfrac{x_0'^2}{y_0'} = -\dfrac{2x_0^2}{y_0}$.

Substituting in the parametric equation, we obtain

$$x_0' = 2t_0'^2 = \quad 8t_0^2 = \ 4 \cdot (2t_0^2) = \ 4x_0,$$
$$y_0' = 4t_0'^3 = -32t_0^3 = -8 \cdot (4t_0^3) = -8y_0.$$

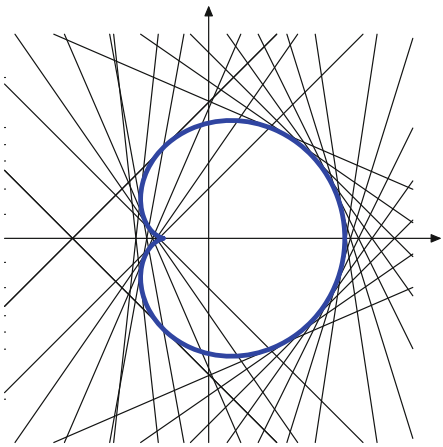


Fig. 8.3 Example 8.12

The coordinates of P' are thus

$$P' = (x_0', y_0') = (4x_0, -8y_0).$$

Figure 8.2, (right) shows the tangent F_{t_0} at point $M = (x_0, y_0)$, the second line $F_{t_0'}$ of the family F also intersecting the envelope at P, and the point $P' = (x_0', y_0')$, whose associated tangent is $F_{t_0'}$.

We can say, that the algebraic envelope of this problem does not really separate the internal and the external part of the set of lines, and so it does not exist a proper separating real envelope.

Example 8.12 The next problem is also to be compared with Problem (5.7.2) in [12], from which we take the approach. Let

$$F(t, x, y) = x(2\sin t \cos t - \sin t) + y(\cos t + \sin^2 t - 1) - \sin t.$$

To use our methods we have to substitute $s = \sin t$, $c = \cos t$ and add the restriction $s^2 + c^2 - 1 = 0$. You can see the picture in Fig. 8.3. Using the algorithms we have:

```
> LIB "grobcov.lib";
> ring R=(0,x,y),(s,c),dp;
> short=0;
> poly F=x*(2*s*c-s)+y*(c+2*s^2-1)-s;
> ideal C=s^2+c^2-1;
> def Env=envelop(F,C); Env;
[1]:
   [1]:
      _[1]=(27*x^4+54*x^2*y^2-18*x^2-8*x+27*y^4-18*y^2-1)
   [2]:
      [1]:
         _[1]=1
   [3]:
```

```
      [1]:
         1
      [2]:
         Normal
      [3]:
         _[1]=s^2+c^2-1
[2]:
   [1]:
      _[1]=(x-1)
   [2]:
      [1]:
         _[1]=1
   [3]:
      [1]:
         0
      [2]:
         Special
      [3]:
          _[1]=c^2+c-2
         _[2]=s^2-c+1
> def E=Env[1][1]; E;
E[1]=(27*x^4+54*x^2*y^2-18*x^2-8*x+27*y^4-18*y^2-1)
> def A=AssocTanToEnv(F,C,E); A;
[1]:
   [1]:
      _[1]=c
      _[2]=s
   [2]:
      _[1]=4*c+(-9*x^2-9*y^2+5)
      _[2]=(4*y)*s+(9*x^3-3*x^2+9*x*y^2-5*x-3*y^2-1)
   [3]:
      [1]:
         [1]:
            _[1]=(27*x^4+54*x^2*y^2-18*x^2-8*x+27*y^4-18*y^2-1)
         [2]:
            [1]:
               _[1]=(y)
               _[2]=(3*x+1)
            [2]:
               _[1]=(y)
               _[2]=(x-1)
            [3]:
               _[1]=(3*y^2+4)
               _[2]=(x-1)
[2]:
   [1]:
      _[1]=c
      _[2]=s^2
   [2]:
      _[1]=(2*x)*c+(-x-1)
      _[2]=s^2
   [3]:
      [1]:
         [1]:
```

```
                _[1]=(y)
                _[2]=(x-1)
            [2]:
                [1]:
                    _[1]=1
        [2]:
            [1]:
                _[1]=(y)
                _[2]=(3*x+1)
            [2]:
                [1]:
                    _[1]=1
[3]:
    [1]:
        _[1]=s
        _[2]=c^2
    [2]:
        _[1]=2*s+(-y)*c+(y)
        _[2]=c^2+c-2
    [3]:
        [1]:
            [1]:
                _[1]=(3*y^2+4)
                _[2]=(x-1)
            [2]:
                [1]:
                    _[1]=1
```

The envelope has one "Normal" component

$$E_1 = 27x^4 + 54x^2y^2 - 18x^2 - 8x + 27y^4 - 18y^2 - 1,$$

and one "Special" component

$$E_2 = x - 1,$$

whose anti-image is $c = 1, s = 0$. But $F(x, y, c = 1, s = 0) \equiv 0$, and there is no line of the family corresponding to these values. So we tend to discard this "Special" component.

On the other hand, the parameter values of the associated tangent to the "Normal" envelope component given by the algorithm, are

$$c_0 = \frac{1}{4}(9x_0^2 + 9y_0^2 - 5)$$

$$s_0 = \frac{1}{4y_0}(-9x_0^3 + 3x_0^2 - 9x_0y_0^2 + 5x_0 + 3y_0^2 + 1)$$

for every point of $\mathbf{V}(E_1)$ except for the two special real points $P_1(-1/3, 0)$, $P_2 = (1, 0)$, and a pair of complex points in which we are not interested. For P_1 and P_2 we

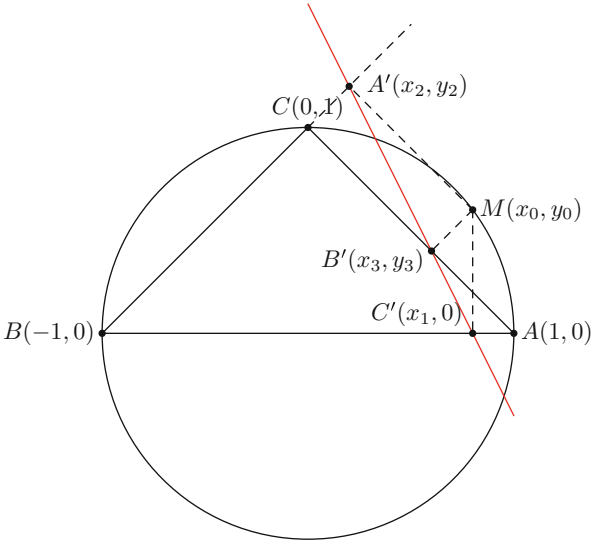


Fig. 8.4 Example 8.13 for point M on the circumcircle of ABC

obtain double solutions, namely $(c = 1, s = 0)$ for P_1 corresponding to the tangent line $y = 0$, and $(s = 0, c = -1)$ for P_2 corresponding to the tangent line $x = 1$. We leave the study of the lines passing through the points of E_1 as an exercise. From the geometrical perspective, we expect to have only the tangent line and another real line passing through each point $(x_0, y_0) \in \mathbf{V}(E_1)$, as in Example 8.11. The reader can try to use the technique used there to compute it.

Example 8.13 Consider a triangle $A = (1, 0)$, $B = -(1, 0)$, $C = (0, 1)$ and a free point $M(x_0, y_0)$. Let ℓ, m, n be the lines from M perpendicular to sides BC, AC and AB of the triangle. Let A', B', C' be the intersections of lines ℓ, n, m with the sides of the triangle. (See Fig. 8.4).

The following equations relate the positions of points A', B', C' with that of the free point $M(x_0, y_0)$:

$$\begin{aligned} H = & x_2 - y_2 + 1, x_2 - x_0 + (y_2 - y_0), && \#\, BC, MA' \\ & x_3 + y_3 - 1, y_3 - y_0 - x_3 + x_0, && \#\, AC, MB' \\ & y_1, x_1 - x_0. && \#\, AB, MC' \end{aligned}$$

We can express the coordinates of A', B', C' in terms of $M(x_0, y_0)$ computing the following `grobcov`

```
> LIB "grobcov.lib";
> ring R0=(0,x0,y0),(x3,y3,x2,y2,x1,y1),lp;
> ideal H=x2-y2+1, x2-x0+(y2-y0),
```

```
        x3+y3-1, y3-y0-x3+x0,
        y1,x1-x0;
> def J=grobcov(H);   J;
[1]:
   [1]:
      _[1]=y1
      _[2]=x1
      _[3]=y2
      _[4]=x2
      _[5]=y3
      _[6]=x3
   [2]:
      _[1]=y1
      _[2]=x1+(-x0)
      _[3]=2*y2+(-x0-y0-1)
      _[4]=2*x2+(-x0-y0+1)
      _[5]=2*y3+(x0-y0-1)
      _[6]=2*x3+(-x0+y0-1)
   [3]:
      [1]:
         [1]:
            _[1]=0
         [2]:
            [1]:
               _[1]=1
> ideal H2=J[1][2]; H2;
H2[1]=y1
H2[2]=x1+(-x0)
H2[3]=2*y2+(-x0-y0-1)
H2[4]=2*x2+(-x0-y0+1)
H2[5]=2*y3+(x0-y0-1)
H2[6]=2*x3+(-x0+y0-1)
```

Solving H_2 for the set of coordinates of points A', B', C', we have

$$\begin{aligned} &y_1 = 0, && x_1 = x_0, \\ &y_2 = 1/2(x_0 + y_0 + 1), && x_2 = 1/2(x_0 + y_0 - 1), \\ &y_3 = 1/2(-x_0 + y_0 + 1), && x_3 = 1/2(x_0 - y_0 + 1). \end{aligned} \tag{8.10}$$

Our first problem is determining for which positions of M points A', B', C' are aligned. This will happen if the following determinant is zero

$$T \equiv \begin{vmatrix} x_1 & y_1 & 1 \\ x_2 & y_2 & 1 \\ x_3 & y_3 & 1 \end{vmatrix} = x_1y_2 + x_2y_3 + x_3y_1 - x_3y_2 - x_2y_1 - x_1y_3. \tag{8.11}$$

We can use the command ADGT (see Chap. 6, Sect. 6.2) for automatic discovering the geometric theorem $H_2 \Rightarrow T$.

```
> ideal T=x1*y2+x2*y3+x3*y1-x3*y2-x2*y1-x1*y3;
> def Alig=ADGT(H2,T,1,1); Alig;
[1]:
   [1]:
      1
   [2]:
      [1]:
         [1]:
            _[1]=(x0^2+y0^2-1)
         [2]:
            [1]:
               _[1]=1
```

This proves the following

Theorem 8.14 *Points A', B' and C' are aligned if and only if the mover point $M(x_0, y_0)$ lies on the circumcircle $c : x_0^2 + y_0^2 - 1 = 0$ of ABC.*

Remark 8.15 The preceding theorem can be generalized for any non degenerate triangle. (We let the study for the reader.)

To determine the adequate expressions of the family of lines F and the constraints C in terms of (x_0, y_0), we continue the computations adding the equations of c and those of the lines determined by $B'C'$, $A'C'$, $A'B'$, and computing their Gröbner basis

```
> ring R1=0,(x,y,x3,y3,x2,y2,x1,y1,x0,y0),lp;
> def HH=imap(R0,H2);
> ideal HP=x0^2+y0^2-1,
    (x1-x3)*y+(y3-y1)*x+x3*y1-x1*y3, // B'C';
    (x1-x2)*y+(y2-y1)*x+x2*y1-x1*y2, // A'C'
    (x3-x2)*y+(y2-y3)*x+x2*y3-x3*y2; // A'B'
> HH=HH,HP;
> def GH=std(HH); GH;
GH[1]=x0^2+y0^2-1
GH[2]=y1
GH[3]=x1-x0
GH[4]=2*y2-x0-y0-1
GH[5]=2*x2-x0-y0+1
GH[6]=2*y3+x0-y0-1
GH[7]=2*x3-x0+y0-1
GH[8]=x*y0+x+y*x0-x0*y0-x0
GH[9]=x*x0-y*y0+y+y0^2-1
> def F1=GH[8];
> def F2=GH[9];
> def C=GH[1];
> F1;
x*y0+x+y*x0-x0*y0-x0
> F2;
x*x0-y*y0+y+y0^2-1
> C;
x0^2+y0^2-1
```

We observe that the formula determining that the line passes through A', B' and C' is given by two polynomials instead of a single one. It represents thus an I-regular function $F = \{F_1, F_2\}$. The geometrical reason is that for M on a vertex, for example at point C, points A' and B' become equal to C and so the line $A'B'$ is undefined. So one needs another expression that uses C' and either A' or B' for a complete description.

Nevertheless, the command `envelop` for determining the envelope of the family of lines F does not accept an I-regular function for the family, and we must restrict our computation to the use of a generic representation, i.e. either F_1 or F_2. This will cause some troubles as we shall see.

The same happens for the command `AssocTanToEnv` for obtaining the the tangent element at a point of an envelope component. We shall see, instead, how to use the command `FamElemsAtEnvCompPoints` with the full representation of F.

We choose

$$\begin{aligned} F_0 &= xy_0 + x + yx_0 - x_0y_0 - x_0, \\ C_0 &= x_0^2 + y_0^2 - 1. \end{aligned} \tag{8.12}$$

Having eliminated the variables of points A', B', C', we continue the computation as follows:

```
> ideal GGH=GH[1],GH[8],GH[9];
> ring R=(0,x,y),(x0,y0),lp;
> ideal RH=imap(R1,GGH); RH;
RH[1]=x0^2+y0^2-1
RH[2]=-x0*y0+(y-1)*x0+(x)*y0+(x)
RH[3]=(x)*x0+y0^2+(-y)*y0+(y-1)
> poly F1=RH[2];
> poly F2=RH[3];
> ideal C1=RH[1];
> def E=envelop(F1,C1); E;
[1]:
   [1]:
      _[1]=(x^4+2*x^2*y^2+10*x^2*y-x^2+y^4-6*y^3+12*y^2-8*y)
   [2]:
      [1]:
         _[1]=1
   [3]:
      [1]:
         1
      [2]:
         Normal
      [3]:
         _[1]=x0^2+y0^2-1
[2]:
   [1]:
      _[1]=(y)
   [2]:
      [1]:
```

```
        _[1]=1
   [3]:
      [1]:
         0
      [2]:
         Special
      [3]:
         _[1]=y0^2+y0
         _[2]=x0^2-y0-1
> def E0=E[1][1][1]; E0;
(x^4+2*x^2*y^2+10*x^2*y-x^2+y^4-6*y^3+12*y^2-8*y)
```

We obtain the "Normal" envelope component

$$E_0 = x^4 + 2x^2y^2 + 10x^2y - x^2 + y^4 - 6y^3 + 12y^2 - 8y, \tag{8.13}$$

representing the Steiner Deltoid, and a "Special" component $y = 0$. The "Special" component comes from considering only the generic representation of F in (8.12) instead of its full representation. So we discard this component (Fig. 8.5).

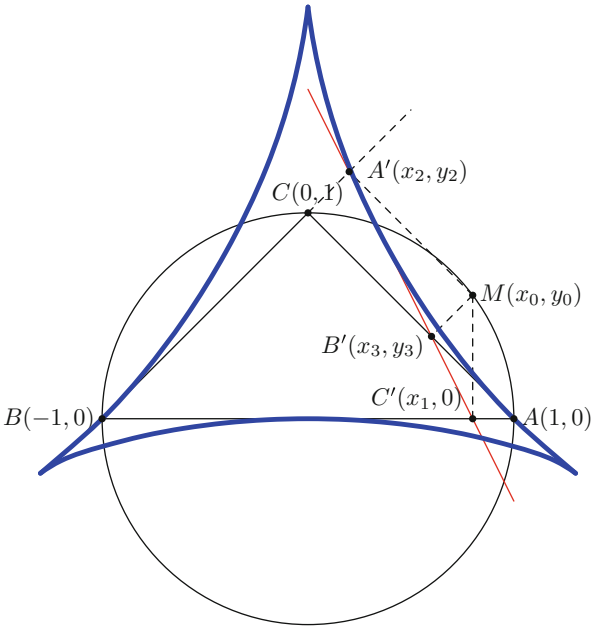


Fig. 8.5 Envelope of Example 8.13: Deltoid

Now we can determine the associated tangent line at a point $P(x, y)$ of the "Normal" envelope component E_0. We proceed as follows:

```
> def A=AssocTanToEnv(F1,C1,E0);  A;
[1]:
   [1]:
      _[1]=y0
      _[2]=x0
   [2]:
      _[1]=(4*y+1)*y0+(-x^2-y^2-3*y+1)
      _[2]=(4*y^2-7*y-2)*x0+(x^3+x*y^2+11*x*y+x)
   [3]:
      [1]:
         [1]:
            _[1]=(x^4+2*x^2*y^2+10*x^2*y-x^2+y^4-6*y^3+12*y^2-8*y)
         [2]:
            [1]:
               _[1]=(y)
               _[2]=(x)
            [2]:
               _[1]=(y-2)
               _[2]=(x)
            [3]:
               _[1]=(4*y+1)
               _[2]=(16*x^2-27)
[2]:
   [1]:
      _[1]=y0^2
      _[2]=x0
   [2]:
      _[1]=(8*y+1)*y0^2+(4*y)*y0+(-3*y-1)
      _[2]=(13*y+1)*x0+(x)*y0+(2*x)
   [3]:
      [1]:
         [1]:
            _[1]=(y)
            _[2]=(x)
         [2]:
            [1]:
               _[1]=1
      [2]:
         [1]:
            _[1]=(4*y+1)
            _[2]=(16*x^2-27)
         [2]:
            [1]:
               _[1]=1
[3]:
   [1]:
      _[1]=y0^2
      _[2]=x0*y0
      _[3]=x0^2
   [2]:
```

```
      _[1]=y0^2-2*y0+1
      _[2]=x0*y0-x0
      _[3]=x0^2+2*y0-2
   [3]:
      [1]:
         [1]:
            _[1]=(y-2)
            _[2]=(x)
         [2]:
            [1]:
               _[1]=1
> def Mt=A[1][2];
> Mt;
_[1]=(4*y+1)*y0+(-x^2-y^2-3*y+1)
_[2]=(4*y^2-7*y-2)*x0+(x^3+x*y^2+11*x*y+x)
```

Reading the output of `AssocTanToEnvComp`, the value of $M_t(x_0, y_0)$ for the associated tangent line at point $P(x, y) \in E_0$ is

$$\begin{aligned} x_0 &= -\frac{x^3 + xy^2 + 11xy + x}{4y^2 - 7y - 2}, \\ y_0 &= \frac{x^2 + y^2 + 3y - 1}{4y + 1} \end{aligned} \tag{8.14}$$

except for some special points arising from the use of the generic representation of F, that we also discard.

Now, we want to determine all the family elements passing through a point $P(x, y)$ of the envelope component. Here we can use a trick to include the full representation of F. Since the command `FamElemsAtEnvCompPoints` only uses the union of F and C, we can include in C the second representation F_2 of F. So we proceed as follows

```
> ideal C2=C1,F2;
> C2;
_[1]=x0^2+y0^2-1;
_[2]=(x)*x0+y0^2+(-y)*y0+(y-1)
>  def FEAECP=FamElemsAtEnvCompPoints(F1,C2,E0);
[1]:
   [1]:
      _[1]=y0^3
      _[2]=x0
   [2]:
      _[1]=y0^3+(-2*y+1)*y0^2+(x^2+y^2-1)*y0+(x^2-y^2+2*y-1)
      _[2]=(y^4-6*y^3+12*y^2-8*y)*x0+(-x^3-2*x*y^2-10*x*y+x)*y0^2
         +(x^3*y+2*x*y^3+10*x*y^2-x*y)*y0
         +(-x^3*y+x^3-2*x*y^3-8*x*y^2+11*x*y-x)
   [3]:
      [1]:
         [1]:
            _[1]=(x^4+2*x^2*y^2+10*x^2*y-x^2+y^4-6*y^3+12*y^2-8*y)
```

```
          [2]:
             [1]:
                _[1]=(y)
                _[2]=(x)
             [2]:
                _[1]=(y-2)
                _[2]=(x)
[2]:
   [1]:
      _[1]=y0^2
      _[2]=x0*y0
      _[3]=x0^2
   [2]:
      _[1]=(y-1)*y0^2+(-y)*y0+1
      _[2]=(y-1)*x0*y0-x0
      _[3]=(y-1)*x0^2+(y)*y0+(-y)
   [3]:
      [1]:
         [1]:
            _[1]=(y)
            _[2]=(x)
         [2]:
            [1]:
               _[1]=1
      [2]:
         [1]:
            _[1]=(y-2)
            _[2]=(x)
         [2]:
            [1]:
               _[1]=1
```

Thus the lines of the family passing through point $P(x, y) \in E_0$ verify

$$f_1 = y_0^3 + (-2y+1)y_0^2 + (x^2+y^2-1)y_0 + (x^2-y^2+2y-1),$$

$$f_2 = (y^4 - 6y^3 + 12y^2 - 8y)x_0 + (-x^3 - 2xy^2 - 10xy + x)y_0^2 + (x^3y + 2xy^3 + 10xy^2 - xy)y_0 - (x^3y - x^3 + 2xy^3 - 8xy^2 - 11xy + x).$$

Regarding Fig. 8.6, we expect that through a point of the envelope only two lines of the family pass: the associated tangent, and another line of the family F tangent at another point of E_0. As f_1 has degree 3 in y_0 and is independent of x_0, and f_2 has degree 1 in x_0, the system has three solutions.[1]

As in previous Example 8.11, we expect that f_1 has the double solution of y_0 corresponding to the associated tangent line. Thus we divide f_1 by $((4y+1)y_0 - (x^2+y^2+3y-1))^2$ corresponding to the value of the associated tangent

[1]If we do not use the full representation of F but only the generic one, we obtain an extra point y=1, that must be discarded.

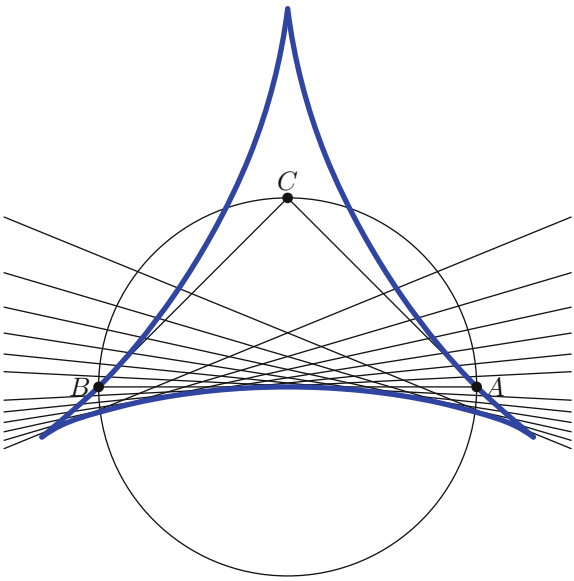


Fig. 8.6 Deltoid and some family lines of Example 8.13

(formula (8.14)). The result (obtained in Maple) is

$$> q := \text{simplify}(\text{quo}(f_0, ((4y+1)y_0 - (x^2+y^2+3y-1))^2, y_0, 'r')) :$$

$$q = \frac{-6y^2 + (4y_0+8)y + 2x^2 + y_0 - 1}{(4y+1)^3}$$

$$> \text{factor}(-r);$$

$$\frac{(2x^2 - 6y^2 - 12yy_0 - 3y_0 - 3)E_0}{(4y+1)^3}.$$

The remainder has the envelope E_0 as factor. Thus, for the points of E_0, the remainder is 0, and we conclude that `FamElemsAtEnvCompPoints` gives the double solution of the associated tangent plus another line of the family determined by the position of $M_n(x_0', y_0')$ at a point that we obtain isolating y_0 from q and substituting it in f_2 and then solving for x_0

$$y_0' = -\frac{2x^2 - 6y^2 + 8y - 1}{4y+1},$$
$$x_0' = \frac{(4(x^2 - y^2 + (5/2)y - 1))(x^2 - 3y^2 + 6y)(x^2 + 2y^2 + 10y - 1)x}{(4y+1)^2 y(y-2)^3}. \tag{8.15}$$

The result can be illustrated (see Fig. 8.7) plotting both lines for a particular point of the Deltoid, for instance $x = -0.6$. Then we determine the corresponding value of y on E_0 (the computations are done using Maple) and obtain

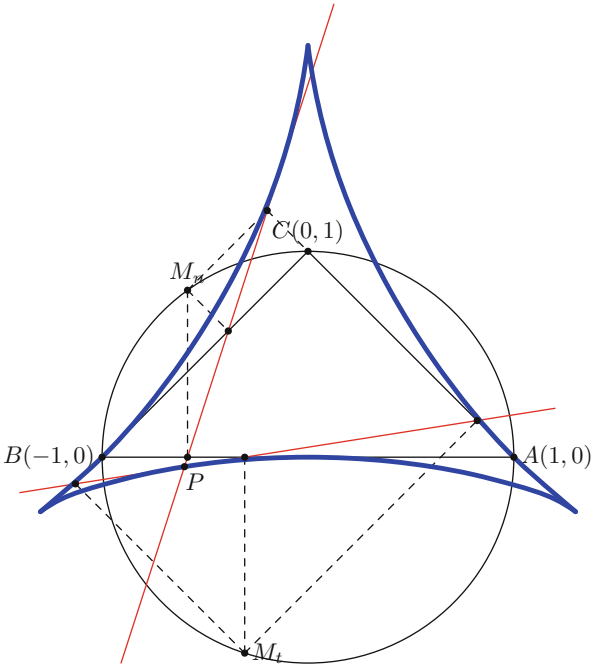


Fig. 8.7 The pair of family lines passing through a point P of the Deltoid

$P(x, y) = (-0.6, -0.4608841247)$. Using Eqs. (8.14), the tangent line corresponds to $M_t(x_0, y_0) = (-0.3074454515, -0.9515657073)$, and using Eqs. (8.15), the second line passing through $P(x, y)$ corresponds to $M_n = (x_0, y_0) = (-0.5851090972, 0.8109545882)$. Replacing in (8.12), the corresponding lines become:

$$F_t = -0.3074454515x + 1.951565707y - 0.0945227047,$$
$$F_n = -0.5851090972x + 0.1890454118y - 0.34356559,$$

which are plotted in Fig. 8.7. We have also added points A'_t, B'_t, C'_t and A'_n, B'_n, C'_n.

Example 8.16 Consider the set of ellipsoids centred at $(0, 0, 0)$ with semi-axes a, b, c of constant volume $V = \frac{4}{3}\pi abc$. To fix ideas, we take $C(a, b, c) = abc - 8$, fixing the value of the volume to $\frac{32}{3}\pi$. The equation of the family can be written

$$F(x, y, z, a, b, c) = b^2c^2x^2 + a^2c^2y^2 + b^2c^2z^2 - a^2b^2c^2.$$

Computing the envelope, we find a single "Normal" component equal to

$$E = 27x^2y^2z^2 - 64,$$

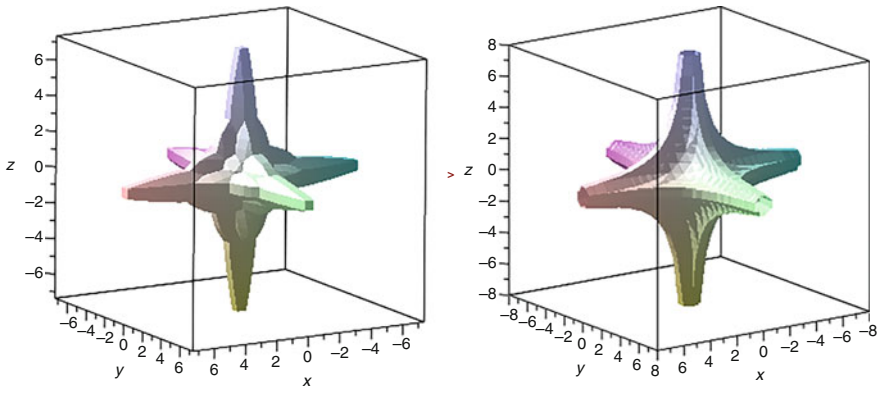


Fig. 8.8 Ellipsoids of constant volume and their envelope (Example 8.16)

with the associated tangent ellipsoid at point (x_0, y_0, z_0) being $a_0 = \sqrt{3}|x_0|$, $b_0 = \sqrt{3}|y_0|$, $c_0 = \sqrt{3}|z_0|$, which is also the unique ellipsoid of the family passing through this point. (See Fig. 8.8). (Verify this assertion using the algorithms.)

Example 8.17 The next example of a simple envelope contains a new type of component. Consider the offset at distance 1 of a cylinder with axis the z-axis and radius 1. The equations of the family and the constraint ideal are

$$F = (x - x_1)^2 + (y - y_1)^2 + (z - z_1)^2 - 1,$$
$$C = x_1^2 + y_1^2 - 1.$$

Using the algorithms we obtain

```
> LIB "grobcov.lib";
> ring R=(0,x,y,z),(x1,y1,z1),lp;
> poly F=(x-x1)^2+(y-y1)^2+(z-z1)^2-1;
> ideal C=(x1)^2+(y1)^2-1;
> short=0;
> def Env=envelop(F,C); Env;
[1]:
   [1]:
      _[1]=(x^2+y^2-4)
   [2]:
      [1]:
         _[1]=1
   [3]:
      [1]:
         2
      [2]:
         Normal
      [3]:
```

```
          _[1]=x1^2+y1^2-1
[2]:
   [1]:
      _[1]=(y)
      _[2]=(x)
   [2]:
      [1]:
         _[1]=1
   [3]:
      [1]:
         1
      [2]:
         Accumulation
> def E=Env[1][1];  E;
E[1]=(x^2+y^2-4)
> AssocTanToEnv(F,C,E);
[1]:
   [1]:
      _[1]=z1
      _[2]=y1
      _[3]=x1
   [2]:
      _[1]=z1+(-z)
      _[2]=2*y1+(-y)
      _[3]=2*x1+(-x)
   [3]:
      [1]:
         [1]:
            _[1]=(x^2+y^2-4)
         [2]:
            [1]:
               _[1]=1
> FamElemsAtEnvCompPoints(F,C,E);
[1]:
   [1]:
      _[1]=y1^2
      _[2]=x1
   [2]:
      _[1]=16*y1^2+(-4*y)*y1*z1^2+(8*y*z)*y1*z1
      +(-4*y*z^2-16*y)*y1+z1^4+(-4*z)*z1^3+(6*z^2+8)*z1^2
      +(-4*z^3-16*z)*z1+(4*y^2+z^4+8*z^2)
      _[2]=(2*y^2-8)*x1+(-2*x*y)*y1+(x)*z1^2
      +(-2*x*z)*z1+(x*z^2+4*x)
   [3]:
      [1]:
         [1]:
            _[1]=(x^2+y^2-4)
         [2]:
            [1]:
               _[1]=(y+2)
               _[2]=(x)
            [2]:
               _[1]=(y-2)
               _[2]=(x)
```

```
[2]:
   [1]:
      _[1]=y1
      _[2]=x1^2
   [2]:
      _[1]=(2*y)*y1-z1^2+(2*z)*z1+(-z^2-4)
      _[2]=16*x1^2+z1^4+(-4*z)*z1^3+(6*z^2+8)*z1^2
      +(-4*z^3-16*z)*z1+(z^4+8*z^2)
   [3]:
      [1]:
         [1]:
            _[1]=(y+2)
            _[2]=(x)
         [2]:
            [1]:
               _[1]=1
      [2]:
         [1]:
            _[1]=(y-2)
            _[2]=(x)
         [2]:
            [1]:
               _[1]=1
> def E1=Env[2][1];  E1;
E1[1]=(y)
E1[2]=(x)
> AssocTanToEnv(F,C,E1);
[1]:
   [1]:
      _[1]=z1
      _[2]=x1^2
   [2]:
      _[1]=z1+(-z)
      _[2]=x1^2+y1^2-1
   [3]:
      [1]:
         [1]:
            _[1]=(y)
            _[2]=(x)
         [2]:
            [1]:
               _[1]=1
```

The result has a very simple geometric interpretation and is represented in Fig. 8.9. There exist two components:

$$E_1 = x^2 + y^2 - 4 \text{ “Normal”},$$
$$E_2 = [x, y] \qquad \text{“Accumulation”}.$$

The “Normal” component represents the external cylinder, and the “Accumulation” component represents the cylinder axis.

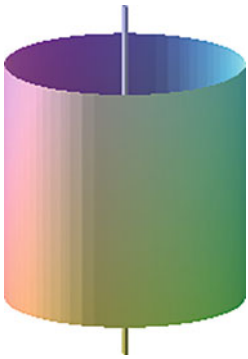


Fig. 8.9 Envelope of Example 8.17

For the "Normal" component, the associated tangent sphere corresponds to $(x_1, y_1, z_1) = (x_0/2, y_0/2, z_0)$. And following the technique used in Example 8.11, we can see that the equations $[z_1^2 - 2x_0x_1 - 2y_0y_1 - 2z_0z_1 + z^2 + 16 = 0, x_1^2 + y_1^2 - 4 = 0]$ determine, as a unique solution, the double solution corresponding to the associated tangent line.

The "Accumulation" component is a one dimensional line. As predicted in Theorem 8.7, there exit infinitely many surfaces of the family which are tangent to the component at a point $P(0, 0, z_0)$ of the axis. The associated tangent surfaces are all the family elements (spheres of radius 1) with center on the circle $z_1 - z_0, x_1^2 + y_1^2 - 1$.

8.5 New Examples

In this section we use a technique for obtaining families of hypersurfaces with at least n degrees of freedom. Let us start with a classical problem with the family $F(\mathbf{x}; \mathbf{y})$ and constraint ideal $C(\mathbf{y})$. Then replace the variables $\mathbf{x}$ in F with new parameter values $\mathbf{y}' = \mathbf{x}$ and add the equation $F(\mathbf{y}', \mathbf{y})$ to the constraints. The new constraint ideal will be $C_2 = C + \langle F(\mathbf{y}', \mathbf{y}) \rangle$. Then consider a new family F_2 formed, for example, by a set of spheres centred at the points $\mathbf{y}'$ and with its radius depending on $\mathbf{y}'$, i.e.,

$$F_2 = (x_1 - y_1')^2 + \cdots + (x_n - y_n')^2 - R(\mathbf{y}').$$

This will generate a family with more degrees of freedom for which the new algorithm ENVELOP will be necessary. If the initial family has s degrees of freedom, then the new family will have $s + n - 1$ degrees of freedom.

Let us give some examples.

Example 8.18 We start from Example 8.9, for which we had

$$F = (x - 2t)^2 + (y - t)^2 - t^2.$$

Using the technique described above, we can generate a new family F_2 taking $C_2 = F(x_1, y_1, t)$ and a new F_2, for example, to be formed by the set of circles of radius 1 centred at the points (x_1, y_1). We then have

$$F_2 = (x - x_1)^2 + (y - y_1)^2 - 1$$

depending on the parameters (x_1, y_1, t) that are constrained by

$$C_2 = (x_1 - 2t)^2 + (y_1 - t)^2 - t^2.$$

The new family represents the family of circles of radius 1 whose centers (x_1, y_1) lie on a circle of radius t centred on the point $(2t, t)$. (See Fig. 8.10.) The family depends on 3 parameters with one constraint, and so it has two degrees of freedom. The new family is not a classical case since $s = n = 2$. From now one, let us denote F_2 and C_2 simply as F and C.

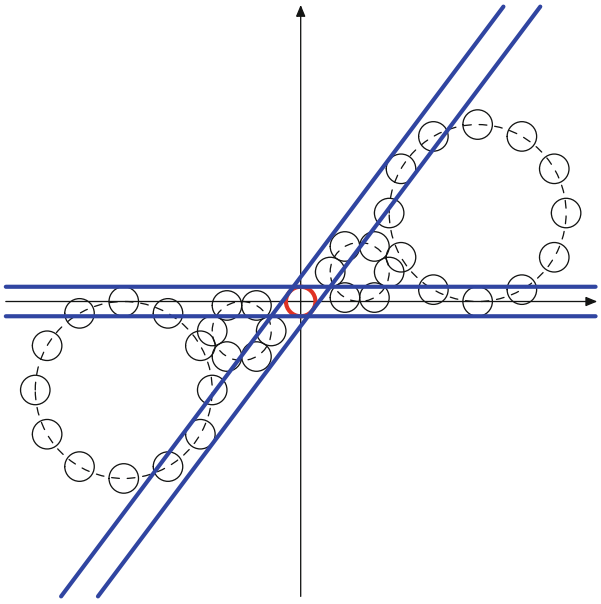


Fig. 8.10 Family of circles of Example 8.18 and its envelope

1. *Algebraic envelope* Following Theorem 8.6, to guarantee that the rank of the Jacobian matrix

$$J = \begin{pmatrix} \dfrac{\partial F}{\partial x_1} & \dfrac{\partial F}{\partial y_1} & \dfrac{\partial F}{\partial t} \\ \dfrac{\partial C}{\partial x_1} & \dfrac{\partial C}{\partial y_1} & \dfrac{\partial C}{\partial t} \end{pmatrix}$$

is less than 2, Algorithm ENVELOP will automatically incorporate into the system S the following three minors

$$J_1 = \begin{vmatrix} \dfrac{\partial F}{\partial x_1} & \dfrac{\partial F}{\partial y_1} \\ \dfrac{\partial C}{\partial x_1} & \dfrac{\partial C}{\partial y_1} \end{vmatrix} = 0,\ J_2 = \begin{vmatrix} \dfrac{\partial F}{\partial y_1} & \dfrac{\partial F}{\partial t} \\ \dfrac{\partial C}{\partial y_1} & \dfrac{\partial C}{\partial t} \end{vmatrix} = 0,\ J_3 = \begin{vmatrix} \dfrac{\partial F}{\partial x_1} & \dfrac{\partial F}{\partial t} \\ \dfrac{\partial C}{\partial x_1} & \dfrac{\partial C}{\partial t} \end{vmatrix} = 0.$$

to compute `locus`(S).

Substituting F_2 and C_2 into the Jacobians we have

$$\begin{aligned} J_1 &= 2ty_1 - tx_1 + (x - 2y)t - xy_1 + yx_1, \\ J_2 &= -4tx_1 + 4xt + y_1x_1 - xy_1 + 2x_1^2 - 2xx_1, \\ J_3 &= -4ty_1 + 4yt + y_1^2 + 2y_1x_1 - yy_1 - 2yx_1. \end{aligned}$$

Applying `envelop`, taking (x, y) as parameters and (t, x_1, y_1) as variables, will generate J_1, J_2, J_3 automatically. The result (the reader is invited to verify the computation) leads to the following five components

$$\begin{aligned} E_1 &= \mathbf{V}(4x - 3y - 5) \text{ “Normal”}, \\ E_2 &= \mathbf{V}(x^2 + y^2 - 1) \text{ “Special”}. \\ E_3 &= \mathbf{V}(4x - 3y + 5) \text{ “Normal”}, \\ E_4 &= \mathbf{V}(y - 1) \quad\quad\ \text{“Normal”}, \\ E_5 &= \mathbf{V}(y + 1) \quad\quad\ \text{“Normal”}. \end{aligned}$$

We obtain four “Normal” and one “Special” component. In Fig. 8.11 one can see some circles of the family, as well as the components (blue = “Normal” and red =“Special”) of the envelope.

The “Special” component represents a circle of the family F that is moreover, a solution to the envelope equations. It remains entirely in the interior of the real circle’s zone and has as real anti-image $(x_1 = 0, y_1 = 0)$.

Let P_i’s be the intersection between pairs of components.

In order to determine the associated tangent circles at the points of the envelope components, we have to apply `AssocTanToEnv` to each component. The result (to be verified by the reader) is that, except for the special points P_1 to P_8, the associated tangent parameters x_1, y_1, t_1 at point (x_0, y_0) of a components are

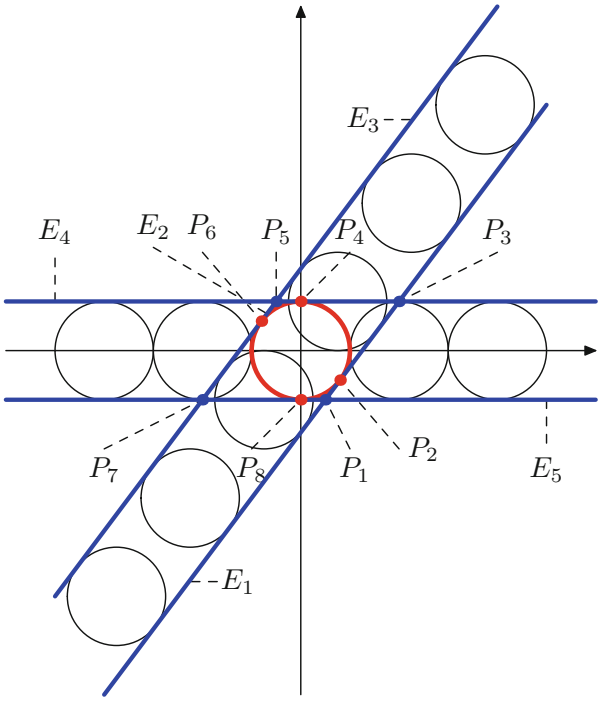


Fig. 8.11 Circles of Example 8.18 tangents to the envelope

Point	Intersection of components
$P_1 = (1/2, -1)$	E_1 and E_5
$P_2 = (4/5, -3/5)$	E_1 and E_2
$P_3 = (2, 1)$	E_4 and E_1
$P_4 = (0, 1)$	E_4 and E_2
$P_5 = (-1/2, 1)$	E_3 and E_4
$P_6 = (4/5, -3/5)$	E_3 and E_2
$P_7 = (-2, -1)$	E_5 and E_3
$P_8 = (0, -1)$	E_5 and E_2

The corresponding circles of the family being

$(x_0, y_0) \in E_i \setminus P_k'$s	Values of the associated tangent
$E_1 \setminus (P_1 \cup P_2 \cup P_3)$	$t_1 = (5y_0 + 3)/8, x_1 = (15y_0 + 9)/20, y_1 = y_0 + 3/5,$
$E_2 \setminus (P_2 \cup P_4 \cup P_6 \cup P_8)$	$t_1 = x_1 = y_1 = 0,$
$E_3 \setminus (P_5 \cup P_6 \cup P_7)$	$t_1 = (5y_0 - 3)/8, x_1 = (15y_0 - 9)/20, y_1 = y_0 - 3/5$
$E_4 \setminus (P_3 \cup P_4 \cup P_5)$	$t_1 = x_0/2, x_1 = x_0, y_1 = 0,$
$E_5 \setminus (P_7 \cup P_8 \cup P_1)$	$t_1 = x_0/2, x_1 = x_0, y_1 = 0.$

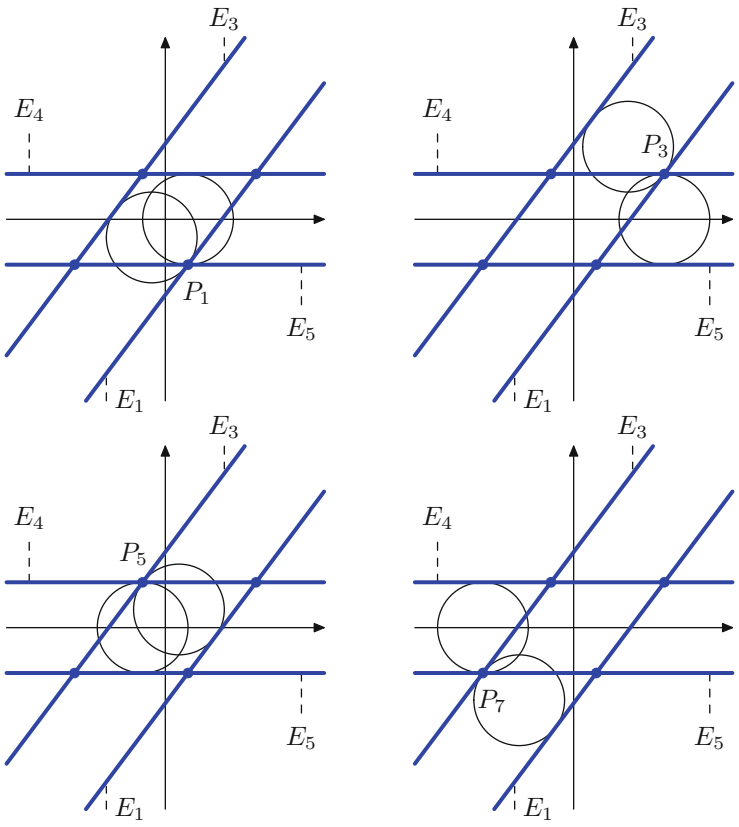


Fig. 8.12 Associated tangent circles at the special points

$E_1 \setminus (P_1 \cup P_2 \cup P_3)$,	$F^{(1)}_{(x_0,y_0)} = (x - x_0 + 4/5)^2 + (y - y_0 - 3/5)^2 - 1$,
$E_2 \setminus (P_2 \cup P_4 \cup P_6) \cup P_8$,	$F^{(2)}_{(x_0,y_0)} = x^2 + y^2 - 1$,
$E_3 \setminus (P_5 \cup P_6 \cup P_7)$,	$F^{(3)}_{(x_0,y_0)} = (x - x_0 - 4/5)^2 + (y - y_0 + 3/5)^2 - 1$,
$E_4 \setminus (P_3 \cup P_4 \cup P_5)$,	$F^{(4)}_{(x_0,y_0)} = (x - x_0/2)^2 + y^2 - 1$,
$E_5 \setminus (P_7 \cup P_8 \cup P_1)$,	$F^{(5)}_{(x_0,y_0)} = (x - x_0/2)^2 + y^2 - 1$.

2. *Special points* For the 8 special points P_i's, intersection of two components (see Fig. 8.12), we have to regard the bases of the second segment of `AssocTanToEnv` applied to E_1, E_2 and E_3.

The basis of the second segment of `AssocTanToEnv`(F, C, E_1) is

$$B_1 = \{(126y^2 - 46)x_1^2 + (-100y^2 - 120y - 36)x_1 + (45y^2 + 102y + 45),$$
$$(28y^2 - 12)y_1 + (-4y^2 + 20y + 16)x_1 + (-15y^2 - 34y - 15),$$
$$(91y^2 - 24y - 51)t + (-55y^2 + 20y + 35)x_1 + (-10y - 6)\}.$$

Specializing B_1 for $P_1 = (1/2, -1)$, the basis becomes

$$B_{11} = \{(10x_1 + 3)(2x_1 - 1), (4y_1 - 2x_1 + 1), (16t - 10x_1 + 1)\},$$

that leads to two solutions

$$(x_1 = -3/10, y_1 = -2/5, t = -1/4) \text{ and } (x_1 = 1/2, y_1 = 0, t = 1/4),$$

corresponding to the family curves

$$C_1 : (x + 3/10)^2 + (y + 2/5)^2 - 1 = 0,$$
$$C_2 : (x - 1/2)^2 + y^2 - 1 = 0.$$

C_1 is tangent to component E_1 and C_2 is tangent to E_5.

Specializing B_1 for $P_3 = (2, 1)$, the basis becomes

$$B_{13} = \{(5x_1 - 6)(x_1 - 2), y_1 + 2x_1 - 4, t - 1\},$$

leading to two solutions

$$(x_1 = 6/5, y_1 = 8/5, t = 1) \text{ and } (x_1 = 2, y_1 = 0, t = 1),$$

whose corresponding circles are

$$C_3 : (x - 6/5)^2 + (y - 8/5)^2 - 1 = 0,$$
$$C_4 : (x - 2)^2 + (y)^2 - 1 = 0.$$

C_3 is tangent to component E_1 and C_4 is tangent to E_4.

For points P_2, P_4, P_6 and P_8 we use the basis of the second segment of `AssocTanToEnv`(F, C, E_2). The result is always the unique circle E_2

$$C_0 : x^2 + y^2 - 1 = 0,$$

corresponding to the "Special" component. C_0 is tangent to E_1 at P_2, to E_3 at P_6, to E_4 at P_4 and to E_5 at P_8, and to itself on every point of E_2.

The basis of the second segment of `AssocTanToEnv`(F, C, E_3) is

$$B_3 = \{(12y^2 - 46)x_1^2 + (100y^2 - 120y + 36)x_1 + 45y^2 - 102y + 45,$$
$$(28y^2 - 12)y_1 + (-4y^2 - 20y + 16)x_1 + 15y^2 - 34y + 15,$$
$$(91y^2 + 24y - 51)t + (-55y^2 - 20y + 35)x_1 - 10y + 6\}.$$

Specializing B_3 for $P_5 = (-1/2, 1)$, the basis becomes

$$B_{35} = \{(4(2x_1 + 1))(10x_1 - 3), 16y_1 - 8x_1 - 4, 64t - 40x_1 - 4\},$$

leading to two solutions

$$(x_1 = -1/2, y_1 = 0, t = -1/4) \text{ and } (x_1 = 3/10, y_1 = 2/5, t = -1/4),$$

whose corresponding family circles are

$$C_5 : (x + 1/2)^2 + y^2 - 1 = 0,$$
$$C_6 : (x - 3/10)^2 + (y - 2/5)^2 - 1 = 0.$$

C_5 is tangent to component E_4 and C_6 is tangent to E_3.

Specializing B_3 for $P_7 = (-2, -1)$, the basis becomes

$$B_{37} = \{(16(x_1 + 2))(5x_1 + 6), 16y_1 + 32x_1 + 64, 16t + 16\},$$

leading to two solutions

$$(x_1 = -6/5, y_1 = -8/5, t = -1) \text{ and } (x_1 = -2, y_1 = 0, t = -1)$$

corresponding to the family curves

$$C_7 : (x + 6/5)^2 + (y + 8/5)^2 - 1 = 0,$$
$$C_8 : (x + 2)^2 + (y)^2 - 1 = 0.$$

C_7 is tangent to component E_3 and C_8 is tangent to E_5.

At the singular points of the envelope P_1, P_3, P_5, P_7 where two components intersect, there are two associated tangent circles, one associated to each component. For points P_2, P_4, P_6, P_8 in E_2 there is a single double solution.

The discussion up to here can be automatized. Nevertheless, it is not sufficient for obtaining the discussion over the reals. We will show, in this example, how to approach the real envelope.

3. *Parameters domain* To obtain real solutions from the constraint equation $C \equiv x_1^2 + y_1^2 + 4t^2 - 4tx_1 - 2ty_1 = 0$, the discriminant with respect to t must be non-negative. So we have

$$\Delta_1 = y_1(-3y_1 + 4x_1) \geq 0.$$

There are two solutions:

$$\begin{aligned} &(1) \quad y_1 \geq 0 \quad \text{and} \quad -3y_1 + 4x_1 \geq 0 \quad \text{and} \quad t \geq 0, \\ &(2) \quad y_1 \leq 0 \quad \text{and} \quad -3y_1 + 4x_1 \leq 0 \quad \text{and} \quad t \leq 0. \end{aligned}$$

representing the two regions of Fig. 8.13 with a common special point $x_1 = y_1 = t = 0$ where the parameters are limited.

This suggests a nice division of the family F into two subfamilies (see Fig. 8.13), namely:

$$\begin{aligned} &(1)\ F_1 : F \text{ restricted by } C \text{ and } t \geq 0 \\ &(2)\ F_2 : F \text{ restricted by } C \text{ and } t \leq 0. \end{aligned}$$

Considering the geometry of the problem, the envelope of family F_1 is generated by the circles of radius 1 running on the semi-line $4x_1 - 3y_1 = 0$ from $x_1 = y_1 = 0$ for $x_1 \geq 0$ (and also $y_1 \geq 0$), and the circles of radius 1 running on the semi-line $y_1 = 0$ from $x_1 = y_1 = 0$ for $x_1 \geq 0$. There is a common circle of both parts, namely the circle $x^2 + y^2 - 1 = 0$ corresponding to the "Special" component associated to the point $(x_1 = y_1 = 0)$. The envelope of the first semi-line will be the semi-line parallel to $4x_1 - 3y_1 = 0$ at distance 1 on the semi-plane $4x_1 - 3y_1 \leq 0$, which is $4x_1 - 3y_1 + 5 = 0$ for $x \geq -3/5$ and $y \geq 4/5$. The envelope of the second semi-line will be the semi-line parallel to $y = 0$ for $y \leq 0$ at distance 1, which is $y + 1 = 0$ for $x \geq 0$. Both semi-lines connect with the arc of the circle $x^2 + y^2 - 1$ corresponding to $x_1 = 0$, $y_1 = 0$ between the points P_2 and P_4.

The envelope of family F_2 is symmetric to the envelope of family F_1 with respect to the origin.

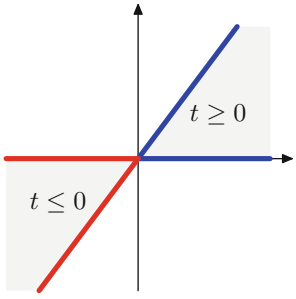


Fig. 8.13 Domains of the parameters of Example 8.18

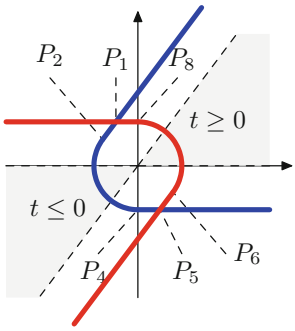


Fig. 8.14 Real envelopes of both families of Example 8.18

Summarizing, we obtain two families and two continuous and differentiable real envelopes:

(1) Family $F_1 = (F, \text{ for } t \geq 0)$

$E_3 : 4x - 3y + 5 = 0$ for $(x \geq -4/5,\ y \geq 3/5)$ till P_2

$E_2 : x^2 + y^2 - 1 = 0$ from P_2 till P_4 running clockwise

$E_4 : y - 1 = 0$ for $x \geq 0$ starting at P_2.

(2) Family $F_2 = (F, \text{ for } t \leq 0)$

$E_5 : y + 1 = 0$ for $x \leq 0$ starting at P_1

$E_2 : x^2 + y^2 - 1 = 0$ from P_1 till P_8 running anticlockwise

$E_1 : 4x - 3y - 5 = 0$ for $(x \leq 4/5\ y \leq -3/5)$ till P_6.

Both families are represented in Fig. 8.14.

4. *Circles intersecting the envelope* Let us come back to the family F and obtain the results of point 4 using algebraic methods. In point 4 we made somewhat excessive use of intuition in building the envelope of families F_1 and F_2. We can certify the correctness of the results in point 4, as well as determining the true real envelope of F, using the `FamElemsAtEnvCompPoints` command.

We will study the component $E_3 = 4x - 3y + 5$ and leave the study of the remaining components as an exercise. The library also contains a command `discrim` for factoring discriminant equations of degree two, because, as we have seen, this can be useful for determining real solutions. To find out the curves of family F passing through E_1, after the computation of E_1, we call

```
> ring R1=(0,x,y),(t,x1,y1),lp;
> short=0;
> poly F=(x-x1)^2+(y-y1)^2-1;
> ideal C=(x1-2t)^2+(y1-t)^2-t^2;
> poly E3=4*x-3*y+5;
> discrim(C[1],t);
```

```
[1]:
   _[1]=4
   _[2]=4*x1-3*y1
   _[3]=y1
[2]:
   1,1,1
> def FE1=FamElemsAtEnvCompPoints(F,C,E3);  FE1;
[1]:
   [1]:
      _[1]=x1^2
      _[2]=t^2
   [2]:
      _[1]=16*x1^2+(-24*y+40)*x1+16*y1^2+(-32*y)*y1
      +(25*y^2-30*y+9)
      _[2]=64*t^2-64*t*x1-32*t*y1+(24*y-40)*x1+(32*y)*y1
      +(-25*y^2+30*y-9)
   [3]:
      [1]:
         [1]:
            _[1]=(4*x-3*y+5)
         [2]:
            [1]:
               _[1]=1
> def H=FE1[1][2][1];
> discrim(H,y);
[1]:
   _[1]=-64
   _[2]=-4*x1+3*y1
   _[3]=-4*x1+3*y1-10
[2]:
   1,1,1
```

So we obtain two discriminants, namely[2]

$$\Delta_1 = \text{discriminant}(C, t) = (4x_1 - 3y_1)y_1,$$
$$\Delta_2 = \text{discriminant}(H, y) = -(4x_1 - 3y_1)(4x_1 - 3y_1 + 10).$$

We have already discussed the discriminant of C (point 4), which delimits the values of (x_1, y_1) and t in the regions:

$$R_1 = \{t \geq 0, y_1 \geq 0, 4x_1 - 3y_1 \geq 0\},$$
$$R_2 = \{t \leq 0, y_1 \leq 0, 4x_1 - 3y_1 \leq 0\}.$$

Even if there are complex circles at each point of the component E_1, there will exist a real circle of the family F with parameters (x_1, y_1, t) passing through a point $P(x_0, y_0) \in E_1$ only if $\Delta_1 \geq 0$ and $\Delta_2 \geq 0$.

[2]It is important to choose the order of the ring variables adequately in the definition of the ring in order to obtain this interesting result. We tried other alternatives and the result was not obvious.

Consider first the circles for which $y_1 \geq 0$. In order to have real solutions we need $(4x_1 - 3y_1) \geq 0$ and $(4x_1 - 3y_1)(4x_1 - 3y_1 + 10) \leq 0$. Both inequalities are only compatible if $4x_1 - 3y_1 = 0$, and then both discriminants become 0. From the equation of $C = 4t^2 - (4x_1 + 2y_1)t + x_1^2 + y_1^2 = 0$ we find that $t = \dfrac{2x_1 + y_1}{4}$ and using $4x_1 - 3y_1 = 0$ results in $t = \dfrac{5y_1}{8} = \dfrac{5x_1}{6}$. We then have $y_1 \geq 0$, $y_1 = \dfrac{8t}{5}$ and $x_1 = \dfrac{6t}{5}$, and there exists a unique double solution. Substituting the value $x_1 = \dfrac{4y_1}{3}$ in the equation H, and solving with respect to y_1, we obtain $y_1 = y_0 - \dfrac{3}{5}$ and thus $x_1 = x_0 + \dfrac{4}{5}$, which corresponds to the unique associated tangent circle at point $(x_0, y_0) \in E_3$.

So for the family F_1 that restricts the values of $t \geq 0$, at all points $(x_0, y_0) \in E_3$ generated by the points in the line $4x - 3y = 0$, for $x > 0$ and $y > 0$, there exists only one circle of the family passing through them which is the associated tangent circle, corresponding to $y_1 = y_0 - \dfrac{3}{5}$, $x_1 = x_0 + \dfrac{4}{5}$. Repeating the computations for E_4, will completely justify the assertions in point 4.

Consider now the circles for which $y_1 \leq 0$. In order to have real solutions we need $(4x_1 - 3y_1) \leq 0$ and $(4x_1 - 3y_1)(4x_1 - 3y_1 + 10) \geq 0$. This leads to the constraint

$$0 \leq -4x_1 + 3y_1 \leq 10. \tag{8.16}$$

So there are infinitely many intersecting circles for $y_1 < 0$. Thus, the points $(x_0, y_0) \in E_1$ for which there exist solutions of the inequalities (8.16) do not form part of the true real envelope of F. Let us determine them. From the inequalities (8.16), setting $4x_1 + 3y_1 = a$ (for $0 \leq a \leq 10$), substituting $x_1 = \dfrac{3y_1 - a}{4}$ in H, and solving with respect to y, we obtain two solutions for each value of a, namely:

$$y_1 = y_0 - 3/5 + (3/25)a \pm (4/25)\sqrt{(-a + 10)a}.$$

We search for the smallest value of y_1 in the above expression. If it is greater than 0, then no solution verifies both inequalities in (8.16), and at the point (x_0, y_0) only the associated tangent circle passes through it, and so it is part of the true envelope. At the remaining points, there are infinitely many real circles passing through them and so they do not form part of the true envelope. Let us solve the problem. We have to find the smallest value of y_1 depending on a. From the two solutions, the smallest will be

$$y_1 = y_0 - 3/5 + (3/25)a - (4/25)\sqrt{(-a + 10)a}. \tag{8.17}$$

Differentiating with respect to a, we have

$$y_1' = 3/25 - (2/25)\frac{-2a + 10}{\sqrt{-a^2 + 10a}}.$$

Now solving with respect to a, we obtain $a = 2$. And replacing in formula (8.17), we obtain for the minimal value of y_1

$$\min(y_1) = y_0 - 1.$$

Thus if $y_0 > 1$, the minimal value of y_1 is positive and the point belongs to the true envelope. The limit point is thus $P_1 = (-1/2, 1)$.

That is, the semi-line of E_1 starting from P_1 for $y \geq 1, x \geq -1/2$, is part of the true real envelope.

We can proceed similarly for all other "Normal" components, and we leave the computation as an exercise. We obtain the following results.

6. *The true real envelope* E_1 and E_5 intersect at P_1, and E_3 and E_4 intersect at P_5. Repeating the computations for all the components, we prove that the points of the semi-line of E_5 starting from P_8 for $x \geq 0$ are in the true real envelope. The same applies for the points of E_4 starting from P_4 for $x \geq 0$, and for the points of E_3 starting from P_6 for $x \leq 4/5$ and $y \leq -3/5$ and for the points of E_1 for $(x \leq 1/2, y \leq -1)$.

So we can assert that, as the geometry of the problem shows, the true "envelope" consists of

$$\begin{cases} 4x - 3y + 5 = 0 \text{ for } x \geq -1/2, \ y \geq 1, \\ y - 1 = 0 \qquad \text{for } x \leq -1/2, \\ y + 1 = 0 \qquad \text{for } x \geq 1/2, \\ 4x - 3y - 5 = 0 \text{ for } x \leq 1/2, \quad y \leq -1. \end{cases} \tag{8.18}$$

Equations (8.18) represent the true real envelope. We realize that the necessary conditions determined by S are not sufficient to find the true points of what we would desire to obtain.

The problem is that the procedure described here for this particular example cannot be automatically generalized for other problems. So we must accept that we are only able to solve the first part of the problem automatically, and we must try to solve by hand, using also the algorithms, which points of this envelope form the true envelope.

We have also realized that, for some problems, it can be useful to divide the initial family into subfamilies considering different regions of the parameter values, in order to obtain more geometrically accurate real envelopes.

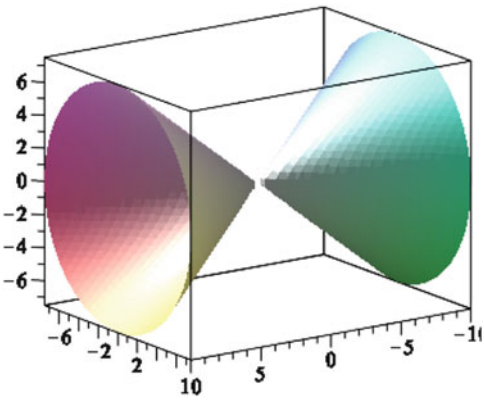


Fig. 8.15 Envelope E of family F in Example 8.19

Example 8.19 Consider now a problem similar to Example 8.10 but in $3d$. Let F be the family of spheres centred at point $(5t, 0, 0)$ and radius $3t$

$$F(x, y, z; t) = (x - 5t)^2 + y^2 + z^2 - 9t^2.$$

Applying the same techniques, we obtain a "Normal" component

$$E = 9x^2 - 16y^2 - 16z^2,$$

representing a cone (See Fig. 8.15). The associated tangent sphere of the family at $(x_0, y_0, z_0) \in \mathbf{V}(E)$ corresponds to $t_0 = \frac{5}{16}x_0$ and is thus

$$F_{t_0} : \left(x - \frac{25}{16}x_0\right)^2 + y^2 + z^2 - 9\left(\frac{5}{16}x_0\right)^2 = 0.$$

Using `FamElemsAtEnvCompPoints` we can also prove that, through each point of E, there only passes a single sphere of the family (double solution of the associated tangent sphere).

Let us now apply the same technique used in Example 8.18 to obtain a new family with more degrees of freedom. For the new family, we have new parameters (x_1, y_1, z_1) and we add $F(x_1, y_1, z_1, t)$ to the constraint ideal C (which in this example is empty). Then consider as a new family the set of spheres centred at the points (x_1, y_1, z_1) with radius 1. The new family will have $1 + 3 - 1 = 3$ degrees

of freedom. We have

$$C_2 = \langle (x_1 - 5t)^2 + y_1^2 + z_1^2 - 9t^2 \rangle,$$
$$F_2 = (x - x_1)^2 + (y - y_1)^2 + (z - z_1)^2 - 1.$$

The Jacobian is a 4×2 matrix. Thus, the algorithm will add 6 determinants 2×2 denoted J_c, and then compute the $\texttt{locus}(C_2 + \langle F_2 \rangle + \langle J_c \rangle)$.

Let's apply the algorithms:

```
> LIB "grobcov.lib";
> ring R=(0,x,y,z),(x1,y1,z1,t),lp;
> short=0;
> poly F=(x-x1)^2+(y-y1)^2+(z-z1)^2-1;
> ideal C=(x1-5*t)^2+y1^2+z1^2-9*t^2;
> def Env=envelop(F,C,"anti-image",0); Env;
[1]:
   [1]:
      _[1]=(9*x^2+30*x-16*y^2-16*z^2+25)
   [2]:
      [1]:
         _[1]=(y^2+z^2)
         _[2]=(3*x+5)
   [3]:
      [1]:
         2
      [2]:
         Normal
      [3]:
         _[1]=0
[2]:
   [1]:
      _[1]=(9*x^2-30*x-16*y^2-16*z^2+25)
   [2]:
      [1]:
         _[1]=(y^2+z^2)
         _[2]=(3*x-5)
   [3]:
      [1]:
         2
      [2]:
         Normal
      [3]:
         _[1]=0
[3]:
   [1]:
      _[1]=(x^2+y^2+z^2-1)
   [2]:
      [1]:
         _[1]=1
   [3]:
      [1]:
         0
```

```
      [2]:
         Special
      [3]:
         _[1]=4*t^3+t
         _[2]=25*y1^2*t+25*z1^2*t-16*t
         _[3]=25*y1^4+50*y1^2*z1^2-16*y1^2+25*z1^4-16*z1^2
[4]:
   [1]:
      _[1]=(z)
      _[2]=(y)
      _[3]=(3*x+5)
   [2]:
      [1]:
         _[1]=1
   [3]:
      [1]:
         0
      [2]:
         Accumulation
[5]:
   [1]:
      _[1]=(z)
      _[2]=(y)
      _[3]=(3*x-5)
   [2]:
      [1]:
         _[1]=1
   [3]:
      [1]:
         0
      [2]:
         Accumulation
> def E=Env[1][1]; E;
E[1]=(9*x^2+30*x-16*y^2-16*z^2+25)
> def A=AssocTanToEnv(F,C,E); A;
[1]:
   [1]:
      _[1]=t
      _[2]=z1
      _[3]=y1
      _[4]=x1
   [2]:
      _[1]=16*t+(-5*x-3)
      _[2]=(5*y^2+5*z^2)*z1+(3*x*z-5*y^2*z-5*z^3+5*z)
      _[3]=(5*y^2+5*z^2)*y1+(3*x*y-5*y^3-5*y*z^2+5*y)
      _[4]=5*x1+(-5*x-3)
   [3]:
      [1]:
         [1]:
            _[1]=(9*x^2+30*x-16*y^2-16*z^2+25)
         [2]:
            [1]:
               _[1]=(y^2+z^2)
               _[2]=(3*x+5)
```

```
            [2]:
               _[1]=(16*y^2+16*z^2-25)
               _[2]=(x)
            [3]:
               _[1]=(25*y^2+25*z^2-16)
               _[2]=(5*x+3)
[2]:
   [1]:
      _[1]=t^2
      _[2]=z1
      _[3]=y1
      _[4]=x1
   [2]:
      _[1]=(6416*y^2+6416*z^2-4121)*t^2+(-225*y^2-225*z^2+144)
      _[2]=(641*y^2+641*z^2-425)*z1+(-48*y^2*z-48*z^3+75*z)*t
      +(-225*y^2*z-225*z^3+144*z)
      _[3]=(641*y^2+641*z^2-425)*y1+(-48*y^3-48*y*z^2+75*y)*t
      +(-225*y^3-225*y*z^2+144*y)
      _[4]=5*x1-16*t
   [3]:
      [1]:
         [1]:
            _[1]=(16*y^2+16*z^2-25)
            _[2]=(x)
         [2]:
            [1]:
               _[1]=1
      [2]:
         [1]:
            _[1]=(25*y^2+25*z^2-16)
            _[2]=(5*x+3)
         [2]:
            [1]:
               _[1]=1
[3]:
   [1]:
      _[1]=1
   [2]:
      _[1]=1
   [3]:
      [1]:
         [1]:
            _[1]=(y^2+z^2)
            _[2]=(3*x+5)
         [2]:
            [1]:
               _[1]=(z)
               _[2]=(y)
               _[3]=(3*x+5)
[4]:
   [1]:
      _[1]=t
      _[2]=y1^2
      _[3]=x1
```

```
    [2]:
       _[1]=3*t+1
       _[2]=25*y1^2+25*z1^2-16
       _[3]=15*x1+16
    [3]:
       [1]:
          [1]:
             _[1]=(z)
             _[2]=(y)
             _[3]=(3*x+5)
          [2]:
             [1]:
                _[1]=1
>  A[1][2][1];
  16*t+(-5*x-3)
```

From the computation of `envelop`, we obtain two "Normal" components, one "Special", and two "Accumulation" points. The "Normal" components are represented in Fig. 8.16. The result is geometrically easy to interpret: The "Normal" components are two cones corresponding to a kind of offset of the envelope of the initial example, the "Accumulation" points are the vertices of the cones, and the "Special" component is a sphere in the center. Let us analyze the details.

In the computation we selected for a discussion the first "Normal" component

$$E_1 = 9x^2 + 30x - 16y^2 - 16z^2 + 25 = (3x + 5)^2 - 16(y^2 + z^2).$$

From the computation of `AssocTanToEnv`, and taking into account that $E(x_0, y_0, z_0) = 0$, we obtain that the associated tangent sphere at a point

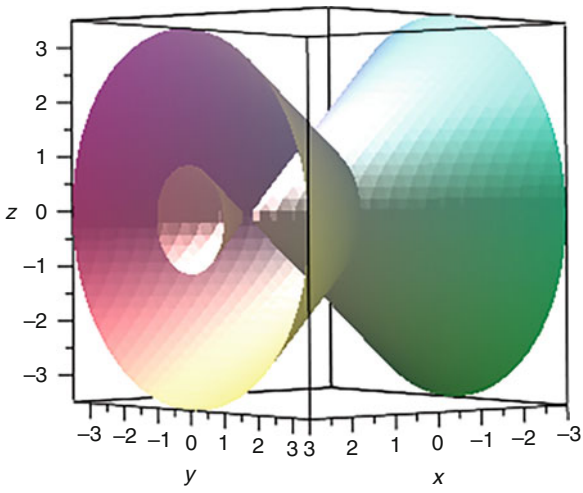


Fig. 8.16 Envelope of family F_2 in Example 8.19

$(x_0, y_0, z_0) \in \mathbf{V}(E)$ corresponds to the parameter values

$$t = \frac{5x_0 + 3}{16}, \qquad x_1 = x_0 + \frac{3}{5},$$
$$y_1 = y_0\left(1 - \frac{16}{5(3x_0 + 5)}\right), \; z_1 = z_0\left(1 - \frac{16}{5(3x_0 + 5)}\right)$$

except for the following special point and lines

$$P_1 = \left(-\frac{5}{3}, 0, 0\right),$$
$$\ell_1 = \{x = 0, 16(y^2 + z^2) - 16 = 0\},$$
$$\ell_2 = \{x = -\frac{3}{5}, 25(y^2 + z^2) - 16 = 0\}.$$

P_1 is the vertex of the cone, ℓ_1 is the intersection with the cone E_2, and ℓ_2 is the intersection with the "Special" sphere component E_3.

8.6 Open Problems

In Example 8.18, using DISCRIM we where able to determine on which points of the envelop component the FAMELEMATENVCOMPPOINTS have a unique real solution and on which points there are infinitely many ones.

In difficult problems and in 3d or higher dimension, it is not evident how to prove the results. We can approach the solution guided by the images and using numerical computations on level curves.

For our Example 8.19, we leave as a challenging problem to prove that for $x \geq -\frac{3}{5}$ there exists a unique real sphere of the family passing through a point of E_1 that corresponds to the associated tangent one, whereas there exist infinitely many real spheres for $x \leq -\frac{3}{5}$.

As in Example 8.18, this suggests dividing family F_2 into two families, namely

$$F_{21} = \{F_2 \text{ for } t \geq 0\},$$
$$F_{22} = \{F_2 \text{ for } t \leq 0\}.$$

Figure 8.17 represents the envelope of family F_{21}, namely

$$\text{for } -1 \leq x \leq -\frac{3}{5}, \; E_3 = x^2 + y^2 + z^2 - 1,$$
$$\text{for } x \geq -\frac{3}{5}, \qquad E_1 = (3x - 5)^2 - 16(y^2 + z^2).$$

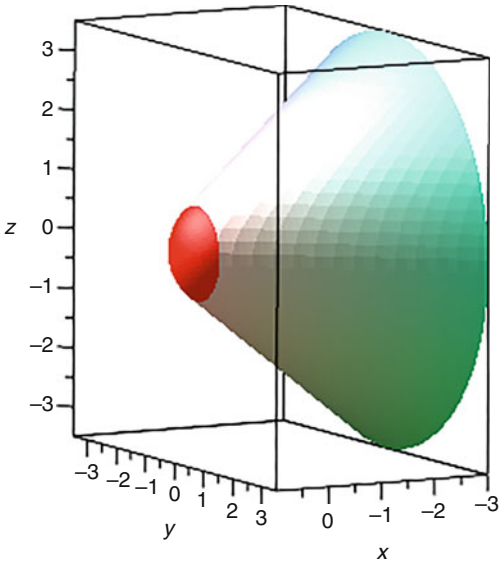


Fig. 8.17 Envelope of family F_{21}

Family F_{22} is symmetric to F_{21} for $x \to -x$. It is not evident how to prove the above result using only algebraic methods.

We can further consider a new embedded family adding the family F_2, substituting the point (x, y, z) by (x_2, y_2, z_2), to the constraints C_2, and considering a new F_3, for example spheres centred at (x_2, y_2, z_2) with fixed radius 1. We will have

$$\begin{aligned} C_3 =& (x_1 - 5t)^2 + y_1^2 + z_1^2 - t^2, \\ & (x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2 - 1; \\ F_3 =& (x - x_2)^2 + (y - y_2)^2 + (z - z_2)^2 - 1. \end{aligned}$$

The new family has $7-2 = 5$ degrees of freedom, becoming a very hard problem, and having no particular interest. This method allows, in principle, for obtaining new embedded families with $7, 9, \ldots$ degrees of freedom.

The study is left as a new problem.

Example 8.20 Consider the family of circles of radius $\sqrt{t-s}$ centred on the points of a circle of center $(2t, s)$ and radius $\sqrt{s}$ for all possible values of s and $t \geq s \geq 0$. The equation of the family and the ideal of constraints are

$$\begin{aligned} F &= (x - x_1)^2 + (y - y_1)^2 - (t - s), \\ g &= (x_1 - 2t)^2 + (y_1 - s)^2 - s. \end{aligned}$$

This problem has three degrees of freedom (4 parameters and 1 constraint), and not a single one as in usual two dimensional envelope problems. The `envelop`

algorithm will automatically incorporate to the system the six 2×2 minors of the Jacobian. Applying `envelop`(F, g), we have

```
> LIB "grobcov.lib";
> ring R=(0,x,y),(x1,y1,s,t),lp;
> short=0;
> poly F=(x-x1)^2+(y-y1)^2-(t-s);
> ideal g=(x1-2*t)^2+(y1-s)^2-s;
> def En=envelop(F,g,"anti-image",0);  En;
[1]:
   [1]:
      _[1]=(512*x^3-1024*x^2*y^2-2560*x^2*y-640*x^2+4096*x*y^3
         +4864*x*y^2-704*x*y+984*x-4096*y^4+1536*y^3-16*y^2
         +144*y+289)
   [2]:
      [1]:
         _[1]=1
   [3]:
      [1]:
         1
      [2]:
         Normal
      [3]:
         _[1]=256*s^4-512*s^3*t+32*s^3+256*s^2*t^2+96*s^2*t
         +65*s^2-128*s*t^2-56*s*t+16*t^2
> poly E=En[1][1][1]; E;
       (512*x^3-1024*x^2*y^2-2560*x^2*y-640*x^2+4096*x*y^3
       +4864*x*y^2-704*x*y+984*x-4096*y^4+1536*y^3-16*y^2
       +144*y+289)
```

There exist a single "Normal" envelope component

$$E = 512x^3 - 1024x^2y^2 - 2560x^2y - 640x^2 + 4096xy^3 + 4864xy^2 - 704xy + 984x - 4096y^4 + 1536y^3 - 16y^2 + 144y + 289.$$

The set of equations $\{E, \frac{\partial E}{\partial x}, \frac{\partial E}{\partial y}\}$ has a unique solution, and so the envelope $E = 0$ has a unique singular point $(x_s, y_s) = (4.231421832, 1.057913009)$, that is a cusp singular point.

Figure 8.18 shows the envelope component. Even if it is described by an irreducible polynomial, the graphic shows that it has two parts. We want to delimit both parts, that we denote E_{int} and E_{ext}. The singular point is the vertex of E_{int}.

Now, the problem to be solved is determining which circles of the family pass at a fixed point $P(x, y)$ of the plane. With this objective we consider the ideal formed by $[F, g]$ and determine its "`grobcov`". From it we can determine the dimension of the set of lpp (which is equal to the dimension of the basis), as well as a discriminant of the first polynomial in the basis with respect to y_1, which is of degree 2 in y_1.

```
> ideal H=g;
> H[size(H)+1]=F;
> def Gr=grobcov(H); Gr;
  [1]:
```

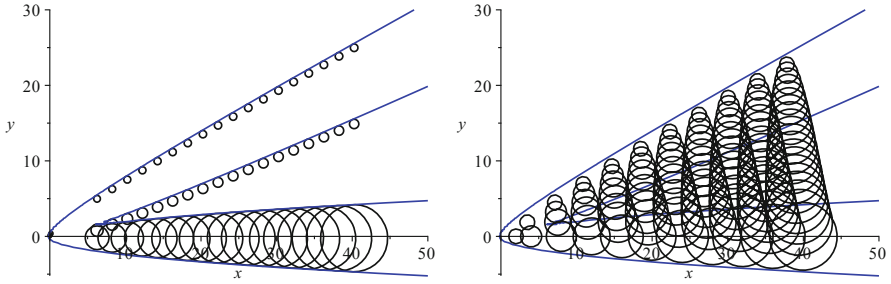


Fig. 8.18 Example 8.20. Envelope, associate tangent circles, and some family circles

```
      [1]:
         _[1]=y1^2*s^2
         _[2]=x1*t
         _[3]=x1*y1*s
         _[4]=x1^2
      [2]:
         _[1]=4*y1^2*s^2+(-8*y)*y1^2*s+16*y1^2*t^2+  (-16*x)*y1^2*t+(4*x^2
         +4*y^2)*y1^2-4*y1*s^3+(4*y+8)*y1*s^2-16*y1*s*t^2+(16*x-4)*y1*s*t
         +(-4*x^2+4*y^2-8*y)*y1*s+(-16*y)*y1*t^2+(16*x*y+4*y)*y1*t
         +(-4*x^2*y-4*y^3)*y1+s^4-4*s^3+8*s^2*t^2+(-8*x+2)*s^2*t
         +(2*x^2-2*y^2+4)*s^2-4*s*t+(4*y^2)*s+16*t^4+(-32*x-8)*t^3
         +(24*x^2+8*x+8*y^2+1)*t^2+(-8*x^3-2*x^2-8*x*y^2-2*y^2)*t
         +(x^4+2*x^2*y^2+y^4)
       _[2]=4*x1*t+(-2*x)*x1+2*y1*s+(-2*y)*y1-s^2+2*s-4*t^2-t+(x^2+y^2)
       _[3]=8*x1*y1*s+(-8*y)*x1*y1-4*x1*s^2+8*x1*s+(-2*x+4*y^2)*x1
        -16*y1^2*t+(8*x)*y1^2+8*y1*s*t+(-12*x+2)*y1*s+(24*y)*y1*t
        +(-4*x*y-2*y)*y1-4*s^2*t+(6*x-1)*s^2-8*s*t+(-4*x+2)*s-16*t^3
        +(24*x+8)*t^2+(-12*x^2-2*x-12*y^2-1)*t+(2*x^3+x^2+2*x*y^2+y^2)
       _[4]=x1^2+(-2*x)*x1+y1^2+(-2*y)*y1+s-t+(x^2+y^2)
   [3]:
      [1]:
         [1]:
            _[1]=0
         [2]:
            [1]:
               _[1]=1
> def lpp=Gr[1][1];
> def B=Gr[1][2];
> dim(lpp);
  // ** _ is no standard basis
  2
> discrim(B[1],y1);
  [1]:
     _[1]=-16
     _[2]=-2*t+(x)
     _[3]=s^4+(-4*y)*s^3+8*s^2*t^2+(-8*x-2)*s^2*t+(2*x^2+6*y^2+4)*s^2
     +(-16*y)*s*t^2+(16*x*y+4*y-4)*s*t+(-4*x^2*y-4*y^3)*s+16*t^4
     +(-32*x-8)*t^3+(24*x^2+8*x+8*y^2+1)*t^2
     +(-8*x^3-2*x^2-8*x*y^2-2*y^2)*t+(x^4+2*x^2*y^2+y^4)
[2]:
   1,2,1
```

We see that for every point $P(x, y)$ the system provides a two dimensional solution set. The question is, for which points P the solution is real. We obtain a discriminant

$$\begin{aligned}\Delta_0 =& s^4 + (-4y)s^3 + 8s^2t^2 + (-8x-2)s^2t + (2x^2+6y^2+4)s^2 \\ &+ (-16y)st^2 + (16xy+4y-4)st + (-4x^2y-4y^3)s + 16t^4 \\ &+ (-32x-8)t^3 + (24x^2+8x+8y^2+1)t^2 \\ &+ (-8x^3-2x^2-8xy^2-2y^2)t + (x^4+2x^2y^2+y^4)\end{aligned}$$

telling us that for the values of (x, y, s, t) for which $\Delta_0 \geq 0$, the solutions can be real, but if $\Delta_0 < 0$ the solutions are complex. But this is not sufficient to delimit the region with points $P(x, y)$ for which they pass real circles of the family.

We proceed first determining the associated tangent circles to the envelope.

```
> def A=AssocTanToEnv(F,g,E); A;
[1]:
   [1]:
      _[1]=x1
      _[2]=y1
      _[3]=s
      _[4]=t
   [2]:
      _[1]=4*x1+(-4*x-1)
      _[2]=(1024*y^3-768*y^2+192*y-556)*y1+(192*x^2+128*x*y^2
      -832*x*y-136*x-1024*y^4+1024*y^3+272*y^2+48*y-29)
      _[3]=(5120*y^3-3840*y^2+960*y-2780)*s+(1024*x^2*y
       +704*x^2-2048*x*y^3-1920*x*y^2-4288*x*y+984*x-1024*y^4
       +4352*y^3-816*y^2-880*y+357)
      _[4]=(10240*y^3-7680*y^2+1920*y-5560)*t+
         (-512*x^2*y+128*x^2-4096*x*y^3+5120*x*y^2-896*x*y
       +1948*x-2048*y^4-2176*y^3+3008*y^2+80*y+1139)
   [3]:
      [1]:
         [1]:
            _[1]=(512*x^3-1024*x^2*y^2-2560*x^2*y-640*x^2
              +4096*x*y^3+4864*x*y^2-704*x*y+984*x-4096*y^4
              +1536*y^3-16*y^2+144*y+289)
         [2]:
            [1]:
               _[1]=(256*y^3-192*y^2+48*y-139)
               _[2]=(24*x-64*y^2-16*y-13)
[2]:
   [1]:
      _[1]=x1
      _[2]=y1^2
      _[3]=s
      _[4]=t
   [2]:
```

```
    _[1]=24*x1+(-64*y^2-16*y-19)
    _[2]=9*y1^2+(-6*y+6)*y1+(y^2-2*y+1)
    _[3]=90*s+(-96*y-66)*y1+(-32*y^2-8*y-41)
    _[4]=720*t+(192*y-48)*y1+(-896*y^2-224*y-293)
[3]:
   [1]:
      [1]:
         _[1]=(256*y^3-192*y^2+48*y-139)
         _[2]=(24*x-64*y^2-16*y-13)
      [2]:
         [1]:
            _[1]=1
```

Isolating the variables (x_1, y_1, s, t) from segment [1] of the output (subsection [2] for the lpp and [3] for the equations), the associated tangent circle at a point $(x, y) \in \mathbf{V}(E)$ is given by

$$
\begin{aligned}
x_1 =& x + 1/4,\\
y_1 =& -\frac{192x^2 + 128xy^2 - 832xy - 136x - 1024y^4 + 1024y^3}{1024y^3 - 768y^2 + 192y - 556}\\
&-\frac{272y^2 + 48y - 29 + 1024y^3 + 272y^2 + 48y - 29}{1024y^3 - 768y^2 + 192y - 556},\\
s =& -\frac{1024x^2y + 704x^2 - 2048xy^3 - 1920xy^2 - 4288xy}{5120y^3 - 3840y^2 + 960y - 2780}\\
&-\frac{984x - 1024y^4 + 4352y^3 - 816y^2 - 880y + 357}{5120y^3 - 3840y^2 + 960y - 2780},\\
t =& -\frac{-512x^2y + 128x^2 - 4096xy^3 + 5120xy^2 - 896xy}{10240y^3 - 7680y^2 + 1920y - 5560}\\
&-\frac{1948x - 2048y^4 - 2176y^3 + 3008y^2 + 80y + 1139}{10240y^3 - 7680y^2 + 1920y - 5560}.
\end{aligned}
\tag{8.19}
$$

In order to determine and plot circles of the family we proceed as follows.

1. For x varying in the interval $0 \le x \le x_{max}$ we determine the corresponding values of y in the envelope using (in Maple)

$$\texttt{fsolve(subs}(x = i \cdot x_{max}/n, E), y),$$

and then determining the parameters of the associated tangent circles at these points. We plot then the corresponding circles. The result is shown in Fig. 8.18 left.

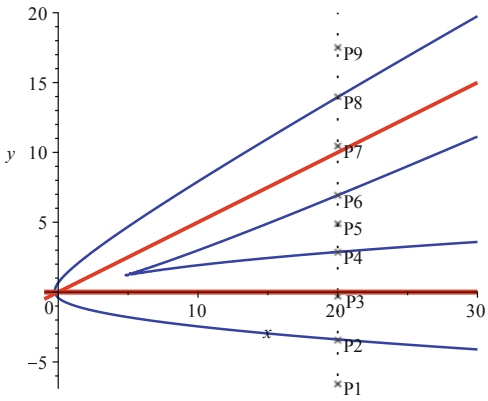


Fig. 8.19 Envelope and separating lines

2. For obtaining circles of the family, we have the limitation $0 \le s \le t$ for the circles of $[F, g]$ to belong to the family. Fixing s, t, then from $g = 0$ we have

$$x_1 = 2t + \sqrt{s}\,\cos\alpha,\ y_1 = s + \sqrt{s}\,\sin\alpha$$

where $0 \le \alpha \le 2\pi$. There are thus 3 parameters (s, t, α) to vary with the limitations on s, t, α already given. In order to plot them, in Fig. 8.18 right we fixed $\alpha = 2\pi/3$ and let vary s, t. The result is Fig. 8.18 right.

The first thing to do now is to separate both parts of the component $E = 0$. With this objective, let us analyze the asymptotic behavior of $E = 0$. Dividing E by x^4 and considering only the terms in $\ell = y/x$, because for $x \to \infty$ the remaining terms have limit 0, we obtain:

$$-1024\,\ell^2(1 - 4\ell + 4\ell^2) = 0$$

The equation has two double solutions, namely $\ell = 0$ and $\ell = 1/2$, that correspond well to the directions observed in the picture for the four parts of the component at infinity. In Fig. 8.19 are plotted the corresponding asymptotic lines $y = 0$ and $y = x/2$, and we observe that they separate the internal and the external parts of the envelope. To prove this result we determine the intersections of $E = 0$ with $y = 0$ and with $y = x/2$. Using Maple we obtain

```
> fsolve({E,2*y-x},{x,y});
              {x = -0.2210442075, y = -0.1105221037}
> fsolve({E,y},{x,y});
                   {x = -0.2464192098, y = 0.}
```

showing that each of both lines has a unique intersection with the envelope, and namely with E_{ext}. This shows that these lines really separate the internal and the

external part of the envelope. In consequence the two parts are

$$E_{ext} = \{(x, y) : (E = 0) \ \wedge ((2y - x) \geq 0 \ \vee (y \leq 0))\}$$
$$E_{int} = \{(x, y) : (E = 0) \ \wedge ((2y - x < 0) \ \wedge (y > 0))\}$$

In order to delimit the different regions in which the envelope $E = 0$ divides the plane, we compute the intersections of $E = 0$ with the line $x = 20$, and determine the signs of E (and of $2y - x$ and y) in the different zones, substituting in E. See Fig. 8.19.

```
> PointsE := proc (a)
     local x, y, E, y0, P0, P1, P2, P3, P4, P5, P6, P7, P8;
     E :=512*x^3-1024*x^2*y^2-2560*x^2*y-640*x^2+4096*x*y^3
         +4864*x*y^2-704*y*x+984*x-4096*y^4+1536*y^3-16*y^2
         +144*y+289;
    y0:=[fsolve(subs(x = a, E), y)];
    P1:=[a,y0[1]];
    P3:=[a,y0[2]];
    P5:=[a,y0[3]];
    P7:=[a,y0[4]];
    P2:= [a,(1/2)*y0[1]+(1/2)*y0[2]];
    P4:=[a,(1/2)*y0[2]+(1/2)*y0[3]];
    P6:= [a,(1/2)*y0[3]+(1/2)*y0[4]];
    P0:= [a,(3/2)*y0[1]-(1/2)*y0[2]];
    P8:=[a,(3/2)*y0[4]-(1/2)*y0[3]];
    return [P0, P1, P2, P3, P4, P5, P6, P7, P8]
  end proc;
> PE:=PointsE(20);
   PE:=[[20, -6.520470103], [20, -3.389330985],[20, -.258191867],
     [20, 2.872947250], [20, 4.903384449],
     [20, 6.933821648], [20, 10.44569187],
     [20, 13.95756209], [20, 17.46943232]]
```

Substituting in the different points we obtain the signs of E.

$$E(P_1) < 0,\ E(P_2) = 0,\ E(P_3) > 0,$$
$$E(P_4) = 0,\ E(P_5) < 0,\ E(P_6) = 0,$$
$$E(P_7) > 0,\ E(P_8) = 0,\ E(P_9) < 0.$$

On points P_2, P_4, P_6, P_8 the value of E is approximately 0, because they are points of the envelope. On the intermediate points the sign are alternated. Taking into account the sign of the equations of the separating lines, envelope $E = 0$ divides the plane into three disjoint regions namely

$$S_1 = \{(x, y) \in \mathbb{R}^2 : (E < 0) \wedge ((2y - x > 0) \ \vee (y < 0))\}$$
$$S_2 = \{(x, y) \in \mathbb{R}^2 : (E < 0) \wedge ((2y - x) < 0) \ \wedge (y > 0))\}$$
$$S_3 = \{(x, y) \in \mathbb{R}^2 : (E > 0)\}$$

We expect that there exist an infinite set, (with two degrees of freedom), of circles of the family passing through every point of $S_2 \cup S_3 \cup E_{int}$, and no circle on S_1.

There are three conjectures that have to be proved:

1. For each point $(u, v) \in E_{int} \cup S_2 \cup S_3$, there exist infinitely many real circles of the family passing through it.
2. For every point $(u, v) \in E_{ext}$, there exists a unique real circle of the family passing through it, namely the associated tangent circle.
3. There does not exists any real circle of the family passing through $(u, v) \in S_1$.

To answer these questions we begin solving the system $[F, g]$ for a circle passing through a fixed point $P(u, v)$, having center in $Q(x_1, y_1)$ in order to determine the values of x_1, y_1, s, t. Setting $m = t - s$, equation $F = 0$ gives

$$m = (u - x_1)^2 + (v - y_1)^2$$

and substituting $s = t - m$ in $g(u, v, x_1, y_1, s, t)$ we have

$$m^2 - 2mt + 2my_1 + 5t^2 - 4tx_1 - 2ty_1 + x_1^2 + y_1^2 + m - t = 0.$$

This is an equation of degree 2 in t. Its discriminant with respect to t is

$$\Delta = -16m^2 + 16mx_1 - 32my_1 - 4x_1^2 + 16x_1y_1 - 16y_1^2 - 16m + 8x_1 + 4y_1 + 1,$$

If $\Delta \geq 0$ then it exists a solution of the system $[F, g]$ for a given point $P(u, v)$, and the values of t, s are then

$$t = \frac{1}{10}(2m + 4x_1 + 2y_1 + 1 + \Delta), \quad s = t - m.$$

We can now evaluate the discriminant for points P_1 to P_9 with center at $Q(x_1, y_1)$. We build the simple Maple routine `pars` for determining the value of the discriminant for a circle with center $Q(x_1, y_1)$ that passes through point $P(x, y)$. When the discriminant Δ is non-negative the circle will belong to the family $[F, g]$. To show the output we apply it here as example to point P_4.

```
> pars:=proc(P,x1,y1)
     local  u,v,m,dis;
     global x,y;
     u:=P[1];v:=P[2];
     m:=(u-x1)^2+(v-y1)^2;
     dis:=-16*m^2+16*m*x1-32*m*y1-4*x1^2+16*x1*y1-16*y1^2-16*m
          +8*x1+4*y1+1;
     return([dis,(x-x1)^2+(y-y1)^2-m]);
  end:
> pars(P4,x1,y1);
     [-16*((20-x1)^2+(2.872947250-y1)^2)^2+(16*((20-x1)^2
     +(2.872947250-y1)^2))*x1-32*y1*((20-x1)^2+(2.872947250-y1)^2)
```

```
-16*y1^2-16*(20-x1)^2-16*(2.872947250-y1)^2+8*x1+4*y1+1,
(x-x1)^2+(y-y1)^2-(20-x1)^2-(2.872947250-y1)^2]
```

Substituting (x, y) by point P_i, equation $\Delta = 0$ determines a curve delimiting regions with opposite sign of Δ. In the interior region the discriminant will be positive and thus, for the points $Q(x_1, y_1)$ inside, there will exist circles of the family, whereas there will not exist circles of the family for values of $Q(x_1, y_1)$ outside.

We build another routine `cirs` that traveling point $Q(x_1, y_1)$ in the rectangle with extreme vertices (x_a, y_a) and (x_b, y_b) in n points per line and column, applies the previous routine to them. If Δ is non-negative then it considers the point, and if not then it refuses the point. So `cirs` obtains a list of points (with option `opt=1`) or the corresponding circles (with option `opt=0`), for which there exist a circle of the family passing at $P(u, v)$.

```
> cirs:=proc(P,xa,xb,ya,yb,n, opt)
     local L,C,i,j,x1,y1,c,dis;
     global x;
     C:={};
     for i to n+1 do
       for j to n+1 do
          x1:=xa+(i-1)*(xb-xa)/n;
          y1:=ya+(j-1)*(yb-ya)/n;
          L:=pars(P,x1,y1);
          dis:=L[1];
          c:=L[2];
          if dis>=0 then
            if(opt=0) then c:=L[2]; C:={op(C),c}
            else c:=[x1,y1]; C:=[op(C),c]  fi;
          fi;
       od;
    od;
    return(C);
 end;
```

For a given point P_i, we can plot the curve $\Delta = 0$ separating the points Q for which there exist a circle of the family passing at P from those for which it does not. Then we can apply the routine `cirs` in the rectangle where the curve is located and obtain the corresponding circles (with `opt=0`) or the centers (with `opt=1`). As an example Fig. 8.20 shows the result for point P_4, where we plotted the curve $\Delta = 0$ and the points that correspond to centers of circles of the family that pass at P_4.

We observe that the points in the finite interior region correspond to centers of circles of the family. Thus it exists a set of points $Q(x_1, y_1)$ with two degrees of freedom that are centers of a circle of the family passing through P_4. We have plotted as well the corresponding zones for points P_3, P_4, P_5, P_6 and P_7 in Fig. 8.21. Points P_4 and P_6 belong to E_{int}, and we see that there exist infinitely many circles of the family passing through them, as states conjecture (1). This is not a complete proof of conjecture (1) for every point of E_{int}, but gives evidence of it.

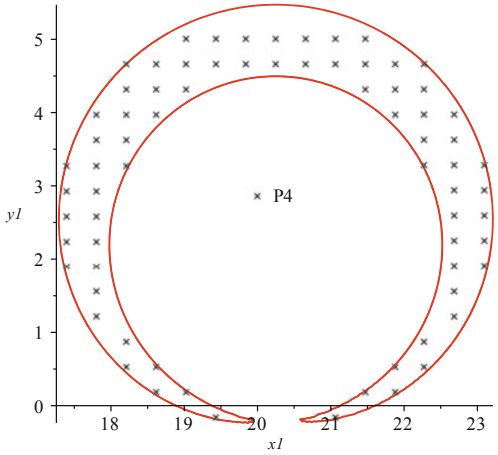


Fig. 8.20 $\Delta(P_4, Q) = 0$ and some points $Q(x_1, y_1)$ for which $\Delta \geq 0$

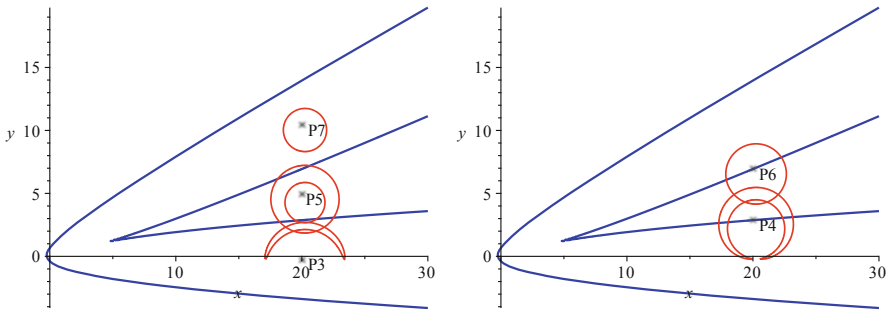


Fig. 8.21 Regions for which $\Delta(x_1, y_1) \geq 0$ relatives to points P_3 to P_7

For the external points P_1 and P_9, using the routines `pars` and `cirs`, we do not found any circle of the family passing through them, i.e., $\Delta = 0$ has no real point. This gives evidence of conjecture (3). The same happens for P_2 and P_8 that lie in E_{ext}. But in this case the reason comes from using numerical approximation. For them, we know that there exist the associated tangent circle passing through them which are tangent. This gives also evidence of conjecture (2).

In Fig. 8.22 are plotted some circles of the family passing through the points P_3, P_4, P_5, P_6 and P_7, and the associated tangent circles passing through points P_1 and P_8.

A second approach to the problem consists of considering two subfamilies of $[F, g]$ obtained by fixing the extreme values of s, i.e., $s = 0$ and $s = t$. The two families are

1. Family F_1. For $s = 0$: $g(t, 0) = (x_1 - 2t)^2 + y_1^2 = 0 \Rightarrow$
   $\{x_1 = 2t, y_1 = 0\}$, $F_1 = (x - 2t)^2 + y^2 - t$, $C_1 = 0$.

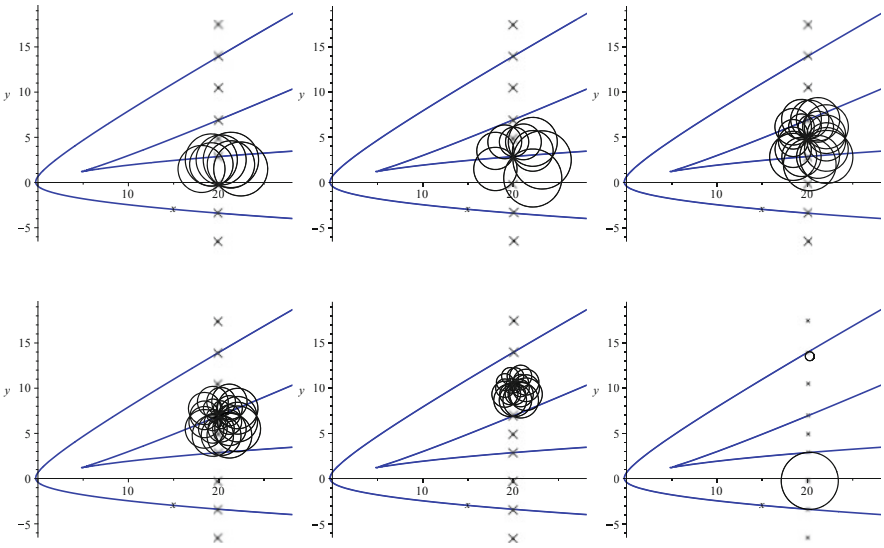


Fig. 8.22 Some circles of family F passing through points P_3, P_4, P_5, P_6, P_7 and associate tangent circles through P_2 and P_8

2. Family F_2. For $s = t$: $F(t = s) = (x - x_1)^2 + (y - y_1)^2 = 0 \Rightarrow$
   $\{x_1 = x, y_1 = y\}$, $F_2 = g(x = x_1, y = y_1, s = t)$
   $= (x - 2t)^2 + (y - t)^2 - t$, $C_2 = 0$.

   F_1 corresponds to F with the constraint $g_1 = \{g, s\}$.
   F_2 corresponds to g with the constraint $g_2 = \{F, t - s\}$.
   Using `envelop` it is simple to obtain the envelopes of F_1 and F_2:

```
> ring R2=(0,x,y),(t),lp;
> short=0;
> poly F1=(x-2*t)^2+(y)^2-t;
> ideal C1=0;
> def E1e=envelop(F1,C1):
> poly E1=E1e[1][1][1]; E1;
      (8*x-16*y^2+1)
> E1e[1][3][2];
   Normal
> poly F2=(x-2*t)^2+(y-t)^2-t;
> def E2e=envelop(F2,C1);
> poly E2=E2e[1][1][1]; E2;
       (4*x^2-16*x*y-8*x+16*y^2-4*y-1)
> E2e[1][3][2];
   Normal
```

   The envelopes are respectively:

1. $E_1 = 8x - 16y^2 + 1$, "Normal".
2. $E_2 = 4x^2 - 16xy - 8x + 16y^2 - 4y - 1$, "Normal".

To determine the points (x, y) for which there are circles of F_1 we have

$$F_1 = (x - 2t)^2 + y^2 - t = 4t^2 - (4x + 1)t + x^2 + y^2$$

The discriminant with respect to t is

$$\Delta_1 = (4x + 1)^2 - 16(x^2 + y^2) = 8x - 16y^2 + 1 = E_1$$

In consequence, for a point $P(x, y)$ such that $E_1 > 0$, i.e. in the interior region, there exist two circles of F_1 passing through it. If $(x, y) \in E_1$, then it exist a unique tangent circle passing through it, and if $E_1 < 0$ then no circle of F_1 passes through it. Moreover, the values of t of the circles of F_1 passing through (x, y) are

$$t_1 = \frac{4x + 1 \pm \sqrt{E_1}}{8}.$$

Similarly for F_2 we have

$$F_2 = (x - 2t)^2 + (y - t)^2 - t = 5t^2 - (4x + 2y + 1)t + x^2 + y^2$$

and the discriminant with respect to t is

$$\Delta_2 = -4x^2 + 16xy - 16y^2 + 8x + 4y + 1 = -E_2$$

In consequence, for $P(x, y)$ such that $E_2 < 0$, i.e. in the interior region, there exist two circles of F_2 passing through it. If $(x, y) \in E_2$, then it exist a unique tangent circle passing through it, and if $E_2 > 0$ then no circle of F_2 passes through it. The values of t of the circles of F_2 passing through (x, y) are

$$t_2 = \frac{4x + 2y + 1 \pm \sqrt{-E_2}}{10}$$

In Fig. 8.23 are represented the envelopes $E = 0$, $E_1 = 0$ and $E_2 = 0$. The union of $\mathbf{V}(E_1)$ and $\mathbf{V}(E_2)$ is very similar to $\mathbf{V}(E)$, but it is not identical to $\mathbf{V}(E)$. All the circles included in family F_1 are included in F. Concerning the circles of F_2 they are included in g and in consequence also in $S_{int} = S_2 \cup S_3 \cup E_{int}$. For these reasons, $\mathbf{V}(E_1)$ as well as $\mathbf{V}(E_2)$ and their interior regions must be included in S_{int}.

So the real envelope of F is E_{ext}, and is also determined by

$$\begin{aligned}
&\text{Real envelope of } F = E_{ext} = \\
&= \{(x, y) \in \mathbb{R}^2 : ((E = 0) \wedge (E_1 \leq 0) \wedge (E_2 \geq 0))\} \\
&= \{(x, y) \in \mathbb{R}^2 : ((512x^3 - 1024x^2y^2 - 2560x^2y - 640x^2 + 4096xy^3 \\
&\quad + 4864xy^2 - 704xy + 984x - 4096y^4 + 1536y^3 - 16y^2 + 144y + 289 = 0) \\
&\quad \wedge (8x - 16y^2 + 1 \leq 0) \wedge (4x^2 - 16xy - 8x + 16y^2 - 4y - 1 \geq 0))\}
\end{aligned}$$

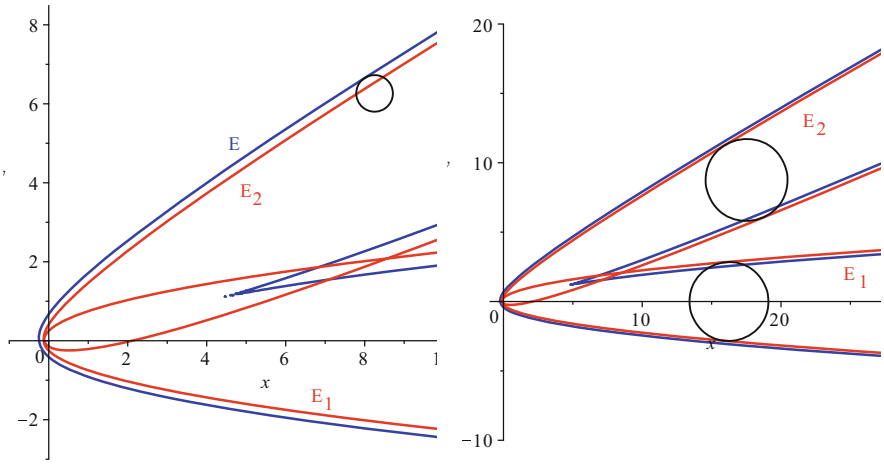


Fig. 8.23 Envelopes E, E_1 and E_2 and one associated tangent circle to each one

As a consequence there exist at least two real circles of family F for the points in $E_1 \geq 0$ and for the points in $E_2 \leq 0$, and thus also for a substantive part of points of $S_{int} = E_{int} \cup S_2 \cup S_3$. This gives evidence that the two dimensional set of solutions that we obtained determining the `grobcov` of $[F, g]$ are real on $(E_1 \geq 0) \vee (E_2 \leq 0)$, and by continuity, there are also real in the whole interior part of E_{ext}, i.e. on S_{int}, which would prove conjecture (1).

Example 8.21 This is a new example in which we can observe the kind of difficulties for determining the real points of the envelope, and how can we try to do it.

Consider the family of spheres of radius 1, centred at point (x_1, y_1, z_1) of a sphere with center $(0, 0, t)$ and radius $\sqrt{t}$. We have the following family and restrictions

$$F = (x - x_1)^2 + (y - y_1)^2 + (z - z_1)^2 - 1,$$
$$g = \langle x_1^2 + y_1^2 + (z_1 - t)^2 - t \rangle.$$

The problem has 3 degrees of freedom.

Applying `envelop(F,g)` one obtains 2 components: In Fig. 8.24 are represented the two disjoint parts of the "Normal" envelope component, and the "Accumulation" point.

$$E_1 = \mathbf{V}((16x^6 + 48x^4y^2 + 16x^4z^2 - 32x^4z - 56x^4 + 48x^2y^4 + 32x^2y^2z^2$$
$$- 64x^2y^2z - 112x^2y^2 - 32x^2z^3 - 24x^2z^2 + 29x^2 + 16y^6$$

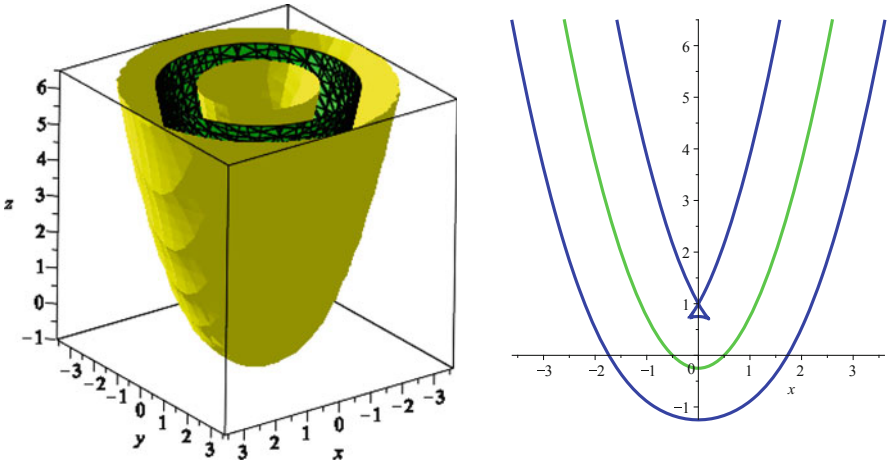


Fig. 8.24 Envelope, separating paraboloid and 2d-section at $y = 0$

$$+ 16y^4z^2 - 32y^4z - 56y^4 - 32y^2z^3 - 24y^2z^2 + 29y^2$$
$$+ 16z^4 - 24z^3 - 15z^2 + 38z - 15)) \setminus \mathbf{V}(z - 1, x^2 + y^2)$$

Normal

$$E_2 = \mathbf{V}(z - 1, y, x)$$

Accumulation

As g has degree 2 in t, we can determine its discriminant with respect to t and isolate t in order to obtain the real envelope.

$$\Delta(x_1, y_1, z_1) = \texttt{discrim}(g, t) = 4z_1 + 1 - 4x_1^2 - 4y_1^2,$$
$$t = z_1 + \frac{1}{2} \pm \frac{1}{2}\sqrt{\Delta(x_1, y_1, z_1)}.$$

We deduce that all the spheres of the family have their center in the interior region $\Delta \geq 0$. In Fig. 8.24, we have represented the envelope and the separating paraboloid $\Delta(x, y, z) = 0$, on which lie the centers of the spheres of the family having discriminant is zero.

We denote E_{ext} (external) and E_{int} (internal), the parts of E_1:

$$E_{ext} = \left\{(x, y, z) \in \mathbf{V}(E_1) : z < x^2 + y^2 - \frac{1}{4}\right\}$$
$$E_{int} = \left\{(x, y, z) \in \mathbf{V}(E_1) : z > x^2 + y^2 - \frac{1}{4}\right\}$$

We consider now the subfamily of spheres of $[F, g]$ with center in the paraboloid $\Delta = 0$, i.e., for $t = z_1 + 1/2$. For the subfamily we have

$$g_1 = g\left(x_1, y_1, z_1, z_1 + \frac{1}{2}\right) = \Delta(x_1, y_1, z_1).$$

Computing the envelope of the subfamily, we obtain the same envelope E_1 as for the initial problem.

Dividing E_1 by $\Delta(x, y, z)$, first with respect to z and then with respect to x we obtain the following decomposition of E_1

$$\begin{aligned}
E_1 &= (q + q_1)\Delta + r,\\
q &= 4z^3 - (4x^2 + 4y^2 + 7)z^2 - (12x + 12y + 2)z - 20x^4 - 40x^2y^2\\
&\quad - 20y^4 + x^2 + y^2 + 10 - 64x^6 - 192x^2y^4 - 192x^2y^4 - 64y^6\\
&\quad - 32x^4 - 64x^2y^2 - 32y^4 + 68x^2 + 68y^2 - 25,\\
q_1 &= 16x^4 + (32y^2 + 16z + 12)x^2 + 16y^4 + 16y^2z + 12y^2 + 16z^2 + 16z - 14,\\
r &= -64z^3 - 80z^2 + 40z - 11.
\end{aligned}$$

This shows that on the hyperboloid $\Delta(x, y, z) = 0$ the value of E_1 is r which is a polynomial in z whose unique real zero is $z = -1.682257884$. As the minimal value of z for all points of $\Delta(x, y, z) = 0$ is $z = -1/4$, this shows that E_1 is negative on all points of $\Delta(x, y, z) = 0$ (Fig. 8.25).

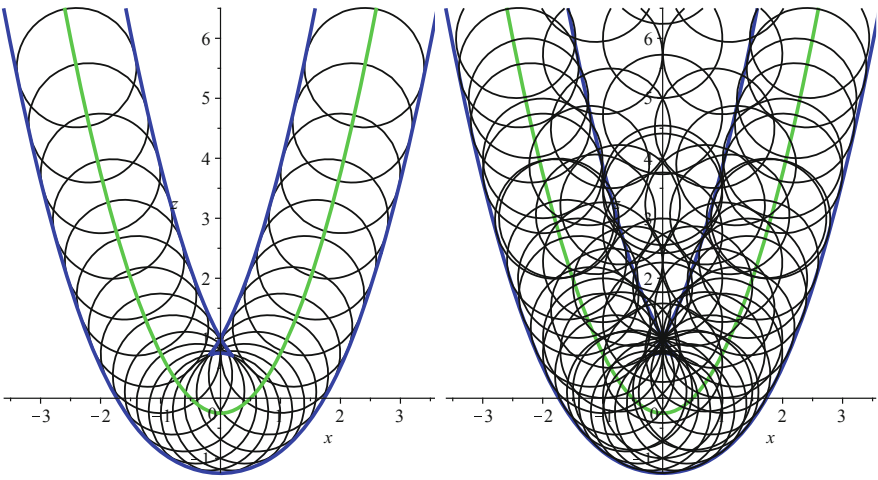


Fig. 8.25 2d-section at $y = 0$ of spheres of the subfamily with $\Delta = 0$ (left) and of the whole family (right)

The envelope E_1 divides the space into four regions plus the two parts of the envelope E_{int} and E_{ext}:

$$S_1 = \{(x, y, z) \in \mathbb{R}^3 : E_1 > 0 \cap \Delta(x, y, z) < 0\}$$

$$S_2 = \{(x, y, z) \in \mathbb{R}^3 : E_1 < 0\}$$

$$S_3 = \{(x, y, z) \in \mathbb{R}^3 : E_1 > 0 \cap \Delta(x, y, z) > 0 \cap z < 1\}$$

$$S_4 = \{(x, y, z) \in \mathbb{R}^3 : E_1 > 0 \cap \Delta(x, y, z) > 0 \cap z > 1\}$$

There are no spheres of the family on S_1. All the spheres of the subfamily are included in $S_2 \cup S_3$. On S_4 there are no spheres of the subfamily, but there are infinitely many spheres of the whole family.

We conclude that the real envelope of $[F, g]$ is E_{ext}, verifying

1. For their points pass a unique sphere of the family, namely the associated tangent one having discriminant is zero,
2. E_{ext} separates the region S_1 where there are no spheres of the family, from the interior region $S_2 \cup S_3 \cup S_4 \cup E_{int}$ where there are infinitely many spheres of the family depending on two parameters passing through each point.

Considering a point $P(x, y, z)$ of the "Normal" envelope component E_1, the command `AssocTanToEnv`(F, C, E_1) determines the values of the parameters (t, x_1, y_1, z_1) of the associated tangent sphere at point P. We obtain

$$t = -\frac{12x^4 + 24x^2y^2 - 4x^2z^2 - 28x^2z - 31x^2 + 12y^4 - 4y^2z^2 - 28y^2z - 31y^2 - 16z^4 - 28z^3}{16z^3 + 24z^2 + 12z - 25} + \frac{-3z^2 + 10z + 10}{6z^3 + 24z^2 + 12z - 25}$$

$$z_1 = -\frac{24x^4 + 48x^2y^2 - 8x^2z^2 - 56x^2z - 62x^2 + 24y^4 - 8y^2z^2 - 56y^2z - 62y^2 - 32z^4 - 40z^3}{32z^3} + \frac{18z^2 + 32z - 5}{32z^3}$$

$$y_1 = -\frac{6x^4yz + 32x^4y + 32x^2y^3z + 64x^2y^3 + 16x^2yz^3 - 16x^2yz^2 - 100x^2yz - 116x^2y + 32y^5}{32z^4 + 16z^3 - 24z^2 - 74z + 50} + \frac{16y^5z - 16y^3z^3 - 16y^3z^2 - 100y^3z - 116y^3 - 48yz^4 - 68yz^3}{32z^4 + 16z^3 - 24z^2 - 74z + 50}$$

$$x_1 = -\frac{16x^5z + 32x^5 + 32x^3y^2z + 64x^3y^2 + 16x^3z^3 - 16x^3z^2 - 100x^3z - 116x^3 + 16xy^4z}{32z^4 + 16z^3 - 24z^2 - 74z + 50} + \frac{32xy^4 - 16xy^2z^3 - 16xy^2z^2 - 100xy^2z - 116xy^2 - 48xz^4 - 68xz^3 - 36xz^2 + 36xz + 35x}{32z^4 + 16z^3 - 24z^2 - 74z + 50}$$

For example, at point $(-3.519505319, 0, 6)$ of the envelope, the center of the associated tangent sphere is $(-2.538358523, 0., 6.193264030)$.
This sphere is also the associated tangent sphere of the family at point $(-1.557188274, 0, 6.386408905)$, and is represented in Fig. 8.26.

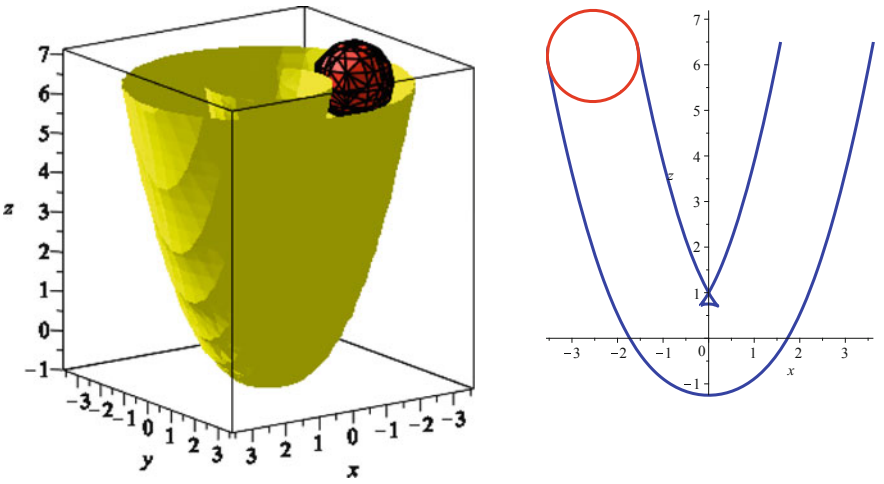


Fig. 8.26 Envelope, one associated tangent sphere and 2d-section at $y = 0$

The description given above can be established in a more formal form. We can suggest new strategies for determining the real envelope, but we let the problem here.

8.7 Loci with Two Movers

We analyze here non standard loci problems in the plane. We consider problems with two mover points. If there exists a reasonable solution, it will contain a bi-dimensional domain. Even so, we can also use the Gröbner Cover to approach these problems.

Example 8.22 Consider two mover points $M_1 = (x_1, y_1)$ and $M_2 = (x_2, y_2)$ gliding respectively on the circles $c_1 : (x_1-4)^2+y_1^2-16$ and $c_2 : (x_2+4)^2+y_2^2-1$, and consider the locus of middle points of M_1M_2. See Fig. 8.27. We have to solve the following set of equations:

$$F = [(x_1 - 4)^2 + y_1^2 - 16, (x_2 + 4)^2 + y_2^2 - 1, 2x - x_2 - x_1, 2y - y_2 - y_1].$$

In a first approach to solve this problem, we use directly the `grobcov` command. As we expect of a two dimensional set, the generic segment has solutions. Nevertheless, these solutions are not real for every value of all real values of (x, y).

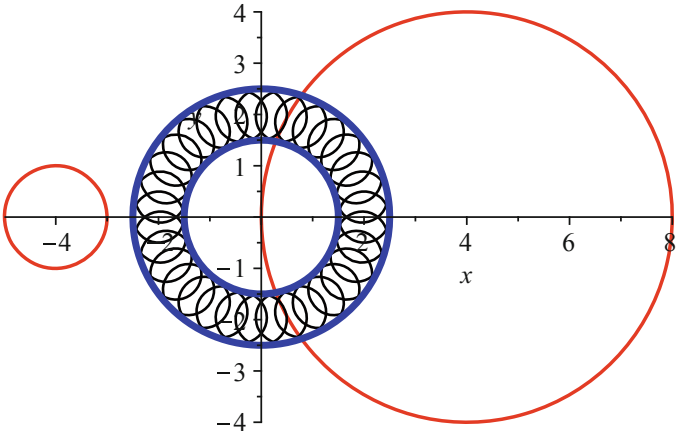


Fig. 8.27 Bi-dimensional locus of Example 8.22

Let us apply `grobcov`:

```
> LIB "grobcov.lib";
> ring R=(0,x,y),(x1,y1,x2,y2),lp;
> short=0;
> ideal F=(x2+4)^2+y2^2-1,(x1-4)^2+y1^2-16,2*x-x2-x1,2*y-y2-y1;
> def G=grobcov(F);
> G;
[1]:
   [1]:
      _[1]=y2^2
      _[2]=x2
      _[3]=y1
      _[4]=x1
   [2]:
      _[1]=(16*x^2+16*y^2)*y2^2+(-32*x^2*y-32*y^3+120*y)*y2
           +(16*x^4+32*x^2*y^2-136*x^2+16*y^4-120*y^2+225)
      _[2]=(4*x)*x2+(4*y)*y2+(-4*x^2+16*x-4*y^2+15)
      _[3]=y1+y2+(-2*y)
      _[4]=(4*x)*x1+(-4*y)*y2+(-4*x^2-16*x+4*y^2-15)
   [3]:
      [1]:
         [1]:
            _[1]=0
         [2]:
            [1]:
               _[1]=(x)
            [2]:
               _[1]=(x^2+y^2)
[2]:
   [1]:
      _[1]=y2
      _[2]=x2
```

```
      _[3]=y1
      _[4]=x1
   [2]:
      _[1]=(120*y)*y2+(16*y^2+225)
      _[2]=(120*y^2)*x2+(16*x*y^2-225*x+480*y^2)
      _[3]=(120*y)*y1+(-256*y^2-225)
      _[4]=(120*y^2)*x1+(-256*x*y^2+225*x-480*y^2)
   [3]:
      [1]:
         [1]:
            _[1]=(x^2+y^2)
         [2]:
            [1]:
               _[1]=(y)
               _[2]=(x)
[3]:
   [1]:
      _[1]=1
   [2]:
      _[1]=1
   [3]:
      [1]:
         [1]:
            _[1]=(y)
            _[2]=(x)
         [2]:
            [1]:
               _[1]=1
[4]:
   [1]:
      _[1]=y2
      _[2]=x2^2
      _[3]=y1
      _[4]=x1
   [2]:
      _[1]=(4*y)*y2+(-4*y^2+15)
      _[2]=(16*y^2)*x2^2+(128*y^2)*x2+(16*y^4+120*y^2+225)
      _[3]=(4*y)*y1+(-4*y^2-15)
      _[4]=x1+x2
   [3]:
      [1]:
         [1]:
            _[1]=(x)
         [2]:
            [1]:
               _[1]=(y)
               _[2]=(x)
```

As expected, the generic segment has solutions apparently for every value of (x, y) except for $x = 0$ or $x^2 + y^2 = 0$, where the solutions are described in segments 2,3 and 4. But the solutions of the first segment are not always real in the variables

(x_1, y_1, x_2, y_2). We can compute the discriminant of the first polynomial on the basis of the first segment following with the above computation:

```
> def g1=G[1][2][1];
> g1;
  (16*x^2+16*y^2)*y2^2+(-32*x^2*y-32*y^3+120*y)*y2
  +(16*x^4+32*x^2*y^2-136*x^2+16*y^4-120*y^2+225)
> discrim(g1,y2)
[1]:
   _[1]=-64
   _[2]=(x)
   _[3]=(4*x^2+4*y^2-9)
   _[4]=(4*x^2+4*y^2-25)
[2]:
   1,2,1,1
```

It shows that the real solutions must verify $(4x^2 + 4y^2 - 9)(4x^2 + 4y^2 - 25) \leq 0$. And this can only happen if $x^2 + y^2 \leq \left(\frac{5}{2}\right)^2$ and $x^2 + y^2 \geq \left(\frac{3}{2}\right)^2$, because the opposite signs are impossible. Thus we obtain the two-dimensional locus:

$$L = \left\{(x, y) : x^2 + y^2 \leq \left(\frac{5}{2}\right)^2, x^2 + y^2 \geq \left(\frac{3}{2}\right)^2\right\}.$$

We can also use a different strategy that is easier to be automated. We can first consider $M_1(x_1, y_1)$ as a fixed parametric point and consider the locus of the middle points of M_1 and M_2 when M_2 varies on c_2. We consider the parametric ring $R = \mathbb{C}[x_1, y_1, x, y][x_2, y_2]$ and compute the parametric locus of

$$S = [(x_2 + 4)^2 + y_2^2 - 1, 2x - x_2 - x_1, 2y - y_2 - y_1].$$

The parametric locus will contain curves in the plane for each value of the parameters (x_1, y_1). Now we can consider the envelope of this locus when the parametric point M_1 is constrained to move on the circle c_1. Using the *grobcov.lib* library we have the following:

```
> LIB "grobcov.lib";
> ring R=(0,x1,y1,x,y),(x2,y2),lp;
> short=0;
> ideal S=(x2+4)^2+y2^2-1,2*x-x2-x1,2*y-y2-y1;
> def L=locus(S);
>  L;
[1]:
   [1]:
      _[1]=(x1^2-4*x1*x-8*x1+y1^2-4*y1*y+4*x^2+16*x+4*y^2+15)
   [2]:
      [1]:
         _[1]=1
   [3]:
      [1]:
         1
      [2]:
```

```
         Special
      [3]:
         _[1]=x2^2+8*x2+y2^2+15
> poly F0=L[1][1][1];
> F0;
(x1^2-4*x1*x-8*x1+y1^2-4*y1*y+4*x^2+16*x+4*y^2+15)
> ring RR=(0,x,y),(x1,y1),lp;
> def F=imap(R,F0);
> ideal C=(x1-4)^2+y1^2-16;
> envelop(F,C);
[1]:
   [1]:
      _[1]=(4*x^2+4*y^2-25)
   [2]:
      [1]:
         _[1]=1
   [3]:
      [1]:
         1
      [2]:
         Normal
      [3]:
         _[1]=x1^2-8*x1+y1^2
[2]:
   [1]:
      _[1]=(4*x^2+4*y^2-9)
   [2]:
      [1]:
         _[1]=1
   [3]:
      [1]:
         1
      [2]:
         Normal
      [3]:
         _[1]=x1^2-8*x1+y1^2
```

The resulting envelope contains two components (circles) that limit the bi-dimensional region between both. The region is

$$D = \left\{(x, y) :\ 9/4 \leq x^2 + y^2 \leq 25/4\right\}.$$

One can also invert the roles of M_1 and M_2 and even if the parametric loci are not the same, the envelopes are.

In Fig. 8.27 the circles c_1 and c_2 are represented in red, the circles forming the parametric locus are in black, and the two envelope circles that limit the $2d$ locus area D are in blue.

Example 8.23 Determine the locus of the middle point between the mover point M_1 on the parabola $p_1 : x_1 - 4 - y_1^2$ and the mover point M_2 on the circle $c_2' : (x_2 + 4)^2 + y_2^2 - 1$. We use the strategy of the envelope.

We fix a parametric point $M_2(x_2, y_2)$ on the circle c_2', and compute the parametric locus when M_1 moves on the parabola p_1. Then we determine the envelope of all the parametric loci varying M_2 on the circle c_2'. We have

```
> LIB "grobcov.lib";
> ring R=(0,x2,y2,x,y),(x1,y1),lp;
> short=0;
> ideal S=x1-4-y1^2,2*x-x2-x1,2*y-y2-y1;
> def L=locus(G);
> "locus(S)="; L;
[1]:
   [1]:
      _[1]=(x2+y2^2-4*y2*y-2*x+4*y^2+4)
   [2]:
      [1]:
         _[1]=1
   [3]:
      [1]:
         1
      [2]:
         Special
      [3]:
         _[1]=x1-y1^2-4
> poly F0=L[1][1][1];
> F0;
(x2+y2^2-4*y2*y-2*x+4*y^2+4)
> ring R2=(0,x,y),(x2,y2),lp;
> def F=imap(R,F0);
> ideal C=(x2+4)^2+y2^2-1;
> envelop(F,C);
[1]:
   [1]:
      _[1]=(256*x^4-1024*x^3*y^2-320*x^3+1024*x^2*y^4
      +36*x^2-1280*x*y^4+48*x*y^2+80*x+1024*y^6-752*y^4
      +112*y^2-25)
   [2]:
      [1]:
         _[1]=1
   [3]:
      [1]:
         1
      [2]:
         Normal
      [3]:
         _[1]=x2^2+8*x2+y2^2+15
```

In this case, the envelope has a unique irreducible component delimiting the whole locus region. The component is

$$E = 256x^4 - 1024x^3y^2 - 320x^3 + 1024x^2y^4 + 36x^2 - 1280xy^4 + 48xy^2 + 80x + 1024y^6 - 752y^4 + 112y^2 - 25,$$

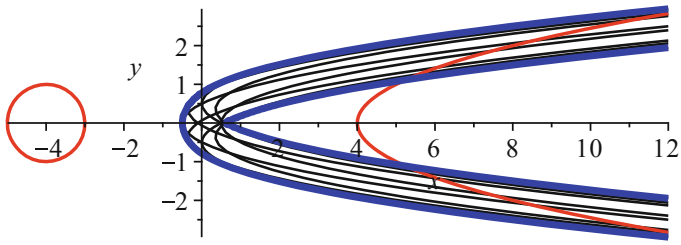


Fig. 8.28 Bi-dimensional locus of Example 8.23

the $2d$-locus being determined by the interior of the curve E, namely

$$L = \{(x, y) : 256x^4 - 1024x^3y^2 - 320x^3 + 1024x^2y^4 + 36x^2 - 1280xy^4 + 48xy^2 + 80x + 1024y^6 - 752y^4 + 112y^2 - 25 \leq 0\}.$$

One can proceed similarly computing first the parametric locus fixing a point $M_1(x_1, y_1) \in p_1$, varying $M_2 \in c_2'$, and then, the envelope of the parametric loci varying $M_1(x_1, y_1)$. Even if the parametric loci are different, the envelopes are equal.

In Fig. 8.28 the first approach is used. One can see the parabola p_1 and the circle c_2' (in red), the parabolas of the loci constraining the mover point $M_2(x_2, y_2)$ on a fix point in c_2' varying the point $M_1 \in p_1$ (in black), and the unique irreducible envelope component (in blue).

8.8 Exercises

We propose some exercises to be solved using the SINGULAR *grobcov.lib* commands for envelopes.

(1) Consider the circle $x_1^2 + y_1^2 - 9 = 0$, and a mover point $M(x_1, y_1)$ on it, and the fix point $Q(5, 0)$. Let F be the family of lines QM. Determine its envelope.
Answer: Two "Special" components:

$$C_1 = \mathbf{V}(3x + 4y - 15) \setminus \mathbf{V}(y, x - 5)$$
$$C_2 = \mathbf{V}(3x - 4y - 15) \setminus \mathbf{V}(y, x - 5)$$

and an "Accumulation" point $(5, 0)$.

(2) Determine the offset of the paraboloid $f : z_1 - 1 - x_1^2 - y_1^2$ at distance 1, i.e., the envelope of the spheres of radius 1 centred on the points of the paraboloid. Determine also the real envelope.

Answer: A single "Normal" component:

$$\begin{aligned} L = &\mathbf{V}(16x^6 + 48x^4y^2 + 16x^4z^2 - 72x^4z + 9x^4 + 48x^2y^4 + 32x^2y^2z^2 \\ &- 144x^2y^2z + 18x^2y^2 - 32x^2z^3 + 96x^2z^2 - 90x^2z + 54x^2 + 16y^6 \\ &+ 16y^4z^2 - 72y^4z + 9y^4 - 32y^2z^3 + 96y^2z^2 - 90y^2z + 54y^2 + 16z^4 \\ &- 104z^3 + 225z^2 - 162z). \end{aligned}$$

(3) Study of a tri-dimensional locus in $3d$ space. Consider the spheres $s_1 : x_1^2 + y_1^2 + (z_1 - 4)^2 - 4$ and $s_2 : x_2^2 + y_2^2 + (z_2 + 4)^2 - 1$. We want to obtain the locus of midpoints between a mover point $M_1(x_1, y_1)$ in s_1 and a mover point $M_2(x_2, y_2)$ in s_2.

Hint: Follow the method of Example 8.23.

Answer: $L = \{(x, y, z) \in \mathbb{R}^3 : \ 1/4 \leq x^2 + y^2 + z^2 \leq 9/4\}$.

(4) In the above examples, study the associated tangent elements at the points of the components of the envelope using the command `AssocTanToEnv` and also the elements of the family at the points of the envelope using the command `FamElemsAtEnvCompPoints`.

Appendix
The BUILDTREE Algorithm

The algorithm BUILDTREE was designed to obtain a disjoint reduced CGS. It is the result of the evolution of previous algorithms that the author, in collaboration with M. Manubens [54–56], developed starting in 2000 till its implementation in the first design of the Gröbner Cover in 2008.

We include it here for historical reasons, because in the recent implementation of the library "grobcov.lib", it has been replaced by the more efficient KSW algorithm [43] described in Chap. 3.

We will start by describing a special representation of locally closed sets, that is adapted for the particular locally closed sets considered in the BUILDTREE algorithm.

A.1 R-Representation of Special Locally Closed Sets

The BUILDTREE algorithm uses special simple locally closed sets for which the R-representation described here is sufficient.

Definition A.1 (R-representation) Let $S \subset L^m$ be a locally closed subset of the form

$$S = S((\mathfrak{a}, h)) = \mathbf{V}(\mathfrak{a}) \setminus \mathbf{V}(h),$$

where $\mathfrak{a} \subset K[\mathbf{u}]$ is an ideal and $h \in K[\mathbf{u}]$. We say that the pair $(\mathfrak{a}, h)$ is an R-representation of S if $\mathfrak{a}$ is a radical ideal and h is not contained in a proper prime ideal of $\mathfrak{a}$.

If $(\mathfrak{a}, h)$ is an R-representation (REP) of S, then the condition that h is not contained in a proper prime ideal of $\mathfrak{a}$ means that $\overline{S} = \mathbf{V}(\mathfrak{a})$. So the ideal $\mathfrak{a}$,

© Springer Nature Switzerland AG 2018

A. Montes, *The Gröbner Cover*, Algorithms and Computation in Mathematics 27, https://doi.org/10.1007/978-3-030-03904-2

$(\mathfrak{a}, h) \leftarrow \mathbf{Rrep}(\mathfrak{a}', h')$
Input:
$\mathfrak{a}' \subset K[\mathbf{u}]$ ideal
$h' \in K[\mathbf{u}]$
Output:
$(\mathfrak{a}, h)$: the R-representation of $S = \mathbf{V}(\mathfrak{a}') \setminus \mathbf{V}(h')$

begin
$h = \text{RADICAL}(h')$ # not strictly necessary (it suffices $h = h'$)
$\{\mathfrak{p}'_1, \ldots, \mathfrak{p}'_s\} = \text{PRIMEDECOMP}(\mathfrak{a}')$
$J = \{\mathfrak{p}_1, \ldots, \mathfrak{p}_r\} = \{\mathfrak{p}'_i :\in \{1, \ldots, s\}\ h \notin \mathfrak{p}'_i\}$
$\mathfrak{a} = \bigcap_{i \in J} \mathfrak{p}_i$
end

Algorithm A.1: RREP Algorithm

associated to S in the RREP, is the same as the top of $\text{CREP}(S) = (\mathfrak{a}, \mathfrak{b})$ (see Chap. 2, Sect. 2.2).

Let $S = \mathbf{V}(\mathfrak{a}') \setminus \mathbf{V}(h')$ with $\mathfrak{a} \subset K[\mathbf{u}]$ an ideal and $h' \in K[\mathbf{u}]$. Then Algorithm A.1: RREP computes an R-representation of S.

In an R-representation, $(\mathfrak{a}, h)$ the ideal $\mathfrak{a}$ is unique since $\overline{S} = \mathbf{V}(\mathfrak{a})$ but the polynomial h is not uniquely determined.

The usefulness of reduced representations comes from the following.

Proposition A.2 (R-representation) *Let $(\mathfrak{a}, h)$ be an R-representation of the locally closed set $S = \mathbf{V}(\mathfrak{a}) \setminus \mathbf{V}(h) \subset L^m$ and $f \in K[\mathbf{u}]$. Then*

(i) *$f(a) = 0$ for all $a \in S$ if and only if $f \in \mathfrak{a}$.*
(ii) *$f(a) \neq 0$ for all $a \in S$ if and only if $\text{RREP}(\mathfrak{a} + f, h) = (\langle 1 \rangle, h)$.*
(iii) *If neither $f(a) = 0$ nor $f(a) \neq 0$ hold for all $a \in S$, then S is the disjoint union of the two non empty disjoint locally closed set*

$$S_0 = S\,(\text{RREP}(\mathfrak{a} + f, h)) \text{ and } S_1 = S\,(\text{RREP}(\mathfrak{a}, hf))\,,$$

and $f(a) = 0$ for all $a \in S_0$ whereas $f(a) \neq 0$ for all $a \in S_1$.

Proof

(i) Obviously, if $f \in \mathfrak{a}$, then $f(a) = 0$ for all $a \in S((\mathfrak{a}, h))$. For the reciprocal, if $f(a) = 0$ for all $a \in S((\mathfrak{a}, h))$, then f also vanishes on the closure $\overline{S((\mathfrak{a}, h))} = \mathbf{V}(\mathfrak{a})$. Thus, as $\mathfrak{a}$ is radical, by Hilbert's Nullstellensatz, it follows that $f \in \mathfrak{a}$.
(ii) The set of all points of $S = \mathbf{V}(\mathfrak{a}) \setminus \mathbf{V}(h)$ where f vanishes is $\mathbf{V}(\mathfrak{a} + f) \setminus \mathbf{V}(h)$. Thus $f(a) \neq 0$ for all $a \in S((\mathfrak{a}, h))$ if and only if $\mathbf{V}(\mathfrak{a} + f) \setminus \mathbf{V}(h) = \emptyset$, and this is equivalent to $\text{RREP}(\mathfrak{a} + f, h) = (\langle 1 \rangle, h)$.
(iii) Obvious from Definition A.1.

□

A.2 Algorithms for Splitting the Discussion

Starting from the pair $(B, (\mathfrak{a}, h))$, where B is the basis of an ideal of $K[\mathbf{u}, \mathbf{x}]$ to be discussed over the segment determined by the $\text{RREP}(S) = (\mathfrak{a}, h)$, our interest is to split into two new sets $(B_0, (\mathfrak{a}_0, h_0))$ and $(B_1, (\mathfrak{a}_1, h_1))$ so that the segments $\mathbf{V}(\mathfrak{a}_0) \setminus \mathbf{V}(h_0)$ and $\mathbf{V}(\mathfrak{a}_1) \setminus \mathbf{V}(h_1)$ are a disjoint partition of S, and B_0 and B_1 are specialized bases of B on the respective segments. So, the partition is more advanced icomputing of the CGS. The first splitting decision depends on a polynomial $f \in K[\mathbf{u}][\mathbf{x}]$, and is given by Algorithm A.2: SPLIT, which is a consequence of Proposition A.2.

Proposition A.3 (Split algorithm) *Let $(\mathfrak{a}, h)$ be an R-representation of $S = \mathbf{V}(\mathfrak{a}) \setminus \mathbf{V}(h) \subset L^m$, and $f \in K[\mathbf{u}]$ a new polynomial that does not lie in $\mathfrak{a}$. Then the Algorithm A.2: SPLIT outputs two new R-representations $(\mathfrak{a}_0, h_0)$ and $(\mathfrak{a}_1, h_1)$ that split S into two disjoint sets $S_0 = \mathbf{V}(\mathfrak{a}_0) \setminus \mathbf{V}(h_0)$ and $S_1 = \mathbf{V}(\mathfrak{a}_1) \setminus \mathbf{V}(h_1)$ such that*

(i) $S_0 \cup S_1 = S$ *and* $S_0 \cap S_1 = \emptyset$,
(ii) $f(a) = 0$ *for all* $a \in S_0$ *and* $f(a) \neq 0$ *for all* $a \in S_1$,
(iii) $\mathfrak{a}_0 = \langle 1 \rangle$ *if and only if* $S_0 = \emptyset$, *so that no splitting is necessary.*

Proof The proof is obvious using Proposition A.2. □

Define a vertex data as $((\mathfrak{a}, h), B, \ell, P)$, where $(\mathfrak{a}, h)$ represents the segment, B the actual basis, ℓ is an integer which tell us that the $\text{lc}(g_i)$, for i till ℓ have been tested to be different form 0, and P is a list of pairs of integers that represent the set of S-polynomials of the basis that have not yet been eliminated following the standard Buchberger strategy.

$((\mathfrak{a}_0, h_0), (\mathfrak{a}_1, h_1)) \leftarrow$ **Split**$(f, (\mathfrak{a}, h))$
`Input:`
  $(\mathfrak{a}, h)$: an R-representation of $S = \mathbf{V}(\mathfrak{a}) \setminus \mathbf{V}(h)$
  $f \in K[\mathbf{u}]$: a new polynomial not in $\mathfrak{a}$

`Output:`
  $(\mathfrak{a}_0, h_0)$: RREP of the points $a \in S$ with $f(a) = 0$
  $(\mathfrak{a}_1, h_1)$: RREP of the points $a \in S$ with $f(a) \neq 0$

begin
  $(\mathfrak{a}_0, h_0) = \text{RREP}(\mathfrak{a} + f, h)$
  if $\mathfrak{a}_0 = \langle 1 \rangle$ **then**
    $(\mathfrak{a}_1, h_1) = (\mathfrak{a}, h)$
  else
    $(\mathfrak{a}_1, h_1) = \text{RREP}(\mathfrak{a}, hf)$
  end if
end

Algorithm A.2: SPLIT Algorithm

$(B', (\mathfrak{a}_0, h_0), l_0, P_0, (\mathfrak{a}_1, h_1), l_1, P_1) \leftarrow$ **DiscussPolys**$((\mathfrak{a}, h), B, l, P)$
Input:
$((\mathfrak{a}, h), B, l, P)$: the current vertex data
Output:
$((\mathfrak{a}_0, h_0), B', \ell_0, P_0)$: a new vertex data making a new null assumption
$((\mathfrak{a}_1, h_1), B', \ell_1, P_1)$: a new vertex data making a new non-null assumption

begin
  $B' = B$
  $split = false$
  while $split = false$ and $l < |B'|$ **do**
    $f =$ PNORMALFORM$(B'_{\ell+1}, (\mathfrak{a}, h))$
    if $f = 0$ **then** $B' = B'$ with $B'_{\ell+1}$ deleted
    else $B' = B'$ with $B'_{\ell+1}$ replaced by f
      $((\mathfrak{a}_0, h_0), (\mathfrak{a}_1, h_1)) =$ SPLIT(lc(f), $(\mathfrak{a}, h)$)
      if $\mathfrak{a}_0 \neq \langle 1 \rangle$ **then** $split = true$
      $\ell_0 = \ell$; $\ell_1 = \ell + 1$;
      $P_1 = P \cup \{(j, \ell_1) : 1 \leq j < \ell_1,$
        $(B_j, B_{\ell_1}) \in$ Buchberger pair selection$\}$
      $P_0 = P$
      else $\ell = \ell + 1$
      $P = P \cup \{(j, \ell) : 1 \leq j < \ell,\ (B_j, B_\ell) \in$ Buchberger pair selection$\}$
      end if
    end if
  end while
  if $split = false$ **then**
    $(\mathfrak{a}_1, h_1) = (\mathfrak{a}, h)$; $(\mathfrak{a}_0, h_0) = (\langle 1 \rangle, h)$; $\ell_0 = |B'|$; $\ell_1 = |B'|$;
    $P_0 = P$; $P_1 = P$
  end if
end

Algorithm A.3: DISCUSSPOLYS algorithm

Assume we are given such a vertex. The task of the following Algorithm A.3: DISCUSSPOLYS is to obtain new information about the vanishing behavior of the leading coefficients of the next polynomials $B_{\ell+1}, B_{\ell+2}, \ldots$ in the list until splitting is necessary. The result of DISCUSSPOLYS is summarized in the following:

Lemma A.4 (DiscussPolys algorithm) *Suppose that* DISCUSSPOLYS *is called with the vertex data* $((\mathfrak{a}, h), B, \ell, P)$. *Then two new vertices with data* $((\mathfrak{a}_0, h_0), B', \ell_0, P_0)$ *and* $((\mathfrak{a}_1, h_1), B', \ell_1, P_1)$ *with the following properties will be obtained:*

(i) $S = S_0 \uplus S_1$ *where* $S = S((\mathfrak{a}, h))$, $S_0 = S((\mathfrak{a}_0, h_0))$ *and* $S_1 = S((\mathfrak{a}_1, h_1))$,
(ii)
- *either* $\mathfrak{a}_0 = \langle 1 \rangle$, *i.e.,* $S_0 = \emptyset$, $S_1 = S$ *and then* $\ell_1 = |B'|$, *which means that all the leading coefficients of polynomials in* B' *have been tested and are non-null on all of* $S = S_1$,
- *or* $\mathfrak{a}_0 \neq \langle 1 \rangle$ *and then* $\mathfrak{a}_0 \supsetneqq \mathfrak{a}$, $\ell_0 = \ell_1 - 1$.

(iii) *P_1 and P_0 are updated using the standard strategy and the Buchberger criterium of eliminating the pairs with a disjoint set of variables of their* lpp.[1]

Proof First, we note that DISCUSSPOLYS will only be called with $\ell < |B|$. The list B' of polynomials is initially equal to B. The algorithm starts with testing if $B_{\ell+1}$ specializes to zero for every point of S. If this is the case, we can simply delete $B_{\ell+1}$ from our list of polynomials and continue with considering the next polynomial $B_{\ell+2} = B'_{\ell+1}$ in the list. If we eventually find a polynomial which does not vanish identically on S, i.e., $f \neq 0$, then we use algorithm SPLIT to test if there is an $a \in S$ with $\mathrm{lc}(f)(a) = 0$, i.e., $\mathfrak{a}_0 \neq \langle 1 \rangle$. If this is the case, we have found a proper splitting, and the two appropriate new vertex data are returned. If $\mathrm{lc}(f)(a) \neq 0$ for all $a \in S$, i.e., $\mathfrak{a}_0 = \langle 1 \rangle$, then no splitting is necessary and the algoroithm continues with the next polynomial in the list.

If we reach the end of the list, then we must have $split = false$ and the last "if"-statement guarantees that we get back the correct result.

So (i) is a direct consequence of Proposition A.3, and the remaining claims are immediate from the algorithm. □

The Algorithm A.4: DISCUSSSPOLYS has some similarities with DISCUSSPOLYS. However, DISCUSSPOLYS is always called with a vertex data $((\mathfrak{a}, h), B, \ell, P)$ satisfying $\ell < |B|$ whereas the vertex data for DISCUSSSPOLYS always satisfies $\ell = |B|$. In other words, if DISCUSSSPOLYS is called with vertex data $((\mathfrak{a}, h), B, \ell, P)$ then $\mathrm{lc}(p)(a) \neq 0$ for all $p \in B$ and $a \in S = S((\mathfrak{a}, h))$. The task of DISCUSSSPOLYS is simply to carry out the usual Buchberger algorithm until the next splitting is necessary, i.e., until a leading coefficient which vanishes on some but not at all points of S is found.

Lemma A.5 (DiscussSPolys algorithm) *Suppose that* DISCUSSSPOLYS *is called with the vertex data $((\mathfrak{a}, h), B, \ell, P)$. Then two new vertex data $((\mathfrak{a}_0, h_0), B', \ell_0, P_0)$ and $((\mathfrak{a}_1, h_1), B', \ell_1, P_1)$ with the following properties will be obtained:*

(i) *$S = S_0 \uplus S_1$ where $S = S((\mathfrak{a}, h))$, $S_0 = S((\mathfrak{a}_0, h_0))$ and $S_1 = S((\mathfrak{a}_1, h_1))$,*
(ii) – *either $\mathfrak{a}_0 \neq \langle 1 \rangle$ and then $\mathfrak{a}_0 \supsetneq \mathfrak{a}$, $\ell_0 = |B'| - 1$, $\ell_1 = |B'|$,*
– *or $\mathfrak{a}_0 = \langle 1 \rangle$, i.e., $S_0 = \emptyset$, $S_1 = S$ and then $\mathrm{lc}(p)(a) \neq 0$ for all $a \in S = S_1$ and $p \in B'$. Also $P_1 = \emptyset$, so that all the S-polynomials of pairs of elements of B' reduce to zero over $S = S_1$. In particular B' specializes to a Gröbner basis of I_a for every $a \in S$.*
(iii) *P_1 and P_0 are updated using the current strategies.*

Proof We recall that $\mathrm{lc}(p)(a) \neq 0$ for all $p \in B$ and $a \in S = S((\mathfrak{a}, h))$. The algorithm starts with picking a pair of polynomials of $B' = B$ specified in $P_1 = P$. This pair is removed from the list and we test if the reduction of the corresponding S-polynomial modulo B' vanishes identically on S, i.e., if $f = 0$. If this is the case,

[1] This should be improved using better strategies such as those developed in [32, 34, 37].

$(B', (\mathfrak{a}_0, h_0), \ell_0, P_0, (\mathfrak{a}_1, h_1), \ell_1, P_1) \leftarrow$ **DiscussSPolys**$((\mathfrak{a}, h), B, \ell, P)$
Input:
$(B, (\mathfrak{a}, h), \ell, P)$: the current vertex data
Output:
$((\mathfrak{a}_0, h_0), B', \ell_0, P_0)$: a new vertex data making a new null assumption
$((\mathfrak{a}_1, h_1), B', \ell_1, P_1)$: a new vertex data making a new non-null assumption

begin
$B' = B$; $P_1 = P$
$split = false$
while $split = false$ and $P_1 \neq \emptyset$ **do**
Pick $(i, j) \in P_1$ #{standard choice}
$P_1 = P_1 \setminus \{(i, j)\}$
$f = \mathrm{lc}(B_j)B_i - \mathrm{lc}(B_i)B_j$
$f =$ PNORMALFORM(PDIV(f, B'), $(\mathfrak{a}, h)$)
if $f \neq 0$ **then**
$B' = B' \cup \{f\}$
$((\mathfrak{a}_0, h_0), (\mathfrak{a}_1, h_1)) =$ SPLIT$(\mathrm{lc}(f), (\mathfrak{a}, h))$
if $\mathfrak{a}_0 \neq \langle 1 \rangle$ **then** $split = true$
$\ell_0 = |B'| - 1$; $P_0 = P_1$
$\ell_1 = |B'|$; $P_1 = P_1 \cup \{(j, \ell_1) : 1 \leq j < \ell_1,\ (B_j, f) \in \mathrm{BPS}\}$
else $\ell = \ell + 1$; $P_1 = P_1 \cup \{(j, \ell) : 1 \leq j < \ell,\ (B_j, f) \in \mathrm{BPS}\}$
end if
end if
end while
if $split = false$ **then**
$(\mathfrak{a}_1, h_1) = (\mathfrak{a}, h)$; $(\mathfrak{a}_0, h_0) = (\langle 1 \rangle, h)$;
$\ell_0 = |B'|$; $\ell_1 = |B'|$; $P_0 = \emptyset$; $P_1 = \emptyset$
end if
end

Algorithm A.4: DISCUSSSPOLYS algorithm

then the algorithm continues by picking the next pair from P_1. Otherwise, i.e., if $f \neq 0$, then we add f to the basis and use the algorithm SPLIT to test if there is an $a \in S$ with $\mathrm{lc}(f)(a) = 0$, i.e., $\mathfrak{a}_0 \neq \langle 1 \rangle$. If this is the case, we have found a proper splitting, and the two appropriate new vertex data are returned. If $\mathrm{lc}(f)(a) \neq 0$ for all $a \in S$, i.e., $\mathfrak{a}_0 = \langle 1 \rangle$, then no splitting is necessary and we continue by picking the next pair in P_1.

If we have to remove the last element from P_1, then we must have $split = false$ and the last "if"-statement guarantees that the algorithm returns the correct result. We note that only in this case we will have $\mathfrak{a}_0 = \langle 1 \rangle$. When the list P_1 is empty, this means that for each pair from the current basis $B' = \{p_1, \ldots, p_r\}$ the corresponding S-polynomials reduce to zero modulo B' over S. In other words, for every $a \in S$ the polynomials $\{p_1(a, \mathbf{x}), \ldots, p_r(a, \mathbf{x})\}$ satisfy Buchberger's criterion and thus are a Gröbner basis of I_a. □

$B' \leftarrow$ **ReduceGB**(B)
`Input`: B: a finite subset of $K[\mathbf{u}][\mathbf{x}]$ such that for every a in a
certain locally closed subset S of L^m we have
$\mathrm{lc}(p)(a) \neq 0$ for all $p \in B$ and
$B(a) \subset K[\mathbf{x}]$ is a Gröbner basis.
`Output`: B': a finite subset of $K[\mathbf{u}][\mathbf{x}]$ such that $B'(a)$ is
(up to normalization) the reduced Gröbner basis of
$\langle B(a) \rangle \subset K[\mathbf{x}]$ for every $a \in S$.

begin
Let $B' \subset B$ be the set of all polynomials in B with minimal lpp.
for $p \in B'$ **do**
$B' = B' \setminus \{p\}$
$p = \mathrm{PDIV}(p, B')$
$B' = B' \cup \{p\}$
end do
end

Algorithm A.5: REDUCEGB algorithm

The Algorithm A.5: REDUCEGB, does the usual final action of reducing the basis. First, it minimizes the Gröbner basis and then fully reduces the minimized Gröbner basis.

A.3 BUILDTREE Algorithm

We present first the main algorithm BUILDTREE, and then the recursive algorithm RECBUILDTREE called by BUILDTREE. RECBUILDTREE uses the two sub-algorithms DISCUSSPOLYS and DISCUSSSPOLYS.

BUILDTREE is a Buchberger-like algorithm for computing a Gröbner basis. As here the coefficients of the polynomials are polynomials in the parameters, the algorithm branches every time that it has to deal with a polynomial of the basis or an S-polynomial whose leading coefficient vanishes at some, but not at all points of the locally closed set under consideration. It builds up a dichotomic binary tree, whose branches at each vertex correspond to the annihilation or not of a new polynomial of $K[\mathbf{u}]$. So, at a vertex, some polynomials, say $N \subset K[\mathbf{u}]$ have been assumed to be null and some others, say $W \subset K[\mathbf{u}]$, have been assumed to be non-null. This determines a locally closed subset S of L^m, of the special kind for which R-representations can be used (see Sect. A.1). That is,

$$S = \mathbf{V}(N) \setminus \mathbf{V}(h) \subset L^m, \quad \text{with} \quad h = \prod_{w \in W} w \in K[\mathbf{u}].$$

The position of a vertex V of the tree is given by a list of zeros and ones describing its position in the tree. At each vertex of the tree, BUILDTREE stores $(\alpha, (\mathfrak{a}, h), B, \ell, P)$, where

(1) $\alpha =$ list of zeroes and ones indicating its position,
(2) $(\mathfrak{a}, h)$ is an R-representation of $S = V(\mathfrak{a}) \setminus \mathbf{V}(h)$,
(3) B is a finite list of polynomials in $K[\mathbf{u}, \mathbf{x}]$ such that, for every $a \in S$, the polynomials obtained from B by specialization are a generating set of I_a, being B_i the i-th element in this list,
(4) ℓ is an integer such that $0 \leq \ell \leq |B|$ and for $i = 1, \ldots, \ell$, we have $\mathrm{lc}(B_i)(a) \neq 0$ for all $a \in S$, and so far the algorithm has not obtained information about the vanishing behavior of $\mathrm{lc}(B_t)$ on S for $t > \ell$,
(5) P is a list of pairs of elements of $\{1, \ldots, \ell\}$ such that for each pair $(i, j) \in P$, the S-polynomial of B_i and B_j has not yet been considered in the algorithm.

Using R-representations, the algorithms A.3: DISCUSSPOLYS and A.4: DISCUSSSPOLYS allow to split recursively into two dichotomic branches when the algorithm has to decide if a new polynomial $f \in K[\mathbf{u}]$ is null or non-null on the given locally closed set S. This is done by the recursive Algorithm A.7: RECBUILDTREE that uses Algorithm A.1: SPLIT, already discussed in Sect. A.1.

Theorem A.6 (BUILDTREE algorithm) *Given a finite set $F \subset K[\mathbf{u}][\mathbf{x}]$ of homogeneous polynomials generating the ideal I, the Algorithm A.6:* BUILDTREE *builds a finite binary tree T with root such that at each terminal vertex v of T its data $(\alpha_v, (\mathfrak{a}_v, h_v), B_v, \ell_v, P_v)$ verify*

(i) *$(\mathfrak{a}_v, h_v)$ is an R-representation of the locally closed set $S_v = S((\mathfrak{a}_v, h_v))$ and B_v is a finite subset of $K[\mathbf{u}, \mathbf{x}]$.*
(ii) *S_v is parametric, $\mathrm{lpp}(B_v)$ is the minimal generating set of $\mathrm{lpp}(S_v)$ and B_v specializes to the reduced Gröbner basis of I_a (up to normalization) for every $a \in S_v$.*

```
T ← BuildTree(F)
Input: F ⊂ K[u][x] a finite set of homogeneous polynomials
       generating the ideal I
Output: T: the tree where all data is stored. At the terminal
       vertices these data form a disjoint Gröbner cover.

begin
    T=empty global variable which will be filled with all the
        vertices of the tree by RecBuildTree
    r = (∅, (⟨0⟩, 1), F, 0, ∅)  # the root node
    RecBuildTree(r)
    return(T)
end
```

Algorithm A.6: BUILDTREE algorithm

RecBuildTree$(\alpha, (\mathfrak{a}, h), B, \ell, P)$
Input:
$(\alpha, (\mathfrak{a}, h), B, \ell, P)$: vertex of the tree T at which RECBUILDTREE is called.
RECBUILDTREE builds recursively the whole tree T under the calling vertex. At each step it stores the calling vertex, and the algorithm calls himself with the values of the two new generated vertices.
Output:
After the algorithm has finished, the global tree variable $T = \{v_1, \ldots, v_s\}$ will contain all the vertices of the tree $v_i = (\alpha_i, (\mathfrak{a}_i, h_i), B_i, \ell_i, P_i)$.
T is a global variable containing the tree in construction

begin
$T = T \cup (\alpha, (\mathfrak{a}, h), B, l, P)$
$B' = B$
if $l < |B|$ **then**
$(B', (\mathfrak{a}_0, h_0), \ell_0, P_0, (\mathfrak{a}_1, h_1), \ell_1, P_1) =$ DISCUSSPOLYS$((\mathfrak{a}, h), B, \ell, P)$
$\ell = \ell_0$
end if
if $\ell = |B'|$ and $B' \neq \emptyset$ **then**
$(B', (\mathfrak{a}_0, h_0), \ell_0, P_0, (\mathfrak{a}_1, h_1), \ell_1, P_1) =$ DISCUSSSPOLYS$(B', (\mathfrak{a}, h), l, P)$
end if
if $\mathfrak{a}_0 \neq \langle 1 \rangle$ **then**
Create two new vertices v_0 and v_1 descending from v
RECBUILDTREE$(v_0, (\mathfrak{a}_0, h_0), B', \ell_0, P_0)$
RECBUILDTREE$(v_1, (\mathfrak{a}_1, h_1), B', \ell_1, P_1)$
else # {then $P = \emptyset$}
$B' =$ REDGB(MINGB(B'))
Store $((\mathfrak{a}, h), B')$ in v.
end if
end

Algorithm A.7: RECBUILDTREE algorithm

(iii) *The S_v's are pairwise disjoint and cover the whole L^m (as v ranges over all terminal vertices).*
(iv) $\ell = |B|$.
(v) $P = \emptyset$.

So in essence the terminal vertices of BUILDTREE *give a disjoint Gröbner cover (CGS) of L^m with respect to I. Nevertheless, it is not yet the canonical Gröbner cover.*

Proof The Algorithm BUILDTREE: A.6 begins creating the empty global variable T and calls the recursive algorithm RECBUILDTREE with the values of the root vertex $r = ((0), (\langle 0 \rangle, 1), F, 0, \emptyset)$.

When RECBUILDTREE is called at the vertex $(\alpha, (\mathfrak{a}, h), B, \ell, P)$, then either v becomes a terminal vertex or the algorithm has to split, so that v has two successor vertices $(\alpha, 0)$ and $(\alpha, 1)$ and RECBUILDTREE calls itself at $(\alpha, 0)$ and $(\alpha, 1)$ with the new vertex data $((\alpha, 0), (\mathfrak{a}_0, h_0), B_0, \ell_0, P_0)$, $((\alpha, 1), (\mathfrak{a}_1, h_1), B_1, \ell_1, P_1)$ respectively, which are the output of DISCUSSPOLYS. We note that in the second case we have $\mathfrak{a}_0 \neq \langle 1 \rangle$ and $\mathfrak{a}_0 \supsetneqq \mathfrak{a}$ by Lemmas A.4 and A.5.

If we follow RECBUILDTREE along a path from the root down the tree, then we find that essentially it performs the usual Buchberger algorithm: First, RECBUILDTREE repeatedly calls DISCUSSPOLYS until we obtain a basis B such that the leading coefficient of every polynomial in B is non-zero at every point of the current locally closed set S (i.e $l = |B|$). Then it calls DISCUSSSPOLYS and advances in the Buchberger algorithm. If at some vertex v, it takes the right branch to v_0 when DISCUSSSPOLYS has found a splitting, then the new vertex data $(\mathfrak{a}_0, h_0, B_0, \ell_0, P_0)$ at v_0 satisfies $l_0 = |B_0| - 1$ and DISCUSSSPOLYS will be called. If it takes the right branch to v_1, then the new vertex data $((\mathfrak{a}_1, h_1), B_1, \ell_1, P_1)$ satisfies $\ell_1 = |B_1|$ and DISCUSSSPOLYS will advance in the Buchberger algorithm.

Claim (i) is obvious and Claim (iii) is immediate from the algorithm and Lemmas A.4 and A.5.

Suppose the algorithm eventually reaches a terminal vertex v. This can only happen if the current list P is empty, and we see from Lemma A.5, that for each a in the current locally closed subset $S = S_v$, the current basis B' specializes to a Gröbner basis of I_a. Now it only remains for RECBUILDTREE to minimize and to reduce the basis. The algorithm stores the new basis in v and quits. Thus, as claimed in (ii), we see that $\mathrm{lpp}(B_v)$ is the minimal generating set of $\mathrm{lpp}(I_a)$ and that B_v specializes, up to normalization, to the reduced Gröbner basis of I_a for every $a \in S_v$. Next we will prove that S_v is parametric. In general the elements of B_v need not lie in I, because we have reduced them modulo the assumed null-conditions. But by construction for every $p \in B_v$, there are polynomials $p' \in I$ and $q' \in K[\mathbf{u}]$ such that $q'(a) \neq 0$ and $q'(a)p(a, \mathbf{x}) = p'(a, \mathbf{x})$ for all $a \in S_v$, and so

$$\frac{p(a, \mathbf{x})}{\mathrm{lc}(p)(a)} = \frac{q'(a)p(a, \mathbf{x})}{q'(a)\,\mathrm{lc}(p)(a)} = \frac{p'(a, \mathbf{x})}{\mathrm{coef}(p', \mathrm{lpp}(p))(a)}$$

for all $a \in S_v$, proving that S_v is parametric (cf. Remark 5.2). □

A.4 Comparison of BUILDTREE and KAPUR-SUN-WANG Algorithms

We have already commented that when the library "grobcov.lib" was implemented for the first time in 2008, we used the BUILDTREE algorithm for obtaining the previous CGS needed for the `grobcov` command. Since 2012, we have replaced it by the more efficient KSW algorithm.

The GROBCOV algorithm builds the canonical object. It follows the following steps:

(1) Homogenizes the ideal.
(2) Computes a disjoint CGS with constant lpp's.
(3) Groups the segments by lpp of the reduced Gröbner basis.
(4) Dehomogenizes and reduces the bases.
(5) Computes the homogenized-lpp-segments grouped in (3) by adding together all the segments with the same lpp.
(6) Computes the generic (and the extended) basis for the lpp segments.

Step (2) computes a CGS, which needs to contain disjoint segments with constant lpp's, and must be described in a canonical form in order to be able to be added together with the other segments having the same lpp. By Wibmer's Theorem ([85]), their union is locally closed and allows a unique basis $B_i \subset \mathcal{O}_{S_i}[X]$. In several papers [56, 58], and more precisely in [63], we presented the algorithm (BUILDTREE) for this purpose. But since 2012, we have replaced it in the "grobcov.lib" library by the KSW algorithm.

We tested many examples for comparing efficiency of both algorithms. Here are some results for problems of medium difficulty. Computations have been done with a MacBook Pro, Version 10.7.4, with 2.3 GHz Intel Core i7 processor and 4GB 1600 MHz DDR3 memory. Time is given in seconds. The examples can also be used as interesting problems to be studied using GROBCOV.

Performance of the Gröbner Cover algorithm using KSW/BUILDTREE inside.

	Using Buildree		Using KSW		GrobCov	Time ratio
System	Time	Segments	Time	Segments	Segments	KSW/BT
S10	1.78	6	0.24	6	3	0.13
S11	0.40	6	1.26	23	6	3.15
S12	8.28	31	0.98	24	11	0.12
S15	61.70	93	2.26	31	22	0.04
S16	1.98	36	0.43	16	5	0.22
S20	0.51	8	0.30	7	7	0.59
S36	0.85	8	0.17	6	6	0.20
S47	15.22	18	0.52	9	7	0.03
S53	1.75	9	0.29	7	5	0.17
S54	4.23	23	0.66	2	2	0.16
S58	7.71	88	1.33	31	12	0.17
S59	3.72	66	0.89	25	12	0.33
S64	–	–	15.18	47	34	0.00
S82	1.30	28	0.89	22	10	0.68
S86	1.83	15	0.51	13	9	0.28
S92	0.35	5	0.40	2	2	1.14
S93	–	–	3.42	14	9	0.00

The examples are taken from real applications or from test problems with special difficulties. S_{10} discusses the inverse kinematic problem of a simple robot arm [58]. S_{11}, S_{12} and S_{15} also come from robotics ([24, 38, 73]). S_{16} studies a tensegrity problem [68]. S_{20} studies an electrical network [57]. S_{47}, S_{53}, S_{54}, S_{64}, S_{93} are examples of automatic demonstration of theorems, some of which have already been discussed in the book, in Chap. 6, Sect. 6.4, using the more elaborated `ADGT` command. S_{47} is an automatic proof of the nine points circle theorem. S_{53} is the problem of automatically discovering the conditions for having an isosceles ortic triangle. S_{54} is also an automatic discovering theorem problem, proposed in the pastimes section of the French journal "Le Monde" in the printed edition of Jan. 8, 2007 as a problem of two skaters. Both problems S_{53} and S_{54} are discussed in [60]. Problem S_{64} comes from Mechanical Geometry Theorem Proving [23]. S_{92} is an example of the Casas-Alvero Conjecture and S_{93} is the proof of the Generalized Steiner-Lehmus Theorem (see Chap. 6). S_{58} and S_{59} are problems where bases containing I-regular functions described by multiple polynomials are expected and was proposed by the author.

In the following examples, parameters appear inside parenthesis. The parametric ideals are:

$$
\begin{aligned}
S_{10} =& s_1 s_2 l - c_1 c_2 l - c_1 + (r), -s_1 c_2 l - s_1 - c_1 s_2 l + (z), s_1^2 + c_1^2 - 1, \\
& s_2^2 + c_2^2 - 1; \\
S_{11} =& (r d_3 d_4 - r + Z - r_2^2 d_3 d_4 + r_2^2 - d_3^3 d_4 + d_3^2 d_4^2 - d_3 d_4^3 + d_3 d_4) t^4 \\
& + (-2 r r_2 d_4 + 2 r_2^3 d_4 + 2 r_2 d_3^2 d_4 - 4 r_2 d_3 d_4^2 + 2 r_2 d_4^3 + 2 r_2 d_4) t^3 \\
& + (-2r + 2Z + 4 r_2^2 d_4^2 + 2 r_2^2 - 2 d_3^2 d_4^2 + 4 d_4^2) t^2 \\
& + (-2 r r_2 d_4 + 2 r_2^3 d_4 + 2 r_2 d_3^2 d_4 + 4 r_2 d_3 d_4^2 + 2 r_2 d_4^3 + 2 r_2 d_4) t \\
& + (-r d_3 d_4 - r + Z + r_2^2 d_3 d_4 + r_2^2 + d_3^3 d_4 + d_3^2 d_4^2 + d_3 d_4^3 - d_3 d_4); \\
S_{12} =& (-\ell_2) c_2 + (-\ell_1) c_1 + (a), \ (-\ell_2) s_2 + (-\ell_1) s_1 + (b), \ c_1^2 + s_1^2 - 1, \\
& c_2^2 + s_2^2 - 1; \\
S_{15} =& (d) s_1 + (a), (-d) c_1 + (b), (l_3) c_3 + (l_2) c_2 + (-d), (l_3) s_3 + (l_2) s_2 + (-c), \\
& c_1^2 + s_1^2 - 1, c_2^2 + s_2^2 - 1, c_3^2 + s_3^2 - 1; \\
S_{16} =& (x - y - z) w_5, (z) w_4 + (-z) w_5, (x + y - 1) w_4 + (-z + 1) w_5, \\
& (y - 1) w_3 + (y) w_4 + (-2z + 1) w_5, (x - z) w_3 + (-y) w_4, \\
& (z - 1) w_2 + (z) w_3 + (2z - 1) w_5, (y - 1) w_2 + (y + 2z - 1) w_5, \\
& (x - 1) w_2 + (z) w_3 + (y + 2z - 1) w_5;
\end{aligned}
$$

$$
\begin{aligned}
S_{20} =& -12e_2 - 110f_2 - 2e_3 - 10f_3 + (-P_1 + 14),\\
& -2200e_2 + 240f_2 - 200e_3 + 40f_3 + (-20Q_1 + 2397),\\
& 16e_2^2 + 16f_2^2 - 4e_2e_3 + 20f_2e_3 - 20e_2f_3 - 4f_2f_3 - 12e_2 + 110f_2 + (-P_2),\\
& 2599e_2^2 + 2599f_2^2 - 400e_2e_3 - 80f_2e_3 + 80e_2f_3 - 400f_2f_3 - 2200e_2\\
& -240f_2 + (-20Q_2);\\
S_{36} =& (ab)x^2y + z,\ (ac)yz + z,\ (a)x + 2y - z;\\
S_{47} =& x_0^2 + y_0^2 - r_2, x_0^2 + y_0^2 + (-2a-2)x_0 + (-2b)y_0 - r_2 + (a^2 + 2a + b^2 + 1),\\
& x_0^2 + y_0^2 + (-2a+2)x_0 + (-2b)y_0 - r_2 + (a^2 - 2a + b^2 + 1),\\
& x_0^2 + y_0^2 + (-4a)x_0 - r_2 + (4a^2),\\
& x_1^2 + y_1^2 - 2x_1x_0 + x_0^2 - 2y_1y_0 + y_0^2 + (2a)x_1 + (2b)y_1 + (-2a)x_0\\
& + (-2b)y_0 - r_2 + (a^2 + b^2), x_1 + (-a), (a+1)x_1 + (b)y_1 + (-a-1),\\
& x_1^2 + y_1^2 - 2x_1x_0 + x_0^2 - 2y_1y_0 + y_0^2 + 2x_1 - 2x_0 - r_2 + 1,\\
& x_1^2 + y_1^2 - 2x_1x_0 + x_0^2 - 2y_1y_0 + y_0^2 - 2x_1 + 2x_0 - r_2 + 1,\\
& (b)x_2 + (-a-1)y_2 + (2b), (a+1)x_2 + (b)y_2 + (-2a-_2),\\
& x_2^2 + y_2^2 - 2x_2x_0 + x_0^2 - 2y_2y_0 + y_0^2 - r_2,\\
& (b)x_3 + (-a+1)y_3 + (-2b), (a-1)x_3 + (b)y_3 + (2a-2),\\
& x_3^2 + y_3^2 - 2x_3x_0 + x_0^2 - 2y_3y_0 + y_0^2 - r2;\\
S_{53} =& (-b)x_2 + (a-1)y_2 + (b), (a-1)x_2 + (b)y_2 + (a-1),\\
& (b)x_3 + (-a-1)y_3 + (b), (a+1)x_3 + (b)y_3 + (-a-1),\\
& -x_2^2 + x_3^2 - y_2^2 + y_3^2 + (2a)x_2 + (-2a)x_3;\\
S_{54} =& x_1^2 + y_1^2 + (-2a)x_1 - 2y_1, x_2^2 + y_2^2 + (2b)x_2 - 2y_2,\\
& (a)x_1 + y_1 + (a^2c_v - a^2 + c_v - 1), (-b)x_2 + y_2 + (b^2c_w - b^2 + c_w - 1),\\
& -x_1 + (a)y_1 + (a^2s_v + s_v), -x_2 + (-b)y_2 + (b^2s_w + s_w),\\
& -y_1x_2 + x_1y_2 - 2x_1 + 2x_2;\ a^2 + 1 \neq 0, b^2 + 1 \neq 0, a + b \neq 0;\\
S_{58} =& (a_0)x^2 + (b_0)y + (c_0), (a_1)x^2 + (b_1)y + (c_1);\\
S_{59} =& (a_0)x^2 + (b_0)xy + (c_0)y^2, (a_1)x^2 + (b_1)xy + (c_1)y^2, (a_2)x + (b_2)y;
\end{aligned}
$$

$$
\begin{aligned}
S_{64} =&(-2u_3)x_2 + (2u_1 - 2u_2)x_3 + (-u_1^2 + u_2^2 + u_3^2),\\
&(-2u_3)x_4 + (-2u_2)x_5 + (u_2^2 + u_3^2),\\
&(-u_1^2 + u_1u_2)x_2 + (-u_1u_3)x_3\\
&+ (-x_1u_1^2 + 2x_1u_1u_2 - x_1u_2^2 - x_1u_3^2 + u_1^2u_3),\\
&(-u_1u_2)x_4 + (u_1u_3)x_5 + (-x_1u_2^2 - x_1u_3^2),\\
&- x_2x + x_3y,\ -x_4x + x_5y + (u_1)x_4 + (-u_1)y,\\
&(-x_1 + u_3)x + (\frac{1}{2}u_1 - u_2)y + (x_1u_2 - \frac{1}{2}u_1u_3);\\
S_{82} =&(a)x^2 + (b)xy + (c)z^2, (d)x^2 + (e)xy + (f)z^2;\\
S_{86} =&(-c)xz + (c)yw - x + (a),\ (-c)yz + (-c)xw - y + (b),\\
&x^2 + y^2 - 1,\ z^2 + w^2 - 1;\\
S_{92} =&x1^5 + (5a_4)x1^4 + (10a_3)x_1^3 + (10a_2)x_1^2 + (5a_1)x_1 + (a_0),\\
&x_1^4 + (4a_4)x_1^3 + (6a_3)x_1^2 + (4a_2)x_1 + (a_1),\\
&x_2^5 + (5a_4)x_2^4 + (10a_3)x_2^3 + (10a_2)x_2^2 + (5a_1)x_2 + (a_0),\\
&x_2^3 + (3a4)x2^2 + (3a3)x2 + (a2),\\
&x_3^5 + (5a_4)x_3^4 + (10a_3)x_3^3 + (10a_2)x_3^2 + (5a_1)x3 + (a_0),\\
&x_3^2 + (2a_4)x_3 + (a_3),\\
&x_4^5 + (5a_4)x_4^4 + (10a_3)x_4^3 + (10a_2)x_4^2 + (5a_1)x_4 + (a0),\\
&x_4 + (a_4);\\
S_{93} =&p^2 + (-x^2 - y^2), (-y)a + bp + (x)b,\ (-y)a + (x-1)b + (y),\\
&r^2 - 2r + (-x^2 + 2x - y^2),\ (-y)m + nr + (x-2)n + (y),\\
&(-y)m + (x)n,\ a^2 + b^2 - m^2 + 2m - n^2 - 1.
\end{aligned}
$$

A.5 Conclusions

It can be observed that, in problems of medium-high difficulty, in general the performance of the Gröbner Cover algorithm using KSW is much better than with BUILDTREE. The speed increases up to 30 times in the best cases even if there are particular problems for which nothing is gained. One of the reasons for the better performance lies not only in the speed of the KSW algorithm but also in the fact

that, in general, KSW produces fewer number of segments than BUILDTREE, and this makes the remaining parts of the algorithm less expensive.

For problems of small difficulty there are no particular differences between both methods, and even BUILDTREE can be more efficient, this is not significant. It can be observed that efficiency increases considerably when the number of segments of the CGS is smaller for KSW than for BUILDTREE.

Another interesting result of the comparison between both algorithms is that it provides evidence of the canonical character of the Gröbner Cover algorithm. Even if the CGS computation produces very different results in both procedures, the output of the Gröbner Cover algorithm after steps (3), (4), (5), (6) is always the same.

Bibliography

1. M. A. Abanades, F. Botana, A. Montes, T. Recio, An algebraic taxonomy for locus computation in dynamic geometry, Computer Aided Geometrical Design 56 (2014) 22–33.
2. W. W. Adams, P. Loustaunau, An Introduction to Gröbner Bases, vol. 3, 1st ed., American Mathematical Society, 1994.
3. M. R. Adhikari, A. Adhikari, Basic Modern Algebra with Applications, Springer, New Delhi, 2014.
4. J. Allouche, Note on the constructible sets of a topological space, in: Papers on general topology and applications (Gorham, ME, 1995), 1–10, vol. 806, Annals of New York Academy of Sciences, 1996, pp. 1–10.
5. P. Aubry, D. Lazard, M. Moreno-Maza, On the theories of triangular sets, Journal of Symbolic Computation 28 (1999) 105–124.
6. T. Becker, V. Weispfenning, Gröbner Bases: A Computational Approach to Commutative Algebra, 1st ed., Springer-Verlag, Berlin, 1993.
7. F. Botana, Bringing more intelligence to dynamic geometry by using symbolic computation, in: Symbolic Computation and Education, World Scientific, 2007, pp. 136–150.
8. F. Botana, On the parametric representation of dynamic geometry constructions, in: B. Murgante, O. Gervasi, A. Iglesias, D. Taniar, B. Apduhan (eds.), Computational science and its applications–ICCSA 2011, Lecture Notes in Computer Science, Springer, 2011, pp. 342–352.
9. F. Botana, Z. Kovacs, New tools in GeoGebra offering novel opportunities to teach loci and envelopes, Computing Research Repository CoRR abs/1605.09153.
10. F. Botana, T. Recio, Computing envelopes in dynamic geometry environments, Annals of Mathematics and Artificial Intelligence (2016) 1–18.
11. F. Botana, J. L. Valcarce, A software tool for the investigation of plane loci, Mathematics and Computers in Simulation 61 (2) (2003) 139–152.
12. J. Bruce, P. Giblin, Curves and Singularities, Cambridge University Press, 1984.
13. J. M. Brunat, A. Montes, Computing the canonical representation of constructible sets, International Journal of Mathematics in Computer Science 10 (2016) 165–178.
14. B. Buchberger, Ein Algorithmus zum Auffinden der Basiselemente des Restklassenrings nach einem nulldimensionalen Polynomideal, Ph.D. Dissertation, University of Innsbruck, 1965.
15. B. Buchberger, An algorithm for finding the basis elements of the residue class ring of a zero dimensional polynomial ideal, translation of Ph.D. Thesis, Journal of Symbolic Computation 41 (2006) 471–511.
16. Cabri, J.M. Laborde and F. Bellemain, http://www.cs.wichita.edu/ye, Personal web page (2014).

© Springer Nature Switzerland AG 2018

A. Montes, *The Gröbner Cover*, Algorithms and Computation in Mathematics 27, https://doi.org/10.1007/978-3-030-03904-2

17. C. Chen, J. H. Davenport, M. Moreno-Maza, B. Xia, R. Xiao, Computing with semi-algebraic sets represented by triangular decomposition, in: A. Leykin (ed.), Proceedings of ISSAC'2011, ACM Press, New York, 2011, pp. 75–82.
18. C. Chen, O. Golubitsky, F. Lemaire, M. Moreno-Maza, W. Pan, Comprehensive triangular decomposition, in: Proceedings of CASC 2007, Lecture Notes on Computational Science and Engineering, Springer, Heidelberg, 2007, pp. 73–101.
19. C. Chen, F. Lemaire, L. Li, M. M. Maza, W. Pan, Y. Xie, The ConstructibleSetTools and ParametricSystemTools modules of the RegularChains library in Maple, in: Proceedings of ICCSA'2008, IEEE Computer Society, 2008, pp. 342–352.
20. C. Chen, L. Li, M. Moreno-Maza, W. Pan, Y. Xie, On the representation of constructible sets, in: Proceedings of Milestones in Computer Algebra 2008, Trinidad and Tobago, 2008, pp. 103–108.
21. C. Chen, M. Moreno-Maza, Algorithms for computing triangular decomposition of polynomial systems, Journal of Symbolic Computation 47 (6) (2012) 610–642.
22. C. Chen, M. Moreno-Maza, Solving parametric polynomial systems by RealComprehensive-Triangularize, in: Mathematical Software-ICMS 2014, Springer, 2014, p. 1st Chapter.
23. S.-C. Chou, Mechanical Geometry Theorem Proving, chap. Example 5.7. p.71. Paterson. Example 5.9., Springer, 1987, p. 73.
24. M. Coste, Classifying serial manipulators: Computer algebra and geometric insight, in: Proceedings of EACA-2004, Santander, 2004, pp. 323–324.
25. D. Cox, J. Little, D. O'Shea, Ideals, Varieties and Algorithms, 3rd ed., Springer, New-York, 1992.
26. D. Cox, J. Little, D. O'Shea, Using Algebraic Geometry, Springer, New-York, 1998.
27. G. Dalzotto, T. Recio, On protocols for the automated discovery of theorems in elementary geometry, Journal of Automated Reasoning 43 (2) (2009) 203–236.
28. W. Decker, C. Lossen, Computing in Algebraic Geometry. A Quick Start using SINGULAR, Springer, Berlin-Heidelberg-New York, 2006.
29. A. Dolzmann, T. Sturm, Redlog: Computer algebra meets computer logic, ACM Sigsam Bulletin 31/2 (1997) 2–9.
30. D. Eisenbud, Commutative Algebra with a View Toward Algebraic Geometry, corrected 3rd printing (1999) ed., Springer, New York, 1994.
31. J. Escribano, F. Botana, M. Abánades, Adding remote computational capabilities to dynamic geometry systems, Mathematics and Computers in Simulation 80 (2010) 1177–1184.
32. J. Faugère, A new efficient algorithm for computing Gröbner bases without reducing to zero (f_5), in: Proceedings of ISSAC'02, ACM, 2002, pp. 75–83.
33. X. Gao, Automated geometry diagram construction and engineering geometry, in: ADG 1998, vol. 1669 of Lecture Notes in Artificial Intelligence, Springer, 1999, pp. 232–257.
34. R. Gebauer, H. Möller, On an installation of Buchberger's algorithm, Journal of Symbolic Computation 6 (1988) 275–286.
35. GeoGebra, web page, http://www.geogebra.org (2014).
36. P. Gianni, B. Trager, G. Zacharias, Gröbner bases and primary decomposition of polynomial ideals, Journal of Symbolic Computation 6 (1988) 149–167.
37. A. Giovinni, T. Mora, G. Niesi, L. Robbiano, C. Traverso, One sugar cube, please: OR selection strategies in the Buchberger algorithm, in: Proceedings ISSAC'91, ACM, 1991, pp. 49–54.
38. M. J. González-López, T. Recio, Computer Algebra in Industry, chap. The ROMIN inverse geometric model and the dynamic evaluation method, Wiley & Sons, 1993, pp. 117–141.
39. G.-M. Greuel, G. Pfister, A Singular Introduction to Commutative Algebra, 2rd (2004) ed., Springer, 2000.
40. Java, Geometry Expert, http://www.cs.wichita.edu/ye (2014).
41. M. Kalkbrenner, On the stability of Gröbner bases under specializations, Journal of Symbolic Computation 24 (1997) 51–58.
42. D. Kapur, Principles and Practices of Constraints Programming, chap. An Approach for Solving Systems of Parametric Polynomial Equations, MIT Press, 1995, pp. 217–244.

43. D. Kapur, Y. Sun, D. Wang, A new algorithm for computing comprehensive Gröbner systems, in: Proceedings of ISSAC'2010, ACM Press, New York, 2010, pp. 29–36.
44. G. Kemper, Preprint: Morphisms and constructible sets: Making two theorems of Chevalley constructive (2007).
45. Z. Kovacs, The Relation Tool in GeoGebra 5, Springer International Publishing Cham, 2015, pp. 53–71.
46. Z. Kovacs, Real-time animated dynamic geometry in the classroom by using fast Gröbner basis computation, Mathematics in Computer Science 11.
47. Z. Kovacs, T. Recio, P. Velez, Reasoning about linkages with dynamic geometry, talk at the Conference on Geometry: Theory and Applications, Pilsen, Czech Republic (2017).
48. D. Lazard, F. Rouillier, Solving parametric polynomial systems, Journal of Symbolic Computation 42 (2007) 636–667.
49. A. Leykin, Constructibility of the set of polynomials with a fixed Bernstein-Sato polynomial: an algorithmic approach, Journal of Symbolic Computation 32(6) (2001) 663–675.
50. A. López, A. Villa, Geometría Diferencial, CLAGSA, 1997.
51. R. Losada, T. Recio, J. L. Valcarce, Sobre el descubrimiento automático de diversas generalizaciones del teorema de Steiner-Lehmus, Boletín de la Sociedad Puig Adam 82 (2009) 53–76.
52. R. Losada, T. Recio, J. L. Valcarce, On the automated discovery of Steiner-Lehmus generalizations, in: J. Richter-Gebert, P. Schrek (eds.), Proceedings of ADG'2010, Springer, München, 2010, pp. 171–174.
53. R. Losada, T. Recio, J. L. Valcarce, Equal bisectors at a vertex of a triangle, in: B. Murgante, et al. (eds.), Proceedings of ICCSA'2011, vol. 6785 of Lecture Notes in Computer Science, Springer, Berlin, 2011, pp. 328–341.
54. M. Manubens, Ph. Thesis: Parametric Polynomial System Discussion: Canonical Comprehensive Gröbner Systems, Universitat Politècnica de Catalunya, 2008.
55. M. Manubens, A. Montes, Improving dispgb algorithm using the discriminant ideal, Journal of Symbolic Computation 41 (2006) 1245–1263.
56. M. Manubens, A. Montes, Minimal canonical comprehensive Gröbner systems, Journal of Symbolic Computation 44 (5) (2009) 463–478.
57. A. Montes, Algebraic solution of the load-flow problem for a 4-nodes electrical network, Mathematics and Computers in Simulation 45 (1998) 163–174.
58. A. Montes, A new algorithm for discussing Gröbner bases with parameters, Journal of Symbolic Computation 22 (2002) 183–208.
59. A. Montes, Using Kapur-Sun-Wang algorithm for the Gröbner Cover, in: R. Sendra, C. Villarino (eds.), Proceedings of EACA 2012, Universidad de Alcalá de Henares, 2012, pp. 135–138.
60. A. Montes, T. Recio, Automated discovery of geometry theorems using minimal canonical comprehensive Gröbner systems, in: Proceedings of ADG'2006, vol. 4869 of Lecture Notes in Artificial Intelligence, Springer, 2007, pp. 103–138.
61. A. Montes, T. Recio, Generalizing the Steiner-Lehmus theorem using the Gröbner cover, Mathematics and Computers in Simulation 104 (2014) 67–81.
62. A. Montes, H. Schönemann, Singular "grobcov.lib" library D.2.4, http://www.singular.uni-kl.de, Computer Algebra System for polynomial computations. Center for Computer Algebra, University of Kaiserslautern, free software under the GNU General Public Licence (2017).
63. A. Montes, M. Wibmer, Gröbner bases for polynomial systems with parameters, Journal of Symbolic Computation 45 (2010) 1391–1425.
64. T. Mora, Solving Polynomial Equation Systems I: The Kronecker-Duval Philosophy, vol. 88 of Encyclopedia of Mathematics and its Applications, Cambridge University Press, 2003.
65. T. Mora, Solving Polynomial Equation Systems II: Macaulay's Paradigm and Gröbner Technology, Encyclopedia of Mathematics and its Applications, Cambridge University Press, 2005.
66. K. Nabeshima, A speed-up of the algorithm for computing comprehensive Gröbner systems, in: Proceedings of ISSAC'2007, ACM Press, New York, 2007, pp. 299–306.
67. J. O'Halloran, M. Schilmoeller, Gröbner bases for constructible sets, Communications in Algebra 30:11 (2002) 5479–5483.

68. D. Orden, M. de Guzmán, Finding tensegrity structures: geometric and symbolic approaches, in: Proceedings of EACA-2004, 2004, pp. 167–172.
69. M. Pesh, Computing Comprehensive Gröbner Bases using MAS, User Manual, Passau University, 1994.
70. H. Pottmann, M. Peternell, Envelopes - Computational Theory and Applications, in: B. Falcidieno (ed.), Proceedings of the Spring Conference on Computer Graphics 2000, Comenius University, Bratislava, 2000, pp. 3–23.
71. T. Recio, P. Vélez, Automated discovery of theorems in elementary geometry, Journal of Automated Reasoning 23 (1999) 63–82.
72. T. Recio, P. Vélez, Numerical and symbolic scientific computing: progress and prospects, chap. An Introduction to automated discovery in geometry through symbolic computation, Texts and Monographs in Symbolic Computation, Springer, Wien, 2011, pp. 257–271.
73. M. Rychlik, Complexity and applications of parametric algorithms of computational algebraic geometry, in: R. de la Llave, L. Petzold, J. Lorenz (eds.), Dynamics of Algorithms, vol. 118 of IMA Volumes in Mathematics and its Applications, Springer, 2000, pp. 1–29.
74. Sage, web page, http://www.geogebra.org, Sage Cell Server.
75. Y. Sato, A. Suzuki, An alternative approach to comprehensive Gröbner bases, Journal of Symbolic Computation 36:3–4 (2003) 649–667.
76. T. Sillke, http://www.mathematik.uni-bielefeld.de/~sillke/PUZZLES/steiner-lehmus.
77. Sketchpad, J. Jachiw, https://sketch.io/sketchpad/, the Geometer Sketchpad v. 4.0 (2014).
78. M. Spivak, Calculus on Manifolds, Addison-Wesley Publishing Company, 1965.
79. Sutherland, http://www.cl.cam.ac.uk/techreports/ (2014).
80. A. Suzuki, Y. Sato, A simple algorithm to compute comprehensive Gröbner bases, in: Proceedings of ISSAC'2006, ACM, New York, 2006, pp. 326–331.
81. A. Suzuki, Y. Sato, Implementation of CGS and CGB on Risa/Asir and other computer algebra systems using Suzuki-Sato algorithm, ACM Communications in Computer Algebra 41:3.
82. D. Wang, Elimination practice: software tools and applications, Imperial College Press, London, 2004.
83. V. Weispfenning, Comprehensive Gröbner bases, Journal of Symbolic Computation 14 (1992) 1–29.
84. V. Weispfenning, Canonical comprehensive Gröbner bases, Journal of Symbolic Computation 36:3–4 (2003) 669–683.
85. M. Wibmer, Gröbner bases for families of affine or projective schemes, Journal of Symbolic Computation 42 (2007) 803–834.
86. W. T. Wu, X. L. Lü, Triangles with equal bisectors, People's Education Press, Beijing, 1985, in Chinese.
87. R. Zalaya, Tesis doctoral: Escultura Matemática, Universidad Politécnica de Valencia, 2005.

Index

Abánades, Miguel Angel, viii, xii, 126, 153
Algorithm
  AD, 139
  AddPart, 30
  ADGT, 113
  AssocTanToEnv, 187
  Buchberger, 7
  BuildTree, 253, 260
  Canonical Gröbner Cover, 88
  CGSDR, 89
  Combine, 71, 89
  CompLev, 111
  ConsLevels, 27
  Crep, 16
  Delta, 71
  DiscussPolys , 256
  DiscussSpolys, 258
  Envelop, 186
  Extend, 71
  FamElemsAtEnvCompPoints, 187
  FirstLevel, 27
  GCover, 88
  Grobcov, 90
  KSW, 58
  LCUnion, 30, 89
  Levels, 28
  Locus, 139
  Locus2, 137
  Nabeshima, 55
  Nonnull, 45
  NS, 140
  Pdivi, 47
  Pnormalf, 46
  Prep, 19
  primary decomposition, 10
  radical, 44
  RecBuildTree, 261
  Reduce, 4
  ReducedGröbnerBasis, 7, 44
  ReduceGB, 259
  Rrep, 254
  SimplifyUnion, 29
  Split, 255
  stdlocus, 129
  Suzuki-Sato, 55
  WLemma, 81
Automatic Deduction of Geometric Theorems, 99
Automatic Demonstration of Geometric Theorems, 119

Botana, Francisco, viii, xii, 126, 145, 153, 177
Brunat, Josep M., xii, 15, 172
Buchberger, Bruno, vii, 4

Canonical discussion, 12
Canonical Gröbner Cover, 87
Casas-Alvero conjecture, 90
Characterization of solutions, 8
Coefficient, 2
Command
  ADGT, 113
  AssocTanToEnv, 186
  cgsdr, 60
  Conslevels, 35
  Crep, 32
  discrim, 219
  envelop, 186, 188

© Springer Nature Switzerland AG 2018

A. Montes, *The Gröbner Cover*, Algorithms and Computation in Mathematics 27, https://doi.org/10.1007/978-3-030-03904-2

extendGC, 76
extendpoly, 76
FamElemsAtEnvCompPoints, 190
grobcov, 76, 90, 103
Levels, 28, 35
locus, 139
locusdg, 177
locusto, 177
pdivi, 49
pnormalf, 46
Prep, 32
PtoCrep, 32
WLemma, 81, 83
Comparison between BUILDTREE and KSW, 262
Complex Gröbner Cover, 128
Complex locus, 128
Comprehensive Gröbner Basis, 54
Comprehensive Gröbner System, 53
Constructible set
canonical representation, 26
definition, 20
levels, 26

Definition
Accumulation locus component, 133
algebraic envelope, 182
Canonical Gröbner Cover, 87
Comprehensive Gröbner Basis, 54
Comprehensive Gröbner System, 53
constructible set, 20
Degenerate locus component, 133
family of hypersurfaces, 182
Gröbner Cover, 80
I-regular function, 67
locally closed set, 16
NonComparable, 56
Normal locus component, 133
parametric set, 79
regular function, 67
R-representation, 253
Special locus component, 133
Degree, 2
Dolzmann, Andreas, 41

Envelope
"Accumulation" component, 183
algebraic, 213
algebraic definition, 182
algorithm ASSOCTANTOENV, 187
algorithm ENVELOP, 186
algorithm FAMELEMSATENVCOMP-POINTS, 187
intersecting family elements, 219
"Normal" component, 183
parameters domain, 218
real, 181
special points, 215
Euler's circle, 120
Exponent vector, 1

Faithful, 44
Feuerbach circle, 120
Field
algebraically closed extension, 1
computable, 1

Generic representation, 69
GeoGebra, 131, 178
Geometer's Sketchpad, 131
Geometric locus, 125
Gianni, Patricia, 16
Graf von Bothmer, Hans-Christian, 90
grobcov.lib, 11, 32, 42, 49, 53, 54, 60, 76, 83, 90, 103, 148, 188, 192
Gröbner basis, 4, 8
Gröbner Cover, 11, 262
Gröbner, Wolfgang, 4

Hole, 16

Ideal
homogeneous, 84
of variety, 2
parametric, 79
I-regular function
full representation, 70
generic representation, 70

Java Geometry Expert, 131

Kalkbrenner, Michael, 41
Kapur, Deepak, viii, xii, 41
Kovacs, Zoltan, ix, xii, 145, 180

Labs, Oliver, 90
Lario, Joan Carles, 172
Leading
coefficient, 3
monomial, 3
power product, 3
power products of I over S, 79
term, 3

Lemma
COMBINE, 72
DELTA, 71
DISCUSSPOLYS, 256
DISCUSSSPOLYS, 257
Corolari of Wibmer's Lemma, 84
homogeneous versus non-homogeneous GB, 86
Kapur-Sun-Wang, 56
locally closed sets, 21
Suzuki-Sato, 55
Wibmer, 80
Locally closed set
canonical C-representation, 15
canonical P-representation, 15
definition, 16
Gröbner Cover, 84
R-representation, 253
Locus
"Accumulation" component, 133
complex, 128
"Degenerate" component, 133
detecting bad mover positions, 153
mover point, 128
non-normal, 132
normal, 132
"Normal" component, 133
of points, 125
Pascal's Limaçon, 134
Pascal's Limaçon 3*d*, 171
pedal of an ellipsoid, 171
real, 128
"Special" component, 133
taxonomy, 127
tracer point, 128
with two movers, 245
web implementation, 177
lpp-segments, 86

Manubens, Montserrat, xi, 41
Miguel de Guzmán, 180
Minimal generating set, 8, 79
Monomial, 1
Monomial order, 3
Mover point, 126, 128

Nabeshima, Katsusuke, 41
Non-faithfull, 44
Non-normal locus, 132
Normal locus, 132

Offset, 208, 227, 251
Option of `grobcov`
`ext`, 91, 95, 103
`nonnull`, 112
`null`, 112
`rep`, 91
`showhom`, 91
Order
graded reverse lexicographic, 3
lexicographical, 3
monomial, 3
product, 42
term, 3

Parameters, 1, 42
Parametric polynomial, 2
Parametric set, 79, 84
Performance of Gröbner Cover, 262
Pesh, Michael, 41
Power product, 1
Proposition
AddPart, 28
canonical C-representation, 17
canonical P-representation, 18
constructible sets, 22, 23
dimension of a monomial ideal, 9
dimensions in the representations, 19
EXTEND, 73
homogeneous versus non-homogeneous parametric set, 86
loci are constructible sets, 132
SPLIT, 255

Rabinowitz trick, 45, 56
Real Gröbner Cover, 128
Real locus, 128
Recio, Tomás, viii, 99, 126, 145
Reduced Gröbner Basis, 79
Regular function
full representation, 70
generic representation, 69
Ring, parametric, 1
R-representation, 253
Rychlik, Michael, 62
Rychlik robot, 61

Sato, Yosuke, xi, 41, 55
Schönemann, Hans, vii, xi
Schicho, Joseph, 90

Segment
of a CGS, 54
of a constructible set, 26
generic, 11
of a Gröbner Cover, 80
lpp, 11
Singular, vii
grobcov.lib library, xi, 32, 90
Specialization, 42
Specialized function
faithful representation, 44
non-faithful representation, 44
Steiner Deltoid, 198
Steiner-Lehmus
classical Theorem, 100
generalized Theorem, 109
Sturm, Thomas, 41
Sun, Yao, viii, xii, 41
Support, 2
Sutherland, 134
Suzuki, Akira, 41, 55

Taxonomy of locus components, 127
Term, 2
Term order, 3
Theorem
"Accumulation" envelop component, 185
Canonical Gröbner Cover, 85
dimension of a variety, 10
Kalkbrenner, 55
Kapur-Sun-Wang, 57
Nine-Points Circle, 119
normal envelop component, 184
Nullstellensatz, 2
PDIVI and specialization, 49
R-representation, 254
Steiner-Lehmus, 100, 148
Wibmer, 84
Top, 16
Tracer point, 126, 128
Trager, Barry, 17
Trianguar sets, 12

Van de Woestijne, Chritiaan, 90
Variables, 1
Variety
dimension, 8
irreducible, 3
Vélez, Pilar, 145

Wang, Dingkang, viii, xii, 41
Weierstrass form, 11
Weispfenning, Volker, vii, 41
Wibmer, vii
Wibmer
Lemma, 80
Michael, 11, 42
Theorem, 84

Zacharias, 17
Zariski topology, 3, 15, 42